

// ZWIRE //

# zwire

v0.6.60

## THE COMPANION

the Blink base · the rebrand · the HUD · the fork



MENKE  
TECHNOLOGIES

# Copyright and Attribution

**zwire** — a Chromium/Blink browser forked into a tiling HUD

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews and for the source excerpts identified below, which carry their own upstream licenses.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by Google LLC, the Chromium project, its maintainers, the Rust Foundation, or any other project named in these pages.

## The software this book describes

**zwire** is released under the MIT License, Copyright © 2026 MenkeTechnologies. The source is at [github.com/MenkeTechnologies/zwire](https://github.com/MenkeTechnologies/zwire).

## The upstream this book is a fork of

**zwire** is a fork of **Chromium**. The Chromium source is not vendored in the **zwire** repository: [fork/](#) carries the pinned upstream revision in [CHROMIUM\\_VERSION](#), the fetch script that retrieves it, the build arguments, and the patch series that is applied on top. Chromium and its Blink rendering engine are the work of the Chromium authors and contributors and remain under their own license terms; the patches in [fork/patches/](#) are the part of the fork that is copyright MenkeTechnologies.

Short excerpts of Chromium's source are quoted in this book for the purpose of commentary, criticism, and technical explanation. Every such excerpt is cited to the file it came from.

## The fonts this book's software ships

The typefaces bundled with **zwire** are vendored so the HUD renders offline; their licensing is recorded in [fonts/LICENSE](#) in the **zwire** repository.



MT PUBLISHING



MT APP

# Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| Copyright and Attribution                                   | 1  |
| Before You Begin                                            | 6  |
| A Browser-Engine Primer                                     | 9  |
| Chapter 1 — The Pitch                                       | 14 |
| Chapter 2 — Why a Real Blink Base                           | 16 |
| Chapter 3 — The Architecture                                | 18 |
| Chapter 4 — The Base                                        | 21 |
| Chapter 5 — The Runtime Rebrand                             | 24 |
| Chapter 6 — The New-Tab HUD                                 | 26 |
| Chapter 7 — The Internal-HUD Extension: The Injection Model | 29 |
| Chapter 8 — The Eight Schemes and the Sync Bus              | 34 |
| Chapter 9 — Vim on Every Page                               | 40 |
| Chapter 10 — tmux in the Browser                            | 43 |
| Chapter 11 — The Powerline Statusbar                        | 49 |
| Chapter 12 — The Command Palette                            | 52 |
| Chapter 13 — Fuzzy Find                                     | 55 |
| Chapter 14 — The HUD Internal Pages                         | 58 |
| Chapter 15 — The Native Host                                | 62 |
| Chapter 16 — The Terminal                                   | 65 |
| Chapter 17 — zgui-core, the Shared Toolkit                  | 68 |
| Chapter 18 — zpwrchrome, the Power-Tool                     | 70 |
| Chapter 19 — The Launcher and State Layout                  | 73 |
| Chapter 20 — The CDP Verification Layer                     | 76 |
| Chapter 21 — The Source Fork                                | 80 |
| Chapter 22 — Dev Loop, Testing, and CI                      | 87 |

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| Chapter 23 — Packaging and Self-Contained Installs                       | 90  |
| Chapter 24 — Cross-Platform, Updating, and Operations                    | 95  |
| Chapter 25 — The Trust Boundaries and the Threat Model                   | 98  |
| Chapter 26 — The Performance Budget                                      | 100 |
| Chapter 27 — Extending the HUD                                           | 103 |
| Chapter 28 — The Keyboard Arbitration Model                              | 105 |
| Chapter 29 — The Six Boundaries and Their Transports                     | 108 |
| Chapter 30 — Living Inside MV3                                           | 111 |
| Chapter 31 — The Storage Schema and Its Ownership Rules                  | 114 |
| Chapter 32 — When a Hop Fails                                            | 117 |
| Chapter 33 — The Cost of Reuse                                           | 120 |
| Chapter 34 — The Lineage: The Cyberpunk Idiom and the zpwr Fleet         | 123 |
| Chapter 35 — Positioning: zwire Among the Custom Browsers                | 125 |
| Chapter 36 — Failure Modes and Recovery                                  | 127 |
| Chapter 37 — Text Input, Composition, and the Keyboard That Is Not Yours | 130 |
| Chapter 38 — Living on the Open Web                                      | 134 |
| Chapter 39 — Accessibility and the Neon Surface                          | 138 |
| Chapter 40 — Two Browsers, One Host, One Theme File                      | 142 |
| Chapter 41 — Debugging a System With Six Boundaries                      | 145 |
| Chapter 42 — Incognito, and the Extension That Is Not There              | 149 |
| Chapter 43 — The Dead Zones                                              | 152 |
| Chapter 44 — Zoom, Density, and the Geometry That Is Not Yours           | 154 |
| Chapter 45 — The Long Session                                            | 157 |
| Chapter 46 — What Leaves the Machine                                     | 159 |
| Chapter 47 — Teardown, Portability, and the Second Machine               | 162 |
| Chapter 48 — Version Numbers as a Contract                               | 165 |
| Chapter 49 — Lifecycle Hooks: stryke on Browser Events                   | 168 |
| Chapter 50 — Output Triggers: Regex on What the Page Renders             | 171 |
| Chapter 51 — Pane Pipelines: A Dataflow Edge Between Two Panes           | 173 |
| Chapter 52 — The Audio Dashboard                                         | 175 |

|                                                         |     |
|---------------------------------------------------------|-----|
| Chapter 53 — The Later HUD Pages                        | 177 |
| Chapter 54 — The Fourth Transport                       | 179 |
| Chapter 55 — The Verb Surface                           | 182 |
| Chapter 56 — Reversibility as a Declared Property       | 185 |
| Chapter 57 — The Journal the Host Cannot Keep           | 189 |
| Chapter 58 — One Action, Three Deliveries               | 192 |
| Chapter 59 — A Protocol Mirrored by Hand                | 195 |
| Chapter 60 — The Identity Chain                         | 198 |
| Chapter 61 — The Mesh, and the Boundary at Its Edge     | 201 |
| Chapter 62 — Two Info.plists and a Signature            | 204 |
| Chapter 63 — Reading the Series                         | 208 |
| Chapter 64 — The HUD’s Own UI Layer                     | 212 |
| Chapter 65 — The Extension Behind the Clock             | 216 |
| Chapter 66 — Two Bases, One Launcher                    | 219 |
| Chapter 67 — What a Version Bump Breaks                 | 222 |
| Chapter 68 — The Stamp, Audited                         | 225 |
| Chapter 69 — One Installer Stamps; Two Do Not           | 228 |
| Chapter 70 — What the Frame Reader Does With a Lie      | 230 |
| Chapter 71 — Six Ways In, and What Each One Returns     | 233 |
| Chapter 72 — Two Daemons, One Path                      | 236 |
| Chapter 73 — SIGINT, and What Dies With the Host        | 239 |
| Chapter 74 — The Suite, Counted and Run                 | 242 |
| Chapter 75 — What CI Does Not Look At                   | 245 |
| Chapter 76 — Exit Zero, and the Things It Does Not Mean | 248 |
| Chapter 77 — The New Tab Becomes a Layout               | 251 |
| Appendix A — File and Directory Map                     | 255 |
| Appendix B — The State Directory (\$ZWIRE_STATE)        | 257 |
| Appendix C — Keybinding Reference                       | 258 |
| Appendix D — The Eight-Scheme Palette                   | 259 |
| Appendix E — Launch Flag Reference                      | 261 |

|                                                 |     |
|-------------------------------------------------|-----|
| Appendix F — The Chrome WebUI Token Map         | 262 |
| Appendix G — The Native-Messaging Protocol      | 265 |
| Appendix H — The Seeded Keyword Commands        | 266 |
| Appendix I — The Storage Keys                   | 268 |
| Appendix J — The Rust Host Modules              | 270 |
| Appendix K — The Launcher and Build Scripts     | 271 |
| Appendix L — The CDP Verification Probes        | 273 |
| Appendix M — The zpwchrome Manifest             | 274 |
| Appendix N — The Fork Patch Targets             | 275 |
| Appendix O — The Three Extension Manifests      | 276 |
| Appendix P — The Content-Script Injection Order | 278 |
| Appendix Q — The Fuzzy Matcher                  | 279 |
| Appendix R — Visual Effects and Readline        | 280 |
| Appendix S — The System-Stats Sample            | 281 |
| Appendix T — The Palette Destinations           | 282 |
| Appendix U — The Command Bus                    | 283 |
| Appendix V — The Frecency Model                 | 284 |
| Appendix W — The Extension Messaging Graph      | 285 |
| Appendix X — The zgui-core Toolkit Surface      | 286 |
| Appendix Y — The HUD Internal Pages             | 287 |
| Appendix Z — The Dev-Loop Scripts               | 288 |
| Appendix AA — Three Paths, End to End           | 289 |
| Appendix AB — Glossary                          | 290 |
| Appendix AC — The Statusbar Segments            | 291 |
| Appendix AD — Grounding and Edition             | 292 |
| Colophon                                        | 295 |
| Reproducing the Book’s Claims                   | 296 |
| Also from MenkeTechnologies                     | 297 |

# Before You Begin

This is the book about **zwire** — the one you read in a chair, front to back, to understand not only *what* **zwire** does but *why* it is shaped the way it is. It is not the README, which enumerates every switch, path, and patch and which you reach for when you need one fact fast. This book is the other thing: the continuous account of the browser — from the unbranded Blink snapshot it starts from to the neon toolbar the source fork compiles — and of the deliberate line it draws between what a runtime rebrand can do and what only a patched Chromium can.

**zwire** is a **Chromium/Blink browser forked into a tiling cyberpunk HUD**. It is not a WebView wrapper and not a new engine: it is a real Blink base, patched at the seams a compiled fork can reach and extended with a keyboard-driven workspace. The repository states the intent on its own front page — [\[FORKED CHROMIUM · TILING HUD · CYBERPUNK\]](#), with the one-line creed the rest of this book unpacks: “*Not Chrome with a skin — a real Blink engine turned into a keyboard-driven, tiling, cyberpunk workspace.*” It carries a tmux-style tiling overlay, a `⌘K` command palette, durable sessions, a browser-wide audio engine, and the **zpwchrome** power-tool preloaded against a dedicated profile so it never touches your system Chrome.

The reason **zwire** needs a *real* Blink engine, rather than a WebKit shell like Tauri or Servo, is concrete and it drives every decision downstream: the **zpwchrome** extension is Manifest V3 and depends on `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, `webRequest`, and a service-worker background, with `minimum_chrome_version: 127`. None of that runs anywhere but a genuine Chromium engine. The base is therefore a plain, **unbranded Chromium snapshot** — no Google logo, no “for automated testing” stripe — which also happens to retain the `--load-extension` switch that branded Chrome removed in version 137. That single retained switch is what lets **zwire** preload its extensions at all; the book returns to it in Chapter 2.

**zwire** ships two build paths, and the whole shape of the project is the boundary between them. The **runtime rebrand** (the default, no compile) applies HUD colors, the HUD new-tab, the internal-page skin, and your extensions on top of a prebuilt Chromium base — tab shapes stay stock, because a Chrome theme extension can change colors and nothing else. The **source fork** (`fork/`, which compiles Chromium) removes that limit: it restyles the native chrome itself — tab shapes, UI fonts, the neon toolbar, native menus and dialogs, and a browser-wide audio engine — through a series of twenty-seven authored patches against a pinned Chromium tag. This book is honest about which capabilities live on which side of that line, because the line *is* the design.

The whole thing is two commands to stand up and one to run:

```
git clone --recurse-submodules https://github.com/MenkeTechnologies/zwire.git
cd zwire
scripts/install.sh          # fetch base + link `zwire` on PATH + rebrand (macOS)
zwire                      # launch with the HUD new tab
```

This is fast-moving software: the edition string on the cover tracks `zwire/package.json (v0.6.59` at the time of writing), and where a feature needs the compiled fork, or a native host must be installed separately, this book marks it rather than rounding up. **zwire** is a MenkeTechnologies project, built on the Chromium project’s engine, and this book is grounded in that code.

## Provenance

Every figure, file:line citation, and transcript in this edition was read out of one tree at one commit, and both are named here so a reader can reproduce them rather than trust them.

```
tree      MenkeTechnologiesMeta/zwire      (the only checkout of zwire; there is
                                             no Publications/src/zwire counterpart)
branch    main
commit    0097c66abd5401a88e150f0dedbcb8465251091d    2026-08-10 13:06:55 -0400
subject   zwire 0.6.60
```

To recover exactly the tree the later chapters were measured against:

```
git -C zwire fetch origin
git -C zwire checkout 0097c66abd5401a88e150f0dedbcb8465251091d
git -C zwire submodule update --init --recursive
git -C zwire log -1 --format='%H %ci %s'      # must print the line above
```

Submodule figures — the Rust host, the shared toolkit, the power-tool — are read at the pointers that commit records, not at their own tips: [extensions/hud-internal/native/zwire-host](#) at [5622539668](#), [extensions/hud-internal/lib/zgui-core](#) at [f3330e7e95](#), [extensions/zpwchrome](#) at [f7d9737c29](#), with [package.json](#) at [0.6.60](#) and the host's [Cargo.toml](#) at [0.3.15](#). Where a number in this book disagrees with a checkout, check the commit before checking the book: the two most common causes are a submodule left at a newer tip than the superproject records, and a second checkout of the same project somewhere else on the machine.

## How to read this book

The book runs bottom-up, in seventy-seven chapters and thirty appendices. Chapters 1 through 3 are the pitch, the justification, and the architecture in one view — what zwire is, why it must be Chromium, and how the pieces sit against each other. Then it descends the stack: the base, the rebrand, the new tab, and the large internal-HUD extension (Chapters 4 through 7), which gets a chapter per feature it layers on every page — the eight schemes, vim, the tmux overlay, the powerline statusbar, the command palette, the fuzzy find, the HUD internal pages, the native host, and the terminal, in Chapters 8 through 16. Chapter 17 is the shared toolkit all of that is built from. After the internal HUD come the power-tool submodule, the launcher, and the CDP verification probes (18 through 20), then the source fork, the dev-and-CI loop, the packaging, and the operations runbook (21 through 24).

Chapters 25 through 48 are the second half, and they are analytical rather than descriptive: the trust boundaries and the performance budget, the keyboard arbitration model, the six boundaries and their transports, living inside MV3, the storage schema, what happens when a hop fails, the cost of reuse, the lineage and the positioning, the failure modes, text input and composition, living on the open web, accessibility, two browsers under one host, debugging across six boundaries, incognito, the dead zones, zoom and density, the long session, what leaves the machine, teardown and portability, and version numbers as a contract. Chapters 49 through 53 cover the automation and content layer the HUD grew on top of that base — stryke lifecycle hooks, output triggers, pane pipelines, the audio dashboard over the browser-wide DSP engine, and the later HUD pages — and 54 through 67 open the automation bus, its transactional half, the identity chain, the packaging, and the patch series as artifacts.

Chapters 68 through 76 are a different kind of chapter and are marked here so they are not mistaken for the rest. Everything before them describes how zwire is built. Those nine describe what it *does when something is wrong*, and every figure in them is the output of a command run against the tree named in the Provenance section above: the version stamp audited at HEAD and across twenty release commits, the three installers counted for the stamps only one of them carries, the frame reader fed seventeen malformed streams, the six ways the host can be started and what each returns, two daemons racing for one socket path, a signal arriving mid-session, the whole test suite counted and run, what CI does not look at, and the collected set of paths that exit zero without having done the thing their caller believed. They correct the earlier text in several places, and those corrections are listed in Appendix AD.

Chapter 77 stands apart for a different reason: it replaces a chapter rather than extending one. The new tab stopped being a page with tiles and became a library of layouts — a port of Vivaldi's Start Page — and Chapter

6 describes the page that used to be there. Read 6 for the extension's shape, its vendored fonts and its focus trick, all of which still hold, and 77 for what it now renders.

The reference appendices at the back run A through AD. A–N are the core tables: the file map, the state directory, the full keybinding tables, the eight-scheme palette (dark and light), the launch flags, the Chrome WebUI token map, the native-messaging protocol, the seeded keyword commands, the storage keys, the Rust host modules, the launcher and build scripts, the CDP probes, the power-tool manifest, and the fork patch targets. O–AD carry the rest: the three extension manifests, the content-script injection order, the fuzzy matcher, the visual effects and readline, a system-stats sample, the palette destinations, the command bus, the frequency model, the extension messaging graph, the zgui-core toolkit surface, the HUD internal pages, the dev-loop scripts, three paths traced end to end, a glossary, the statusbar segments, and a closing note on grounding and edition.

Every claim in the chapters is grounded in a file in the [zwire](#) repository, and where the book says “the code comments say,” it is quoting the source.

One unnumbered section comes before Chapter 1: a browser-engine primer, for the reader who uses a browser daily and has never had reason to take one apart. It covers what an engine is and what its stages are, how Blink, V8, Skia and the content layer fit together, why one browser is many processes, and what forking Chromium actually commits you to. If those are already familiar, skip it — the book does not depend on it. Let's start.

# A Browser-Engine Primer

Everything after this section assumes you know roughly what is inside a browser — that “Blink” is a thing, that a tab is a process, that a fork of Chromium is a serious commitment rather than a weekend. If you do, skip to Chapter 1; nothing here is new to you. If you do not, this is the half hour that makes the rest of the book legible. It is not a reference: Appendix AB is the glossary, Appendix N is the fork’s patch-target map, and the Chromium project’s own docs are the authority for everything upstream.

## What is shown here, and where it came from

zwire is a graphical application. It opens windows; it is not a program you can put in a pipeline and watch print. So this primer is mostly prose, and its few transcripts are the few things that genuinely print and exit without drawing anything: a version string and a file the launcher reads. Nothing was screenshotted and no browser window was opened to produce this section. Everything else is sourced rather than demonstrated. Statements about Chromium, Blink, V8 and Skia are quoted from those projects’ own documentation and attributed in the sentence that makes them; this book has no business asserting facts about an engine it did not write. Statements about zwire are read out of the tree named in the Provenance section, or out of the chapters that follow. Where a transcript shows a path under your home directory it is written as `$ZWIRE_STATE` — the variable Appendix B documents.

## The engine, and the program around it

A browser is two programs sharing one name. The outer program owns the window: the tab strip, the address bar, the menus, the bookmarks, the profile, the extension system. The inner program owns the rectangle below the toolbar — given bytes arriving from a network, produce pixels. That inner program is the *engine*, and it is the part nobody sane rewrites. Hold onto that split while reading this book: zwire changes the outer program comprehensively and the inner one not at all, and every chapter that follows sits on one side of the line or the other.

The engine’s work is a pipeline, and Chrome’s own “Inside look at modern web browser” series names the stages. First the response is parsed: “the main thread begins to parse the text string (HTML) and turn it into a Document Object Model (DOM).” Then style — “the main thread parses CSS and determines the computed style for each DOM node.” Then layout, “a process to find the geometry of elements,” where every box lands in real pixels at this window size and this zoom. Then paint: “the main thread walks the layout tree to create paint records,” an ordered list of drawing operations rather than the drawing itself. Then compositing, “a technique to separate parts of a page into layers, rasterize them separately, and composite as a page in a separate thread called compositor thread.”

Read as a chain: **network** → **HTML parse** → **DOM** → **style** → **layout** → **paint** → **composite**. It is a pipeline, so a change late in it is cheap and a change early in it is not — which is why the HUD skins pages with CSS rather than rewriting markup — and it is re-entrant, running again on every resize, zoom change and DOM mutation, which is why Chapter 44 has anything to say about zoom and density and why the tiling overlay of Chapter 10 is careful about when it moves geometry.

## Four names: Blink, V8, Skia, and the content layer

Those stages are not one codebase. Four names cover almost everything you will meet in this book.

**Chromium** is the open-source browser project; Google Chrome is a *branded* build of it. `zwire`'s own build config toggles exactly that distinction with one GN argument, `is_chrome_branded`, which Chapter 21 quotes set to `false`.

**Blink** is the rendering engine: “Blink is the name of the rendering engine used by Chromium and particularly refers to the code living under `src/third_party/blink`.” The DOM, style, layout and paint stages above live there. The same page notes that “Blink is implemented on top of an abstract platform and thus cannot be run by itself” — which matters for the “can I just embed the engine?” question below.

**V8** is the JavaScript engine, defined on its own site as “Google’s open source high-performance JavaScript and WebAssembly engine, written in C++. It is used in Chrome and in Node.js, among others.” Blink quarantines its V8 usage in a `bindings/` directory because, its README says, “V8 APIs are complex, error-prone and security-sensitive.”

**Skia** is the drawing library — what turns paint records into pixels: “an open source 2D graphics library which provides common APIs that work across a variety of hardware and software platforms. It serves as the graphics engine for Google Chrome and ChromeOS, Android, Flutter, and many other products.”

The fourth name is not an engine but a layer, and it explains why “just embed the engine” is not a plan. Chromium’s `content/README.md` describes `content/` as “the core code needed to render a page using a multi-process sandboxed browser. It includes all the web platform features (i.e. HTML5) and GPU acceleration” — and is equally clear about the omission: `content` “does not include Chrome features, e.g. extensions/autofill/spelling etc.” Those live in the `chrome/` layer above it.

```
chrome/           the browser application: tabs, toolbar, menus, settings -
                  and, per content/README.md, extensions, autofill,
                  spellcheck, sync, safe browsing, translate
content/          "the core code needed to render a page using a
                  multi-process sandboxed browser ... all the web platform
                  features (i.e. HTML5) and GPU acceleration"
third_party/blink/ the rendering engine (chromium.org/blink)
```

That split is why Chapter 2 exists. The extension system — what `zpwchrome` and the internal HUD are built out of — is a `chrome/`-layer feature. An embedder that takes only `content/` gets the web platform and no extension model at all, which is the gap Chapter 2 walks through when it rules out the WebView-shaped alternatives.

It is also the map for reading the fork. The patch targets in `fork/patches/series` are overwhelmingly browser-application files — `tab_style_views.cc`, `toolbar_view.cc`, `location_bar_view.cc`, `chrome_color_mixer.cc` — with only a handful reaching into `content/` and below, and nothing at all touching Blink’s layout or paint code. `zwire` restyles the program around the engine and leaves the engine alone; the patch list is the evidence.

## Why one browser is many processes

Open a browser, open a process monitor, and count: one browser is not one process, and one tab is not one either. That is not an accident, and it explains most of the awkward parts of Chapters 15, 29 and 41.

Chromium’s multi-process architecture document names the two main roles: “we refer to the main process that runs the UI and manages renderer and other processes as the ‘browser process’ or ‘browser,’” and “the processes that handle web content are called ‘renderer processes’ or ‘renderers’.” Chrome’s developer series adds the third: a GPU process, which “handles GPU tasks in isolation from other processes.”

The reason for the split is stated plainly in the same document: “Given the renderer is running in a separate

process, we have the opportunity to restrict its access to system resources via sandboxing” — restricting network, filesystem and display access so as to “significantly limit what a compromised renderer process is able to accomplish.” Untrusted code from the open web runs in the renderer; the sandbox makes that survivable, and a sandbox needs a process boundary to be a boundary at all.

Once security was the driver the process count went up rather than down. Site Isolation makes it so that “pages from different websites are always put into different processes, each running in a sandbox that limits what the process is allowed to do”; the Chromium security pages record it as enabled by default for all sites in Chrome 67 on Windows, Mac, Linux and Chrome OS, and Chrome’s developer series names the trigger — “with Meltdown and Spectre, it became even more apparent that we need to separate sites using processes.” So the rough shape is one **browser process** holding the window and the privileged work, **many renderer processes** — at least one per site, sandboxed — a **GPU process** doing the drawing, and utility processes for services the browser factored out.

This is why zwire is not one program either. A content script from Chapter 7 runs inside a renderer, sandboxed, with no filesystem. The extension’s service worker (Chapter 30) is elsewhere again and is suspended when idle. The native host of Chapter 15 is a separate OS process outside the browser entirely, which is the *only* reason the HUD can have a real terminal — a sandboxed renderer cannot spawn a shell and no extension API will let it. Even the fork’s audio work hits this: patch 0024 exists because the sandboxed audio service could not read a config file and needed the unsandboxed browser process to read it and push the value across. Chapter 29 counts six such boundaries and names the transport for each; Chapter 41 debugs across them. None of it is zwire’s invention — it is the shape of the browser it lives in.

## What “a Chromium fork” actually commits you to

“Fork” is a cheap word in most contexts. Here it is not, and the book is direct about the bill.

**The tree.** [fork/README.md](#) puts the first build at “~100 GB checkout” and “1–4 hours” of compiling, with “~150 GB” of disk once you have source plus one output directory — and that checkout is already fetched with history omitted to keep it that small. Rebuilding after one patch is minutes; the first one is an afternoon.

**The patch series is the artifact.** A fork of this kind is not a copy of the tree with edits in it; it is an ordered list of diffs against a pinned upstream tag — [fork/patches/series](#) — plus a script that applies and reverses them. That is what makes it maintainable, and what you actually rebase.

```
$ cat fork/CHROMIUM_VERSION
150.0.7871.46
$ ls fork/patches/*.patch | wc -l
27
```

**The treadmill.** Upstream does not stand still to let you catch up. Chrome’s developer blog states that “Chrome has shipped a new milestone every four weeks” since 2021, and a post dated 3 March 2026 announces a move to “a two-week release cycle” from the stable release of Chrome 153. Whatever the number, a pinned tag is a fixed point on a line that keeps moving, and the distance grows on a schedule you do not control.

That is what a rebase costs here. Chapter 21 is not romantic about it: the anchors the patches attach to “shift between Chromium milestones, so most releases you re-anchor most patches.” The procedure is reverse the series off the old tree, bump the pinned version, fetch the new tag, re-apply, fix the rejects, rebuild. What you get for it is a tab strip, a toolbar and a set of native menus that belong to you — the whole trade, stated in full, with Chapter 35 the place zwire measures itself against the others who took it.

## Rebrand versus fork

The last concept the internals chapters assume is the difference between the two things zwire ships, because they are not degrees of the same operation. A **rebrand** changes nothing about the binary’s code. It changes the binary’s *identity* and its *cargo*: the name and icon the desktop shows (Chapter 5), the profile it runs against,

the extensions it loads, the flags it starts with. Every one of those is a supported, documented surface, so nothing can rot — there is no diff to rot against. A **fork** changes the code. It reaches things no extension API exposes — the shape of a tab, the font the UI is drawn in, the colors native menus resolve to — and pays for that reach with the compile and the treadmill above.

|                         | runtime rebrand       | source fork                 |
|-------------------------|-----------------------|-----------------------------|
| engine                  | stock snapshot        | compiled from patched src   |
| name, icon, profile     | yes                   | yes                         |
| new tab, internal pages | yes (extensions)      | same                        |
| vim / tiling / palette  | yes (content scripts) | same                        |
| frame + toolbar colors  | theme layer only      | native color mixer (0002)   |
| tab shape, UI font      | no                    | yes (0001, 0003)            |
| native menus + dialogs  | no                    | yes (0020, 0021)            |
| browser-wide audio DSP  | no                    | yes (0022-0024)             |
| cost to install         | a download            | ~100 GB and 1-4 hours       |
| cost to keep            | re-fetch a snapshot   | re-apply 27 patches per tag |

That line is the book's organizing principle, not a caveat it apologizes for: Chapters 4 and 5 are the rebrand tier, Chapter 21 is the fork tier, Chapter 3 is how one launcher serves both.

## What is different about zwire specifically

Three things to carry into Chapter 1.

**It is both tiers, and the launcher cannot tell them apart.** `bin/zwire` resolves the browser binary from an environment override and then one recorded file, and does nothing else with it — so the same launcher runs a downloaded snapshot or a compiled fork with no flag and no branch. The only on-disk evidence of the tier is a prefix on the recorded version string. The machine this edition was written on is on the fork tier:

```
$ cat "$ZWIRE_STATE/base.version"
fork-150.0.7871.46
$ "$ZWIRE_STATE/base/zwire.app/Contents/MacOS/zwire" --version
zwire 150.0.7871.46
```

That the binary answers to `zwire` rather than `Chromium` is patch 0006, the branding-strings patch — the shortest demonstration that the fork tier is real rather than aspirational.

**The engine has to be genuine and current, and something enforces it.** Chapter 2's argument is not stylistic. The power-tool extension declares `minimum_chrome_version: 127` along with a permission set — `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, `webRequest`, a service-worker background — that exists only inside a real Chromium extension runtime, and the browser refuses to load the extension otherwise. That is a floor written into a manifest, not a preference the project could argue itself out of.

**Nothing it does is felt by the browser you already had.** `zwire` runs against its own user-data directory, stages its own copies of the extensions it loads, and installs its native-messaging host profile-relative rather than system-wide. That host is an ordinary local binary, which makes it the other thing in this system you can check from a terminal:

```
$ ./target/debug/zwire-host --version
zwire-host 0.3.15
```

## Where to go from here

You now know enough to read the rest of the book. Chapter 1 states what `zwire` is; Chapter 2 is the capability argument this primer sketched, in full, with the manifests as evidence; Chapter 3 lays out the seven layers and

is the map for everything after it. Chapters 4 and 5 are the base and the rebrand described informally above, and Chapter 21 is the fork.

If the multi-process section was the part that landed, read Chapter 29 next — the six boundaries and what crosses each — then Chapter 41, which debugs across all six at once. If the rebrand/fork line landed, read Chapter 21 and then 35. Appendix AB is the glossary for any term this primer used but did not define.

# Chapter 1 – The Pitch

A browser is the least likely thing on the stack to be rewritten, and zwire does not rewrite it. This chapter states what zwire *is* — not “a browser from scratch,” but Chromium **owned** at the seams a rebrand and a patch series can reach: a real Blink engine wearing the cyberpunk HUD, carrying the author’s extensions, on a profile that never collides with system Chrome.

## What zwire is – a rebrand and an extension host, not a new engine

The lazy framing is “a custom browser,” which usually means either a WebView wrapper (WebKit, no MV3) or a fresh engine (a decade of work nobody finishes). zwire is neither. It takes a real Chromium and changes the two things that matter for a daily driver — how it looks and what it runs — while inheriting the engine, the security model, and the update cadence for free.

The mechanics are unglamorous on purpose. `bin/zwire` is a shell script. It reads one path (`~/.zwire/base.path`, written when the base was fetched), and it `execs` that binary against a dedicated profile with a fixed set of extensions loaded. There is no daemon, no supervisor, no packaged Electron runtime, no bundled Node. The “browser” is a Chromium snapshot Google itself builds and hosts; zwire’s contribution is the profile, the extension set, the icon, the name, and — in the fork — twenty-seven C++ patches. Everything else is Blink, unmodified, doing what Blink does. That is the entire trick, and it is why zwire exists in kilobytes of shell and JavaScript while a from-scratch browser would be millions of lines of C++ nobody finishes.

Two consequences follow immediately, and they run through the whole book. First, zwire tracks upstream Chromium *for free*: a security fix that lands in the Blink tree is one `scripts/fetch-base.sh` away, because the base is a stock snapshot, not a frozen fork of one. (The fork path pays for its native restyle by giving that up — Chapter 21 is honest about the rebase cost.) Second, every capability zwire adds on the runtime path is an *extension* or a *launch flag* — nothing patched, nothing recompiled — so the runtime rebrand degrades gracefully: delete an extension directory and the launcher simply skips it, and you still have a working browser.

## What the rebrand buys, and what only the fork buys

A Chrome theme extension can repaint the frame, the toolbar background, and the tab strip — and nothing else. Tab *shape*, the UI *font*, the toolbar *border*, the omnibox field *geometry*: those are native C++ in Chromium’s Views layer, not exposed to any extension API. This is not a zwire limitation; it is the boundary Chromium draws around what a theme is allowed to touch. The runtime rebrand accepts that boundary and does everything it can reach on the right side of it — which turns out to be a great deal, because zwire pushes the color and content surface much further than a theme extension normally does:

- **Colors** it changes through the theme layer and, more aggressively, through the internal-HUD content script that skins `chrome://` pages directly.
- **Content** it owns completely: the new tab is a full HUD page (`chrome_url_overrides`), and several internal pages (`chrome://extensions`, `chrome://history`, `chrome://bookmarks`, `chrome://settings`, `chrome://version`) are redirected to HUD-styled replacements served from the extension.

- **Behavior** it layers on with content scripts: vim keybindings on every page, a tmux-style tiling overlay, a powerline statusbar, a `⌘K` command palette, a fuzzy find bar that replaces Chrome's own, and an xterm terminal over a real PTY.

What stays out of reach on the runtime path is precisely the native Views chrome: the pill-shaped tabs, the system UI font, the rounded omnibox. The `fork/` path compiles a patched Chromium to reach those — trapezoid HUD tabs, a Share Tech Mono UI font, a neon under-toolbar line, HUD-colored frame, and the eight HUD schemes wired into the DevTools Theme dropdown. `zwire` ships *both* paths rather than forcing the ~100 GB Chromium checkout on everyone: the rebrand is the daily driver, and the fork is there for whoever wants the whole browser in the HUD and will pay the build-and-rebase cost. The boundary between them is not a wart to be apologized for — it is the map of what a browser lets you change without becoming its maintainer.

## Why it never touches system Chrome

`zwire` runs against `~/.zwire/profile` — a dedicated `--user-data-dir` — and never your system Chrome profile. This is not politeness; it is a hard requirement of loading unpacked extensions with elevated flags. The launcher passes `--load-extension`, `--extensions-on-chrome-urls`, and `--test-type`; you do not want those flags, those specific unpacked extensions, or the developer-mode banner they raise anywhere near the profile that holds your real logins, cookies, and saved passwords. A separate user-data-dir keeps the two worlds from ever sharing state: `zwire`'s bookmarks, history, sessions, and extension metadata live under `~/.zwire/profile`, and Chrome's live where they always did.

The isolation goes one level deeper than the profile. The extensions the launcher loads are staged into a per-user `~/.zwire/ext/` copy at launch, not run in place from the `.app` bundle, because Chromium writes extension `_metadata/` indexes into the extension directory and every user on the machine needs their own writable copy. And the native-messaging host — the bridge that powers the HUD terminal, the system-stats stream, and the scheme sync — is installed *profile-relative*, into `~/.zwire/profile/NativeMessagingHosts/`, so it is wired for `zwire` and invisible to system Chrome. The whole design keeps `zwire` a guest that brought its own everything: its own profile, its own extension copies, its own native host, its own icon and name. Nothing it does can be felt by the browser you already had.

## The two tiers, in the launcher's own words

The whole shape of the project is stated at the top of `bin/zwire` itself, and it is worth reading before the chapters take the stack apart: “Chromium, rebranded, with the `zpwrcchrome` power-tool + cyberpunk new-tab + the internal-HUD skin preloaded. Runtime rebrand: a plain Chromium snapshot binary launched against a dedicated profile with a fixed extension set. Nothing to compile (the optional whole-chrome HUD lives in `fork/`).” Two tiers, named in the first four lines of the launcher: a runtime rebrand that compiles nothing and reaches everything an extension and a launch flag can reach, and an optional source fork that recompiles Chromium to reach the native chrome. The parenthetical is the honest boundary the rest of the book draws — “the optional whole-chrome HUD lives in `fork/`” — and it is why the same `bin/zwire` launches both: the launcher reads only `base.path`, so it does not care whether that path points at a downloaded snapshot or a compiled fork. The runtime tier is the daily driver, the fork tier is the whole-browser HUD for whoever will pay the build cost, and the seam between them is a design decision, not a limitation to apologize for.

# Chapter 2 – Why a Real Blink Base

This chapter is the technical justification for the whole project: the exact set of Manifest V3 capabilities `zpwchrome` requires, why each one rules out the non-Chromium alternatives, and why the base must be *unbranded* Chromium rather than Google Chrome or Chrome for Testing.

## The MV3 capability set `zpwchrome` requires

`zpwchrome` declares `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, `webRequest`, a service-worker background, and `minimum_chrome_version: 127`. Each is load-bearing, and the combination exists only on a genuine Blink engine. Take them one at a time:

- `userScripts` is Manifest V3's supervised replacement for the old arbitrary-code injection that `tabs.executeScript` used to allow. It lets an extension register user-authored scripts that run in page context. It is a Chromium-only API — it does not exist in Safari's WebExtensions surface, and it has no analogue in a WebView, which exposes no extension model at all.
- `declarativeNetRequestWithHostAccess` is the MV3 network-rules engine: the extension ships declarative match/modify rules that the browser applies in its own network stack, rather than round-tripping every request through a JavaScript listener. The `WithHostAccess` variant grants the dynamic, host-scoped rules `zpwchrome` needs. This is the modern content-blocking and request-rewriting substrate, and it is implemented in Chromium's network service — not reachable from a wrapper that only owns a WebView. (zwire's own `hud-internal` extension uses the plain `declarativeNetRequest` form for its frame-busting rule set, discussed in Chapter 10.)
- `nativeMessaging` is the bridge to a local process: the extension launches a registered native binary and speaks a length-prefixed JSON protocol over its stdio. `zpwchrome`'s `pass` integration and segmented downloads need it to talk to a native host; zwire's own HUD host (Chapter 15) uses the same mechanism for the terminal and the system-stats stream. Native messaging requires the browser to read a host manifest from a known directory and spawn the process — something only a full browser (not a WebView) does.
- `webRequest` is the observe/block network API. Even in MV3, where the *blocking* form is constrained in favor of declarativeNetRequest, the observational surface remains, and `zpwchrome` declares it.
- **A service-worker background** is the MV3 background model: no persistent background page, an event-driven worker that the browser wakes on demand and suspends when idle. Every piece of zwire that coordinates across tabs — the scheme bridge, the `⌘K` command relay, the terminal open-state — is written for this suspend-any-time worker, and it only exists inside a Chromium extension runtime.
- `minimum_chrome_version: 127` is the extension declaring, in its own manifest, that it will not load on anything older than Chromium 127. That is not a preference; it is a floor. It rules out not only WebKit and Servo but any Chromium old enough to predate the APIs above.

The point is cumulative. Any *one* of these might be worked around; the *set* can only be satisfied by a current, genuine Chromium/Blink engine running a real extension runtime. Laid against the alternatives:

| Capability need       | WebView (Tauri) | Servo | Safari WebExtensions | Chromium        |
|-----------------------|-----------------|-------|----------------------|-----------------|
| userScripts           | no ext model    | no    | no                   | yes             |
| declarativeNetRequest | no              | no    | partial              | yes             |
| nativeMessaging       | no (own IPC)    | no    | limited              | yes             |
| service-worker bg     | no ext model    | no    | yes                  | yes             |
| --load-extension      | n/a             | n/a   | n/a                  | yes (unbranded) |

That is why zwire is a Chromium rebrand and not a Tauri app, a Servo experiment, or a Safari extension. The engine is not an implementation detail zwire happened to pick — it is the one requirement everything else hangs from.

## Why unbranded Chromium, not Chrome or Chrome for Testing

The base is a plain Chromium snapshot from the public snapshot index — no Google logo, no “for automated testing” banner. This narrows the field to exactly one build, and it is worth seeing why the two obvious alternatives are both wrong.

**Google Chrome** is the branded build, and branded Chrome removed the `--load-extension` command-line switch in version 137. That switch is the entire mechanism by which zwire preloads its extensions on launch — the launcher’s `--load-extension="$LOAD"` line. Without it there is no way to load the HUD new-tab, the internal-HUD skin, and `zpwchrome` unpacked from disk at startup. So branded Chrome is disqualified by a removal that is specific to the *branded* build; the upstream Chromium engine still honors the switch.

**Chrome for Testing** is Google’s dedicated automation build, and it keeps `--load-extension` — but it wears a permanent “for automated testing” info-bar stripe across the top of every window, because that is its entire purpose: a build you cannot mistake for a user’s real browser. That stripe is exactly what you do *not* want on a daily driver you have gone to the trouble of rebranding with a cyberpunk icon and name. So Chrome for Testing is disqualified by the banner.

**Plain unbranded Chromium** — the continuous-build snapshot Google hosts at [storage.googleapis.com/chromium-browser-snapshots](https://storage.googleapis.com/chromium-browser-snapshots) — threads the needle: it is a real Chromium/Blink build with no Google product branding *and* no testing banner, and it retains `--load-extension`. `scripts/fetch-base.sh` resolves the correct platform id (`Mac_Arm`, `Mac`, `Linux_x64`, `Win_x64`) and, by default, the latest snapshot revision from the `LAST_CHANGE` marker; pass an explicit revision as the one argument to pin an exact build. The resolved revision is recorded so a launch is reproducible. Chapter 4 opens the fetch mechanics; the takeaway here is that the choice of base is not aesthetic — it is the only build that satisfies both constraints at once.

## The manifest is the evidence

This chapter’s argument is not abstract; it is written down in the power-tool’s `manifest.json`, and it is worth pointing at the exact lines. `zpwchrome` declares `manifest_version: 3`, `minimum_chrome_version: 127`, and nineteen permissions, four of which — `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, and `webRequest` — are the ones that exist only on a current Chromium engine running a real extension runtime. The `minimum_chrome_version` line is not a preference the launcher could talk it out of; it is a hard floor the browser itself enforces, so any base older than Chromium 127, and every non-Chromium engine, refuses to load the extension at all. The internal-HUD manifest adds its own load-bearing declarations — `declarativeNetRequest` for the frame-busting rule, `developerPrivate` and `management` for the extensions page, `settingsPrivate` for the settings page, `system.display` for the worker’s window math — none of which a WebView exposes and none of which Safari’s WebExtensions surface fully implements. The whole of Chapter 2 reduces to a claim you can check by reading two manifests: the *set* of capabilities these extensions require is satisfiable on exactly one engine, and that engine is a genuine, current Chromium. Everything else about zwire is downstream of that single, verifiable fact. (Appendix M is the full manifest.)

# Chapter 3 – The Architecture

zwire is seven cooperating layers. This chapter names each one and its single responsibility before the later chapters open them up: **Base**, **Rebrand**, **Theme**, **New tab**, **Internal HUD**, **Power-tool**, and **Launcher**, with the optional **Fork** underneath them all.

## The seven layers, top to bottom

Read the stack from the engine up:

- **Base** — a pinned, plain Chromium snapshot downloaded by `scripts/fetch-base.sh` into `~/.zwire/base`, with its resolved binary path recorded in `~/.zwire/base.path` and its revision in `~/.zwire/base.version`. This is the engine, unmodified. Everything else is skin and cargo loaded onto it. On the fork path this same slot holds the compiled patched Chromium instead; the launcher does not care which, because it reads only `base.path`. (Chapter 4.)
- **Rebrand** — `scripts/rebrand-macos.sh`, the in-place bundle patch that makes the Dock, `⌘-Tab` switcher, and menu bar show “zwire” with the cyberpunk icon. It sets `CFBundleName/CFBundleDisplayName`, installs the `.icns`, deletes the key that would override it, re-signs the bundle ad-hoc, and re-registers it with LaunchServices. On Linux there is no bundle to patch — the launcher name *is* the brand — so this layer is macOS-only. (Chapter 5.)
- **Theme** — `theme/`, a Chrome theme extension that maps the HUD palette onto frame, toolbar, and tabs (colors only). It is present in the tree but, deliberately, **not loaded by the launcher** — the reason is the whole point of the palette pipeline and gets its own section below.
- **New tab** — `newtab/`, a `chrome_url_overrides.newtab` extension: the full HUD new tab, with a neon clock, an omnibox, quick-launch tiles, and its fonts vendored locally so it renders offline. (Chapter 6.)
- **Internal HUD** — `extensions/hud-internal`, the MV3 content-script suite that skins Chrome’s own internal pages, redirects several of them to HUD replacements, and layers vim keys, a tmux overlay, a powerline statusbar, the `⌘K` palette, a fuzzy find bar, and a terminal onto every page — plus the floating eight-scheme picker and its native host. This is the biggest layer, and Chapters 7 through 16 open it one feature at a time.
- **Power-tool** — `extensions/zpwrchrome`, the Manifest V3 power-tool, loaded as a **submodule** rather than vendored, so zwire tracks it upstream instead of freezing a copy. (Chapter 18.)
- **Launcher** — `bin/zwire`, the single entry point that assembles the flags, ensures the native host, makes the profile, and `execs` the base. (Chapter 19.)
- **Fork** (optional, underneath) — `fork/`, the source build that compiles a patched Chromium so the native chrome itself lives in the HUD. It is not a layer the runtime path touches; it is an alternate *base*. (Chapter 21.)

The layering has a property worth naming: every layer above the base is optional and independently removable. Delete `theme/` (already unloaded), delete `newtab/`, delete the `zpwrchrome` submodule — the launcher includes only the extension directories that actually contain a `manifest.json`, so a missing layer degrades to a plainer browser rather than a broken one. That graceful-skip rule is in the launcher itself, and it is what lets a fresh clone without submodules still run.

## Why the static theme is present but not loaded

The `theme/` extension maps the HUD palette onto frame, toolbar, and tabs — but the launcher does not load it, and the launcher says so in a comment where it builds the extension list. Two things own the chrome palette ahead of it, and a static theme would fight both:

1. **The fork's native color mixer (patch 0002).** On the fork path, the cyberpunk palette is forced onto frame, toolbar, tab background/foreground, and the omnibox directly in Chromium's `chrome_color_mixer.cc`, keyed to the active scheme. That is the authoritative source of the chrome colors when the fork is in use.
2. **The internal-HUD skin.** On the runtime path, the `hud-internal` content scripts drive the scheme, and the scheme is mirrored through a native file so the whole HUD — pages, statusbar, palette — follows one picker.

A Chrome custom theme extension applies **last** in Chromium's color pipeline: it is designed to win, so the user's chosen theme is what they see. If `zwire` loaded `theme/`, it would override both the native mixer and the scheme-driven skin, pinning the chrome to one static palette and defeating the eight-scheme picker. So the theme extension is kept in the tree — it is a correct, working Chrome theme, useful as a reference and loadable by hand — but excluded from the launcher's default set precisely because it is *too* authoritative. Understanding that ordering is understanding the whole palette pipeline: the scheme picker is the source of truth, the native mixer and the content-script skin are its two renderers, and a static theme is deliberately kept out of the way.

## One source of truth, four renderers

It is worth stating the palette architecture as a whole here, because it is the shape every later chapter reconnects to. There is exactly one source of truth for the active scheme — the native file `~/.zwire/hud-scheme`, owned by the host — and there are four renderers that all reconcile to it. The internal-HUD content scripts read a mirror of it in `chrome.storage.local` (`zb_scheme`) and style every ordinary page and every skinned `chrome://` page from it. The new-tab extension, a different origin that cannot share that storage, polls the native file directly and styles itself. The power-tool receives it over `runtime.sendMessage` and colors its own surfaces. And on the fork path, the compiled color mixer reads the same native file and colors the native chrome. Four renderers, one file, no primary — pick a scheme anywhere and the file changes, and every renderer follows within its own refresh. The reason the static `theme/` extension is excluded is now precise: it would be a *fifth* renderer that applies last and ignores the file, pinning the chrome to one palette and breaking the picker for the other four. Keeping it in the tree but out of the launcher is not an oversight; it is the one place a working Chrome theme would actively fight the architecture.

The layering has a second property the launcher enforces: every layer above the base is independently removable. The extension list is built by including only directories that contain a `manifest.json`, so a missing `zpwchrome` submodule, a deleted `newtab`, or an uninitialized checkout simply drops that layer and the browser runs with what remains. The native host is ensured but never required. Nothing above the stock engine is load-bearing for launch — which is why a fresh clone without submodules still opens a working browser, just a plainer one.

## The layers at a glance

The seven layers, their one file or directory, and the chapter that opens each:

| layer        | where                           | responsibility                                               | ch   |
|--------------|---------------------------------|--------------------------------------------------------------|------|
| Base         | \$ZWIRE_STATE/base + base.path  | the stock (or forked) engine                                 | 4    |
| Rebrand      | scripts/rebrand-macos.sh        | Dock name + icon (macOS only)                                | 5    |
| Theme        | theme/ (NOT loaded)             | colors-only Chrome theme                                     | 3    |
| New tab      | newtab/                         | the HUD new-tab override                                     | 6    |
| Internal HUD | extensions/hud-internal/        | skin + vim + tmux + palette +<br>find + statusbar + terminal | 7-16 |
| Power-tool   | extensions/zpwichrome/ (submod) | the MV3 power-tool                                           | 18   |
| Launcher     | bin/zwire                       | assemble flags, exec the base                                | 19   |
| Fork         | fork/ (optional base)           | compiled whole-chrome HUD                                    | 21   |

Every row above the base is optional and independently removable; the launcher reads only `base.path`, so it neither knows nor cares whether the base is a snapshot or a compiled fork, and it loads only the extension layers that actually carry a manifest. That single indifference is what lets one launcher serve both build tiers and lets a partial checkout still run.

# Chapter 4 – The Base

The engine layer: how `scripts/fetch-base.sh` resolves and pins a Chromium snapshot, where it lands under `~/.zwire/base`, and how `base.path` / `base.version` record what was resolved so a launch is reproducible.

## Fetching and pinning a snapshot

`scripts/fetch-base.sh` is the whole of the base layer, and it is a small, readable script. Its job is to turn “the host I am running on” into “the correct Chromium snapshot binary on disk,” and to record what it resolved.

It begins by mapping the host to a snapshot platform. `uname -s/uname -m` select one of four rows:

|                      |              |                  |              |
|----------------------|--------------|------------------|--------------|
| Darwin/arm64         | -> Mac_Arm   | chrome-mac.zip   | chrome-mac   |
| Darwin/x86_64        | -> Mac       | chrome-mac.zip   | chrome-mac   |
| Linux/x86_64   amd64 | -> Linux_x64 | chrome-linux.zip | chrome-linux |
| MINGW/MSYS/CYGWIN    | -> Win_x64   | chrome-win.zip   | chrome-win   |

Each row names the snapshot platform id, the zip filename Google publishes for it, and the subdirectory that zip extracts to. Anything else exits with an “unsupported platform” message rather than guessing.

The revision comes next. With no argument, the script fetches the `LAST_CHANGE` marker for the platform — the latest continuous-build revision — and uses that. Pass a revision as the single argument and it pins that exact build instead:

|                                            |                                              |
|--------------------------------------------|----------------------------------------------|
| <code>scripts/fetch-base.sh</code>         | <code># latest snapshot (LAST_CHANGE)</code> |
| <code>scripts/fetch-base.sh 1656770</code> | <code># pin an exact revision</code>         |

This is the pinning story in full. There is no lockfile and no separate manifest: the “pin” is the revision string you pass, and the record of what you got is the `base.version` file the script writes. Pinning matters because Chromium snapshots are continuous — the latest changes hour to hour — and a reproducible `zwire` means knowing exactly which Blink you are running. For a machine you want to keep stable, fetch a revision once and never pass no-argument again until you choose to roll forward.

For total control there is a bypass. `bin/zwire` honors a `ZWIRE_BASE` environment variable pointing at any executable Chromium binary, checked before it consults `base.path`. That lets you point `zwire` at a Chromium you built yourself, a distro package, or a second snapshot, without going through `fetch-base.sh` at all — useful when developing the fork, where you want the launcher’s extension loading but your own compiled base.

## What lands on disk

The download itself is a `curl` of `storage.googleapis.com/chromium-browser-snapshots/<PLAT>/<REV>/<ZIP>` into `~/.zwire`, an `unzip` into `~/.zwire/base`, and a delete of the zip. Then the script resolves the actual executable inside the extracted tree, because the path differs per platform:

- On macOS it finds the Chromium binary at `.../Chromium.app/Contents/MacOS/Chromium` inside the extracted `chrome-mac`.
- On Linux the binary is `.../chrome-linux/chrome`.

- On Windows/MSYS it is `.../chrome-win/chrome.exe`.

With the binary located, the script writes exactly two small files and nothing else:

```
~/.zwire/base.path      the absolute path to the resolved binary (bin/zwire reads this)
~/.zwire/base.version   the revision, as "r<REV>" (reproducibility)
~ record)
```

`base.path` is the contract with the launcher: `bin/zwire` reads that one line to find the engine. `base.version` is the human-and-tooling record of which snapshot is installed. The `base/` directory holds the extracted browser itself — roughly 325 MB, the number the dev-loop scripts cite when they warn about what a `nuke` re-downloads. That is the entire on-disk footprint of the base layer — one directory and two one-line files — and it is deliberately flat so that a base upgrade is just “run `fetch-base.sh` again,” which overwrites the extracted tree and rewrites the two files. (On macOS an upgrade also means re-running the rebrand, because the fetch replaces the freshly-patched bundle with a stock one; Chapter 5 explains why.)

## The script, exactly

`fetch-base.sh` is small enough to walk in full. Its own header names the base it targets: “a plain Chromium continuous-build snapshot (Google-hosted): a real Chromium/Blink build with NO ‘Google Chrome’ product branding and NO ‘for automated testing’ banner ... Unbranded Chromium also retains the `--load-extension` switch.” It maps `uname -s/-m` to a `PLAT/ZIP/SUBDIR` triple through the four-row `case` tabulated above, then, with no revision argument, `curls` the platform’s `LAST_CHANGE` sentinel from `storage.googleapis.com/chromium-browser-snapshots/<PLAT>` to learn the latest continuous-build number; a revision passed as the single argument pins that build instead. The download is a `curl ... --progress-bar` of `<BASE_URL>/<REV>/<ZIP>` into the state dir, an `unzip -q -o` into `$ZWIRE_STATE/base` (after an `rm -rf` of any prior extraction), and a delete of the zip.

Then it resolves the executable, which differs per platform: on macOS a `find` for the `Chromium` binary inside `*Chromium.app/Contents/MacOS/*`, on Linux `<SUBDIR>/chrome`, on MSYS `<SUBDIR>/chrome.exe`. It writes the two contract files — `printf '%s\n' "$BIN" > base.path` and `printf 'r%s\n' "$REV" > base.version` — and prints where it installed. That `r`-prefix on `base.version` is the marker Chapter 19 reads to tell a snapshot (`r...`) from a compiled fork (`fork-...`). The `~325 MB` figure the dev-loop scripts cite for the extracted browser is not hardcoded in this script; it lives in `localinstall.sh` and `nuke.sh` as the size those operations warn about re-downloading.

## Snapshots are not releases, and what pinning actually buys

It is worth being precise about what a continuous-build snapshot *is*, because the word “version” does two different jobs in this book and confusing them makes the pinning story look sloppier than it is. A Chrome release — the thing with a four-part version like the tag the fork pins — is a branch that was cut, stabilized, and shipped on a schedule. A snapshot is not that. It is a build of the Chromium trunk at one revision, produced continuously, with no stabilization step and no promise that any particular revision is better than the one before it. The revision number is a monotonic counter over commits, not a version in any semantic sense: a higher number means later, and nothing else.

That has three practical consequences the base layer is designed around. First, “latest” is a moving target measured in hours, not weeks, which is why `LAST_CHANGE` is a *sentinel* the script reads rather than a version the script knows. Second, there is no release-notes surface to consult, so the only meaningful statement about which Blink you are running is the number itself — hence `base.version`, written on every fetch, as the one durable record. Third, and least obvious: because a snapshot is a trunk build, it can carry a regression that a released Chrome would never have shipped, and the correct response is not to wait for a fix but to fetch a

different revision. Pinning is therefore not a conservatism knob; it is the mechanism by which you *roll back*, and it is the reason the script takes a revision as its single argument rather than hiding the choice behind a flag.

The rollback procedure is the pinning procedure run backwards, and it is worth stating because nothing else in the tree does it for you. There is no lockfile, no manifest of known-good revisions, and no history of what was installed before — `base.version` holds exactly one number, the current one. So a machine you want to be able to roll back must record the number somewhere the fetch does not overwrite, which in practice means noting it before an upgrade. That is a deliberate thinness rather than an oversight: a version database would be a second source of truth about the base, and the base layer's whole design premise is that there is exactly one — the binary on disk, described by two one-line files beside it.

## The three ways a base gets there, and why the launcher cannot tell

The launcher reads `base.path` and nothing else, and that indifference is what lets three quite different things occupy the same slot. A snapshot fetched by the script is the common case. A compiled fork installed by the fork's packaging step is the second, and it writes the same two files with a `fork--`-prefixed version so the *only* on-disk evidence of which tier you are running is that prefix. The third is the environment override, an executable path in `ZWIRE_BASE` consulted ahead of `base.path`, which bypasses the fetch mechanism entirely and points the launcher at any Chromium you like — a distro package, a second snapshot, or an incremental fork build you have not packaged yet.

The third case is the one that earns its keep during fork development. A fork build produces a browser in the build output directory long before anyone wants to install it over the working base, and the override lets you run that build with the full extension set, the real profile, and the real native host, without disturbing the snapshot the same machine still uses as its daily driver. That is a genuinely different workflow from “install and hope,” and it exists because the launcher's base resolution is a precedence rule — an executable override, then the recorded path, then a clear error — rather than a lookup in a single place.

The error case is worth one sentence of its own, because it is the only fatal condition in the entire launcher. If neither the override nor the recorded path resolves to an executable, the launcher prints the exact command that fixes it and exits non-zero. Every other missing component in the system degrades; the base does not, because there is nothing left to run.

## The upgrade's second-order effects

An upgrade is `fetch-base.sh` again, and the script's behavior on an existing install is worth spelling out because two of its effects surprise people. The fetch removes the prior extraction before unzipping, so the base directory is replaced rather than merged — which is correct (a merged Chromium tree of two revisions would be a genuinely dangerous artifact) and which means anything you put inside the base directory is gone. And on macOS the fresh extraction is a stock, Chromium-signed bundle, so every one of the rebrand's four edits is undone at once: the name, the icon, the deleted icon-name key, and the ad-hoc seal. The rebrand is not stored anywhere the fetch could preserve; it is a patch applied to a bundle the fetch just replaced.

What the upgrade does *not* touch is equally important and is why an upgrade is safe to run casually. The profile lives outside the base directory, so bookmarks, history, sessions, and every extension's stored state survive untouched. The per-user staged extension copy is rebuilt at the next launch from the source directories, so it needs no migration. The native host manifest points at a binary outside the base tree, so it is unaffected. And the scheme and the shared preferences live in the fleet-wide store, not under the install root, so the browser comes back up in the same colors it went down in. The base is the one layer where “replace it entirely” is the correct upgrade strategy, and the layout above it is arranged so that replacing it costs one macOS-only command.

# Chapter 5 – The Runtime Rebrand

The no-compile path. On macOS `scripts/rebrand-macos.sh` patches the base bundle's Dock name to `zwire`, swaps in the cyberpunk `.icns`, deletes `CFBundleIconName`, and re-signs ad-hoc so macOS honors the change. This chapter explains why each of those steps is required and why the rebrand must be re-run after a base upgrade.

## The Dock name, the icon, and the ad-hoc re-sign

The rebrand is an in-place edit of the base bundle under `~/zwire/base` — it never touches a system browser install. The script reads `base.path`, walks up to the `.app` directory (stripping `/Contents/MacOS/*`), and patches the bundle's `Info.plist` and `Resources`. Four operations, each of which is there for a specific macOS reason:

1. **The name.** `CFBundleName` and `CFBundleDisplayName` are set to `zwire` via `PlistBuddy` (Set, falling back to Add if the key is absent). These are what the Dock, the `⌘-Tab` switcher, and the menu bar read. Change them and the app stops calling itself “Chromium.”

2. **The icon, under a new filename.** The cyberpunk `branding/zwire.icns` is copied into the bundle's `Resources` as `zwire.icns`, and `CFBundleIconFile` is pointed at `zwire`. It is installed under a *new* filename rather than overwriting Chromium's own icon so it does not collide with the original icon's LaunchServices association — a fresh name is a fresh association.

3. **Deleting `CFBundleIconName`.** This is the non-obvious step, and the script comments it. Modern Chromium ships its icon as `AppIcon` inside a compiled `Assets.car` and references it through `CFBundleIconName`, which takes **precedence** over the `.icns` named in `CFBundleIconFile`. If you set `CFBundleIconFile` but leave `CFBundleIconName` in place, macOS keeps showing the compiled Chromium icon and your `.icns` is ignored. So the script deletes `CFBundleIconName`, forcing macOS to fall back to `zwire.icns`. Without this deletion the icon swap silently does nothing.

4. **The ad-hoc re-sign.** The base bundle is ad-hoc signed. Editing `Info.plist` and `Resources` invalidates that signature, and once the signature is invalid macOS treats the icon and name change as *tampering* and reverts to the cached original. The fix is to re-seal the bundle: `codesign --force --sign - "$APP"` re-signs it ad-hoc (the nested frameworks and helper apps keep their own valid seals, so only the top level needs re-signing). This is what makes macOS *accept* the change rather than roll it back. The script warns but does not fail if the re-sign fails, because a browser with a stale icon is still a working browser.

Finally it nudges LaunchServices to refresh its icon/name cache for this bundle (`lsregister -f "$APP"`) and *touches* the bundle so the change is noticed. The result: `~/zwire/base/Chromium.app` now presents as “zwire” with the cyberpunk icon, everywhere macOS shows an app's identity, without a single line of Chromium recompiled.

The order matters and the script encodes it: patch the plist and resources **first**, re-sign **second**, re-register **third**. Re-signing before the edits would seal the wrong state; re-registering before re-signing would cache a bundle macOS still considers tampered.

## When the icon cache lingers: bust and nuke

macOS caches Dock and `⌘-Tab` icons aggressively by bundle, so occasionally the old Chromium icon or name lingers after a rebrand even when every step above ran. The dev-loop has a dedicated escape hatch for this: `scripts/bust.sh` (“cache-bust”) re-seals and re-registers *both* the base bundle and any `/Applications/zwire.app` wrapper with LaunchServices, forcing the cyberpunk icon and the “zwire” name to refresh immediately. `scripts/nuke.sh` is the heavier option — it removes the wrapper and the ~325 MB base, re-fetches from scratch, rebrands, and reinstalls, while preserving your profile. These exist because the LaunchServices cache is the one part of the rebrand macOS does not always honor on the first try, and a browser whose Dock icon is wrong reads as broken even when it is not.

## Why an upgrade forces a re-run, and why Linux needs none

An *upgrade* forces a re-run of the rebrand: a fresh `fetch-base.sh` unzips a stock, Chromium-signed bundle over the top of the patched one, so every one of the four edits above is gone and must be reapplied. `scripts/install.sh` chains the rebrand after the fetch on macOS for exactly this reason, and the README’s update recipe ends with `rebrand-macos.sh` for the same one.

On Linux an application’s identity — the launcher entry, the taskbar name, the window title — comes from how it was invoked and from a `.desktop` file, not from metadata embedded in the binary. So on Linux the launcher *name itself* is the brand: the user runs `zwire`, and the process and window read `zwire` because that is what started them. There is no in-place bundle rewrite because there is no bundle to rewrite, and `install.sh` only invokes `rebrand-macos.sh` when `uname -s` is `Darwin`. “Rebrand” therefore means two different concrete things on the two platforms — a bundle patch on macOS, a launcher name on Linux — and the project does not pretend otherwise.

## The four operations, as commands

The rebrand is four `PlistBuddy` and `codesign` operations in a fixed order, and the script’s comments give the reason for each. It resolves the `.app` by stripping `/Contents/MacOS/*` off `base.path`, then a `set_key` helper that tries `Set` and falls back to `Add ... string` writes `CFBundleName` and `CFBundleDisplayName` to `zwire`. It copies `branding/zwire.icns` into the bundle’s `Resources` under a new filename and points `CFBundleIconFile` at it, then deletes `CFBundleIconName` — the non-obvious step whose comment explains that modern Chromium references its icon as `AppIcon` inside a compiled `Assets.car` through that key, which “takes PRECEDENCE over the `.icns` in `CFBundleIconFile`,” so the key must go for the custom icon to win. It re-seals the bundle with `codesign --force --sign - "$APP"` (warning but not failing on error, “nested frameworks/helpers keep their own valid seals”), and finally re-registers with `lsregister -f` and `touches` the bundle. The order is load-bearing and the script encodes it: name and resources first, re-sign second, re-register third — re-signing before the edits would seal the wrong state, and re-registering before re-signing would cache a bundle macOS still considers tampered. `scripts/bust.sh` runs exactly the re-seal-and-re-register half against both the base bundle and any `/Applications/zwire.app` when the LaunchServices cache is stubborn.

# Chapter 6 — The New-Tab HUD

`newtab/` is a `chrome_url_overrides.newtab` extension: the full HUD new tab — a neon clock, an omnibox, quick-launch tiles — with its fonts vendored locally so it renders offline and never phones a font CDN. This chapter walks the page, its palette module, and the scheme sync that keeps it in step with the rest of the HUD.

**What this chapter describes was superseded.** The tiles and the `zb.tiles` key below are the page as it stood through 0.6.59. The new tab now renders a *layout* — Speed Dial groups, a widget grid, a navigation rail — and the tiles are migrated into the default layout once, by copy. **Chapter 77** is the current account. Everything else here — the extension's shape, the vendored fonts, the palette module, the focus trick, the scheme sync — is unchanged and still applies.

## The page and its locally-vendored fonts

The new tab is a small MV3 extension whose whole purpose is one override: `chrome_url_overrides.newtab` points at `newtab.html`. When `zwire` opens a new tab, Chromium serves that page instead of Google's. The manifest pins a stable extension id via its embedded key (id `gpoep...gieb`), declares the `nativeMessaging`, `tabs`, and `history` permissions the palette needs, and lists its own files as web-accessible so they load in page context — including from the `tmux` overlay, which frames this page directly.

`newtab.js` is the page's logic, and it is deliberately dependency-free:

- **The clock.** A one-second `setInterval` writes a zero-padded `HH:MM:SS` time and an uppercased `WEEKDAY, MON DD, YYYY` date into the page. No library, just `Date` and `padStart`.
- **The quick-launch tiles.** A default set (GitHub, DuckDuckGo, MDN, crates.io, Hacker News, the docs site) renders as neon tiles, each a two-letter glyph plus a label. The set is overridable at runtime through a `zb.tiles localStorage` key holding an array of `{label, url}` objects, so the tiles are yours to edit without touching the extension.
- **The omnibox.** The page's own search field decides, per input, whether you typed a URL or a query. `looksLikeUrl` is a small heuristic: a scheme (`https://...`) is a URL; whitespace means a search; a bare `host.tld[/path]` or the literal `localhost` is a URL. A URL navigates directly (adding `https://` if you omitted the scheme); anything else goes to the search engine, default DuckDuckGo, overridable via a `zb.engine localStorage` key with a `%s` placeholder.
- **Reclaiming focus.** This is the page's least obvious and most necessary trick. On its own new tab, Chrome parks the text cursor in the `browser` omnibox and defeats the page's `<input autofocus>`, so the page never receives a single keystroke — which would kill both typing and the `⌘K` palette until you clicked in. `newtab.js` calls `q.focus({preventScroll:true})` in JavaScript, which moves `activeElement` to the page field where the attribute could not, and retries it across the first animation frames and again whenever the tab is re-shown. That is why the HUD new tab is typeable the instant it appears.

The fonts are **vendored locally** under `newtab/fonts/`, not linked from a CDN, and this is a design choice with two payoffs. First, the new tab renders correctly **offline** — the display face is present on disk, so a HUD new tab on a plane looks exactly like one on a fiber connection. Second, the page never makes a request to a font host on every new tab, which is both a privacy property (no third party sees a beacon each time you open a tab) and a latency one (no network round-trip in the render path of the most-opened page in the browser). The `fonts/LICENSE` file records the vendored faces' licensing. Vendoring costs a few hundred kilobytes in

the extension; it buys an offline-correct, beacon-free, instant new tab, and for the page you open more than any other that trade is obviously right.

## Palette, schemes, and scheme-sync

The new tab is a *different extension* from `hud-internal`, which means the global `⌘K` command palette that `hud-internal` injects into ordinary pages cannot reach it — a content script from one extension does not run in another extension's page. So `newtab/palette.js` carries its own copy of the same `ZGui.palette`, built on the shared `zgui-core` fuzzy engine, and because it runs as an extension page with the `tabs` permission it can call `chrome.tabs` directly with no worker round-trip. The palette it builds is substantial: dozens of `chrome://` internal pages (extensions, settings, history, downloads, flags, gpu, net-internals, and a long tail of the `-internals` diagnostics pages), the sub-pages inside `chrome://settings`, one entry per color scheme, plus your frecent history (scored by visit count and recency over the last 90 days) and your open tabs, all fuzzy-filtered in one list.

There is a wrinkle in how the palette opens, and it is the same `⌘K`-reservation problem as focus. On the new tab, Chrome reserves `⌘K` at the browser level, so no page `keydown` listener can catch it. The `hud-internal` background worker owns `⌘K` as a `chrome.commands` shortcut; when the active tab is the new tab it sends the new-tab page a `zwireOpenPalette` message (permitted because the new-tab manifest lists the HUD extension in `externally_connectable`), and `palette.js` toggles the palette in response. On an *ordinary* page `⌘K` and `:` are caught directly. So the palette opens the same way everywhere; the plumbing under it differs because the new tab is a privileged surface Chrome guards more tightly.

The colors are kept in step through `schemes.js`, `palette.js`, and `scheme-sync.js`. `schemes.js` is the canonical table of the eight HUD schemes — its full contents are Chapter 8 and Appendix D. `scheme-sync.js` keeps the new tab following the one scheme the user picked anywhere: because separate extensions do not share `chrome.storage`, the scheme travels through the native file the HUD host writes, and the new tab reads it so switching the scheme in the internal-HUD picker recolors the new tab too. `tmux-pane.js` adds a new-tab surface for the tiling overlay. The through-line of the chapter: the new tab is its own extension, so everything shared with the rest of the HUD — the palette, the eight schemes, the current scheme — has to be *re-carried* here and *re-synced* through the one channel two extensions can share, the native file.

## The three support scripts

Behind the page sit three small scripts that make the new tab a full HUD citizen despite being its own extension. `palette.js` is the new tab's copy of the `⌘K` palette, and because it runs as an extension page with the `tabs` permission it calls `chrome.tabs` directly with no worker round-trip — its list carries the same forty-odd `chrome://` destinations, the nineteen settings sub-pages, one entry per scheme, and, appended asynchronously, your open tabs and your frecent history scored by the same 90-day formula the worker uses. It opens the same three ways the ordinary palette does; on the new tab specifically it responds to the worker's `zwireOpenPalette` message, permitted because the new-tab manifest lists the HUD worker in `externally_connectable`.

`scheme-sync.js` is how the colors follow. It cannot share `chrome.storage` with the internal HUD, so instead it polls the native host every 1.5 seconds (`{cmd: 'get'}` to `com.zwire.hud`), applies the returned scheme's variables to `:root`, and applies the returned UI preferences the same way — toggling the `no-scanlines/no-vignette/no-neon-glow/no-anim` classes (an effect is *on* when its class is *absent*, matching the shared effects toggle) and setting light or dark from `ui.light`. `tmux-pane.js` is the forwarder that lets the new tab be a `tmux` pane: when the page is framed as a pane (its window name is `zbtmux`), the overlay lives in the parent frame and this script catches the prefix and command keys and posts them up, mirroring the pane-side half of `zbtmux-pane.js` so the tiling overlay can frame the new tab as reliably as any site.

## Why the focus trick needs four attempts

The focus-reclaim is worth one more sentence than the section above gave it, because the retries are not superstition. Chrome parks the cursor in the *browser* omnibox and overrides the page's `autofocus` after the page has already run, so a single `focus()` call at load loses a race it cannot see. `newtab.js` therefore calls `q.focus({preventScroll:true})` immediately, again on the first animation frame, and again at 60 and 200 milliseconds, and once more on every `visibilitychange` when the tab becomes visible — covering the load race, the post-layout override, and the “switched back to an already-open new tab” case. It also binds a capture-phase `keydown` directly on the field that swallows `⌘K` (stopping the macOS beep and keeping the field from seeing the key) and calls the palette opener, so the palette works even in the instant before focus has fully settled.

## Web-accessible, so the overlay can frame it

One manifest detail earns the new tab its place in the tmux overlay. Fresh tmux panes open the zwire new tab, but `chrome://newtab` cannot be put in an `iframe` — `chrome://` pages are un-embeddable by design — so the overlay frames the new-tab *extension's* page directly, at its stable `chrome-extension://gpoep...gieb/newtab.html` URL. That only works because the new-tab manifest lists its own files as `web_accessible_resources` for `<all_urls>`, which is what lets another page load them in an `iframe`. It is the same page `chrome://newtab` redirects to, reached by its real extension URL instead of the un-framable alias. The manifest's embedded `key` pins that URL to a stable id across installs, so the overlay's hardcoded new-tab address never goes stale — a small manifest choice that is load-bearing for the tiling overlay two chapters later.

# Chapter 7 — The Internal-HUD Extension: The Injection Model

`extensions/hud-internal` is the largest layer on the runtime path, and it is worth understanding its *shape* before the feature chapters open each script. The extension is an MV3 background worker plus a carefully ordered stack of content scripts, a set of HUD replacement pages, a native host, and the shared `zgui-core` library. This chapter is the map; Chapters 8–16 are the territory.

## The manifest as a wiring diagram

The manifest declares a broad permission set — `storage`, `nativeMessaging`, `management`, `tabs`, `webNavigation`, `history`, `downloads`, `bookmarks`, `sessions`, `alarms`, `notifications`, `developerPrivate`, `settingsPrivate`, `system.display`, and `declarativeNetRequest` — plus `<all_urls>` host access. Each permission earns a feature: `management/developerPrivate` power the HUD extensions page, `history/bookmarks/downloads/sessions` power the HUD replacements for those internal pages, `system.display` feeds window math, and `declarativeNetRequest` carries the `frame_bust` rule that lets the `tmux` overlay iframe any site.

The content scripts are registered in several blocks, and the *order within a block* is load-bearing because later scripts depend on globals earlier ones set. The canonical block loads, in order: `schemes.js` (defines `window.ZWIRE_HUD` with the palette table), the `zgui-core` `util.js/fzf.js/command-palette.js` (defines `window.ZGui`), then `zkeys.js` (the keybinding registry `window.ZWIRE_KEYMAP`), `cmd-defaults.js`, `zpalette.js`, and `zvim.js`. By the time `zvim.js` runs it can read both the keymap and the palette; by the time the palette runs `ZGui` exists. Every script also guards against double-injection with a `window.__zb*Loaded` flag, because content scripts can be injected more than once across frame and navigation events.

## Two treatments for `chrome://` pages, one switch that enables both

`chrome://` pages are privileged, and by default no extension content script runs on them — that is why `zwire` launches with `--extensions-on-chrome-urls`, the switch that lets `hud-internal`'s content scripts match `chrome://*/*` at all. With that switch set, the manifest wires two different treatments for internal pages:

- **Redirect.** For a handful of high-traffic pages — `chrome://extensions`, `chrome://settings`, `chrome://history`, `chrome://bookmarks`, `chrome://version` — a `document_start` content script (`redirect.js`) sends you to a full HUD replacement served from the extension's own `pages/` directory. These are not restyles of Chrome's page; they are purpose-built HUD pages that reimplement the surface using the extension's privileged permissions. Chapter 14 is the tour.
- **Skin.** For every *other* `chrome://` page (the ones without a bespoke replacement — flags, discards, dns, password-manager, and the rest), a `document_start` pair — `schemes.js` then `theme.js` — restyles the page in place with the active scheme's palette. Chapter 8 covers what `theme.js` maps.

The excluded set is exactly the redirected pages plus the new-tab URLs (which the new-tab extension owns), so the two treatments never fight over the same page.

## Where each feature lives

The rest of the internal HUD is a set of content scripts and their support files, each of which gets its own chapter:

```
schemes.js      the 8-scheme palette table (window.ZWIRE_HUD)      -> Ch 8
theme.js        the chrome:// WebUI-token skin        -> Ch 8
background.js   the service worker: scheme bridge, tab bus, term flag -> Ch 8,12,16
zvim.js         vim keybindings on every page        -> Ch 9
ztmux-config.js the tmux overlay provider (shared ZGui.tmux WM) -> Ch 10
ztmux-pane.js   the tmux pane-side forwarder (all frames) -> Ch 10
zpowerline.js   the powerline statusbar adapter (shared ZGui.powerline) -> Ch 11
zpalette.js     the global %K command palette        -> Ch 12
zkeys.js        the keybinding registry (single source of truth) -> Ch 9,12
cmd-defaults.js the seeded %K keyword commands        -> Ch 12
zfind.js        the fuzzy find bar (replaces Ctrl+%F) -> Ch 13
pages/          the HUD internal-page replacements    -> Ch 14
native/zwire-host the native host (scheme/ui/sysinfo/PTY/fs/exec/kv) -> Ch 15
lib/term/       the xterm terminal over the native PTY -> Ch 16
lib/zgui-core/   the shared cyberpunk UI toolkit (submodule) -> Ch 17
```

That is the whole internal HUD in one view: a manifest that wires an ordered content-script stack against every page, two treatments for `chrome://`, a background worker that coordinates across tabs, a native host for the machine, and a shared toolkit under it. The feature chapters take them in that order.

## The eleven injection blocks, in order

The manifest's `content_scripts` array is eleven blocks, and the split is not cosmetic — each block has its own match set, run timing, and frame scope, chosen so the right script runs on the right surface at the right moment. Read top to bottom:

| #  | matches                    | run_at         | all_frames | scripts                                                                                                                                                                                                                                                                                               |
|----|----------------------------|----------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | chrome://{5 redirected}    | document_start | no         | redirect.js                                                                                                                                                                                                                                                                                           |
| 2  | http/https/file            | document_idle  | no         | blank-newtab.js                                                                                                                                                                                                                                                                                       |
| 3  | chrome://*/* (minus above) | document_start | no         | schemes.js, theme.js                                                                                                                                                                                                                                                                                  |
| 4  | chrome://{28 debug pages}  | document_start | no         | debug-cloak.js                                                                                                                                                                                                                                                                                        |
| 5  | chrome://{28 debug pages}  | document_idle  | no         | zgui data-table + card +<br>json-view, debug-adapter.js                                                                                                                                                                                                                                               |
| 6  | http/https/file/chrome://  | document_idle  | no         | schemes.js, pages/fzf.js,<br>zfind.js                                                                                                                                                                                                                                                                 |
| 7  | http/https/file/chrome://  | document_start | no         | schemes.js, zgui util/fzf/<br>command-palette, zkeys.js,<br>cmd-defaults.js,<br>palette-cmds.js, zpalette.js,<br>zgui expose + zexpose.js,<br>zpageactions, zreader, zreload,<br>zbreak, zgestures, zpanels,<br>zextras, ztrash, zcookies,<br>zspatial, zspeak, zpeek, zzap,<br>zvim.js, ztriggers.js |
| 8  | http/https/file/chrome://  | document_idle  | no         | schemes.js, powerline.js,<br>zpowerline.js (statusbar)                                                                                                                                                                                                                                                |
| 9  | http/https/file            | document_idle  | yes        | zpipes-core.js,<br>ztmux-pane.js (pane forwarder)                                                                                                                                                                                                                                                     |
| 10 | http/https/file            | document_idle  | no         | zgui util/split-pane/modal/<br>toast/button-bar/tmux,<br>zpipes-core.js, ztmux-config.js                                                                                                                                                                                                              |
| 11 | http/https/file/chrome://  | document_idle  | no         | modal-drag + xterm + terminal<br>+ zterm-boot                                                                                                                                                                                                                                                         |

Four things fall out of the table. Block 1 fires first and only on the five redirected `chrome://` pages, at `document_start`, so the redirect happens before Chrome's own WebUI paints; block 3 skins every *other* `chrome://` page, also at `document_start`, and its exclude set is exactly block 1's match set plus the new-tab URLs, so the redirect and the skin never fight. Blocks 4 and 5 are a pair over the same twenty-eight *static* `chrome://` debug pages — `local-state`, `gpu`, `policy`, `histograms`, the `*-internals` family — and they exist because an extension cannot intercept Chrome's own WebUI before it renders: `debug-cloak.js` hides the native `<table>`/`<pre>` at `document_start`, `debug-adapter.js` rebuilds them at `document_idle` out of real `ZGui.dataTable` / `ZGui.jsonView` / `ZGui.card` widgets, and a hard timeout in the cloak always un-cloaks so content can never be stuck hidden. Block 7 — the keyboard, palette, and page-action stack, and by some distance the largest — runs at `document_start` and in a fixed internal order, because later scripts read globals earlier ones set: `schemes.js` defines the palette table, the zgui-core trio defines `ZGui`, `zkeys.js` defines the keymap, and only then do `zpalette.js`, the page actions, and `zvim.js` load able to read all three. Block 9 is the only one with `all_frames: true`, because the tmux forwarder and the pipe runtime must run inside every framed pane, not just the top document — which is why the shared window manager itself lives one block later, in top-frame-only block 10, alongside the provider (`ztmux-config.js`) that initialises it. Every script also carries a `window.__zb...Loaded` guard, because content scripts can be injected more than once across an SPA's frame and navigation events, and a second injection must be a no-op.

The permission set reads as a feature list once you know which API powers which surface: `management` and `developerPrivate` back the extensions page and the shortcut editors; `history`, `bookmarks`, `downloads`, and `sessions` back the four list pages; `settingsPrivate` backs the settings page; `system.display` feeds the worker's window-tiling math; `notifications` and `alarms` power the CI monitor; `browsingData`, `tabGroups`, `readingList`, and `power` back the automation bus's data-clearing, tab-grouping, reading-list, and keep-awake verbs; `declarativeNetRequest` carries the single `frame_bust` rule that lets the tmux overlay iframe any site; and `<all_urls>` host access is what lets the content-script stack run everywhere. The `externally_connectable` list holds two ids — the power-tool and the new-tab page — the only extensions permitted to message the worker from outside; the new-tab page uses it to bridge its palette into the automation bus described next.

# The automation bus: 109 browser verbs through one executor

The HUD is not only keyboard-driven; it is scriptable. A stryke script — run from a terminal, from a page's ⌘K palette, from the new-tab palette, or from a HUD page — reaches the browser through `zwire-host` and drives it over a semantic action bus. The surface has two halves: 109 `browser.*` verbs that manipulate the browser itself — one branch each in the worker's `execZbCmd` — and the host's own commands, the file, process, job, bus, key-value, PTY, and hook families `zwire-host` dispatches directly.

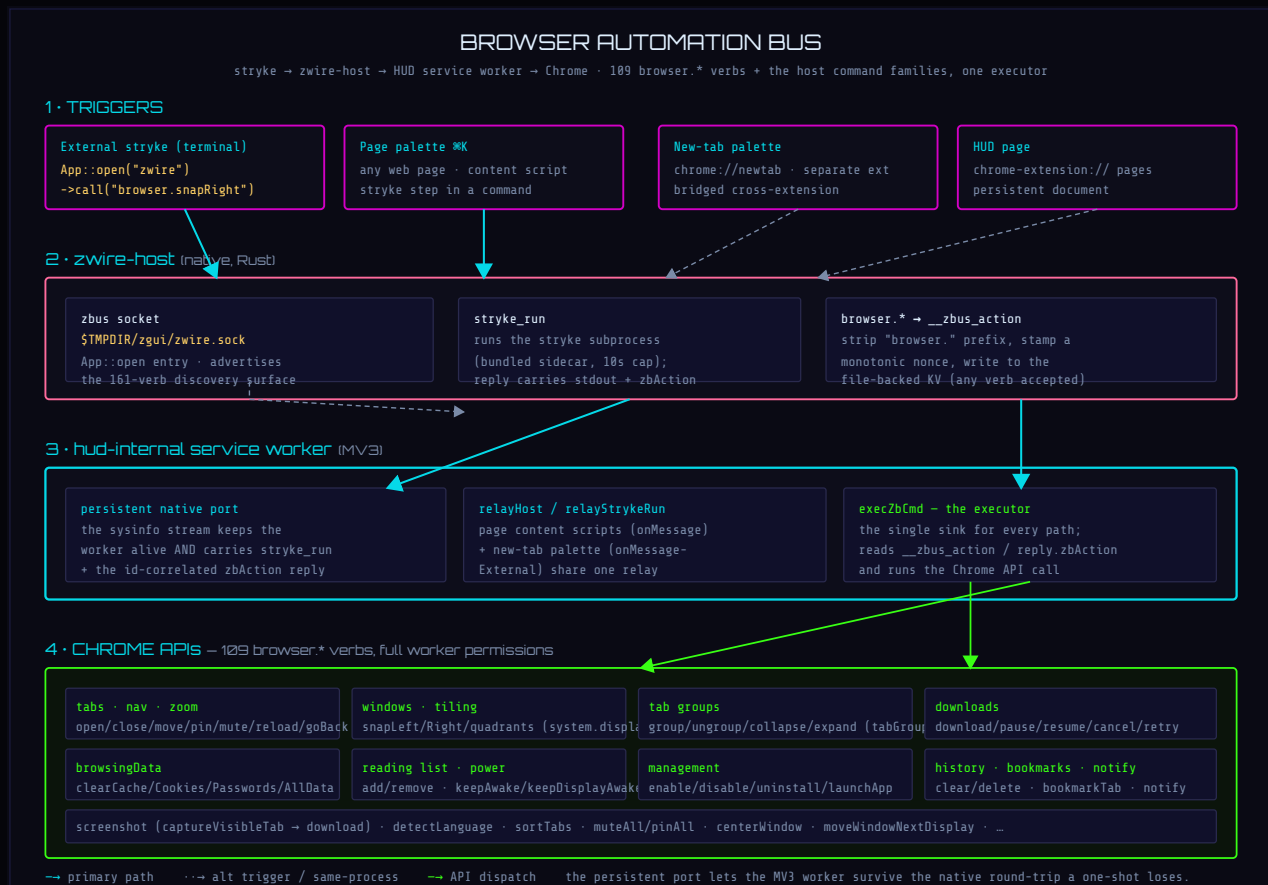


Figure 1: zwire's browser automation bus: a stryke trigger reaches zwire-host, which runs stryke and stamps a `browser.*` action into a file-backed KV; the MV3 service worker's `execZbCmd`, kept alive across the native round-trip by the persistent sysinfo port, is the single executor that runs the Chrome API call across the 109-verb browser surface.

The path has four stages. A trigger runs stryke code containing `App::open("zwire")->call("browser.<verb>")`. `zwire-host` runs the stryke subprocess and, for any `browser.*` verb, strips the prefix and stamps `{a:<verb>, ...args, _n:<nonce>}` into a file-backed key-value entry — it accepts any verb, so the host never needs the verb list: discovery is advisory, execution is open. The MV3 service worker is the single executor. Its `execZbCmd` reads that action — or the `zbAction` piggybacked on a `stryke_run` reply — and issues the matching Chrome API call, with the worker's full `tabs`, `windows`, `downloads`, `bookmarks`, `history`, `tabGroups`, `browsingData`, `readingList`, and `power` permissions behind it.

The hard part is MV3's service-worker lifecycle. A one-shot native message issued from a page's content script loses its reply, because the worker is torn down during the roughly 200-millisecond native round-trip — external pages saw a zero-percent success rate. The fix is the persistent native port that already streams

system stats: it keeps the worker alive and delivers the reply on an already-open channel, correlated by a request id so the browser action fires exactly once. The new-tab page — a separate extension whose content scripts are excluded from `chrome://newtab` and whose storage is isolated from the worker's bus — bridges its palette's host-backed steps into the same executor over `externally_connectable`, so `execZbCmd` stays the one place a browser action is ever run, whichever surface asked for it.

The 109 browser verbs span the browser's own surface: tab lifecycle and navigation, page reload and history motion, per-tab zoom, window state and — using `system.display`'s work area — left/right/quadrant tiling and multi-monitor moves; tab grouping; the full download lifecycle; browsing-data clearing; the reading list; keep-awake; extension and app management; and history, bookmark, and notification actions. Because the executor is one function and the host forwards any verb, adding a capability is a single `execZbCmd` branch plus a single discovery-list entry — the surface grows without a new transport.

# Chapter 8 – The Eight Schemes and the Sync Bus

The HUD has eight color schemes — cyberpunk, midnight, matrix, ember, arctic, crimson, toxic, vapor — and every surface in the browser follows the one the user picks: the ordinary-page content scripts, the `chrome://skins`, the separate new-tab extension, the `zpw:chrome` power-tool, and (on the fork) the native color mixer. This chapter is the palette table, the token map that applies it to Chrome’s own pages, and the cross-extension bus that keeps them all agreeing.

## The palette table

`schemes.js` is the canonical scheme table, `window.ZWIRE_HUD`, and it is the same file loaded by every content script and by the new-tab extension — auto-extracted from the same `hud-theme.js` the wider MenkeTechnologies app suite uses, so the browser cannot drift from the apps. It defines three things: an ordered list of twenty CSS custom-property keys (`SCHEME_VAR_KEYS`), the scheme order (`SCHEME_ORDER`), and the schemes themselves (`COLOR_SCHEMES`). Each scheme has a `label`, a one-line `desc`, a dark `vars` set, and a parallel `lightVars` set:

```
cyberpunk: {
  label: 'Cyberpunk', desc: 'Hot pink + cyan (default)',
  vars: {
    '--accent': '#ff2a6d', '--cyan': '#05d9e8', '--magenta': '#d300c5',
    '--green': '#39ff14', '--red': '#ff073a',
    '--text': '#e0f0ff', '--text-dim': '#7a8ba8',
    '--bg-primary': '#05050a', '--bg-card': '#0d0d1a', '--border': '#1a1a3e',
    /* ...20 keys total, plus the glow/dim variants... */
  },
  lightVars: { /* the same 20 keys, tuned for a light background */ },
}
```

The twenty keys are the HUD’s whole vocabulary: three accent shades, three cyan shades, two magenta, a green pair, a red, three text weights, four background depths, and two border shades. Every HUD surface — the statusbar, the tmux panes, the palette, the find bar, the internal pages — styles itself entirely in terms of these variables, so recoloring the browser is just swapping which scheme’s values are written to `:root`. The eight schemes’ full dark-and-light palettes are Appendix D; the point here is that they are *data*, one table, shared by every consumer, with a matched light variant so the HUD works on a light background too.

## Applying the palette to Chrome’s own pages

`theme.js` is the script that maps the active scheme onto Chrome’s internal pages — the `chrome://` pages that have no bespoke HUD replacement. Its comment is careful about what it is *not*: it deliberately drops all the HUD decoration (the grid lattice, CRT scanlines, neon glow, the floating picker) and maps *only* the scheme’s palette onto Chrome’s WebUI color tokens, plus the mono font, so these pages simply inherit the same theme the rest of the browser uses — no HUD chrome on top. The mechanism is a stylesheet that rewrites Chrome’s own CSS custom properties to point at the HUD variables:

```

:root, :host {
  --color-sys-surface: var(--bg-primary) !important;
  --color-sys-on-surface: var(--text) !important;
  --color-sys-primary: var(--cyan) !important;
  --color-sys-secondary: var(--accent) !important;
  --color-sys-outline: var(--border) !important;
  /* ...plus the --cr-fallback-color-* and --cr-*--background-color families... */
}

```

Chrome's WebUI is built on these `--color-sys-*` and `--cr-*` tokens, so overriding them with `!important` recolors the page's own components without touching their markup. The scheme `theme.js` reads the native source of truth (`~/ .zwire/hud-scheme`), fetched via the background worker — a content script cannot call `sendNativeMessage` itself, so it reads the mirrored value the worker maintains.

## The sync bus: how five consumers agree on one scheme

The hard part is not styling one surface; it is keeping every surface agreeing on one value across three isolation boundaries — content scripts vs the worker, extension vs extension, and the browser vs the native chrome. `hud-internal` solves it with a two-tier store plus a native file as the ultimate source of truth:

- **Content scripts ↔ worker.** The background worker mirrors the current scheme into `chrome.storage.local` under `zb_scheme`, and the content scripts read and observe that key. The worker uses storage rather than an async native round-trip on purpose, and the code comments say why: an MV3 worker can suspend mid-round-trip, so a `sendResponse` across the native host is unreliable, while a storage write always lands.
- **Extension ↔ extension ↔ native chrome.** The scheme travels through `native/zwire-host` and the scheme file it owns. When a page or the picker sets a scheme, the worker relays it to the host (which atomically writes the file) and writes `zb_scheme`. On startup it seeds `zb_scheme` from the file, which may already hold a scheme chosen in a prior session. The new-tab extension reads the same file (the one channel two extensions can share), and on the fork the native color mixer reads it too.
- **The `zpwchrome` bridge.** Because separate extensions cannot share storage, the worker also pushes the scheme to `zpwchrome` over `runtime.sendMessage`, with a guard (`lastPushed`) so a scheme that came from `zpwchrome` is not echoed straight back into a loop. `zpwchrome` can also pull the current scheme on startup (`zb-scheme-get`) and push a user's pick back (`zb-scheme-set`).

A companion file, `~/ .zwire/hud-ui.json`, carries the shared visual-effect and light-mode preferences (light mode, scanlines, vignette, glow, animation) the same way, since those are per-origin `localStorage` on each page and the native file is the only channel that reaches the other extensions. The whole design is one value with one source of truth (the native file) and several caches that all reconcile to it — which is why picking “matrix” in the floating picker recolors the statusbar, the tmux panes, the find bar, the new tab, the `chrome://` pages, and the power-tool, all at once.

## The fleet-wide single source: `zwire-host` and `global.toml`

The per-extension mesh above — `chrome.storage` mirrors plus cross-extension `sendMessage` pushes — is fragile: two surfaces writing at once (a scheme change here, a light toggle there) could desync `newtab`, `zpwchrome`, and the HUD. Newer builds move the *source of truth* off that mesh and onto the native host. The scheme, the light-mode flag, the visual-effect prefs, and any user-defined custom schemes now live in one app-independent file the Rust `zwire-host` owns — `~/ .zwire/global.toml` — read and written by every `zwire-host` client, not just the browser but the wider MenkeTechnologies fleet (Audio-Haxor, `zmax`, `zpw-daw`, and the rest), so a scheme picked in any app repaints all of them. `newtab` and `zpwchrome`

subscribe the host bus directly (a snapshot on connect plus live updates), and the HUD background folds a host theme subscription into its sysinfo port and bridges it to `chrome.storage` with an echo guard — retiring the old push/pull/mirror/seed mesh. This also fixed a real race: two concurrent writes (`{scheme}` and `{ui}`) could clobber each other and silently revert the scheme, so the host now serializes the read-modify-write of `global.toml` behind a cross-process lock (`global.toml.lock`). The legacy `hud-scheme` and `hud-ui.json` files are no longer the bus; the host still writes a plain-text `hud-scheme` projection (and, for light mode, a `hud-light` projection) *beside* `global.toml` purely so the fork's C++ color mixer — which cannot parse TOML at paint time — has a trivial file to read. `zwire-host` also keeps a shared command log, `~/.zwire/hostlog.jsonl`, recording every message it sends and receives (the sysinfo stream included), which the HOST internal page reads back through a `hostlog` command.

Light and dark are part of the same single source. Every HUD surface can toggle light mode — the `⌘K` palette carries a “Toggle light mode” command, the shared command palette from `zgui-core` adds its own light/dark entries, and the app-shell settings modal has a switch — and each writes the light flag through to `global.toml` exactly the way a scheme change does, so light toggled from any surface flows to the one file and every renderer follows. On the fork, the native chrome follows too: the toolbar, tabstrip, and frame read the `hud-light` projection and repaint on launch, so the whole browser, native chrome included, honors one light/dark choice (Chapter 21).

## The background worker's full job list

The scheme relay is one of a dozen things `background.js` does. It is a single MV3 service worker — no persistent page, woken by events and suspended when idle — and it is the coordinator every content script leans on for anything a page cannot do itself. Its own header comment names only the first two jobs (relay the scheme to the native host, mirror it into `chrome.storage.local`), but reading the file top to bottom the worker turns out to own nine distinct responsibilities, each wired to its own set of Chromium events:

- **The scheme fan-out** (Chapter 8's subject): `mirror(scheme)` writes `zb_scheme` and calls `pushToZpwr`, which relays to the power-tool over `runtime.sendMessage` with the `lastPushed` echo guard.
- **The native seed on startup**: `seedFromNative` sends `{cmd: 'get'}` to `com.zwire.hud` and mirrors whatever scheme the file already held, so a scheme picked in a previous session survives a browser restart. It is registered on both `onStartup` and `onInstalled`, and called immediately, because the file “may already hold a scheme picked in a prior session or written before launch.”
- **The `⌘K` command owner**: the worker registers `open-palette` as a `chrome.commands` shortcut (`Command+K` on macOS, `Ctrl+K` elsewhere) and routes it to whichever palette matches the active tab — the content-script palette on an ordinary page (`tabs.sendMessage`), the new-tab extension's palette (cross-ext `sendMessage`), or a HUD page's own palette (a runtime broadcast). Its comment spells out why the browser command must be the single owner: the new-tab page reserves `⌘K` at the browser level before any page JS sees it, so a page listener could never open the palette there, but a `chrome.commands` shortcut fires regardless of focus — at the cost of consuming `⌘K` on every tab, which is why the routing lives here.
- **The sysinfo stream** (Chapter 11): a persistent `connectNative` port that sends `sysinfo_start`, writes each `{sys:{...}}` frame into `zb_sys`, and reconnects five seconds after any disconnect. The comment notes a second reason the port is held open: it keeps the MV3 worker itself alive.
- **The per-tab PTY relay** (Chapter 16): the `zwire-pty` connection handler, covered in its own section below.
- **Four storage mirrors**: the worker keeps `zb_tabs` (every open tab), `zb_exts` (enabled extensions, via `chrome.management`), `zb_shortcuts` (every extension's keyboard shortcuts, via `developerPrivate` — `chrome.management` does not expose commands), and `zb_frecent` (a frequency-scored history list) continuously fresh, each wired to the relevant Chromium event (`tabs.onUpdated`, `management.onInstalled`, `history.onVisited`, and so on). These exist because a content-script palette cannot call `chrome.tabs` or `chrome.management` itself, and — the comment is explicit — “an async worker round-trip” from a page is unreliable when the worker may be asleep, so the worker publishes to storage eagerly and the page reads storage directly.

- **The custom-command seed:** `seedCustomCmds` populates the `%%K` keyword-command registry once, guarded by a `zb_cmds_seeded` flag, “merging defaults with any existing user entries” so a user’s edits and deletions are never clobbered. (Chapter 12 and Appendix H.)
- **The CI monitor:** a GitHub-Actions poller on a `chrome.alarms` heartbeat, covered below.
- **A second tiling engine:** a `chrome.windows`-geometry tmux implementation that tiles *real OS windows* as panes, driven from the same `zb_cmd` bus, distinct from the in-page iframe overlay of Chapter 10.

The frequency list is worth pinning down because the same formula appears in three places (the worker, the new-tab palette, and the HUD pages’ `zg-boot.js`): a `chrome.history.search` over the last 90 days, scored  $(\text{visitCount} + 2 \cdot \text{typedCount}) / (1 + \text{ageDays} \cdot 0.3)$ , `chrome://` URLs filtered out, top 30 kept. A typed visit counts double a clicked one, and the recency divisor decays a site’s score by roughly a third per day of age, so a page you typed yesterday outranks one you clicked ten times last month.

## The storage command bus

The recurring problem the worker solves is that an MV3 service worker can be suspended at any instant, and a `runtime.sendMessage` to a sleeping worker is not guaranteed to wake it in time. The HUD’s answer, used by vim, the palette, and the tmux overlay alike, is the **storage command bus**: a content script writes a `zb_cmd` object into `chrome.storage.local`, and `storage.onChanged` is a reliable MV3 wake-up, so the worker always runs the command. The worker’s comment states the contract directly — “content scripts write `zb_cmd` to storage; `storage.onChanged` is a reliable MV3 wakeup (unlike `sendMessage` to a sleeping worker), so palette navigation / tab-switching always executes.”

A `zb_cmd` carries an action field `a` and a monotonic tag `n` (so a repeated identical command still registers as a change), plus any parameters. The worker’s `onChanged` handler dispatches on `c.a` across a broad tab-and-window vocabulary: `open/openTab` (navigate or new tab), `activate` (switch to a tab and focus its window), `newTab/newWindow/duplicateTab/reopenTab`, `pinTab/muteTab`, `closeTab/closeOthers`, `nextTab/prevTab`, `tmux` (handed to the worker’s own tiling engine), and `ping` (a wake-and-refresh that re-runs all four storage mirrors). This is why a vim `x` can close the current tab and a palette selection can switch to another window even though neither the vim script nor the palette script can touch `chrome.tabs`: they write one storage key and the worker, woken by the write, does the privileged work. The same `onChanged` handler also fans a `zb_scheme` write out to the power-tool, so a scheme picked on the Settings page — which writes storage and the native file but never messages the worker — still reaches `zpw:chrome`.

## The per-tab PTY relay, keyed by tab

The terminal’s bytes flow through the worker because a content script cannot open a native-messaging port that survives navigation. The worker exposes a `zwire-pty runtime.onConnect` port and keeps a `ptySessions` map **keyed by tab id**, so — the comment says — “the shell SURVIVES page navigation.” Each session holds its own `connectNative` port to `com.zwire.hud`, the content-script port currently attached (or `null`), an output buffer, and a `spawned` flag. Three details make it robust: output produced while no page is attached is buffered (capped at 500 messages) and flushed on reconnect; `pty_spawn` is honored only once per tab (`if (sess.spawned) return`), so a re-injected content script re-attaches to the running shell instead of forking a second one; and a content-script disconnect sets `sess.cs = null` rather than killing the native port, “to keep the shell alive across nav,” while an actual `tabs.onRemoved` tears the native port down. The result is a shell session bound to a browser tab, not to a page — you can click a link, load a new site, and keep typing in the same zsh.

## The CI monitor: a poller, an alarm, and a desktop notification

The one worker feature with no page-facing counterpart in the earlier chapters is a continuous-integration monitor. On a `chrome.alarms` heartbeat named `zb-ci-poll` (`periodInMinutes: 10`) the worker runs `pollCi`, which walks your GitHub repositories through the REST API — `/user/repos` with a token, else `/users/<user>/repos` for the default account `MenkeTechnologies` — fetches the latest workflow run per repo, and tallies failing, running, and passing counts into a `zb_ci_status` object the statusbar can show as a dot. The request budget is tuned to GitHub's anonymous cap: the comment “stay under the 60/hr anon cap” sits next to a repo limit that rises from five to twelve only when a token is present. When the failing count rises above the previous poll, and `chrome.notifications` exists, the worker raises a desktop notification titled “CI failing” whose body lists the first four broken repos, drawn with a 16-pixel hot-pink `CI` tile inlined as a base64 SVG “so notifications never depend on an icon file.” This is the only `chrome.alarms` use in the extension, and the only place the HUD reaches out to a network service on its own initiative; the CI page of Chapter 14 is its interactive front end, and the two share the `zb_ci` config key.

## The other tmux: tiling real OS windows

There is a second tiling engine in the worker, and it is easy to miss because it shares the `tmux` vocabulary with the in-page overlay of Chapter 10 while doing something entirely different. The overlay tiles *iframes inside one page*; the worker's engine tiles *real Chromium windows* by geometry. Its own comment states the model: “PANES = real browser windows, tiled by `chrome.windows` geometry. WINDOWS (tmux) = named groups of panes; only the active group is visible, the rest are minimized. SESSIONS = the whole thing, persisted to storage.” State lives in a `TMUX` object mirrored to `zb_tmux_state` so it survives a worker eviction, and the tiling area comes from `chrome.system.display`'s primary work area (falling back to 1440×820 when the display API is unavailable).

The engine is driven from the same `zb_cmd` bus (`{a:'tmux', sub:...}`) and its subcommands mirror tmux: `split` (down splits into rows, else columns), `navigate`, `zoom`, `closePane`, `newWindow`, `nextWindow/prevWindow`, `selectLayout` (cycling `cols`, `rows`, `main-v`, `grid`), `syncToggle`, `killWindow`, and `retile`. `ArectsFor(n, layout)` helper computes the pixel rectangles — a 60% main pane for `main-v`, a  $\lceil\sqrt{n}\rceil$ -column grid otherwise — and `tile` minimizes every window not in the active group, positions the rest, and focuses the active one. Because real windows can be closed or focused outside the engine's knowledge, a `reconcile` step drops closed panes and prunes empty groups on `windows.onRemoved` and `onFocusChanged`, and an `ensureInit` rebuilds the state from `zb_tmux_state` after an eviction, defaulting to “the current window is the sole pane.” The engine even has its own synchronize-panes: a `zbSync` message fans a keystroke out to the active tab of every sibling window in a syncing group. It is a genuine window manager for your browser windows, living in the service worker, distinct from and parallel to the in-page overlay — two answers to “tmux in the browser,” one tiling pages and one tiling windows.

## The color science under the eight names

The eight schemes are not eight arbitrary palettes; they are eight *coordinated* ones, and the coordination is what lets a single content script skin an arbitrary `chrome://` page without auditing it first. Each scheme is a small closed set of roles — a background, a raised surface, a foreground, a dimmed foreground, an accent, and a warning — and the skinning layer paints roles, not pixels. That indirection is the reason the same `theme.js` that colors `chrome://history` also colors `chrome://settings` and every HUD internal page: the page asks for `--zw-accent`, not for `#00e5ff`, and the scheme table decides what the accent is this minute. Change the scheme and every surface that referenced a role moves in lockstep, because there was only ever one place the literal color lived.

The dark schemes share a discipline that keeps neon legible against black, which is the hard case a cyberpunk

palette must survive. A saturated cyan or magenta at full value on a near-black field is the classic mistake — it blooms, it strobes on LCD subpixels, and long-form text set in it is unreadable. The schemes avoid it by never using the accent as body text: the accent lives on borders, the active tab, the statusbar segments, the cursor, and the focus ring, where a thin run of high chroma reads as *signal* rather than as a wall of glow, while the body foreground is a high-value near-white or near-cyan held well below full saturation. The eye reads the accent as an edge and the foreground as a plane, and the two never compete. This is why a scheme can be maximal in name — `matrix`, `synthwave`, `cyberpunk` — and still be a working reading surface: the loud color is rationed to the geometry, not spent on the prose.

The light schemes are the harder engineering, and the book is honest that they came later. Inverting a dark cyberpunk palette does not produce a usable light one — the same cyan that reads as signal on black is nearly invisible on white, and a magenta that glows on a dark field turns muddy on a bright one. The light variants therefore re-derive each role against a light ground rather than flipping a value: the background rises to a warm off-white, the accents drop in value and rise in saturation so they still carry against the paper, and the dimmed foreground — the one role most likely to fail a contrast check — is pinned to a value that clears the readable threshold rather than being computed as a fixed alpha over the background, because a fixed alpha is exactly what fails when the background changes from black to white. The fork's native color mixer reads the same role names from the host's `hud-light` projection, so the eight-times-two matrix stays a single source: eight names, a dark and a light derivation of each, one set of roles, and one table that every surface — content script, new tab, fork chrome, and DevTools dropdown — reads through. The palette is not decoration bolted onto the browser; it is a small, closed color system, and the eight names are its presets.

# Chapter 9 – Vim on Every Page

`zvim.js` puts real vim keybindings on every page: normal mode by default, suppressed while you are typing in an editable field, with motions, link hints, marks, tab operations, and a per-tab off switch. This chapter is the whole keyboard model, and it pairs with `zkeys.js`, the shared registry that makes every key remappable.

## Normal mode, and never stealing keys

The script installs one capture-phase `keydown` listener on the document and decides, per key, whether it owns it. The first rule is restraint: if the active element is editable (an `input`, `textarea`, `select`, or `contentEditable`), vim does nothing except let `Escape` blur the field — so typing in a search box or a comment box is never intercepted. With any of `Alt/Meta/Ctrl` held (except its own combos) it also bows out. And crucially, an *unknown* key is left alone: the listener only calls `preventDefault` when the key maps to a real action, so keys vim does not claim reach the page normally. When it *does* act, it calls both `preventDefault` and `stopImmediatePropagation`, so a single-key site shortcut (a Gmail `j`, say) does not also fire.

## Motions, screens, and the z-family

The core motions are the vim ones, each a small `scrollBy/scrollTo`:

- `j/k` scroll down/up, `h/l` scroll left/right (66px steps), `d/u` half-page down/up.
- `gg` (a two-key sequence via the `g` prefix) jumps to the top; `G` to the bottom.
- `H/M/L` jump to the high/middle/low of the *document* — the vim screen motions, mapped here to top/middle/bottom of the page.
- The `z`-family (`zt/zz/zb`, via the `z` prefix) positions a reference element to the top/center/bottom of the viewport. The reference is the focused element if it is real and on-screen, else whatever is at the center of the window — computed with `getBoundingClientRect` and a `scrollBy`.

Two-key sequences are handled with a small pending-key state and a 900ms timeout: pressing `g`, `z`, `y`, `m`, or ``` arms a prefix, and the next key completes it (or the prefix expires). This is how `gg`, `gt/gT` (tab next/prev), `gi` (focus the first input), `yy` (yank the URL to the clipboard), `m<x>` (set a mark), and ``<x>/'<x>` (jump to a mark) all work from single-character keys without colliding.

## Link hints

`f` and `F` enter hint mode. The script collects every clickable element in the viewport — `a[href]`, `button`, form controls, `[role=button]`, `[onclick]`, `[tabindex]` — filters out the invisible and off-screen ones, and overlays a short letter label on each (one letter when there are few targets, two-letter combinations when there are many, drawn from a home-row-first alphabet). You type the label; a unique match activates immediately. `f` clicks in place; `F` opens links in a new tab (routed through the background worker). Editable targets get focused rather than clicked, and `Escape` or a no-match cancels. Highlighting the labels is a fixed-position overlay themed in the HUD yellow, with the active filter shown in cyan.

## Tabs, history, marks, and the storage bus

Tab operations cannot be done from a content script directly — a page cannot close or switch its own tab — so they route through the background worker over what the code calls the storage command bus: `zvim.js` writes a `zb_cmd` object into `chrome.storage.local`, which reliably wakes the MV3 worker (more reliable than a `runtime.sendMessage` to a suspended worker), and the worker performs the tab action. `J/K` switch tabs, `x` closes, `t` opens, `r` reloads, `[/]` are back/forward. Marks are per-page scroll positions kept in memory. A small toast in the corner confirms actions like a yank or a mark set.

## The keybinding registry and remaps

Every default key above lives in `zkeys.js`, `window.ZWIRE_KEYMAP` — the single source of truth, loaded both as a content script (before `zvim/zpalette/zfind`) and by the Keyboard settings page so both render and dispatch from the same list. It is a categorized table: Scroll, Tabs & history, Jump & hints, Palette & find, and a large Tmux category (every post-prefix key), plus global chorded hotkeys (`⌘K`, `⌘F`) and the fixed native prefix. User remaps live in `chrome.storage.local` under `zb_keys` as `{action: key}`; `zvim` builds its `key` → `action` map by merging those over the defaults and rebuilds it live when the key changes. So rebinding `scrollDown` from `j` to `s` in the settings page immediately changes what `zvim` fires — and, because the tmux overlay reads the same registry, the same mechanism remaps the tmux post-prefix keys too. `Ctrl/⌘+\\` toggles vim off for the tab (and back on), for the moments you want the page's raw keys.

## The handler, in the order it decides

The whole of `zvim.js` is one capture-phase `keydown` listener on the document — registered with the `true` third argument so it sees keys before the page does — and reading it is reading a sequence of guards in a deliberate order. First, if vim is disabled for the tab, only `Ctrl/⌘+\\` (which re-enables it) is honored and everything else returns. Second, hint mode: while link hints are on screen, the listener routes the key to the hint filter before any other rule, so typing a hint label is never mistaken for a motion. Third, the editable guard: inside an `input`, `textarea`, `select`, or `contentEditable` element only `Escape` acts (it blurs the field) and every other key passes through untouched. Fourth, the modifier bailout: any key held with `Alt`, `Meta`, or `Ctrl` is left alone, because those chords belong to the palette, the find bar, and the vim toggle, not to motions. Only after all four does the key reach the prefix state machine and the keymap. The listener is scrupulous about *not* stealing keys: it calls `preventDefault` only when the key maps to a real action — an unknown key gets no interference — and when it does act it calls both `preventDefault` and `stopImmediatePropagation`, whose comment reads “win over site single-key shortcuts,” so a Gmail `j` fires the vim scroll and not Gmail's own handler.

The exact motion amounts are small and fixed: `j/k/h/l` scroll 66 pixels; `d/u` scroll half the viewport height (`innerHeight · 0.5`, rounded); `G` and `L` both jump to the document bottom (they run identical code); `H` to the top; `M` to the vertical middle of the document. The `z`-family positions a *reference element* — the focused element if it is real and on screen, otherwise whatever sits at the center of the window (`elementFromPoint` at the viewport midpoint) — to the top, center, or bottom of the viewport, computed from its `getBoundingClientRect` with a four-pixel margin.

## The two-key prefix machine, marks, and yank

Multi-key commands run through one small state variable and a **900-millisecond** timeout. Pressing `g`, `z`, `y`, `m`, or ``` arms that prefix; the next key completes it or, after 900 ms of silence, the prefix simply clears. Under `g`: `gg` scrolls to the top, `gt/gT` switch to the next/previous tab (through the storage bus), and `gi` focuses the first non-hidden input on the page. Under `y`, `yy` copies the page URL to the clipboard and flashes a toast. Under `m`, `m<x>` records the current scroll position into an in-memory `marks` object keyed by the character you pressed;

under ``` (and `'`, which is aliased to it), ``<x>` scrolls back to that saved position. Marks are per-page and live only in memory — they are scroll bookmarks for the current page, not durable named locations. The toast that confirms a yank or a mark is a single fixed-position element in the corner, themed in the HUD cyan on a translucent black, faded out after 1.2 seconds.

## Link hints, precisely

`f` and `F` enter hint mode. The script collects everything plausibly clickable — the selector is `a[href], button, non-hidden input, select, textarea, [role=button], [role=link], [onclick], and [tabindex]` — then discards anything smaller than two pixels, anything scrolled off screen, and anything with `visibility:hidden, display:none`, or zero opacity. Each survivor gets a label drawn from a home-row-first alphabet (`asdfghjklqwertyuiop...`): one letter while there are 26 or fewer targets, two-letter combinations beyond that. Labels render as fixed-position yellow chips (cyan once you have typed a matching prefix). Typing narrows the set; the moment exactly one label matches, that target activates. `f` clicks in place; `F` routes a link through the storage bus to open in a new tab; an editable target is *focused* rather than clicked; `Escape` or a dead-end keystroke cancels. The hint overlay sits at z-index 2147483645, one below the find bar and palette, so those always draw over it.

## One registry, live remaps

Every default key above is data in `zkeys.js` (`window.ZWIRE_KEYMAP`), and `zvim.js` never hardcodes a key — it builds a `key → action` table by walking the registry's categories and letting a user override win over each default. The overrides live in `chrome.storage.local` under `zb_keys` as `{action: key}`, and the script both reads them at load and re-listens on `storage.onChange`, so rebinding `scrollDown` from `j` to `s` on the Keyboard settings page changes what fires on every open page the instant you save it, with no reload. The same registry and the same `zb_keys` key drive the tmux overlay's post-prefix keys, which is why one settings page remaps both. The per-tab off switch (`Ctrl/⌘+\\`) flips an in-memory flag rather than a stored one, so it is deliberately transient — a page where you have turned vim off gets its raw keys back, and the next page starts with vim on again.

# Chapter 10 – tmux in the Browser

The tmux overlay is the most ambitious thing the extension injects: a real, in-page tiling window manager modeled on tmux and zellij, where a session holds windows which hold panes, the panes are actual webpages rendered in iframes, and you split, zoom, swap, rotate, resize, lay out, copy, paste, and broadcast-type across them — all without a single OS window. It is roughly 1,800 lines of shared toolkit (`ZGui.tmux`, in `zgui-core`'s `webui/tmux.js`) driven by two thin extension-side scripts; the last section of this chapter is how it got there. This chapter is its model and its command set.

## The session/window/pane tree

The whole state is one in-memory object: a session `S` with a list of `windows` and an `active` index. Each window has a binary *tree* of nodes — a `leaf` is a pane (an id, a url, a title), and a `split` has a direction (`row` = side by side, `col` = stacked), a `ratio`, and two children `a/b`. Nesting is arbitrary: a split's child can be another split, so panes tile to any depth. This is the same data model tmux uses internally, rebuilt in JavaScript, and every command is a tree edit:

- `splitPane(dir)` replaces the active leaf, in place, with a new split whose children are the old leaf and a fresh pane.
- `closePane()` finds the active leaf's containing split and promotes its sibling into the split's slot (collapsing the split), killing the window if it was the last pane.
- `swapPane`, `rotatePanes`, and `breakPane` (move a pane out to its own window) are all slot swaps in the tree.
- `resizePane(dir, amt)` walks from the active leaf up to the first ancestor split on the right axis and nudges its `ratio`, clamped to `[0.08, 0.92]`.

A `nodeSlot(root, target)` helper returns the `{parent, key}` such that `parent[key] === target`, and that little abstraction is what makes every structural command a two-line edit rather than a special case.

## Panes are iframes, which is why the fork strips X-Frame-Options

Each pane renders a real webpage in an iframe. The catch is that most sites send `X-Frame-Options` or a frame-ancestors CSP that forbids being framed — so `hud-internal` ships a `declarativeNetRequest` rule (`rules/frame_bust.json`) that strips those headers, and on the fork the native `ancestor_throttle.cc` patch (0008) does the same at the engine level. That is the whole reason patch 0008 exists: without it, the tmux overlay could only tile a handful of framing-friendly sites. Fresh panes open the `zwire` new tab, and because `chrome://newtab` itself cannot be iframed (`chrome://` pages are un-embeddable), the overlay frames the new-tab *extension*'s web-accessible page directly — the same page `chrome://newtab` redirects to.

The overlay runs in *all* frames, and the roles differ. The top frame hosts the overlay UI, the tree, and all the commands. A pane iframe (marked with `window.name='zbtmux'`) runs a thin forwarder: it catches the `Ctrl-b` prefix and the following command key and `postMessages` them up to the top frame, so the prefix works even while your focus is inside a framed page. The prefix is `Ctrl-b` (the real tmux one) or, because the pre-rebuild fork's native split patch used to eat `Ctrl-b`, `⌘-b` (Alt-b) as an alternate nothing else intercepts.

## The command set

After the prefix, the keys are the tmux ones, and every one is remappable through the shared registry (Chapter 9). The full set from `zkeys.js`:

```
% split right      " split down      o next pane      ; last pane
h/j/k/l select pane ← ↓ ↑ →      H/J/K/L resize pane ← ↓ ↑ →
z zoom pane        x close pane      { } swap ← →      O rotate
! break to window  q pane numbers    space cycle layout
F/G/I grid 4/8/16
c new window        n/p next/prev win  Tab last window    w window list
, rename window     . move window      ' go to window      & kill window
f find window       i window info      Enter open a document in this pane
m mark / swap       e synchronize panes E toggle this pane in sync group
[ copy mode         ] paste buffer    = choose buffer
s sessions          S save session    M manage saved layouts (editor)
r reload pane       t clock           : palette          d detach  ? help
0-9 select window  ←→↑↓ focus by direction (Ctrl/Alt + arrows resize)
```

Layouts (`space`) cycle through even-horizontal, even-vertical, main-vertical, main-horizontal, and tiled — each rebuilt by `buildEven/cycleLayout` from the current leaf list. Directional focus (`navDir`) uses the rendered pane rectangles to pick the nearest pane in a direction, so the arrows move focus spatially like tmux's `select-pane -LRUD`.

## Copy mode, paste buffers, and synchronize-panes

Because panes are real scrollable documents, copy mode (`[`) is a genuine scroll-and-select mode inside the focused pane: `j/k/d/u/g/G` scroll, you select text, and `y/Enter` yanks the selection up to the overlay's paste-buffer stack (most-recent-first, capped at 30). `]` pastes the most recent buffer into the focused pane's field, and `=` opens a chooser. Any ordinary copy in a pane is also captured into the stack.

Synchronize-panes (`e`) is the broadcast-typing feature, and it is careful about line editing. When sync is on, a pane forwards each editable keystroke up to the top frame, which relays it to every other pane's focused (or last-focused) field. Plain characters, `Enter`, and `Backspace` forward as-is; the readline combos forward as *semantic tokens* so the broadcast stays correct — `C-w` (kill word) and `C-u` (kill to line start), plus their macOS `⌘/⌘-Delete` twins. Each receiving pane applies the token cursor-aware, deleting the right span and firing a real `input` event so the framed page's own handlers see the change. This is how you type one command into eight terminals-in-tabs at once.

## Persistence

The whole tree lives in memory, so a reload of the host page would wipe it. The overlay snapshots the windows, panes, layout, and paste buffers to per-tab `sessionStorage` (`zbtmux-session-v1`) and restores on re-inject, so the tmux session survives a page reload — the tree, the active window and pane, and the buffers all come back. It is themed entirely with the active scheme's CSS variables, so the tabs, pane borders, dividers, and modals match cyberpunk, ember, matrix, and the rest live. The overlay reserves the bottom 22px for the real powerline statusbar, which is the next chapter.

## Resize and layout, as tree math

Resizing walks the tree. `resizePane(dir, amt)` picks an axis from the direction (`left/right` → a `row` split, `up/down` → a `col` split), builds the path from the active leaf up to the root recording which side each descent came from, and nudges the ratio of the *first* ancestor split on that axis, clamped to `[0.08, 0.92]`.

The nudge size depends on how you asked: a plain `Ctrl+arrow` resizes by 0.03, `Alt+arrow` by 0.08, and the remappable `tmux-resize-*` keys by 0.05; dragging a divider with the mouse is the exception, clamped instead to `[0.06, 0.94]` from the pointer's position within the split. The clamp is what keeps a pane from ever collapsing to nothing or swallowing its sibling.

The five layouts (`space` cycles them) are each a fresh tree built from the current flat list of leaves. `even-h` and `even-v` recursively split the list at its median so the panes tile evenly across a row or a column. `main-v` gives the first leaf 60% of the width and stacks the rest in a right-hand column; `main-h` gives the first leaf 60% of the height and lays the rest in a row beneath — the classic tmux main- pane layouts. `tilled` groups the leaves into rows of `⌈√n⌉` columns. There is a separate grid path, too: `tmux-grid-4`, `tmux-grid-8`, and `tmux-grid-16` (bound to `F/G/I` after the prefix) pad the window with blank panes up to 4, 8, or 16 and build a square-ish tile — a one-key way to summon an even wall of panes.

Directional focus (the arrow keys, `navDir`) is geometric, not tree-order. It reads the rendered percentage rectangle of each pane, takes the active pane's center, and among the panes strictly on the requested side (with a one-unit epsilon) picks the one whose center is nearest by straight-line distance. There is no wrap: if nothing lies to the left, the left arrow does nothing. That is the same “nearest pane in a direction” rule tmux's `select-pane -LRUD` uses, rebuilt from the panes' on-screen geometry.

## The forwarder protocol, and typing into eight terminals

Because each pane is a cross-origin iframe, the overlay cannot listen for keys inside a pane from the top frame — so the pane runs a thin forwarder and the two frames talk by `postMessage`. Every message from a pane up to the top frame carries a `__zbtmux` tag and one of a small set of shapes: `{prefix:1}` (the `Ctrl-b/⌘-b` prefix was pressed inside the pane, so the top frame should arm), `{cmdKey, ctrl, alt}` (the post-prefix command key), `{palette:1}` (`⌘K` inside a pane, so the palette opens in the top frame and drives the active pane rather than opening a stray tab), `{yank:...}` (a copy-mode selection), and `{synckey:...}` (a broadcast keystroke). The top frame replies downward with `{setSync:...}`, `{copyMode:true}`, `{pasteText:...}`, and `{syncapply:...}`. This is why the prefix works while your cursor is inside a framed site: the pane catches it and hands it up.

Synchronize-panes (`e`) is the most careful piece. When sync is on and you are typing in an editable field, the pane forwards each keystroke up as a `synckey` token, and the top frame relays it to every other pane in the sync group. Plain characters, `Enter`, and `Backspace` travel as themselves; the line-editing combos travel as *semantic* tokens — `Ctrl-w` and `Alt-Backspace` both become `C-w` (kill word), `Ctrl-u` and `⌘-Backspace` both become `C-u` (kill to line start). Each receiving pane applies the token cursor-aware against its focused (or last-focused) field: `C-w` walks back over trailing whitespace and then a run of non-whitespace and deletes that span; `C-u` deletes back to the start of the current line (`lastIndexOf('\n')`); a plain character is spliced in at the caret. Every edit dispatches a real `input` event so the framed page's own handlers see the change. That token discipline is what lets one command typed once land correctly in eight different sites' input fields at once, even when their fields disagree about selection state.

## Copy mode, paste buffers, and the chooser

Copy mode (`[`) runs inside the focused pane, because the pane is a real scrollable document. A magenta indicator appears, `j/k` scroll by 64 pixels, `d/u` by half the viewport, `Space/b` by nine-tenths of it, `g/G` jump to the ends, you select text with the mouse, and `y` or `Enter` yanks the selection up to the overlay's paste-buffer stack. The stack is most-recent-first, de-duplicated, and capped at 30 entries; any ordinary copy inside a pane is captured into it too. `]`  pastes the most recent buffer into the focused pane's field, and `=` opens a chooser — a modal listing each buffer's first 90 characters, selectable by click or by digit. The buffers persist with the rest of the session.

## Two kinds of persistence

The overlay has two separate persistence layers, and they answer two different questions. A per-tab `sessionSnapshot` under `zbtmux-session-v1` keeps the live tree, the active window and pane, the open flag, and the paste buffers, so a reload of the host page restores exactly where you were — this is the “survive an accidental refresh” layer, and it migrates an older per-window `sync` boolean to the newer `sync-member-id` array on the way in. The second layer is *named* sessions: `chrome.storage.local` under `zb_tmux_sessions` holds durable, user-named layouts, created with `S`, listed and loaded with `s`, edited in the overlay’s own layouts manager with `M`, and managed in full on the Sessions page of Chapter 14. Loading a named session writes a `zb_tmux_load` trigger the live overlay watches, rebuilds every window’s tree, and re-opens. One layer is a crash-recovery snapshot; the other is a library of layouts you keep.

## A saved window is content plus shape

A saved window used to be its content alone — a name and a flat list of panes with their URLs and titles — and the shape was inferred on load. That inference is `buildEven(list, "row")`: a recursive binary split at the median, entirely in the row direction, which produces even vertical columns and nothing else. So a session saved from a window you had split top-and-bottom came back side-by-side, and there was no representation in which it could have come back otherwise.

Each window therefore now carries a second field, `tree`, holding the tiling *shape* next to `panes[]`, the content:

```
{ name, panes: [{ url, title }, ...], tree }

tree := { t:"split", dir:"row"|"col", ratio, a: tree, b: tree }
        | { t:"leaf" }
```

The two are joined by position, not by id: the tree’s leaves, read depth-first, line up one-to-one with `panes[]` in order — the same order `sessionSnapshot()` writes them in. That is what makes the pair robust in the only way that matters here, because the shape is a *hint* rather than a contract. `reviveTree(snap, list)` counts the leaves in the saved shape first and returns null the moment that count disagrees with the pane list, and every caller falls back to the even-horizontal rebuild. Hand-edit the JSON, import a layout written before shapes existed, add a pane in one editor and open it in another — the count stops matching, the shape is discarded, and you get the old behavior rather than a crash or a mangled tiling. A ratio is sanity-checked the same way: anything outside `(0.02, 0.98)` is replaced with `0.5` on revive, so a corrupt or hand-written ratio cannot produce a pane of zero width.

The structural edits are two functions, and they are the whole reason the shape stays consistent. `shapeSplit(w, pi, dir)` replaces the `pi`-th leaf with a split whose two children are leaves and inserts a slot at `pi + 1` in `panes[]`, so the content and the shape grow together. `shapeClose(w, pi)` removes the `pi`-th leaf — its sibling is promoted into the parent split’s slot, collapsing the split — and splices the same index out of `panes[]`. Both clear the window’s `layout` name, because after a manual split the window is no longer one of the named even/main/tiled layouts. Both are exported through `ZGui.tmux.layout` precisely so the overlay’s built-in editor and any external session manager perform the identical mutation.

## Detach commits the layout

`d` used to mean only “hide the overlay.” It now means what detach means in tmux: the live layout is committed before the surface goes away. `commitLiveSession()` runs first, and it branches on whether the live tree is *attached* to a saved session. If it is — `S.sessId` names one — that session’s `windows` are overwritten with a fresh snapshot and its `updated` stamp advanced, so working inside an attached layout and detaching keeps your changes without a separate save.

If it is not, the layout is a scratch tree with nowhere to go, and the previous behavior simply discarded it on

the next `s`. It is now committed to one reserved entry, id `__auto_last__`, named `last`, which is overwritten on every detach rather than accumulating: a single stable id means repeated detaches leave one entry, not a drawer full of junk. The commit is gated on the tree actually holding something — at least one leaf with a document — so detaching an empty overlay writes nothing. The practical effect is that `d` then `s` gets your working tree back, which is the tmux behavior the key was named after.

## The rest of the command surface

The window and pane operations round out the set, and every one is a tree edit or a small modal. Windows: `c` new, `n/p/Tab` next/previous/last, `,` rename, `.` move (renumber), `'` go-to by prompt, `w` list, `k` kill, and `0-9` jump. Panes: `q` overlays a large number on each pane (type the digit to focus it, auto-dismissed after three seconds), `m` marks the active pane and swaps it with a previously marked one, `!` breaks a pane out into its own window. `r` reloads a pane by re-assigning its `src`; `t` shows a full-screen clock ticking every second; `?` opens a help card built from the same keymap the settings page renders, so the help can never drift from the actual bindings; `d` commits the live layout and detaches (hides) the overlay; `f` searches window names and pane titles, and `i` prints the active window's own record. The overlay writes a small `zb_tmux` record to storage on every change so the real powerline statusbar — a separate content script in the top frame — can show the window/pane counts and light the prefix indicator without the overlay drawing its own bar.

## The pane sandbox and the z-index ladder

Each pane iframe is deliberately sandboxed, and the exact token set is the interesting part. The overlay sets `sandbox="allow-scripts allow-same-origin allow-forms allow-popups allow-popups-to-escape-sandbox allow-modals allow-downloads allow-top-navigation-by-user-activation allow-storage-access-by-user-activation allow-presentation"` — a broad grant so framed sites behave like real tabs, with one pointed omission: it does not grant automatic top-navigation, only the user-activation form. That omission is what defeats JavaScript frame-busting scripts that try to break out by reassigning `top.location` on load; combined with the `frame_bust` header-strip of the earlier section, it is what lets the overlay hold an arbitrary site in a pane without the site either refusing to be framed or escaping the frame. Panes also get `allow='clipboard-read; clipboard-write; fullscreen'` so copy, paste, and fullscreen work inside them.

The overlay's stacking is a fixed z-index ladder, tuned so the right thing always draws on top: the whole overlay sits at 2147483640, a pane's click-cover at 2, the drag dividers at 6, the pane-number badges at 40, a pane's autocomplete at 45, the drag shield at 50, and modals at 60; the copy-mode indicator inside a pane climbs to the maximum 2147483647 so it is never hidden by page content. Every surface is themed from the active scheme by injecting the scheme's variables into an `#zbtmux` CSS prefix — so the tabs, pane borders, dividers, modals, and the marked-pane accent all recolor live when you switch schemes, with the marked pane drawing an accent border and a `◆` and a synced pane a cyan border. It is a small windowing toolkit's worth of CSS, all keyed to the same twenty variables the rest of the HUD uses.

## Now a shared component: `ZGui.tmux`

The overlay described above began as `ztmux.js`, a bespoke content script living only in `hud-internal`. It has since been promoted into the shared `zgui-core` toolkit as `ZGui.tmux` — the same tiling window manager, now the common one the house Tauri apps consume too — and the HUD consumes it rather than carrying its own copy. The model, the command set, the copy-mode and synchronize-panes behavior, and the persistence are all unchanged; what moved is *where the code lives*. In the manifest the overlay is now two thin scripts around the shared WM: the top frame loads `ZGui.tmux` and its dependencies (the shared split-pane, modal, toast, and button-bar widgets) plus `ztmux-config.js`, the provider that calls `ZGui.tmux.init` with iframe panes (an address bar, `openEmptyPane/renderPane`, and `applyKey/copyMode/paste` bridged over

`postMessage`) and reads prefs from `chrome.storage`; and every frame loads `ztmux-pane.js`, the pane-side forwarder extracted from the old script, which relays the prefix, the synced keystrokes, and yanks up to the top frame and runs copy-mode locally over the pane's page. `zgui-core` v1.0.18 added the external-feed API (`prefix/key/relaySync/yank`) precisely so a cross-origin-iframe consumer like this one can feed the shared WM from outside. The promotion has since been completed: `ztmux.js` and `zstatus.js`, the two superseded originals, were deleted from the tree rather than left as reference copies, so there is exactly one implementation of the window manager and one of the statusbar, both in `zgui-core`, and the extension carries only the provider and the pane forwarder.

The API the toolkit exposes has grown past the external-feed entry points, and the additions are what let a *second* surface work on the same layouts. `ZGui.tmux` now publishes `editSessions()` and `editKeymap()` (so a `⌘K` command opens the same editors the `M` and settings paths do), `loadSession(id)` (so an external driver can attach a whole layout into a fresh tab without a keypress — this is the path the HUD Sessions page uses, relaying through the background worker's `tabs.sendMessage`), and a `layout` namespace holding the four pure operations on a *saved* window: `rects(panes, tree, w, h)` returns the geometry `Load` will tile, `dirs(win)` returns the split direction each pane sits in, and `split(win, i, dir)/close(win, i)` mutate the window's shape in place. That namespace exists for exactly one reason: a host that ships its own session manager should draw and edit the same geometry the overlay tiles, rather than reimplementing it against the flat pane list. The next section is what happens when it does not.

# Chapter 11 – The Powerline Statusbar

`zstatus.js` pins a tmux/powerline statusbar to the bottom of every top-level page. The left side is the HUD's own indicators; the right side is a live system monitor fed by the native host. This chapter is its segments, its powerline geometry, and the stream behind the numbers.

## Left and right

The bar sits at `position:fixed; bottom:0; height:22px`, top frame only, themed by the active scheme, in the Share Tech Mono face. Reading left to right:

- **Left:** a `ZW` badge, the `C-b` prefix indicator (lit while a tmux prefix is armed), the current scheme name, the `VIM` status, and `⌘K`.
- **Right:** the machine, streamed from the native host into `chrome.storage`'s `zb_sys` key — CPU, MEM, SWAP, DISK, IO, NET, LOAD, UP (uptime), a temperature, BATT, LAN IP, WAN IP, host name, and a clock.

Each right-side segment is a `<span>` with a `data-*` hook the refresh function writes into: `data-cpu`, `data-mem`, `data-swap`, `data-disk`, `data-io`, `data-net`, `data-load`, `data-up`, `data-temp`, `data-batt`, `data-lip` (LAN), `data-host`, `data-clock`. The refresh formats each from the `zb_sys` object — CPU as a percentage, memory and swap as used/total, disk as a percent, IO and NET as per-second rates with `↑/↓` and `R/W`, battery with a charging-vs-on-battery glyph and a percentage, uptime humanized, and the clock ticking every second.

## Powerline chevrons

The bar draws true powerline chevrons, not glyph characters, using CSS borders. Each block is a `.plr` with a triangular `:before` pseudo-element formed from transparent top/bottom borders and a solid side border, so the segments interlock as arrows. Left-of-the-spacer segments point right (`►`) and right-of-the-spacer segments point left (`◄`), and consecutive blocks alternate between two background shades (`s0/s1`) so the chevrons read as distinct steps. This is exactly how a tmux powerline theme is built in the terminal, rebuilt here in CSS so the browser statusbar looks like the shell statusbar the rest of the stack uses.

## The stream behind the numbers

The numbers are real because `zwire` is a desktop app with a native host that can read the machine. `zstatus.js` does not sample anything itself; it renders whatever is in `zb_sys` and re-renders when that key changes. The native host fills it: Chapter 15 covers `zwire-host`'s `sysinfo_start` mode, which samples the machine through the `sysinfo` crate on a background thread and streams the result, which the background worker writes into `zb_sys`. So the statusbar is a genuine system monitor — CPU load, memory pressure, network throughput, battery, temperature — pinned to the bottom of every web page, not a decorative mock. The bar is toggled from the `⌘K` palette, and (like everything HUD) recolors instantly with the scheme.

## Every segment, and how it is formatted

Each right-hand segment reads one field of the `zb_sys` object the worker writes and formats it with a small, purpose-built formatter, so the numbers read like a tmux theme rather than raw JSON. CPU is a whole-number percent; MEM and SWAP are used-over-total run through a byte-humanizer that steps `B/K/M/G/T` in powers of 1024 and shows one decimal below 100; DISK is a percent; IO is `R<rate> W<rate>` and NET is `↑<rate> ↓<rate>`, both per-second and both humanized the same way; LOAD is the three load averages joined by spaces; UP humanizes uptime as `Xd Yh Zm`; the temperature is `<n>°C` and, when the machine reports no sensor, that segment hides itself entirely rather than showing a dash; BATT prefixes a charging glyph (a bolt ⚡ when plugged, a battery glyph on battery) before the percent; LAN and WAN are the private and public IPs; HOST is the short hostname; and the clock, updated on its own one-second interval independent of the two-second stats stream, shows `MM-DD HH:MM:SS`. Every segment writes into a `data-*` span (`data-cpu`, `data-mem`, and so on down to `data-clock`), so a stats refresh rewrites text in place without rebuilding the bar.

The left side is the HUD's own state, not the machine: a `ZW` badge, the `C-b` prefix indicator, the current scheme name (filled from the scheme table's label), a per-window tmux readout (`i·paneCount`, the active window emphasized, a `⇌` when any window is syncing), a static `VIM` marker, and a clickable `⌘K` that opens the palette. A flexible spacer between the two halves is what splits the bar into a left group and a right group.

## Powerline geometry in CSS

The chevrons are real triangles, not glyphs. The 22-pixel bar height sets up an 11-pixel transparent top and bottom border, and each segment carries a pseudo- element with a solid side border that draws the arrow: left-of-spacer blocks put a `border-left` on their `::after` so they point right, right-of-spacer blocks put a `border-right` on their `::before` so they point left, and consecutive blocks alternate between two background shades (`--bg-card` and `--bg-secondary`) so the interlocking arrows read as distinct steps. The layout function finds the spacer, walks left assigning the point-right class with an alternating shade index, walks right assigning the point-left class, and skips the badge, spacer, and prefix block. This is exactly how a tmux powerline theme is assembled in a terminal, rebuilt in CSS so the browser's bottom line matches the shell's.

The `C-b` prefix block does one extra thing: it lights up. When you press the tmux prefix, the block's background flips from dim to the scheme accent for 1.6 seconds and then fades back — a visible confirmation that the prefix armed. Because the statusbar lives only in the top frame while the prefix can be pressed inside a framed pane, the block listens two ways: directly for `Ctrl-b` keydowns in the top frame, and for the overlay's `zb_tmux.armed` storage write, which reaches it reliably across frames. The bar itself is `position:fixed` at the bottom, 22 pixels tall, top-frame only, with a blurred translucent background, and it re-themes from `zb_scheme` and re-renders from `zb_sys/zb_tmux` on every relevant storage change.

## Now a shared component: `ZGui.powerline`

Like the tmux overlay, the statusbar has been promoted out of the HUD and into `zgui-core`. The bespoke `zstatus.js` is retired; the bar is now `ZGui.powerline` (shipped in `zgui-core` v1.0.17 as `powerline.js/powerline.css`) — the exact same powerline bar the house Tauri apps (`zoffice`, `zphoto`, `zpdf`) render, so the browser's bottom line and the apps' now come from one implementation. What `hud-internal` ships is a thin adapter, `zpowerline.js`, that feeds `ZGui.powerline.init` from `chrome.storage:zb_sys` becomes the system-stats segments, `zb_tmux` the window/pane segment and the armed prefix lamp, `zb_scheme/zb_ui` the scheme (injected onto `:root` so the shared bar inherits the live theme), and `zb_status` the show/hide toggle. The manifest's top-frame statusbar content script now loads `powerline.js` plus `zpowerline.js` (and `powerline.css`) in place of `zstatus.js`. Every segment, formatter, chevron, and behavior described above is preserved — it is the same bar, sourced once and shared — and `zstatus.js`, the superseded original, was deleted rather than kept as a reference copy.

## The sampling contract, and what the numbers do not mean

A statusbar that shows a number implies a contract about what the number describes, and the bar's two halves make different promises. The left half is HUD state — the armed prefix, the scheme name, the pane readout — and it is exact by construction, because the thing it describes is a value the browser itself holds. The right half is a *measurement* of a machine, and measurements have sampling semantics that a two-character-wide segment cannot convey. Reading the bar correctly means knowing three things it does not say.

The first is that nothing on the right side is sampled by the page. The bar renders whatever is in the stats key and re-renders when that key changes, and the sampling happens in the host on its own thread, at the stream's cadence. There is therefore a number on screen that is always at least one sample old, and possibly much older if the stream has stalled — because a stalled stream produces no storage change, and no storage change means no re-render, which means the bar keeps displaying the last value it received with no visual indication that it has gone stale. That is the correct behavior for a bar with no room for a staleness indicator, and it is worth internalizing when diagnosing: blank segments mean the stream never started, while *frozen* segments mean it started and stopped. Chapter 36 treats the first case; the second looks identical to a working bar on an idle machine.

The second is that the clock and the stats are on different clocks in the literal sense. The clock segment updates on its own one-second interval, independent of the two-second stats stream, so the two halves of the right side are never in step: the time advances twice for every stats refresh. This is deliberate and it is the right call — a clock that only moved when the machine reported its CPU would look broken — but it means the timestamp beside a CPU reading is not that reading's timestamp. If you are correlating a spike on the bar against something else, the clock is not the sample time.

The third concerns the rate segments specifically. Chapter 26 records that network and disk throughput are computed as deltas over the elapsed interval, so the host does no work to maintain a counter between ticks and simply subtracts two readings when a tick fires. A delta over an interval is meaningless without a previous reading, which means the first sample after the stream starts — and the first after any resume — has nothing to subtract from. Whatever such a segment displays at that instant is an artifact of the implementation's choice for the undefined case, not a measurement, and it settles within one interval. The practical rule is that the first rate you see after opening a browser is not a rate.

Two smaller pieces of the contract are worth stating because they are the kind of thing a reader assumes wrongly. The temperature segment hides itself entirely when the machine reports no sensor rather than showing a dash, which is a policy decision with a real consequence for layout: the bar's segment count is machine-dependent, so a screenshot from one machine will not have the same chevron sequence as another. And the memory and swap segments show used over total through a byte humanizer stepping in powers of 1024, which is the terminal convention rather than the storage-vendor one — the same reading a shell tool would print, deliberately, because the whole point of the bar is that it looks like the tmux status line the rest of the stack wears.

The through-line is that the bar is a *monitor*, not a dashboard. A dashboard's job is to be authoritative and it earns that with timestamps, units, and staleness indicators, none of which fit in a 22-pixel strip. A monitor's job is to make a change noticeable in peripheral vision, and for that a value that is one sample old and unlabeled is completely sufficient. Knowing which of the two you are reading is what keeps the bar useful rather than misleading.

# Chapter 12 — The Command Palette

The `⌘K` command palette is the HUD's universal launcher: one fuzzy-filtered list that jumps to any internal or native page, switches scheme, runs a keyword search, and surfaces your tabs and frecent history. It exists in **four** implementations, one per execution context, all sharing the same `zgui-core` engine and a single backend-agnostic item source (`palette-cmds.js`, vendored verbatim into each extension so the four can't drift):

- `zpalette.js` — the content-script palette on ordinary `http(s)/chrome://` pages.
- `pages/zg-boot.js` — the palette on the HUD's own extension pages (Settings, Sessions, Host console, and the rest). A web page gets the content-script palette; a HUD page gets this one — both are `hud-internal`, which is why it reads as one but is two.
- `newtab/palette.js` — the New Tab palette (a separate extension).
- `zpwchrome/lib/zpc-palette.js` — the palette on the `zpwchrome` power-tool's own dashboard pages (a second separate extension).

Every surface lists the same rows, including deep-links into `zpwchrome`'s tool pages (Downloads, Pass, Userscripts, Host console, ...). `hud-internal` owns `⌘K` browser-wide as a `chrome.commands` shortcut — a page keydown can never intercept it — and its worker routes each press to the palette matching the active tab. This chapter is what it lists and how it opens.

## What the palette lists

Opened with `⌘K` (or `:` on a page, or `o` in vim mode), the palette registers several sources into one fuzzy list:

- **Internal pages** — dozens of `chrome://` destinations, from the everyday (extensions, settings, history, downloads, bookmarks, flags, gpu) to the deep diagnostics tail (`net-internals`, `serviceworker-internals`, `indexeddb-internals`, `webrtc-internals`, `tracing`, `histograms`, and many more), each opening in a new tab.
- **Settings sub-pages** — the “tabs” inside `chrome://settings` (Appearance, Privacy & security, Performance, Search engine, and the rest), so you can jump straight to one.
- **Schemes** — one entry per color scheme, which sets it through the native bridge.
- **Open tabs** — every tab, with its title and URL, switching to it (and focusing its window) on select.
- **Frecent history** — your history scored by visit count and recency over the last 90 days, top matches first.
- **zwire destinations** — the CI-runs page, the app store, the Chrome Web Store.

The engine is `zgui-core`'s `ZGui.palette` over `ZGui.fzf`, so typing filters fuzzily and the matched characters are highlighted. The styling is inlined into the content script (the `.palette-*` CSS) so it works on arbitrary pages, themed by the active scheme's variables.

## Keyword commands (cmd-defaults.js)

Beyond jumping to pages, the palette runs *keyword commands* — a short keyword plus an argument that expands into a URL. The background worker seeds a default set on first run only (`cmd-defaults.js` and the `CUSTOM_CMD_SEED` in `background.js`), stored in a user-editable `zb_custom_cmds` array so your edits and deletions are never clobbered. The seeds are the everyday web tools:

```
cg ChatGPT      cl Claude      pp Perplexity    wa WolframAlpha
gm Gmail        gd Google Drive cal Google Calendar
tr Translate    img Google Images def Define word
hn Hacker News  li LinkedIn    ...
```

Typing `cg how do futexes work` in the palette expands the `{q}` in the ChatGPT template with your query and opens it. Each rule is `{id, icon, label, keyword, type, value}`, where `type` is `url` (a template) or `action`, and the whole set is edited on the HUD Commands page (Chapter 14). This turns the palette into a custom search-engine bar with your own keywords, seeded sensibly and yours to change.

## How it opens across surfaces

Opening the palette is the same `⌘K`-reservation dance as focus on the new tab. On an ordinary page, `zpalette.js` catches `⌘K/Ctrl+K` (and `:` when not in a field) directly in a capture-phase listener. On the new tab, Chrome reserves `⌘K` at the browser level, so the `hud-internal` background worker owns it as a `chrome.commands` shortcut and messages the new-tab page to open its own palette. And from *inside* a `tmux` pane, `⌘K` is forwarded up to the top frame so the palette opens there and navigates the pane rather than opening a stray new tab. Three surfaces, three plumbing paths, one palette — because `⌘K` is guarded differently on each and the HUD routes around each guard.

## The full source catalog

The list the palette assembles is larger than the summary above suggests. The static half is three tables in `zpalette.js`: a `PAGES` set of the HUD's own replacement pages (Extensions, Settings, History, Bookmarks, CI runs, Shortcuts, Extension shortcuts, Custom commands, System info); a `CHROME` set of roughly forty-four `chrome://` destinations that runs from the everyday (downloads, flags, gpu) down the entire `-internals` diagnostics tail (`net-internals`, `serviceworker-internals`, `indexeddb-internals`, `quota-internals`, `webrtc-internals`, `media-internals`, `histograms`, `tracing`, `ukm`, `predictors`, `memory-internals`, and more), ending at a “chrome://about” entry that lists them all; and a `SETTINGS` set of the nineteen tabs inside `chrome://settings` (Appearance, Privacy, Security, Cookies, Performance, Search engine, and the rest). On top of the static tables the palette registers one entry per scheme, your open tabs (read from `zb_tabs`), your frecent history (from `zb_frecent`), your extensions (each offering a *Tweak* and a *Manage* action), and two external destinations — the Chrome Web Store and the zwire app store. Every page or tab entry routes through the storage command bus so the sleeping worker actually performs the navigation.

## Keyword commands and the search providers

Beyond jumping to a destination, the palette runs *keyword commands*. Two provider functions feed the fuzzy list live as you type. The first is a built-in search table: type `crate serde` and it expands crates.io's search URL with `serde`; a bare keyword opens that site's origin. The second is your own `zb_custom_cmds`, seeded once by the worker and edited on the Commands page, and it is richer than a URL opener — a custom command's `type` may be `url` (substitute the typed argument for `{q}` and open it), `shell` (run the argument in the HUD terminal), `js` (evaluate the argument as the body of a function with the typed text bound to `q`), `action` (a built-in browser action), or `scheme` (switch the color scheme). A generic Google/DuckDuckGo fallback is always registered last, marked so the ranking keeps it beneath every real match. So the same input box is a launcher, a custom search-engine bar, a shell one-liner runner, and a tiny JavaScript scratchpad, disambiguated entirely by the leading keyword. The scheme entries and the `cycle color scheme` action both go through the worker's native bridge, so a scheme picked from the palette lands in the same `~/zwire/hud-scheme` file every other surface reads.

## The provider tail: shortcuts, extensions, and a Chrome cheatsheet

Three more providers round out what the palette can surface, each drawing on a storage mirror the worker keeps fresh. A shortcut provider reads `zb_shortcuts` — every extension’s keyboard commands, published by the worker via `developerPrivate` — so you can find a binding by name. An extension provider reads `zb_exts` and, per extension, offers a *Tweak* entry (opening `chrome://extensions/?id=...`) and a *Manage* entry, so the palette doubles as an extension launcher. And a platform-aware Chrome-shortcut provider carries a curated cheatsheet of the browser’s own key bindings, keyed on whether the machine is a Mac, so typing part of a native action surfaces its real chord. None of these are static text: they are the worker’s live `zb_*` mirrors turned into palette rows, which is why the palette is never stale — it is a view over the same shared storage the statusbar, the settings pages, and the new tab all read.

# Chapter 13 – Fuzzy Find

`zfind.js` hijacks `Ctrl/⌘+F` on every page and replaces Chrome's native find with the HUD's own fuzzy find bar: substring find-in-page with all occurrences navigable, an `zfz` fallback when there is no substring hit, and highlighting that never mutates the page's DOM. This chapter is that one script.

## Substring first, fuzzy second

The find bar is a small fixed-position input themed by the active scheme. As you type, it first does a plain substring find-in-page across all text nodes, collects every occurrence as a navigable match, and lets you cycle them (with a count, like Chrome's own “3/12”). When there is *no* substring hit, it falls back to the same `zfz` fuzzy matcher the palette and the sibling apps use (`ZGui.fzf`), so a misremembered or scattered query still finds the closest text. Substring-first is the right default — exact matches are what you usually want — with fuzzy as the safety net rather than the primary mode.

## The CSS Custom Highlight API

The interesting engineering choice is how it highlights. Instead of wrapping matches in `<mark>` elements — which mutates the page's DOM, can break the page's own scripts and layout, and is expensive to undo — `zfind.js` uses the CSS Custom Highlight API: it builds `Range` objects for each match and registers them as named highlights that the browser paints via a `::highlight()` pseudo-element, with zero DOM mutation. The page's DOM is never touched; the highlights are a paint-layer overlay the browser manages. The script guards for the API (`'highlights' in CSS`) and no-ops where it is absent, but on the Chromium base it targets, it is present. This is the modern, correct way to highlight find matches, and it is why the HUD find bar can run on complex web apps without corrupting them the way a `<mark>`-wrapping finder would.

## Two highlight layers, and the two ranges

The bar keeps two named highlights, not one. All matches are painted underlined in the scheme cyan under the name `zfind`, and the *current* match is painted in the accent color with a translucent background under a second name, `zfind-cur`, which is deleted and rebuilt every time you cycle. Cycling wraps (`Enter` forward, `Shift+Enter` back) and scrolls the current match to the center of the viewport; the counter reads `current/total` exactly like Chrome's own find. Because a `::highlight()` rule cannot read a `var()` off the page's own elements, the script bakes concrete scheme colors into the highlight CSS rather than referencing the HUD variables — a small compromise the code comments call out.

Two implementation choices are worth naming. The match walk uses a `TreeWalker` over text nodes, skipping `SCRIPT/STYLE/NOSCRIPT/TEXTAREA` parents, hidden nodes, and the find bar's own DOM, and it caps at 6000 nodes and 2000 matches so a pathological page cannot hang the search; the fuzzy fallback additionally skips any single text node longer than 300 characters. And the bar's `position:fixed` CSS is injected *synchronously* before the input is focused — the comment explains that focusing an in-flow element would scroll the page down to it, which is the exact “Cmd+F scrolls the page” bug a naïve find bar exhibits. There is no debounce: the search runs on every keystroke, which is affordable precisely because the highlight layer never touches the DOM.

## Why replace the native find at all

Replacing `Ctrl/⌘+F` is a larger claim than it first appears, because Chrome's find bar is not a page-level feature — it is browser chrome, implemented in C++, with behaviors no page script gets for free: it searches across the frame tree, it drives the browser's own match highlighting in the scrollbar, and it survives navigation within the bar's own lifetime. Overriding it means giving those up, and it is worth being explicit about what is bought in exchange, because “our find bar is themed” would not justify the trade.

Three things are bought. The first is the fuzzy fallback: the native find is a substring matcher and nothing else, so a query you half-remember returns nothing, and the correct recovery — try a shorter fragment — is a manual binary search. The HUD bar tries substring first, which is the right default because exact matches are what you usually want, and falls back to the same scored matcher the palette uses when the substring finds nothing. That is a strictly additive capability: everything the native bar would have found is still found, in the same order, and the cases it would have failed now return the closest text instead of an empty count.

The second is theme and keyboard continuity. The bar is a HUD surface styled from the active scheme, and its editing behaves like every other HUD input because the shared toolkit installs readline-style line editing globally on text fields. A user who has `Ctrl-W` and `Ctrl-U` in their fingers gets them in the find bar, which the native bar does not offer.

The third is that the bar is *code you can extend*, which the browser's own find bar is not. The count, the cycling, the highlight styling, and the match-collection policy are all in one script, so a change to any of them is a change to a file rather than a patch to Chromium. On the runtime path that distinction is the whole point of the project.

What is given up is real and the book should say so: the HUD bar searches the top document, so text inside a cross-origin iframe on the page is outside its reach in a way the native bar's frame-tree search is not, and it does not paint the scrollbar tick marks the native bar shows. Those are the honest costs of moving the feature from the browser's C++ into a content script.

## The walk, its caps, and where it gives up

The matching side is a `TreeWalker` over text nodes, and the interesting engineering is in what it refuses to look at. Parents that are `SCRIPT`, `STYLE`, `NOSCRIPT`, or `TEXTAREA` are skipped, because their text is not text the user is reading; hidden nodes are skipped for the same reason; and the bar's own DOM is skipped so a query never matches the query box. Past those, two hard caps bound the work: at most 6000 nodes walked and at most 2000 matches collected, with the fuzzy fallback additionally skipping any single text node longer than 300 characters.

Those caps are the difference between a find bar and a hang. A modern web application can hold tens of thousands of text nodes, and a search that runs on every keystroke with no debounce — which this one does — would be visibly janky long before it was correct if it walked all of them. The caps are chosen so the common case is unaffected (a document with fewer than 6000 text nodes is fully searched) and the pathological case degrades to a partial result rather than a frozen tab. The honest consequence is that on a very large document the match count is a floor rather than a total, and the fuzzy fallback in particular will not consider the longest paragraphs at all — which is the right trade, since a 300-character-plus node is prose, and fuzzy matching prose returns noise.

The absence of a debounce is a deliberate consequence of the highlight design rather than an oversight. Searching on every keystroke is affordable *because* the highlight layer never touches the DOM: registering a set of ranges as a named highlight costs a paint, not a reflow, and discarding the previous set costs nothing at all. A find bar that wrapped matches in elements would have to insert and remove thousands of nodes per keystroke, which is precisely why such implementations debounce — and why they corrupt pages whose own scripts observe the DOM. The two choices are linked: the highlight API is what makes the no-debounce search viable, and the no-debounce search is what makes the bar feel like it is filtering rather than querying.

## Two highlights, one cycle, and the color compromise

The two named highlights are worth one more look because the split is what makes cycling cheap. All matches live in one named highlight painted underlined in the scheme's cyan; the current match lives in a second named highlight painted in the accent with a translucent background. Cycling forward or backward deletes and rebuilds only the second one — a single range — while the first stays exactly as it was. If both were one highlight, every press of `Enter` would rebuild the whole match set to change which member was emphasized, which is the difference between constant work per cycle and work proportional to the match count.

The compromise the code comments call out is the color. A `::highlight()` rule is not matched against the page's own elements, so it cannot read a `var()` off them, which means the highlight CSS cannot reference the HUD's scheme variables the way every other surface does. The script bakes concrete colors into the highlight rules instead. This is the one place in the runtime HUD where a scheme value is duplicated rather than referenced, and the reason it is acceptable is bounded: it affects two rules, both inside one script, both re-emitted when the scheme changes. It is worth flagging precisely because the rest of the HUD is so consistent about painting roles rather than pixels — this is the exception, and it exists because the platform's highlight pseudo-element does not participate in the cascade the way an element does.

# Chapter 14 – The HUD Internal Pages

Several of Chrome's internal pages are not *skinned* but *replaced*: a `document_start` redirect (`redirect.js`) sends `chrome://extensions`, `chrome://settings`, `chrome://history`, `chrome://bookmarks`, and `chrome://version` to purpose-built HUD pages under the extension's `pages/` directory, which reimplement the surface using the extension's privileged permissions. This chapter is the tour.

## Why replace rather than skin

The `chrome://` pages that get replaced are the ones you interact with, not just read — the extension manager, settings, history, bookmarks. Skinning them (the `theme.js` token map) recolors them but leaves Chrome's own UI and its quirks; replacing them lets the HUD present the *data* in its own cyberpunk components with its own fuzzy filtering and keyboard model. The replacements are real, backed by the manifest's permissions: `management` and `developerPrivate` for the extensions page, `history` for history, `bookmarks` for bookmarks, `downloads` for downloads, `sessions` and `system.display` where needed. They are served from the extension, so they are web-accessible pages the redirect can navigate to.

## The pages

From the `pages/` directory, the replacements and HUD-native pages are:

|                                |                                                                                                                                                        |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>extensions.html</code>   | the extension manager (enable/disable/reload/remove), HUD-styled, via <code>management + developerPrivate</code>                                       |
| <code>settings.html</code>     | a HUD settings surface (scheme picker, visual-effect toggles, light mode) writing the <code>hud-ui.json</code> prefs                                   |
| <code>history.html</code>      | history, fuzzy-filterable, via the history permission                                                                                                  |
| <code>bookmarks.html</code>    | bookmarks, via the bookmarks permission                                                                                                                |
| <code>downloads.html</code>    | downloads, via the downloads permission                                                                                                                |
| <code>readinglist.html</code>  | the reading list, via the readinglist permission                                                                                                       |
| <code>version.html</code>      | a HUD <code>chrome://version</code> replacement                                                                                                        |
| <code>keys.html</code>         | the keybinding editor – renders <code>ZWIRE_KEYMAP</code> and writes remaps to <code>zb_keys</code> (Chapter 9), same registry <code>zvim</code> reads |
| <code>commands.html</code>     | the <code>⌘K</code> keyword-command editor – edits <code>zb_custom_cmds</code> (Chapter 12)                                                            |
| <code>extshortcuts.html</code> | extension keyboard-shortcut management                                                                                                                 |
| <code>sessions.html</code>     | the tmux session manager (Chapter 10)                                                                                                                  |
| <code>ci.html</code>           | a zwire CI-runs surface (linked from the palette)                                                                                                      |
| <code>terminal.html</code>     | the standalone terminal page (Chapter 16)                                                                                                              |
| <code>host.html</code>         | the zwire-host console (Chapters 8 and 15)                                                                                                             |
| <code>dashboard.html</code>    | one searchable tile grid launching every HUD page and every <code>chrome://</code> internal page (Chapter 53)                                          |
| <code>hooks.html</code>        | stryke scripts bound to browser lifecycle events (Chapter 49)                                                                                          |
| <code>triggers.html</code>     | regex-on-page-output triggers (Chapter 50)                                                                                                             |
| <code>pipes.html</code>        | reactive dataflow edges between tiled panes (Chapter 51)                                                                                               |
| <code>audio.html</code>        | the browser-wide audio engine's dashboard (Chapter 52)                                                                                                 |
| <code>notes.html</code>        | a folder tree of Markdown notes (Chapter 53)                                                                                                           |
| <code>feeds.html</code>        | an RSS/Atom reader (Chapter 53)                                                                                                                        |
| <code>translate.html</code>    | a translation panel (Chapter 53)                                                                                                                       |
| <code>store.html</code>        | the in-browser product catalog + first-run welcome (Chapter 53)                                                                                        |

Two of these are the HUD's own configuration surfaces and close loops opened earlier in the book: `keys.html` renders the shared keybinding registry and writes remaps into `zb_keys`, the exact key the vim and tmux scripts read, so rebinding a key in the page changes what fires on every page; and `commands.html` edits the `zb_custom_cmds` array the palette's keyword commands come from. The pages share a set of HUD stylesheets (`hud-pages.css`, `ext-page.css`) and helper scripts (`zg-boot.js`, `zg-list.js`, `fzf.js`), so they are one consistent surface, not a dozen snowflakes. `redirect.js` runs at `document_start` so the swap happens before Chrome's own page paints, and the redirected set is exactly the set excluded from the `theme.js` skin so the two never collide.

## What each replacement actually does

These are not skins with a `chrome://` page underneath; each is a working reimplementaion on a privileged Chrome API, and the API is what makes the page more than cosmetic:

- **Extensions** (`extensions.js`, the largest page at 423 lines) is a full extension manager on `chrome.developerPrivate` plus `chrome.management`. It lists every extension including disabled and terminated ones, and it can enable and disable, uninstall (with the confirm dialog), reload an unpacked extension, load a new unpacked directory, pack a directory, check for updates, toggle developer mode, pin to the toolbar, set per-extension site access (`ON_CLICK`, `ON_SPECIFIC_SITES`, `ON_ALL_SITES`), flip the incognito/file-access/error- collection toggles, read an extension's on-disk size, and open its service- worker devtools. A `#shortcuts` sub-view edits every extension's keyboard commands in place, suspending Chrome's own shortcut handling while it captures the chord. One detail in the card grid is worth the sentence because it is the kind of thing that reads as a rendering bug: `developerPrivate` hands back an `chrome://extension-icon/<id>/<px>/<match>` URL sized for a *list* row, and the card tile paints it at `4rem` — 128 device pixels on a HiDPI display — so the icons arrived as upscaled blur. `ExtensionIconSource` serves whatever size the path asks for, so the page rewrites the size segment to 128 rather than upscaling, and passes the result as the card's `icon` (a fixed-size mark) rather than its `thumb` (a full-bleed screenshot). Any other URL shape — a `data:` URI, an app's own icon — falls

through the rewrite unchanged.

- **Settings** (`settings.js`) is built on `chrome.settingsPrivate`, so it renders every Chrome preference the API exposes — grouped by the first dotted segment of the key, each mapped to the right control (a toggle for booleans, a select where the pref advertises allowed values, a number or text field otherwise, a read-only JSON textarea for list and dictionary prefs) and disabled when the pref is policy- controlled. On top of the raw prefs sit the HUD's own Appearance card (the eight- scheme picker and a light-mode toggle) and an Effects card (scanlines, vignette, glow, animations), whose values are pushed to the native `hud-ui.json` file so the separate new-tab extension can follow them.
- **History**, **Bookmarks**, and **Downloads** are thin configurations of a shared filtered-list component (`zg-list.js`) over `chrome.history.search`, `chrome.bookmarks.getTree` (flattened to a folder-path table), and `chrome.downloads.search` respectively, each fuzzy-filterable and each wiring the matching `on*` events for live updates. Downloads opens a completed file on click through `chrome.downloads.open`; the HUD navigation actually points its Downloads entry at native `chrome://downloads`, because the power-tool owns the download manager, which leaves this page a working-but-unlinked alternative.
- **System** (`version.js`) replaces `chrome://version` with a HUD panel that shows the zwire edition, the HUD extension's manifest version, the Chromium version parsed from the user agent, the install and profile paths, and a *live* native- host probe — it sends `{cmd: 'get'}` to `com.zwire.hud` and reports connected with the current scheme and active effects, or “not connected” with the exact error and the setup command to run.

## The two configuration surfaces, and two the book had not named

Two replacement pages are the HUD's own control panels and close loops opened earlier: **Keyboard** (`keys.js`) renders the shared `ZWIRE_KEYMAP` registry as editable chips, writes remaps into `zb_keys` (the exact key vim and tmux read), warns when a new binding collides with an existing one, and also edits the tmux prefix chord and arm-timeout the tmux overlay obeys; **Commands** (`commands.js`) is the CRUD editor for `zb_custom_cmds`, enforcing unique keywords, supporting all five command types (url/shell/js/action/scheme), and — a nice touch — remembering a deleted default in a `zb_cmds_removed` tombstone list so the worker's seeder never re-adds it.

Two more pages exist that the earlier map did not call out. **Sessions** (`sessions.js`) is a durable manager for the tmux overlay's named layouts: it reads and writes `zb_tmux_sessions`, lets you create, duplicate, rename, and delete sessions and edit their windows and panes, renders a small SVG preview of each layout, exports and imports the whole set as JSON, and “attaches” a session by writing the `zb_tmux_load` trigger the live overlay watches.

That preview used to be a hand-written mirror of the overlay's rebuild — a local `tileRects()` reimplementing `buildEven(list, 'row')` against the flat pane list, with a comment instructing the reader to keep it in lockstep — and it was wrong in the way duplicated geometry is always eventually wrong: the page knew only about `panes[]`, so a window saved with a top-and-bottom split previewed, and reloaded, as side-by-side columns. The page now loads `lib/zgui-core/webui/tmux.js` directly and calls `ZGui.tmux.layout` — `rects()` for the preview, `dirs()` for the per-pane direction glyph in the editor rows, `split()` and `close()` for the structural edits behind the new `split → / split ↓` buttons — so the preview is drawn by the same code that tiles the real thing rather than by a copy of it. The `×` button disables itself at one pane rather than silently refusing, `+ pane` is now a row-split of the last pane, imported layouts carry their `tree` through when the file has one, and the whole page keeps working if `tmux.js` ever fails to load: a `TL()` accessor returns null and each call site falls back to the flat even-horizontal behavior it had before. **CI** (`ci.js`) is a full GitHub Actions dashboard — the interactive front end to the worker's background poller of Chapter 8. It reads a `zb_ci` config (GitHub user, optional token, repo limit), fetches each repository's recent workflow runs through the GitHub REST API with the modern `X-GitHub-API-Version` header, renders them as per-repo accordions with status pills and relative timestamps (repos with a failing or in-flight latest run open by default), and handles rate limiting by reading the `x-ratelimit-reset` header and backing off until the window clears. A token

raises the repo budget and switches the repo query from the public account to your full affiliations.

## One shell under all of them

Every page mounts through `zg-boot.js` (`window.ZBHUD`), which is why they read as one surface rather than a dozen. It builds the sticky HUD header and the cross-page navigation row, applies the persisted CRT and effect preferences, and installs the same `%K` command palette the content scripts carry — registered here with the page's own extra commands plus the shared HUD toggles, nav destinations, custom commands, frecent sites, and open tabs. It also owns the pages' half of the scheme bridge: `ZGui.colorscheme.onApply` sends the pick to the native host and mirrors it to `zb_scheme`, and a 1.5-second poll of the native file re-applies an externally chosen scheme so a page left open still follows a scheme picked elsewhere. The three list pages share `zg-list.js`, and the fuzzy matching everywhere is the shared `fzf.js` — so the consistency across a dozen pages is literal shared code, not a dozen parallel implementations.

## The custom-command runner, five ways

The palette that `zg-boot.js` installs on every HUD page is the same one the content scripts carry, and it runs custom commands through one dispatcher, `runCustomBoot`, whose five branches are the concrete meaning of the command types the Commands page edits. A `url` command substitutes the typed argument for `{q}` and opens the result in a new tab. A `shell` command hands the argument to `window.zwireTermRun`, which opens the terminal and types it (Chapter 16). A `js` command evaluates the argument as the body of `new Function('q', value)`, with the typed text bound to `q` — a genuine one-line scripting hook, scoped to the page. An `action` command writes a `zb_cmd` onto the storage bus for the worker to perform. And a `scheme` command sets the color scheme through the native host and the storage mirror at once. The same five types resolve identically in the content-script palette, because both call the same runner against the same `zb_custom_cmds` array — so a command you define once behaves the same whether you trigger it from an ordinary page or from a HUD page. It is the last loop the book closes on the palette: one editor, one storage key, one runner, five behaviors, reachable from every surface.

# Chapter 15 – The Native Host

`native/zwire-host` is the bridge between the browser and the machine — and, as of this edition, a good deal more than a bridge. It is a single self-contained Rust binary (~500 KB, no Python and no `psutil`, `panic = "abort"` and `opt-level="z"` in release) that began as `zwire`'s Chrome native-messaging host and grew into a **universal local endpoint**: a process any app can ask to read the machine, spawn a shell, crawl the filesystem, run a command, or hold a little state. It ships as its own MIT-licensed submodule under `extensions/hud-internal/native/zwire-host`, with its own README, its own CI, and a `src/` tree of 25 files — `lib.rs`, `main.rs`, and 23 capability modules. (Earlier printings of this book described a Python reference host, `hud_host.py`; that file has been removed — the Rust host is now the only one, and where the old text speaks of a “reference,” read it as the historical first cut of the same scheme/stats/PTY path `zwire-host` now owns outright.)

## One dispatcher, four transports

The heart of the host is a single **capability router** (`src/session.rs`, whose own doc line reads “the capability router shared by every transport”). Every request is a JSON object dispatched on its `cmd`; every reply is a JSON object; and a request may carry an `id` that is echoed on its reply, so one connection can multiplex many in-flight requests, streams, and terminals at once. `src/transport.rs` opens with “The three ways to reach the dispatcher” and enumerates three of them; a fourth arrived later in a module of its own, so the count is four. The three in `transport.rs` first:

- **Native messaging** (the default, and what `zwire` uses). Chrome spawns the host and speaks length-prefixed JSON on `stdin/stdout`: a 4-byte little-endian `u32` length followed by that many UTF-8 bytes of body. `src/proto.rs` is the whole framing — `write_all(8{data.len() as u32}.to_le_bytes())` out, a `read_exact` of four bytes then the body in, and `None` on a short read as the port-closed sentinel.
- **Local-socket daemon** (`zwire-host serve`). The same router, reached over newline-delimited JSON (one object per line) on a **Unix domain socket** (macOS/Linux: `$ZWIRE_HOST_SOCKET`, else `$XDG_RUNTIME_DIR/zwire-host.sock`, else `~/.zwire/host.sock`, created `0600` under a `0700` dir because it exposes `exec/fs/pty`) or a **named pipe** on Windows (`\\.\\pipe\\zwire-host-<user>`). This is what lets `tmux`, `emacs`, shell scripts, and any language talk to the host without a client library — `printf '{"cmd":"sysinfo_once"}\n' | nc -U ~/.zwire/host.sock` is a complete client.
- **In-process library** (`zwire_host` as a crate). The same dispatcher, plus `api::walk` / `api::exec` helpers, so a sibling host can embed it and reuse the `crawl/exec/stream` machinery rather than reimplement it. (`transport.rs`'s own third entry is `call`, the one-shot client that connects to the daemon, sends a single request, and prints the reply — the same socket as the second bullet, driven from a shell script.)

The fourth is `src/zbus.rs`, a second listener that does not speak the `{"cmd":...}` request shape at all: it serves the `zgui-bridge` NDJSON protocol so a `stryke` script can reach the host as `App::open("zwire")`. Chapter 54 is that endpoint in full, and Chapter 55 is the verb surface it exposes.

Because all four sit behind the one router, every command in the next section works identically over any of them.

## The capability surface

Each capability is one small module under `src/`, and the twenty-three of them together are the whole of what the host does. The groups zwire itself leans on:

- **Discovery & state** — `hello` feature-tests the host (`{ok,host,version,os, arch,pid,caps:[...]}`), `hostinfo` returns one-shot machine facts, and a per-app key/value store (`src/store.rs`) persists JSON under `~/.<app>/kv/` via `kv_set/kv_get/kv_merge/kv_del/kv_keys`.
- **System stats** (`src/sysmon.rs`) — `sysinfo_once` for a snapshot, `sysinfo_start/sysinfo_stop` to stream one `{sys:{...}}` frame per interval.
- **Filesystem** (`src/fsops.rs`) — `fs_read/fs_write/fs_append/fs_list/fs_stat/fs_mkdir/fs_rm`, plus `fs_walk`, a recursive crawl with `depth/ext/dirs_only/contains` filters.
- **File watching** (`src/watch.rs`) — `fs_watch` streams `created/modified/ removed` events and `fs_tail` streams appended lines (`tail -f`, surviving rotation), each keyed by `id` and cancelled with `watch_stop`.
- **Exec & OS** (`src/exec.rs`, `src/osops.rs`) — `exec` runs a program to completion (base64 `stdout/stderr`), and `open/clipboard_get/clipboard_set/ notify` reach the desktop.
- **Background jobs** (`src/jobs.rs`) — `job_start` spawns a long-running command that survives the connection and fires a desktop notification on completion; `job_list/job_result/job_poll` track and drain them.
- **Process tools** (`src/procs.rs`) — `ps`, `kill`, and `which`.
- **Pub/sub bus** (`src/bus.rs`) — `sub/unsub/pub` turn the host into a coordination hub; the daemon itself publishes on the `scheme` and `ui` topics whenever they change, so a subscribed HUD gets **live theme sync without polling**.
- **Host-to-host peering** (`src/peer.rs`) — daemons run with TCP peering to federate the bus across machines (`peers/peer_connect/remote`), token-gated and single-hop so star and mesh topologies never loop.
- **PTY terminals** (`src/pty.rs`) — the terminal backend, below.

## The scheme and UI bridge

The host's original job survives unchanged as the “legacy zwire scheme/ui” command group. A `{cmd:'get'}` returns the host `version` plus the current scheme (defaulting to `cyberpunk`) and the UI prefs; a `{scheme:'matrix'}` message — validated against the eight allowed names — and a `{ui:{...}}` merge each rewrite their HUD file with a write-to-`.tmp`-then-atomic-rename so a concurrent reader (the fork's color mixer, the new-tab poller) never sees a half-written file. These files are the cross-extension bus of Chapter 8; the host is what owns them on disk, and — via the `scheme/ui` pub topics above — what now pushes a change to every subscriber the instant it lands.

## The system-stats stream

`sysinfo_start` (with an optional `interval_ms`, floored at 250 ms) enters `src/sysmon.rs`, which samples the machine through the cross-platform `sysinfo` crate — no `psutil`, no `pip install` — and sends one `{sys:{...}}` frame per tick:

```
cpu    overall CPU %           mem/swap  used / total (bytes)
load   1/5/15 load average     uptime    seconds since boot
disk   root filesystem used / total  batt      pmset percent + plugged (macOS)
net     bytes sent/recv rate (Δ/dt)    io         disk read/write rate (Δ/dt)
temp   hottest Components sensor    host      short hostname
lip    LAN IP                    pip        public IP via api.ipify.org
```

The one structural change from the old reference matters: the stream runs on its **own background thread**,

so a persistent connection stays free for other RPCs while it ticks — a HUD can hold a sysinfo stream, a PTY, and a file watch on the same pipe at once, which the single-process Python host could not. Rates (`net`, `io`) are deltas over the elapsed wall-clock interval (`dt`, floored at 0.1 s), so they read as true per-second throughput rather than cumulative counters; the public IP is fetched from `api.ipify.org` and refreshed only periodically so the stream never hammers the lookup; and any sensor a platform lacks (no battery off macOS, no temperature) is simply omitted from the frame rather than crashing it. Everything the statusbar shows (Chapter 11) is one field of this sample.

## The PTY relay

`pty_spawn` turns the host into a terminal backend (`src/pty.rs`). It opens a pseudo-terminal through the cross-platform `portable-pty` crate — `native_pty_system().openpty(PtySize{...})` — and runs a program in it: `$SHELL` (falling back to `/bin/sh`) with args defaulting to `["-l"]` for a login shell, at optional `rows/cols` (24×80 default), `cwd`, and `env`. A per-session reader thread pumps the shell's output as `{ev:"output", b64:...}` frames; `pty_write` feeds input, `pty_resize` changes the window size, and `pty_kill` ends it with an `{ev:"exit"}`. The important upgrade over the old single-shell reference is **multiplexing**: sessions are keyed by `id`, so `pty_write/pty_resize/pty_kill` route to the right terminal and one connection can hold several at once (an absent `id` addresses a single default session). It is a genuine login shell in a genuine PTY, driven from a web page — the piece with the most moving parts and the one the CDP probes (Chapter 20) spend most of their assertions verifying. Chapter 16 is the browser end of this pipe.

## Packaging, security, and the library

`zwire` wires the host up as a standard Chrome native-messaging host: a manifest named `com.zwire.hud` (`type: "stdio"`) points `path` at the built binary and lists the allowed extension origins in `allowed_origins`, and `scripts/localinstall.sh` builds `zwire-host` and drops that manifest into the browser's `NativeMessagingHosts/` directory when it packages the `.app`. Because the binary is a single static Rust artifact, the consuming bundle has **zero runtime dependencies** — no system Python to be missing, no `psutil` to `pip install`, nothing to break on a fresh machine.

The daemon transport takes its exposure seriously, because it hands out `exec`, `fs`, and `pty`: the Unix socket is created `0600` inside a `0700` directory (owner-only), and inbound TCP peering is gated by a shared `--token` (or `$ZWIRE_HOST_TOKEN`) that a connection must present before anything privileged — local socket clients are trusted and skip it. `sysinfo` and `portable-pty` abstract the OS, so the same source builds for macOS, Linux, and Windows, and CI runs the four canonical polish gates (`fmt`, `clippy -D warnings`, `doc`, `test`) on all three. And because the whole thing is also a library (`zwire_host`), sibling hosts — `zpwchrome`'s among them — embed it to reuse the crawl/exec/stream machinery rather than reimplement it. Chapter 16 is the browser end of the PTY pipe.

# Chapter 16 – The Terminal

The HUD ships a real terminal: an `xterm.js` front end (`lib/term/`) whose bytes come from the native host's PTY relay, bridged through the service worker, rendered as an overlay on any page and as a standalone page, and kept open across navigation. This chapter is the browser end of the pipe Chapter 15 opened.

## xterm over the service-worker bridge

The terminal is `xterm.js` (`lib/term/xterm.js` + `terminal.js`) plus a boot script. A content script cannot open a native-messaging port that survives page navigation, and the PTY must outlive any single page, so the terminal's bytes flow through the background service worker: the worker holds the `connectNative` port to `com.zwire.hud`, sends `pty_spawn`, and relays `pty_write/pty_resize` down and base64 `output` up. The content-script terminal decodes the base64 into xterm and sends your keystrokes back as `pty_write`. Toggle it with `Ctrl+`` and it drops in as an overlay pane; there is also a standalone `pages/terminal.html`.

## Staying open across navigation

A terminal that vanished on every link click would be useless, so `zterm-boot.js` mirrors the overlay's open state into `chrome.storage.local` under `zb_term_open`. Per-origin `localStorage` does not carry across different sites, so the boot script uses the cross-origin storage key instead: a `MutationObserver` on the pane's `active` class records every open/close, a `pagehide` handler snapshots the state right before navigating, and on the next page the overlay re-injects and re-opens automatically if it was explicitly open — and the worker reconnects the same tab's PTY, so the shell session continues across the navigation.

The open flag is deliberately strict and self-clearing. It re-opens only when `zb_term_open === true` (a real boolean, not a stale truthy value) and the pane is not already showing, and the background worker resets it to `false` on browser startup and on extension reload (Chapter 8's worker also owns this), so a stale “open” flag from a previous session never re-pops the terminal on every page of a fresh one. The result is a terminal that follows you from page to page within a session but does not haunt the next one — the browser-side half of a genuine shell, running the same login `zsh` you have in a real terminal, over the PTY the native host forked.

## The transport shim and the xterm instance

The terminal front end is not `zwire`-original either; `terminal.js` is the shared `zpw-embed-terminal` component, extracted verbatim from Audio-Haxor, and its own header says the only `zwire`-specific change is “a transport shim (auto-detects Tauri vs JUCE).” That shim, `TT`, picks a transport at load: a standalone extension page can `connectNative('com.zwire.hud')` directly, while a content-script overlay — which cannot open a native port — takes the `chrome.runtime.connect({name: 'zwire-pty'})` branch to the worker instead. Both branches speak the same four-message protocol (`pty_spawn`, `pty_write`, `pty_resize`, `pty_kill`) and decode the same base64 `output` frames into a `Uint8Array` that xterm writes directly, so the same component runs identically over the two paths.

The xterm instance itself is configured for the HUD: a blinking block cursor, a 13-pixel Hack Nerd Font (falling back through Share Tech Mono and Menlo), a fully transparent background so the scheme shows through, cyan cursor and selection, a ten-thousand-line scrollbar, and the sixteen ANSI colors set explicitly. There is no fit addon — the component measures the rendered cell size from xterm's own render service and computes

rows and columns by hand, re-fitting on a 50-millisecond- debounced `ResizeObserver` and sending each new size down as `pty_resize`. One piece of the component exists solely for release WebKit: because that engine ignores `setAttribute('style', ...)` on the DOM renderer's rows, a monkey-patch replaces xterm's `_addStyle` to set `el.style.color` and `el.style.backgroundColor` directly, so truecolor output actually paints — the same class of WebKit-release quirk the GUI apps in the suite fight constantly.

## Docks, Ctrl-letters, and shell one-liners

The overlay is a draggable, dockable pane, not a fixed panel. It carries four dock positions (the corners), snaps to the nearest one on release using pixel-positioned zones (because “CSS calc() with percentages doesn't resolve in release WebView,” the comment notes), and persists its geometry. Two keyboard details make it usable as a real terminal inside a browser. First, `Ctrl+`` toggles it — deliberately not `Ctrl+T`, which is “left to Chrome for new-tab.” Second, when the terminal has focus a capture-phase handler forwards `Ctrl+<letter>` combinations straight to the PTY as their control codes (`Ctrl-A`, `Ctrl-E`, `Ctrl-K`, `Ctrl-U`, `Ctrl-W`), because macOS WKWebView would otherwise swallow them for its own Emacs bindings — so readline editing works in the browser terminal the way it does in a real one. And `zwireTermRun`, the entry point behind the palette's `shell` command type, opens the terminal and then polls for up to ten seconds until the PTY is alive before writing the command, so a one-liner fired the instant the terminal opens is never dropped before the shell exists.

## The session's lifetime, and the four ways it ends

A shell inside a browser has a lifetime problem no ordinary terminal has: the thing displaying it — a content script in a document — is destroyed and recreated constantly, by every navigation, while the thing running it must not be. The design's answer is to bind the session to the *tab* rather than to the page, and the consequences of that choice are worth walking, because they explain every otherwise-odd behavior in the terminal.

A session begins when a spawn request reaches the worker for a tab that has none. It is honored once per tab: a second spawn for the same tab returns without doing anything, so a content script re-injected by a navigation re-attaches to the running shell rather than forking a second one behind the same window. This single guard is what prevents the worst failure mode in the feature — two shells writing to one screen, which looks like corruption rather than like a bug.

From there the session ends in exactly four ways, and only one of them is the user quitting. The tab closing tears the native port down, because the tab is the lifetime the session was bound to. The shell exiting sends its exit frame up and the session ends with it. An explicit kill ends it deliberately. And the browser or the host dying ends it by ending the process at one end of the pipe. What is conspicuously *not* on that list is the page navigating: a content-script disconnect nulls the attachment and leaves the native port alive, with output buffering until something re-attaches. That asymmetry — page disconnect is not session end, tab removal is — is the entire mechanism behind “click a link and keep typing in the same zsh.”

The buffer that covers the gap has a bound, and the bound is a real decision. Output produced while nothing is attached is held, capped at 500 messages, and flushed on reconnect. A cap is required because a detached session could produce output indefinitely — a build, a log tail — and an unbounded buffer in the worker is a memory leak with extra steps. The consequence of the cap is that a very chatty command run across a long navigation will lose its earliest output, which is the correct thing to lose: the tail is what the user will read.

## Sizing without a fit addon

The terminal computes its own geometry rather than using xterm's fit addon, and the reason is worth stating because it is the same class of problem the rest of the suite fights. A fit addon measures the container and asks the terminal to match it; that requires the container's resolved size to be trustworthy at the moment it is

asked, which in a release web view — with a dock-positioned, draggable overlay whose CSS is resolving at the same time — it frequently is not.

So the component measures the rendered cell size from xterm's own render service and computes rows and columns by hand, re-fitting on a resize observation debounced at 50 milliseconds and sending each resulting size down to the host as a resize request. Two properties fall out. The measurement is taken from the *terminal's own rendering* rather than from the container's computed style, which makes it independent of whatever the surrounding layout is doing. And the debounce means a drag that changes the size continuously produces a handful of resize messages rather than one per animation frame — which matters because each one is a real `ioctl` on a pseudo-terminal and a `SIGWINCH` to the shell, and a shell receiving hundreds of those per second will redraw its prompt hundreds of times.

The same release-web-view pressure explains two other choices the chapter has already named: the dock zones are pixel-positioned because percentage-based `calc()` does not resolve reliably in the release engine, and a monkey-patch replaces xterm's style application so truecolor output actually paints, because that engine ignores the attribute-based path the renderer normally uses. None of these are xterm bugs or web-view bugs in any interesting sense; they are the ordinary cost of running a component built for one rendering environment inside another, and they are isolated to the smallest possible surface — one measurement path, one geometry helper, one patched method — which is what keeps the shared component shared.

## Why `Ctrl+`` and not `Ctrl+T`

The toggle chord is a small decision with a large amount of reasoning behind it, and it is a good closing illustration of the arbitration model. `Ctrl+T` is the obvious choice for a terminal and is exactly wrong here: it is Chrome's new-tab accelerator, it is claimed at the browser level, and a page listener cannot reliably take it. Binding it would produce a feature that opened a new tab most of the time and a terminal occasionally, which is worse than not binding it at all.

`Ctrl+`` is unclaimed by the browser, is the chord the same gesture uses in several editors, and — usefully — is a key most web applications do not bind, so the claim-only-what-is-mapped discipline costs almost nothing to honor around it. The terminal's second keyboard claim, the `Ctrl+<letter>` forwarding, is scoped to the moments the terminal has focus precisely so that it does not violate the same discipline: outside the terminal those chords belong to the browser and the page, and the terminal does not want them.

# Chapter 17 — zgui-core, the Shared Toolkit

Under the internal HUD sits `lib/zgui-core`, a git submodule that is the shared cyberpunk GUI toolkit for the whole MenkeTechnologies app suite. `zwire` consumes a slice of it — the fuzzy matcher, the command palette, the effect toggles, the fonts — and this chapter is what that library is and why `zwire` loads it from the submodule rather than copying it.

## One toolkit, many consumers

`zgui-core` is, in its own words, the core GUI library shared across the app suite (Audio-Haxor, traderview, `ztranslator`, `zpdf`, `zemail`, `zoffice`, and more) — one canonical implementation of the shell, settings, dialog, table, and notification chrome so the many apps stop drifting their own divergent copies. Each component was extracted from an app that had already built it. It exposes a single namespace, `window.ZGui`, and its `webui/` directory holds on the order of 246 component scripts plus their CSS — a filterable card grid of buttons, inputs, charts, cyberpunk effects, and overlays that its own demo page renders.

## What `zwire` uses

`zwire` does not load the whole library; it loads the handful of modules its content scripts depend on, in the manifest's canonical block:

- `fzf.js` — `ZGui.fzf`, the `fzf`-style fuzzy matcher with configurable scoring weights and a matched-character highlighter (`<mark class="fzf-hl">`). This is the engine behind the command palette (Chapter 12), the fuzzy find fallback (Chapter 13), and the fuzzy filtering on the HUD internal pages (Chapter 14).
- `command-palette.js` — `ZGui.palette`, the palette shell the `%K` list is built on.
- `util.js` — the shared helpers the above need.
- `fonts.css` + `webui/fonts/` — the shared typeface loader: `@font-face` for Orbitron (display) and Share Tech Mono (body/mono), with the `.ttf` files bundled under SIL OFL, so the HUD's font stack resolves the same way it does in every other app in the suite.
- `fx.js` — `ZGui.fx`, the global cyberpunk effect toggles (CRT scanlines, bezel vignette, neon glow, animations), each flipping a `body.no-*` class the theme reacts to — the mechanism behind the visual-effect prefs in `hud-ui.json`.

## Submodule, not copy

`zgui-core`'s own guidance is emphatic: consume it as a git submodule and load the modules directly from the submodule path — *never copy* `webui/*`, because copies go stale and partial-sync. `zwire` follows that: `lib/zgui-core` is a submodule (declared in `.gitmodules`), and the manifest references files under `lib/zgui-core/webui/...` directly. This is the same reuse-not-copy discipline the `zpwchrome` power-tool and the native host follow, and it means an improvement to the shared fuzzy matcher or palette reaches `zwire` with a submodule bump rather than a hand-merge. The HUD's palette and find bar are not `zwire`-original UI code; they are the suite's shared components, wired to `zwire`'s data.

## Inside the fuzzy matcher

`ZGui.fzf` is not a substring test; it is a scored fzf-style matcher, and its scoring is the reason the palette and find bar rank the way they do. Its own comment records the lineage — “EXTRACTED VERBATIM from Audio-Haxor” and “shared so the apps stop drifting their own copies.” The core, `fzfMatch(needle, haystack)`, is a case-insensitive subsequence match that, for each possible starting position of the first character, greedily takes the earliest subsequent matches and scores the run: each matched character is worth a base amount, adjacency to the previous match earns a consecutive bonus, and a gap between matches costs a start penalty plus a per-character extension penalty. On top of that come *position* bonuses that encode what humans mean by a good match — a character right after a non-word character scores a word-boundary bonus, a capital right after a lowercase scores a camelCase bonus, and the first matched character’s bonus is multiplied. The default weights are concrete (match 16, gap-start -3, gap-extend -1, boundary 9, non-word 8, camel 7, consecutive 4, first-char multiplier 2) and are themselves tunable and persisted, which is how the whole suite can retune ranking in one place. Above `fzfMatch` sits an extended query parser — space-separated AND terms, `|` for OR groups, and the fzf sigils `!` (negate), `'` (exact), `^` (prefix), `$` (suffix) — and a highlighter that wraps the matched characters in `<mark class="fzf-hl">`, which is the cyan emphasis you see on the matched letters in every list.

## The palette shell and the shared readline

`ZGui.palette` is the list host the `%K` overlay is built on, and its ranking is a deliberate three tiers: static items and “strong” provider results are fzf-scored together at the top (ties favoring the static entries), “secondary” reference rows sit in the middle, and “fallback” rows — the generic web search — are pinned last no matter what you type, so a real command is never buried under a search suggestion. One more piece of `util.js` reaches every HUD input: on load it installs Emacs/ readline line-editing globally (`^A/^E` to move, `^W/^K/^U` to kill, `M-b/ M-f` by word, and the rest) and disables autocorrect and spellcheck on text fields. That is why the palette input, the find bar, and every settings field share the same terminal-style editing — it is one installed behavior from the shared toolkit, not a per-field reimplementation.

## The visual-effects layer

The cyberpunk *look* — the grid lattice, the CRT scanlines, the bezel vignette, the neon glow — is also `zgui-core`, and it is built on one clean convention worth stating: an effect is *on* when its class is *absent*. `ZGui.fx` maps four effects to four body classes (`no-scanlines`, `no-vignette`, `no-neon-glow`, `no-anim`), and turning an effect off adds its class; the default, with no class present, is everything on. Each toggle persists to its own `zg-fx-...` `localStorage` key, and the Settings page bridges the same four toggles up to the native `hud-ui.json` so the new-tab extension follows.

The CSS behind the effects is unglamorous and worth demystifying, because none of it is magic. The grid lattice is two `linear-gradient` hairlines on a 40-pixel fixed-attachment cell, layered under three faint `radial-gradient` color blooms on the body background. The scanlines are a single `repeating-linear-gradient` on `body::after` — two transparent pixels, two barely-visible cyan pixels, repeated — plus, when the animated variant is on, two injected beam divs that sweep across on twelve- and eighteen-second loops. The vignette is one `radial-gradient` ellipse on `body::before` that is transparent to 58% and then darkens to the edges. And the neon glow is not a filter at all; it is pervasive `text-shadow` and `box-shadow` on themed elements. That last point is why the off switch is so blunt: `body.no-neon-glow *` sets `text-shadow` and `box-shadow` to `none !important`, `body.no-anim *` sets `animation` and `transition` to `none !important`, and the two overlay pseudo-elements are simply `display:none`. The effects are cheap layered CSS with a global kill switch, which is exactly why they can be toggled live without a reflow storm — and why the same look renders identically across every HUD surface that includes the one stylesheet.

# Chapter 18 — `zpwrcode`, the Power-Tool

`extensions/zpwrcode` is the Manifest V3 power-tool, loaded as a **submodule** — reused, not copied — so `zwire` tracks upstream instead of forking a snapshot of it. This chapter covers what the extension provides, the native host its `pass` and segmented-download features need, and the cosmetic developer-extensions banner that unpacked loading produces.

## Loaded as a submodule, not vendored

`zpwrcode` is its own public repository, and `zwire` consumes it through a git submodule at `extensions/zpwrcode` (declared in `.gitmodules`), not by copying its files into the tree. This is the same reuse-not-copy discipline `zgui-core` and the native host follow, and it has a concrete payoff: a fix or feature that lands in `zpwrcode` upstream reaches `zwire` with a submodule bump, and there is exactly one canonical copy of the extension rather than a frozen fork that drifts. `zwire` is a *host* for `zpwrcode`, not a second home for it.

The cost of a submodule is that it can be absent — a clone without `--recurse-submodules`, or a checkout where the submodule was not initialized, leaves `extensions/zpwrcode` empty. `zwire` handles this with the same graceful-skip rule the whole launcher uses: `bin/zwire` includes an extension directory in `--load-extension` only if it contains a `manifest.json`. An empty `zpwrcode` directory has no manifest, so it is silently skipped and `zwire` runs with just the new-tab and internal-HUD extensions. The power-tool is additive, not required; its absence degrades the browser, it does not break it. This is why the README can honestly say a fresh clone still runs.

Because it is a submodule and a public, separately-versioned project, this book does not re-document `zpwrcode`'s own feature surface in depth — that belongs to `zpwrcode`'s own documentation, and duplicating it here would go stale the moment upstream moves. What matters for `zwire` is the *seam*: how the power-tool is loaded (submodule, skipped-if-absent), how it stays color-synced with the rest of the HUD (the background worker pushes the scheme to it over `runtime.sendMessage`, and it can pull the current scheme or push a user pick back through the same bridge, since separate extensions cannot share storage — Chapter 8), and what it needs from the machine.

## Native messaging and the developer-mode banner

Two operational notes close the chapter, both from the README's own notes section.

**Native messaging.** `zpwrcode`'s `pass` integration (the standard Unix password store) and its segmented-download feature both need `zpwrcode`'s native host installed — a separate step, documented in the `zpwrcode` project's setup, not something `zwire`'s launcher does for it. `zwire` wires *its own* HUD host (`com.zwire.hud`, for the terminal, system stats, and scheme file) automatically on launch; `zpwrcode`'s host is `zpwrcode`'s to install. The two hosts are independent, and the `allowed_origins` in the HUD host's manifest list all three extension ids (HUD, `zpwrcode`, new-tab) so the scheme bridge can be reached from any of them.

**The developer-mode banner.** Because the launcher loads the extensions unpacked via `--load-extension`,

Chromium shows its “disable developer-mode extensions” notice on startup. It is cosmetic. The extensions run fully — the banner is Chromium telling you that unpacked extensions are present, which is precisely what *zwire* intends. There is no way to load unpacked extensions on launch *without* the notice on current Chromium; *zwire* accepts it rather than working around it, and this book flags it so it is expected rather than alarming.

## What the power-tool declares

This book does not re-document *zpwirchrome*'s feature surface — that belongs to its own project and would go stale the moment upstream moved — but its manifest is worth reading once, because it is the concrete proof of Chapter 2's argument that only a genuine Chromium engine will do. *zpwirchrome* is Manifest V3 with `minimum_chrome_version: 127`, and it declares nineteen permissions, among them the four that rule out every non-Chromium base: `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, and `webRequest` (with `webRequestAuthProvider`). Alongside those sit `tabs`, `tabGroups`, `sessions`, `bookmarks`, `history`, `scripting`, `webNavigation`, `cookies`, `downloads`, `contextMenus`, `notifications`, `clipboardWrite`, `alarms`, and `storage`, over `<all_urls>` host access, with a module service-worker background and six content scripts. It binds fifty-four keyboard commands (its action opens on `Alt+T`), which is a scale of its own — and part of why *zwire* loads it as a submodule and lets it own its own shortcuts rather than trying to re-carry any of it.

The feature summary, from the power-tool's own README, is a browser toolkit behind one toolbar icon: a browserpass-compatible Unix `pass` integration (fill, copy, OTP, profile and credit-card autofill, through a vendored Rust native host that walks `~/.password-store`), a segmented multi-connection download manager that takes over Chrome's default via a HEAD probe and parallel range requests, a JetBrains-style most-recently-used tab switcher with scenes and an opener tree and fzf history over thousands of entries, a Tampermonkey-equivalent userscript engine, full-page screenshots, a technology detector, a theme injector, a lights-off dimmer, JSON and XML viewers, a user-agent switcher, and find-in-all-tabs. It needs its own native host for the `pass` integration and the segmented downloads — a separate install, independent of *zwire*'s HUD host — and the `allowed_origins` in *zwire*'s own host manifest lists all three extension ids so the scheme bridge is reachable from any of them. What matters to *zwire* is only the seam of Chapter 18: loaded as a submodule, skipped if absent, color-synced over `runtime.sendMessage`, and speaking to the machine through its own host.

## The seam, stated as a contract

Because the power-tool is a separately-versioned project, the durable thing to write down is not its feature list but the *contract* between it and *zwire* — the small set of facts each side may rely on. Stated as four clauses, it is short enough to check against the code and stable enough to survive upstream moving:

*zwire* will load the power-tool from its submodule directory if and only if that directory contains a manifest, and will do so unpacked, against the dedicated profile, with the same elevated flags the other extensions get. That is the loading clause, and its second half is what makes the first half safe: the power-tool is trusted with those flags because it is running in a profile that holds nothing.

*zwire* will push the active scheme to the power-tool over cross-extension messaging, and will accept a scheme the power-tool pushes back, with an echo guard on the *zwire* side so a value that originated in the power-tool is not relayed back to it. That is the color clause, and the guard is on *zwire*'s side because *zwire* is the component that can distinguish its own writes.

*zwire*'s own native host manifest lists the power-tool's extension id in its allowed origins, so the power-tool may open a port to the HUD host if it wants the scheme directly rather than through a message. *zwire* does *not* install the power-tool's own native host, which is a separate binary for separate features and a separate install step. That is the host clause, and its asymmetry is deliberate: *zwire* ensures what *zwire* needs and never reaches into another project's setup.

Everything else is out of scope in both directions. `zwire` does not read the power-tool's storage, does not depend on any of its fifty-odd keyboard commands existing, and does not assume any particular feature is present; the power-tool, for its part, is a complete extension that runs in an ordinary Chrome and does not require `zwire`. That mutual independence is what makes the submodule model honest rather than a fork with extra steps.

## Why the surface is not re-documented here

There is a real temptation, in a book this thorough, to enumerate the power-tool's features as carefully as the HUD's. Resisting it is a documentation-engineering decision, not laziness, and it is the same reasoning that governs the code. A book that described a separately-released project's feature surface would be a copy of that project's documentation, and a copy is stale the moment upstream ships — with the particular hazard that a *book* is a durable artifact a reader trusts more than a README. Describing the seam instead means describing something `zwire` actually controls and can therefore keep true.

The one part of the power-tool this book does document in detail is its manifest, and that is because the manifest is not a feature list — it is the evidence for the argument the entire project rests on. The declaration of a manifest-version-three extension with a minimum Chromium floor and the four permissions that exist only on a genuine Chromium engine is what makes “`zwire` must be Chromium” a checkable claim rather than a preference. That belongs in this book permanently, because it is an argument about `zwire`; the download manager's segment count does not, because it is a fact about another project.

The practical guidance that falls out for a reader: when something about the power-tool behaves unexpectedly, the question to ask first is which side of the seam it is on. A color that does not follow the scheme is a seam question and this book covers it. A missing extension after a fresh clone is a seam question and this book covers it. A password-store integration that cannot find its native host is not a seam question at all — it is the power-tool's own setup, deliberately independent of `zwire`'s, and no amount of `zwire` configuration will affect it.

# Chapter 19 – The Launcher and State Layout

`bin/zwire` is the single entry point: it starts the base against `~/.zwire/profile` with `newtab`, `zpwchrome`, and `hud-internal` loaded and `--extensions-on-chrome-urls` set, passing any extra Chromium flag straight through. This chapter maps the launcher's flag assembly and the `$ZWIRE_STATE` directory layout.

## What the launcher assembles

`bin/zwire` is a short, `set -euo pipefail` shell script, and reading it top to bottom is reading the entire runtime path:

1. **Resolve the base.** It checks `ZWIRE_BASE` (an executable override) first, then the path recorded in `~/.zwire/base.path`. If neither yields an executable binary it prints `zwire: no base browser installed – run ... fetch-base.sh` and exits non-zero. No base, no launch, with a message that names the fix.
2. **Build the extension list.** It iterates the three extension directories — `newtab`, `extensions/zpwchrome`, `extensions/hud-internal` — and includes each only if it contains a `manifest.json`, joining the survivors with commas. This is the graceful-skip rule: a missing submodule or deleted directory drops out of the list instead of breaking the launch. Note what is *not* here: `theme/` is deliberately excluded, for the palette-ordering reason Chapter 3 gave.
3. **Ensure the native host.** It looks for the HUD host manifest at `~/.zwire/profile/NativeMessagingHosts/com.zwire.hud.json` and reads the binary path out of it; if the manifest is missing or its target binary is gone, it runs `scripts/setup-native-host.sh` to build the Rust `zwire-host` and (re)write the manifest. This is what makes a bare `zwire` self-sufficient on Linux and in dev, not just the packaged macOS `.app`. It is never fatal — if the host cannot be built, the browser still launches, with the terminal and system stats disabled and a one-line note on stderr.
4. **exec the base** with a fixed flag set plus your arguments:

```
exec "$BASE" \
  --user-data-dir="$PROFILE" \
  --load-extension="$LOAD" \
  --extensions-on-chrome-urls \
  --test-type \
  --no-first-run \
  --no-default-browser-check \
  --homepage="chrome://newtab" \
  --disable-features=NtpFooter \
  --enable-features=SplitViewHorizontal,SplitViewTabRestore \
  "$@"
```

Each flag earns its place. `--user-data-dir` is the profile isolation the whole design rests on. `--load-extension` is the switch unbranded Chromium keeps and branded Chrome removed — the reason for the whole base choice. `--extensions-on-chrome-urls` lets the internal-HUD content scripts run on `chrome://` pages. `--test-type` suppresses the automation-restriction warnings that would otherwise fire with these flags. `--no-first-run` and `--no-default-browser-check` skip the onboarding

and the default-browser nag. `--homepage=chrome://newtab` points the home button at the HUD new tab. `--disable-features=NtpFooter` removes the new-tab footer that would sit under the HUD page. `--enable-features=SplitViewHorizontal,SplitViewTabRestore` turns on Chromium's side-by-side split view and its restore-on-reopen. And `"$@"` passes *everything you typed* straight through — so `zwire https://github.com` opens a URL, `zwire --incognito` starts incognito, and, relevant to Chapter 20, `zwire --remote-debugging-port=9222` opens the CDP endpoint the verification probes attach to. The launcher owns the HUD-defining flags and gets out of the way of every other Chromium flag.

## Reading the launcher line by line

`bin/zwire` is short enough to hold in the head, and a few details below the summary are worth naming. It resolves its state directory by sourcing `state-dir.sh` and honoring `$ZWIRE_STATE`; then it runs the one-time legacy migration described in the next section. Base resolution has an exact precedence — an executable `$ZWIRE_BASE` first, then the path recorded in `$ZWIRE_STATE/base.path`, then nothing — and if nothing resolves to an executable it prints, verbatim, `zwire: no base browser installed - run: .../scripts/fetch-base.sh` and exits non-zero. The extension list is the three-directory loop of Chapter 3, filtered to directories that actually contain a `manifest.json` and joined with commas, with `theme/` conspicuously absent.

The native-host ensure step is more careful than “run the setup script.” The launcher reads the `"path"` field out of the existing `com.zwire.hud.json` with a one-line `sed`, and it re-runs `setup-native-host.sh` only when the manifest is missing or its recorded binary is no longer executable — a dangling manifest, not just an absent one, triggers a rebuild. And it is never fatal: if the host cannot be built, the launcher prints `zwire: native host unavailable (terminal/sysinfo disabled) - see scripts/setup-native-host.sh` to stderr and launches the browser anyway, minus the terminal and the system stats. That single property — the browser runs even when the host, a submodule, or a whole extension is missing — is the graceful-degradation rule made concrete in the one script every launch goes through.

## The `$ZWIRE_STATE` directory

All of `zwire`'s state lives under `$ZWIRE_STATE`, and — a correction the earlier printings of this book owe the reader — its *default* is not a dotdir in `$HOME`. `scripts/state-dir.sh` resolves the OS's regular application-data location: `~/Library/Application Support/com.menketechologies.zwire` on macOS — named by the reverse-DNS bundle id, matching the `.app`'s `CFBundleIdentifier` and the platform convention that Application Support folders carry the bundle id — and `${XDG_CONFIG_HOME:-~/.config}/zwire` on Linux, with `$ZWIRE_STATE` overriding on every platform. There are two *legacy* locations earlier builds used: the original `~/.zwire` dotdir, and a brief intermediate macOS build that used the bare `~/Library/Application Support/zwire` name before it moved to the bundle id. `bin/zwire` does a one-time `mv` of whichever leftover it finds into the new state dir when no explicit `$ZWIRE_STATE` is set, so an upgrade keeps your profile, base, and scheme intact and then never touches the old dir again. The theme is the one exception to this per-install dir: the shared colour scheme and light/fx prefs live in a *fleet-wide* location, `~/.zwire` (the host's `global.toml`, overridable via `$ZWIRE_GLOBAL_DIR`), read and written by every MenkeTechnologies app so they all follow one look, and the C++ color patch (fork 0002) resolves that same `~/.zwire` natively — so the scheme file the host writes is the file the browser reads. `~/.zwire` is therefore a real, distinct directory now, not merely shorthand for `$ZWIRE_STATE`. The per-install tree, rooted at `$ZWIRE_STATE`, is a small, legible directory:

```

$ZWIRE_STATE/      (macOS: ~/Library/Application Support/com.menketotechnologies.zwire ·
└─ Linux: ~/.config/zwire)
  base/             the extracted Chromium (or the packaged fork)
  base.path         absolute path to the resolved binary (the launcher reads this)
  base.version      the pinned revision, e.g. "r1656770" or "fork-150.0.7871.46"
  profile/          the dedicated --user-data-dir (bookmarks, history, sessions)
    NativeMessagingHosts/com.zwire.hud.json  the HUD native host manifest
    Default/NativeMessagingHosts/com.zwire.hud.json  second location Chromium checks
  ext/             per-user copy of the loaded extensions, staged at launch

```

The theme is deliberately *not* in this tree. The shared scheme, light/fx prefs, and any custom schemes live in the fleet-wide `~/.zwire/global.toml` (with the host's plain-text `hud-scheme/hud-light` projections beside it for the fork's C++ mixer), so every zwire-host client on the machine — the browser, the new tab, `zpwchrome`, and the wider app suite — reconciles to one theme file rather than a per-install copy.

Two of the entries above repay a second look. `profile/` is the isolation boundary — a complete, separate browser profile that never shares a byte with system Chrome, holding your zwire bookmarks, history, sessions, and the extensions' own stored state (`zb_scheme`, `zb_keys`, `zb_custom_cmds`, `zb_term_open`, and the rest). And `ext/` exists because Chromium writes extension `_metadata/` indexes *into the extension directory*, which means loading extensions straight out of a read-only `.app` bundle fails and, on a multi-user machine, two users writing the same directory would collide. So the loaded extensions are staged into a per-user writable copy under `ext/` at launch, and each user gets their own. The `base.version` file is the one place that records whether you are on a snapshot (`r...`) or a compiled fork (`fork-...`), which is the single bit that distinguishes the two build paths on a running machine.

`ZWIRE_STATE` being an environment variable means the whole tree relocates: point it at a different directory and you get a second, independent zwire — a different profile, a different base, a different native host manifest — without touching the first. That is the same isolation principle one level up: as zwire keeps its state out of system Chrome, `ZWIRE_STATE` lets you keep one zwire's state out of another's.

# Chapter 20 – The CDP Verification Layer

The top-level `v*.mjs` scripts — `vkeys`, `vovl`, `vpl`, `vpop`, `vpos`, `vsys`, `vterm`, `vwin` — are **verification probes**. Each connects to a running zwire's Chrome DevTools Protocol endpoint on `127.0.0.1:9222`, attaches to a live page or extension page, and asserts that a specific HUD feature is actually wired and rendering. They are not the overlay itself — the vim keys, the tmux tiling, the statusbar, the palette, and the terminal all live in the `hud-internal` content scripts. The `v*.mjs` scripts are how you *check*, from the outside, that those content scripts did their job in a real browser.

## Talking to a live browser over CDP

Every probe shares the same eight-line bootstrap, and understanding it once explains all of them. The launcher does **not** open a debugging port by default; you opt in by passing it through:

```
zwire --remote-debugging-port=9222
node vterm.mjs
```

With the port open, a probe does this: `fetch http://127.0.0.1:9222/json/version` to get the browser's `webSocketDebuggerUrl`, open a `WebSocket` to it, and speak CDP over that socket. Commands are JSON objects with a monotonic `id`, a `method`, `params`, and (once attached to a target) a `sessionId`; responses come back tagged with the same `id`, and a small map of pending resolvers matches each reply to its request. To reach a page the probe calls `Target.attachToTarget` with `flatten:true` for a session id, enables `Runtime`, and then evaluates expressions in the page with `Runtime.evaluate (returnByValue:true)`, reading back whatever the HUD put in the DOM. Some probes open a fresh target first (`Target.createTarget` at an extension page URL) and some find an existing page in `/json/list` (matching a stable site like `github/amazon/wikipedia`) to attach to. `9222` is the wire because that is the conventional CDP port and what the probes hardcode; the browser only listens on it when you passed the flag.

That these are external CDP probes rather than in-page code is the point: they verify the HUD the way a black-box test would, driving a genuinely running browser and asserting on its real DOM, which is the only way to prove the multi-hop paths (page → content script → service worker → native host → PTY → shell) actually connect end to end.

## The modules and what each verifies

Each probe targets one HUD surface and prints a short list of assertions:

- `vkeys.mjs` opens the Keyboard settings page (`pages/keys.html`), confirms `ZWIRE_KEYMAP` is present, counts the editable key chips and category headers, then *exercises a remap*: it writes `zb_keys.scrollDown='s'` into `chrome.storage.local` and reads it back, proving the remap round-trips through storage where `zvim` would pick it up.
- `vterm.mjs` opens the standalone terminal page (`pages/terminal.html`), checks that xterm's `Terminal` global loaded and the pane rendered, waits several seconds for a heavy zsh login to boot over the native-host PTY, and reports whether rows of real shell text appeared.

- `vovl.mjs` and `vpop.mjs` attach to an ordinary web page, confirm the terminal content script injected a (hidden) `#terminalPane`, synthesize a `Ctrl+`` keystroke via `Input.dispatchEvent`, and verify the overlay becomes `active` and xterm renders — `vpop` waits longer and reads back the booted shell's visible text.
- `vwin.mjs` triggers the terminal from a page's `Ctrl+``, then looks in `/json/list` for the terminal popup window that opened, attaches to it, and confirms xterm rendered and `chrome.runtime.connectNative` is reachable from it.
- `vsys.mjs` waits for the native host's system-stats stream to fill `zb_sys`, then reads each statusbar segment — CPU, MEM, NET, HOST, LAN, UP, BATT, the powerline chevron count, and the clock — proving the real-machine stats path is live.
- `vpos.mjs` and `vpl.mjs` verify statusbar *layout*: `vpos` reads the left-to-right segment order and checks the `C-b` prefix sits left of the scheme segment; `vpl` inspects the powerline chevron pseudo-elements (the `::after` border that draws each chevron) and the prefix block's background, confirming the powerline geometry renders as intended.

Taken together the eight probes cover the runtime HUD's riskiest surfaces — the remap round-trip, the terminal's full native-PTY path, the overlay toggle, the system-stats stream, and the statusbar's powerline geometry — from outside the browser, over the same protocol any external tool would use. They are the runtime path's answer to the fork path's `git apply --check`: a concrete, re-runnable proof that the thing is actually wired, not merely written.

## Two probe shapes, and no test framework

The probes come in two shapes that reflect what they need to reach. `vkeys` and `vterm` create a fresh target at an extension page URL (`chrome-extension://<hud-id>/pages/keys.html` or `pages/terminal.html`) and assert against it, because the keymap page and the standalone terminal are extension pages you can open in isolation. The other six *attach to an existing tab* — they scan `/json/list` for a real page whose URL matches `github`, `amazon`, or `wikipedia`, because the overlay, statusbar, and terminal content scripts only inject into ordinary web pages, and a probe needs a genuine one to inject into. When no such tab is open, those probes exit cleanly rather than failing, since “no stable page” is a setup gap, not a broken feature.

There is no assertion framework: each probe evaluates an expression in the page and `console.logs` the boolean or the count, and pass or fail is read off the printed lines by a human or a CI grep. The overlay probes synthesize the real toggle by dispatching a `Ctrl+`` key event through `Input.dispatchEvent` (with the Ctrl modifier) and then wait — five seconds for `vovl`, seven for `vpop`, six for `vwin` — for a heavy zsh login to boot over the full page → content-script → worker → native host → PTY path before reading back the shell's visible text, which is the only way to prove that multi-hop chain end to end. Two probe lines are honestly dead weight, and this book flags them rather than pretend the suite is airtight: `vterm` reaches for a `window.__zbTermInstance` global that does not exist (the real instance is module-scoped), and `vovl` prints a placeholder string for the `zb_term_open` check instead of actually reading storage. They do not weaken the probes that matter, but they are there, and a book grounded in the code says so.

## What a probe cannot see

The probes are the runtime path's strongest evidence, and knowing their limits is part of reading them honestly. Four categories of failure are outside what an external CDP observer can assert.

The first is **timing that is correct but slow**. A probe waits a fixed number of seconds for a heavy login shell to boot and then reads the visible text; a session that would have appeared a second later reads as a failure, and one that appeared instantly reads the same as one that took the full wait. The probes prove that the path connects, not that it connects promptly, and the fixed waits are calibrated for the slow case rather than measuring anything.

The second is **state the probe cannot reach**. A probe evaluates expressions in a page and reads back what the HUD put in the DOM, which means anything held only in a module's closure is invisible to it. Two probe lines in the suite are honest casualties of this and the book has already flagged them: one reaches for a terminal instance global that does not exist because the real instance is module-scoped, and one prints a placeholder instead of reading a storage flag. Both are dead weight rather than false confidence — they assert nothing — but they illustrate the boundary exactly.

The third is **the negative case**. Every probe asserts that something is present; none asserts that something is *absent*. There is no probe that a content script did not double-inject, that an unclaimed key was left alone for the page, or that a feature turned off is actually off. Those are the assertions that would catch the regressions the earlier chapters worry about most — the idempotence guard failing, a listener claiming a key it does not own — and they are simply not in the suite. Naming that gap is more useful than implying the eight probes are a complete safety net.

The fourth is **anything on the fork path**. The probes attach over the debugging protocol to a running browser and read the page's DOM; the fork's contributions are native chrome — tab shapes, the toolbar line, menu colors, the audio engine — none of which is reachable from a page context at all. The fork's equivalent proof is `git apply --check` against the pinned tree, which is a completely different kind of claim: it proves the diffs match the source, not that the result behaves. The two verification stories cover the two tiers and neither reaches the other's.

## Why no assertion framework

The probes print booleans and counts and let a human or a grep decide, and it is worth defending that rather than treating it as an omission. A test framework buys you three things — structured pass/fail, setup and teardown, and a runner — and each of them is either unavailable or actively unhelpful here.

Structured pass/fail requires a definition of failure, and several probes have a third outcome that is neither: when no ordinary page is open to attach to, the correct behavior is to exit cleanly, because “no stable page” is a setup gap rather than a broken feature. A framework would have to model that as a skip, which is a concept the probes get for free by printing what they found and stopping.

Setup and teardown are impossible in the useful sense. The probes attach to a browser the operator launched with a debugging port they opted into, holding whatever profile state they have; the probes deliberately do not construct that environment, because constructing it would mean testing a browser that no user runs. The realism *is* the value.

And a runner would add a dependency to a project whose entire testing posture is that the base is prebuilt and there is nothing to compile. A probe is a single script with no imports beyond the platform's own fetch and web socket. That property is what lets one be written in an afternoon when a new HUD surface needs covering, which matters more, for a suite this size, than the ergonomics a framework would add.

## What it would take to assert the negative

The gap the previous section names — every probe asserts presence, none asserts absence — deserves more than an acknowledgement, because the assertions that are missing are exactly the ones guarding the invariants the rest of the book leans on hardest. Three of them are worth designing on paper, both because they are achievable over the same protocol and because working out how you would test an invariant is the fastest way to find out whether you have actually stated it.

**Single injection.** The invariant is that a content script injected twice into the same document is a no-op the second time, which Chapter 7 records as the load guard and Chapter 26 records as the thing standing between a long-lived tab and a slow accumulation of stacked handlers. Asserting it externally requires a countable consequence of injection. Counting listeners is not available to a page-context expression, so the assertion has to be phrased against something the script produces exactly once — the count of a HUD element it inserts,

or the count of a marker it sets. The probe shape is: attach to a page, read the count, drive a same-document navigation, read it again, assert it did not move. That is a genuine regression test for the exact failure Chapter 26 says the guards prevent, and it is buildable with the machinery the existing probes already use.

**The unclaimed key space.** The invariant is Chapter 28's: a HUD listener calls `preventDefault` only when the key maps to a real action, so an unknown key reaches the page untouched. This one is testable and the test is elegant, because the debugging protocol can synthesize a key event and the page can observe whether it arrived. Install a listener in the page context that records the keys it sees, dispatch a key that is deliberately absent from the merged keymap, and assert the page's listener recorded it and that the default was not prevented. Then dispatch a key that *is* mapped and assert the opposite. Two dispatches and two assertions cover the invariant in both directions, which is more than any presence-only probe can claim.

**A feature that is off.** The invariant is that a toggle actually toggles — that vim disabled for a tab means the raw keys go back to the page, that a hidden statusbar is absent rather than transparent, that a terminal whose flag was reset does not re-open. Each of these is an assertion that some element is not in the DOM or some key is not claimed, and each is the exact assertion that would catch the class of bug where a feature's disable path was never wired. Chapter 31's real-boolean rule — a flag compared to `true` rather than for truthiness, with a named reset owner — is a rule with no automated enforcement anywhere, and a probe asserting “after reset, absent” is that enforcement.

Two things make these harder than they look, and they are the reason the suite does not already have them. The first is that a negative assertion has a failure mode a positive one does not: it passes when the feature is broken in a way that removes the thing entirely. A probe asserting “the statusbar is absent after hiding it” passes on a build where the statusbar never injected at all. Every negative assertion therefore needs a positive precondition — assert it is present, toggle, assert it is gone — which doubles the choreography and makes the probe stateful in a way the current ones are not.

The second is timing, which the section above already names as the first category of failure a probe cannot see. A negative assertion evaluated too early passes for the wrong reason, because the thing has not appeared yet. The existing probes handle timing with fixed waits calibrated for the slow case, which is adequate for “did it appear” and actively wrong for “is it absent” — a fixed wait before a negative assertion is a coin flip. The correct shape is a poll with a deadline in the positive direction and a settle-then-assert in the negative, which is more machinery than the current probes carry and is precisely the machinery a test framework would have supplied.

That last observation is the honest counterweight to the previous section's defense of having no framework. The defense holds for the assertions the suite makes today: presence checks against a live browser, printed as booleans, read by a human or a grep, with no dependency beyond the platform. It holds less well for the assertions the suite is missing, because negative assertions need setup, teardown, and a retry policy — which are three of the things a framework exists to provide. The complete position, then, is not “a framework would add nothing” but “a framework would add nothing yet,” and the day the suite grows its first negative assertion is the day that calculation changes.

# Chapter 21 – The Source Fork

`fork/` compiles a patched Chromium so the *whole* browser — tab shapes, neon toolbar, HUD-colored frame, native menus and dialogs, and a browser-wide audio engine — lives in the HUD, at the cost of a ~100 GB checkout and a one-to-four-hour first build. All twenty-seven HUD patches are authored against the pinned tag `150.0.7871.46` and verified apply-clean (67 hunks across 32 Chromium files). This chapter is the pipeline and the patch series, with one real hunk quoted so the level of the work is clear.

## The pipeline: fetch, apply, build, package

The fork is four scripts run in order, and each is a thin, honest wrapper around the Chromium build tools:

1. `fork/fetch.sh` installs `depot_tools` if missing, `fetch --no-history chromium` into `$ZWIRE_SRC` (default `~/zwire-chromium`), checks out the exact tag from `fork/CHROMIUM_VERSION` onto a `zwire-<tag>` branch, and `gclient syncs` the deps for that tag. `--no-history` keeps the checkout to “only” ~100 GB rather than far more. This is the long, network-bound step.
2. `fork/apply-patches.sh SRC [--reverse]` reads `fork/patches/series` (an ordered list with in-line comments stripped) and applies each patch with `git apply --3way` from the Chromium `src/` dir; `--reverse` unapplies them in reverse order, which is how you back the patches out before rebasing onto a new Chromium tag.
3. `fork/build.sh SRC [OUT]` copies `fork/args.gn` into `out/<OUT>` (default `zwire`), runs `gn gen`, and then `autoninja -C out/<OUT> chrome`. That last command is the long compile — one to four hours the first time, minutes for an incremental rebuild after a single patch.
4. `fork/package.sh OUTDIR` installs the freshly built app as the `zwire` base: on macOS it finds the built `.app` (named `zwire.app` if the branding patch ran, else `Chromium.app`), copies it into `~/zwire/base` under its own name, derives the real executable from the bundle’s `CFBundleExecutable`, writes `base.path` and a `fork-<tag>base.version`, and then runs the same `scripts/rebrand-macos.sh` the runtime path uses. After this the same `bin/zwire` launches the HUD-chromed fork with the extensions preloaded — same launcher, forked base.

`fork/args.gn` configures the build: an *unbranded* Chromium (`is_chrome_branded=false`), an official (fully optimized) release build (`is_official_build=true`), with the slow parts turned off for a workstation build time — `symbol_level=0` and `blink_symbol_level=0` (no debug symbols, faster link), `chrome_pgo_phase=0` (skip profile-guided optimization), `enable_nacl=false` (drop legacy Native Client), and `use_thin_lto=false` (ThinLTO is slow for iteration). You set `target_cpu` to `arm64` or `x64` for your host. These are the knobs that turn a multi-hour Chromium build into merely a long one.

## What “apply-clean” proves, and what it does not

The fork’s own docs state the boundary plainly, and the book repeats it because it is the honest status of the fork. Each patch was authored by reading its target file at the exact tag on the Chromium GitHub mirror (`raw.githubusercontent.com/chromium/chromium/150.0.7871.46/...`) and is verified with `git apply --check` against that tree. That proves the diff *context matches the pinned source* — the patch will apply without a reject. It does **not** prove the C++ *compiles*; `fork/build.sh` is that gate. So the series’ status is “all twenty-seven authored and apply-clean against `150.0.7871.46`,” which is a real, checkable claim, and

not the stronger “compiled and shipped” claim, which the fork does not make. A CI check, `scripts/check-patch-series.sh`, keeps the `series` file and the patch files on disk consistent so the two never drift.

## The twenty-seven HUD patches

The patches live in `fork/patches/`, are listed in `series`, and each targets one native surface — a look a theme extension cannot restyle, or a behavior it cannot override. They fall into six groups: **nine styling** patches (0001–0009), **seven behavior** patches that force zwire’s defaults over Chromium’s (0010–0015, 0026), **four native-color** patches that bind the native Views surface to the HUD palette (0020–0021, 0025, 0027), **three extension-compatible** patches that keep the HUD’s own pages and the Chrome Web Store working (0016, 0018, 0019), **one quit** patch (0017), and the **three audio** patches that compile a browser-wide DSP engine into the audio service (0022–0024). Read against the target-file map in `fork/patches/README.md`:

**0001 — Tab shape** (`chrome/browser/ui/views/tabs/tab_style_views.cc`, in `TabStyleViewsImpl::GetPath`). The patch is more than a corner-radius tweak: it replaces the rounded-rect tab path with a *hard trapezoid* — the classic-Chrome tab silhouette with straight diagonal sides — while keeping the interior clip and hit-test paths rectangular so the favicon and title are never clipped by the sloped edges. The real hunk:

```
- path.lineTo(tab_left, tab_top + top_left_corner_radius);
- path.arcTo(SkVector(top_left_corner_radius, top_left_corner_radius), 0,
-           SkPathBuilder::kSmall_ArcSize, SkPathDirection::kCW,
-           SkPoint(tab_left + top_left_corner_radius, tab_top));
+ // zwire HUD: diagonal left side up to the inset top-left corner.
+ path.lineTo(tab_left + hud_slope, tab_top);
```

The slope is  $(\text{tab\_bottom} - \text{tab\_top}) * 0.28$ , applied on each side, so the tab is a trapezoid whose top edge is inset from the bottom. The patch also nudges the favicon and title 8px to the right so they sit clear of the hard trapezoid’s sloped left edge rather than crowding into it. This is a real Views-layer restyle, not a color change — exactly the kind of thing the runtime path cannot do.

**0002 — UI colors** (`chrome/browser/ui/color/chrome_color_mixer.cc`, in `AddChromeColorMixer`, before the high-contrast early-return). Forces the cyberpunk palette onto frame, toolbar, toolbar text and icons, tab background and foreground, and omnibox background and border, and recolors the under-toolbar separator neon cyan, before the high-contrast early-return. The patch has since grown to seed the eight HUD schemes as Sys-token palettes and to end with a dead-last `AddZwireHudFinalMixer` that re-asserts the tab colors so the Material/native tab-strip mixers can’t leave the active tab a scheme-independent default. This is the patch that reaches everything the theme extension cannot, and the one the unloaded `theme/` extension would fight if it were loaded. Two later refinements live in the same patch. First, the native chrome now follows **light mode**: the mixer reads the `$ZWIRE_STATE/hud-light` projection the host writes when `[theme.ui].light` flips (Chapter 8), swaps the dark scheme values for the light variant, and repaints at startup so the choice applies on launch rather than only after a toggle. Second, the **focus ring follows the scheme** — the patch overrides Chromium’s stock GM3 red focus ring with the active scheme’s accent, so the ring matches the HUD instead of clashing with it.

**0003 — UI font.** The target is `ui/base/resource/resource_bundle.cc`, in the `InitDefaultFontList` method. It sets `gfx::FontList::SetDefaultFontDescription("Share Tech Mono, Monaco, 13px")` so all native chrome UI text renders in the HUD mono face, with Monaco as a guaranteed-present fallback before Share Tech Mono is installed.

**0004 — Toolbar neon border** (`chrome/browser/ui/views/toolbar/toolbar_view.cc`, in `Init`, the `kNormal` branch). Adds a 2px cyan bottom border via `views::CreateSolidSidedBorder(..., kColorToolbarContentAreaSeparator)`; the separator color itself is set neon cyan back in 0002, so the two patches together make the under-toolbar line glow.

**0005 — Omnibox field.** The target is `chrome/browser/ui/views/location_bar/location_bar_view.cc`, in the `ComputeBorderRadius` method. It clamps the field radius to 2px so the omnibox is the

sharp HUD field instead of the rounded pill; its background and border colors come from 0002.

**0006 — Product strings** (`chrome/app/theme/chromium/BRANDING`). Rewrites the product names `Chromium` → `zwire`, the company to `MenkeTechnologies`, and the bundle id to `com.menketechologies.zwire`. This is the file the unbranded (`is_chrome_branded=false`) build reads, which is why the fork's `package.sh` looks for a `zwire.app` and only falls back to `Chromium.app`.

**0007 — DevTools theme** (three DevTools front-end files: `design_system_tokens.css` appended, `main-meta.ts` for the `ui-theme` setting, and `ThemeSupport.ts` for `applyTheme`). Adds the eight HUD schemes to the DevTools Theme dropdown, default `zbhud-cyberpunk`. `ThemeSupport` maps `zbhud-<scheme>` onto the `dark` base and stamps `[data-zbhud=<scheme>]` on `<html>`; the appended CSS maps the `--sys-color-*` tokens onto per-scheme `--zb-*` key colors. Without `data-zbhud`, stock Light/Dark/Auto are untouched — the patch adds schemes rather than replacing the existing ones.

**0008 — Allow framing** (`content/browser/renderer_host/ancestor_throttle.cc`). Bypasses `X-Frame-Options` and frame-ancestors CSP so the tmux overlay (the `ZGui.tmux` overlay) can iframe any site into its N-pane tiling. This patch *replaces* an earlier `0008-tmux-split-prefix` patch, whose native 2-pane split was superseded by the JS overlay — the old patch consumed the `Ctrl-b` prefix and the next key before the page saw them (the “C-b %%” double-tap bug), so the native split was removed and the real N-pane tiling moved entirely into the content script, which just needs framing allowed.

**0009 — Ext-command focus** (`chrome/browser/ui/views/extensions/extension_keybinding_registry_views.cc`). When an extension command fires while a chrome UI surface (the omnibox or search bar) has focus, this hands focus to the page web contents so the `⌘K` palette overlay is typeable from the omnibox — closing the same focus gap the new-tab page solves in JavaScript, but at the engine level for ordinary pages.

**0010 — Force zwire new-tab** (`chrome/browser/search/search.cc`, in `HandleNewTabURLRewrite`). When the zwire new-tab extension is loaded, `chrome://newtab` always resolves to it instead of the default Google NTP, so the new tab is always the HUD — even the startup tab. This is the engine-level partner to the extension's own `chrome_url_overrides.newtab`: the override alone loses races on the very first tab, so the rewrite pins it.

**0011 — Pinned extensions always show** (`chrome/browser/ui/views/toolbar/toolbar_view.cc`). Holds the extensions container at its preferred width so pinned actions never collapse into the puzzle overflow when an extension is re-pinned at runtime — the zwire toolbar keeps its pinned tools visible instead of hiding them behind the generic extensions menu.

**0012 — Allow frame-src** (`content/browser/renderer_host/navigation_request.cc`). A host page's `frame-src` / `child-src` CSP never blocks a sub-frame navigation, so the tmux overlay can embed any site into a pane even when the top page's CSP would forbid it. This complements 0008: 0008 bypasses the *framed* site's frame-ancestors defense, and 0012 bypasses the *framing* page's frame-src restriction — both directions of the CSP handshake have to yield for arbitrary panes to work.

**0013 — Restore last session** (`chrome/browser/prefs/session_startup_pref.cc`). Sets the default startup preference to restore the last session, so tabs auto-restore on launch rather than opening a single fresh new-tab.

**0014 — No session-crashed bubble** (`chrome/browser/ui/views/session_crashed_bubble_view.cc`). Never shows the “Restore pages?” session-crashed bubble — the restore-last-session default (0013) already brings the tabs back, so the prompt is redundant noise the HUD suppresses.

**0015 — No navigation block** (`extensions/browser/extension_navigation_throttle.cc`). Never blocks a `chrome-extension://` navigation to a not-yet-registered extension, so the new-tab override page always loads at startup even in the window before its extension has finished registering — the engine-side race fix that makes 0010's forced new-tab reliable on the very first paint.

**0016 — Ext private-API allowlist** (`chrome/common/extensions/api/_permission_features.json`). Allowlists the `hud-internal` extension (by hash) for `developerPrivate` — widening the API's `extension_`

`types` to include `"extension"` — and `settingsPrivate`, so the HUD's own Extensions and Settings pages can drive `chrome.developerPrivate` and `chrome.settingsPrivate` the way the built-in `chrome://extensions` and `chrome://settings` do. Without it the reimplemented pages have no engine to talk to.

**0017 — No hold-to-quit** (`chrome/browser/app_controller_mac.mm`, in `runConfirmQuitPanel`). Forces the hold-⌘Q-to-quit gesture *off* — immediate quit — regardless of the `kConfirmToQuitEnabled` pref, which a synced profile can otherwise flip back on. A daily driver quits when you press quit.

**0018 — Null policy-container crash fix** (`extensions/browser/extension_function_dispatcher.cc`). Gates the `is_sandboxed` UMA probe on `RenderFrameHost::IsActive()` so an extension-function IPC arriving from a non-active frame (with a null `policy_container_host_`) cannot reach the `active_sandbox_flags()` `NOTREACHED`. This fixes a real SIGTRAP: a content script calling a `chrome.*` API on `chromewebstore.google.com` mid-navigation would crash the browser without it.

**0019 — Web Store scripting** (`chrome/common/extensions/chrome_extensions_client.cc`, in `IsScriptableURL`). Drops the gallery special-case (`kCannotScriptGallery`) so extension content scripts and `tabs.executeScript` run on the Chrome Web Store domains like any other page — store-privileged JavaScript bindings stay gated by `IsWebstoreOrigin` in `permissions_data.cc`, so this widens *scripting*, not *privilege*.

**0020 — Native menu colors** (`chrome/browser/ui/color/chrome_color_mixer.cc`, in `AddZwireHudFinalMixer`). Binds the `ui::kColorMenu*` family — the app/three-dot menu, context menus, and dropdowns — to the HUD Sys tokens (`kColorSysBaseContainer` and friends) so native Views menus follow the scheme and the light/dark toggle instead of the `NativeTheme` default that left light text on a light background. References, not literals: it depends on 0002's Sys-token block.

**0021 — Native primitive colors** (same file and mixer). Generalizes 0020 to the whole native surface. HUD light mode recolors the Sys tokens but never flips `NativeTheme`, so `core_default_color_mixer` still builds every native-Views *primitive* (`kColorPrimary/Secondary`, the background and midground endpoints) from the OS color mode — which inverts dialogs, bubbles, textfields, and tooltips when the HUD and OS modes disagree. Rebinding those primitives to the HUD-mode-aware Sys tokens makes the entire native tree track HUD light/dark + scheme in one place.

**0022 — Output audio EQ** (`services/audio/output_controller.{cc,h}` plus `chrome/browser/chrome_content_browser_client.cc`). The first of three patches that give zwire a **browser-wide equalizer**. A preamp + RBJ-biquad cascade (`ZwireAudioEq`) is applied in `OutputController::OnMoreData()` — the audio service's universal chokepoint that every output stream passes through before the OS device — so all tab audio (media element, MSE/YouTube, Web Audio, WebRTC) is equalized, not just the subset the renderer mixer sees. The chain is preamp → EQ cascade → gain → `tanh` soft-clip drive per channel, then an equal-power pan / mono-fold across the stereo pair. It is always on: an empty config is unity gain (audio still routed through it), and only *"off"* bypasses. Because the re-signed macOS audio utility is sandboxed and does not inherit the environment, `ChromeContentBrowserClient::AppendExtraCommandLineSwitches` forwards the config as the `--zwire-audio-eq` switch (switches survive the sandbox; env does not).

**0023 — Audio tab-capture** (`chrome/browser/extensions/api/tab_capture/tab_capture_api.cc` plus `chrome/common/extensions/api/_api_features.json`). Lets the HUD Audio page pull a playing tab's *real* output into its analyzer: it exposes `tabCapture` to `hud-internal` without the manifest permission (a second allowlisted feature variant, mirroring 0016) and defaults the allowlisted extension id so `getMediaStreamId` bypasses the `activeTab` picker and prompt. The grant is scoped to that one id.

**0024 — Live audio config + meters** (`services/audio/public/mojom/audio_service.mojom`, `services/audio/service.{cc,h}`, `content/browser/audio/audio_service.cc`). Makes the 0022 EQ **live** with nothing open and no relaunch. The audio sandbox cannot read the config file, so the unsandboxed browser process polls it off-thread and, on change, pushes the spec over a new `mojom.AudioService::SetZwireEqConfig`, which atomically swaps the process-global config the cascade reads each block. The same patch carries the **meters** back-channel: the service computes a compact post-DSP

frame (a Goertzel spectrum, peak/RMS per channel, phase correlation, a decimated stereo scope) of the real output and pumps it — on a dedicated ThreadPool sequence, off the UI thread so trackpad gestures cannot starve it — to `$ZWIRE_STATE/meters`, which the HUD Audio page reads over the native host. Because the page reads the engine's true output rather than a `tabCapture` of the source, watching the meters never mutes or drops the audio.

**0025 — Native custom palette** (`chrome/browser/ui/color/chrome_color_mixer.cc`, again in `AddZwireHudFinalMixer`). This one closes the last gap between the HUD and the native chrome: **custom** schemes. Patch 0002 paints the native surface by matching the active scheme's *name* against a baked eight-entry table (`kHudSchemes`), which works for the eight built-ins and fails silently for anything else — a user-authored scheme is named `custom-N`, that name is in no table, and the native chrome stays stuck on whichever built-in it painted last while the extension surface follows the new colors. 0025 removes the name lookup from the decision. The mixer reads a `hud-palette` base-color projection — a plain text file of `--var=#rrggbb` lines that `zwire-host` writes out of `[theme.palette]` whenever the scheme changes, the same projection mechanism Chapter 8's sync bus uses for `hud-light` — and, when the file is present and complete, paints tab labels and accent, toolbar, omnibox, and frame from those **resolved** colors instead of the table. A name is no longer evidence of anything; the colors are. Two implementation details carry the change: `ActiveHudScheme()` now returns by value (call sites bind a `const&`) because the palette can be rebuilt under them, and `ReadHudPaletteFile()` parses under a lock and repaints on change through the existing `~/.zwire` directory watcher, so a scheme switch repaints the native chrome without a relaunch. It depends on 0002 — it replaces that patch's color *source*, not its wiring — and it is the reason the scheme picker no longer needs its entries to exist in a C++ table to be usable.

**0026 — Reading-list toast** (`chrome/browser/ui/toasts/toast_service.cc`, in the `ToastId::kAddedToReadingList` registration). A behavior patch that exists because the surface it retargets is native and no extension API can reach it. When a page is added to the reading list, Chromium raises a toast whose Open button reveals the read-later *side panel* — the same `chrome.readingList` entries, rendered in unthemed Chromium chrome, next to a HUD that already has its own reading-list page. The patch replaces the action callback's side-panel call with `ShowSingletonTab(window, GURL("chrome-extension://.../pages/readinglist.html"))`: singleton, so repeated Opens re-focus the one HUD tab rather than stacking duplicates, and gated on `!profile->IsOffTheRecord()` so incognito — where the HUD extension may not be enabled at all — keeps the native panel. Only the UI changes; both surfaces read the same store.

**0027 — Tab-strip control buttons** (`chrome_color_mixer.cc`, again in `AddZwireHudFinalMixer`). The last of the “always red” family, one layer out from the tab wells 0002 fixed. `TabStripControlButton` — the new-tab `+`, tab search, the combo and flat-edge buttons — binds its hover ink drop to `kColorTabStripControlButtonInkDrop`, which `AddMaterialTabStripColorMixer` points at `ui::kColorSysStateHeaderHover`, and *that* resolves to `kColorRefPrimary30/...80` straight off the Material reference palette built from the profile's seed color. Nothing in the HUD pipeline rebinds the Ref family, so the seed showed through as a red hover fill on `+`. The idle state had the mirror-image gap: the new-tab-button background, foreground, and focus-ring recipes live in `AddTabStripColorMixer` and *reference the tab ids*, but a `ColorMixer` recipe resolves its inputs against earlier mixers only (`ColorMixer::GetInputColor` walks `previous_mixer_`), so those references resolve to the pre-HUD tab colors and never see the final mixer's overrides. The patch binds the whole family — ink drop and ripple (the alpha *is* the ink-drop opacity, so upstream's 0.14 press ratio is kept verbatim), the plain and CR background/foreground id sets, the tab-search foreground, the focus ring, and the combo-button separator — directly to the scheme. It depends on 0002 and 0025, because it reads the same `ActiveHudScheme()` locals.

## The rebase cost

The rebase story is the fork's ongoing cost and the docs do not hide it: the radius-calc, color-mixer, `Init`, and `ComputeBorderRadius` anchors shift between Chromium milestones, so most releases you re-anchor most patches. The procedure is in `fork/patches/README.md` — bump `CHROMIUM_VERSION`, reverse the series off the old tree, fetch the new tag, re-apply and fix any `.rej`, and re-

build. You can re-verify an anchor without a full checkout by reading the target file at the new tag from [raw.githubusercontent.com/chromium/chromium/<TAG>/<path>](https://raw.githubusercontent.com/chromium/chromium/<TAG>/<path>) and `git apply --check`-ing against it. That is the solo-fork commitment, and it is the price of putting the native chrome in the HUD.

## The build config, and the series discipline

`fork/args.gn` is the whole build configuration, and every knob is a deliberate trade between build quality and build time on a single workstation:

```
is_chrome_branded      = false    # ships as unbranded Chromium, not Google Chrome
is_official_build      = true      # full release-quality optimization
proprietary_codecs     = true      # H.264 / AAC / MPEG in <video>/MSE
ffmpeg_branding        = "Chrome" # bundle the ffmpeg build that carries them
enable_platform_hevc   = true      # allow HEVC where the OS provides it
enable_platform_hevc_decoding = true
is_debug               = false
is_component_build     = false
symbol_level           = 0         # no debug symbols -> much faster link, smaller
blink_symbol_level     = 0
chrome_pgo_phase       = 0         # skip profile-guided optimization (a large time sink)
enable_nacl            = false     # drop legacy Native Client
use_thin_lto           = false     # ThinLTO is slow; off for iteration builds
target_cpu             = "arm64"   # set to your host: "arm64" | "x64"
dcheck_always_on       = false
```

The first two lines are the identity — unbranded so there is no Google logo and the `BRANDING` strings of patch 0006 take effect, official so the result is a real release build; the last of the trailing block are the knobs that turn a multi-hour Chromium compile into merely a long one by dropping symbols, PGO, NaCl, and ThinLTO.

The codec block is the one place the fork's *behavior* differs from a stock Chromium build. Stock Chromium ships `ffmpeg_branding="Chromium"` — open codecs only (VP8/VP9, AV1, Opus, Vorbis) and no H.264 or AAC — so an H.264 stream that plays in Google Chrome plays in the zwire fork's stock base only as silence and a black frame. Setting `proprietary_codecs=true` with `ffmpeg_branding="Chrome"` bundles the ffmpeg build that carries H.264/AAC/MPEG, so the fork decodes what Chrome does: Synology SurveillanceStation web live-view, most IP cameras, and ordinary MP4/AAC media. HEVC (H.265) is *not* software-decoded even here — Chromium routes it to the platform hardware decoder (VideoToolbox on macOS), which the two `enable_platform_hevc[_decoding]` flags wire up. This is a fork-only capability by construction: the runtime rebrand runs the prebuilt open-codec snapshot and cannot add codecs without recompiling.

The patch series has its own discipline, enforced in CI. `fork/patches/series` is the ordered list `apply-patches.sh` reads, and `scripts/check-patch-series.sh` asserts a strict one-to-one correspondence between the entries in that file and the `*.patch` files on disk — a patch on disk but missing from `series` would silently never apply, and a `series` entry with no file would fail to apply, and the check catches both drifts before they ship. Each series entry carries an inline note recording that it was verified apply-clean against `150.0.7871.46`, and one entry is instructive by its absence: an earlier `0008-tmux-split-prefix.patch` is commented out with an explanation, because its native two-pane split “consumed the prefix + the next key before the page saw them (the ‘C-b %%’ double-tap)” and was superseded by the JS overlay — its `0008` number reused by the framing patch. The series file is not just a manifest; it is a small changelog of what the fork tried and backed out.

## The economics of the rebase

The runtime rebrand tracks upstream Chromium for free; the fork does not, and the difference is the single most important thing to understand before choosing the compiled path. A stock snapshot is a security fix one `fetch-base.sh` away because it is not a fork of anything — it is the upstream artifact, re-signed and

renamed. The fork, by contrast, is twenty-seven patches pinned to one tag, and every time Chromium moves, those patches must be carried forward across a codebase that changes under them without warning. That carrying-forward is the recurring cost, and it is worth being precise about where it falls, because it does not fall evenly across the series.

The patches sort into three risk tiers by *what* they touch. The cheapest are the string and resource patches — the `BRANDING` file, the product name, the icon — which live in files that change slowly and rarely conflict; a rebase moves them with a trivial context shift or none at all. The middle tier is the CSS-and-color work: the trapezoid tab shapes, the neon under-toolbar line, the Share Tech Mono UI font, the frame color wired to the scheme. These live in the Views layer, which Google refactors on its own schedule, so they conflict *sometimes* — a renamed method here, a moved constant there — but the conflicts are mechanical, because the patch's intent is legible from its diff and the new upstream shape is usually obvious. The expensive tier is the behavioral C++: the `is_sandboxed` gate in the function dispatcher, the three audio-service patches that reach into `OutputController::OnMoreData` and add a new `mojom` method, the codec branding. These touch code that carries real invariants, and a rebase there is not “fix the context” but “re-understand the surrounding code well enough to know the patch is still correct” — because upstream may have changed the very assumption the patch depended on. The audio EQ that hooks the universal mixing callback is the canonical example: if Chromium reworks how the audio service routes output, the patch does not fail to apply — it applies and is *wrong*, which is the worst outcome a rebase can produce.

This asymmetry is why the fork is framed throughout the book as the tier for whoever will *pay the build-and-rebase cost*, not merely the build cost. The build is a one-time multi-hour compile made merely long by the `symbol_level=0` and no-PGO knobs; the rebase is a recurring tax that scales with how far the pin has fallen behind. The disciplines that make it survivable are exactly the ones this chapter described: the `series` file as a manifest CI keeps honest, the inline apply-clean notes that record which tag each patch was last verified against, and the willingness — visible in the commented-out `0008` — to back a patch out rather than carry a fragile one forward. The honest reading is that the fork is a maintained thing, not a finished one, and the runtime rebrand exists precisely so that the daily driver never has to be. The line between the two tiers is not a wart; it is the boundary between what upstream will keep working for you and what you have chosen to keep working yourself.

# Chapter 22 – Dev Loop, Testing, and CI

zwire is a browser you assemble, not compile, so its “build” tooling is a set of small shell scripts that fetch, rebrand, cache-bust, and reset the base, plus a JS syntax gate and a CI that lints every script and validates every JSON. This chapter is that loop.

## The script set

The `scripts/` directory is a family of dev-loop commands, all sharing one cyberpunk banner wrapper (`cyberpunk.sh`, which just sets ANSI color variables and a banner from `APP_TITLE/APP_SUB`). Reading them as a set:

```
dev.sh      launch the browser from the checkout (extensions unpacked,
            foreground; edits to content scripts hot-reload)
build.sh    "build" == make sure a rebranded base bundle exists + is current
            (fetch the snapshot if missing, then rebrand) – no compile step
clean.sh    remove the /Applications wrapper + staged dist/, keep base + profile
rebuild.sh  clean + build
bust.sh     re-seal + re-register the bundle with LaunchServices (Chapter 5)
nuke.sh     wipe base + wrapper, re-fetch from scratch, rebrand, reinstall
            (preserves the profile)
test.sh     a JS syntax gate over the front-end + each extension's own tests
```

The design point is that none of these compile the browser — the base is a prebuilt snapshot — so “build” and “rebuild” are really “make sure a current, rebranded base exists,” and the destructive ones are careful about what they keep: `clean` and `nuke` never touch your profile, and `clean` does not even touch the 325 MB base (that is `nuke`’s job). The npm `scripts` in `package.json` alias these (`pnpm dev`, `pnpm build`, `pnpm nuke`, and so on) for anyone who reaches for the package manager.

## Testing

`scripts/test.sh` is a JS syntax gate over the repo’s own front-end (the new-tab page, the theme, the root `v*.mjs` scripts) plus a delegation to each bundled extension’s own test script. There is no Rust or compile step at the browser level, because the base is a prebuilt snapshot; the tests that matter for behavior are the CDP verification probes of Chapter 20, which drive a running browser. The syntax gate catches the cheap failures (a broken content script would silently not load in Chromium) before they reach a launch.

## CI

The GitHub Actions `ci.yml` runs three checks on every push and PR, on a plain Ubuntu runner, with the `zpwchrome` submodule deliberately *not* checked out (nothing CI checks needs it):

- `shellcheck` over `bin/zwire`, `scripts/*.sh`, and `fork/*.sh` at warning level — every shell script the project ships is linted.

- **JSON validation** over every tracked `*.json` outside the `zpwchrome` submodule, discovered with `git ls-files` (never hardcoded), each parsed with `python3 -c json.load`.
- **patch-series consistency** via `scripts/check-patch-series.sh`, which asserts the `fork/patches/series` list and the patch files on disk agree.

A second workflow, `pages.yml`, publishes `docs/` to GitHub Pages — the site the README's “docs · online” badge points at — on any push that touches `docs/`, with a single-deployment concurrency guard so an in-flight publish is never cancelled. The CI is intentionally lightweight: lint, validate, check consistency. It does not build the browser (there is nothing to build) or compile the fork (that needs the 100 GB checkout); those are local, deliberate operations.

## Inside the script family

The dev-loop scripts share one banner library (`cyberpunk.sh`, sourced for the ANSI banner keyed off `APP_TITLE`) and each is careful about what it keeps. `dev.sh` runs the browser from the checkout in the foreground — its comment notes that edits to the content scripts hot-reload while a `background.js` change needs the reload glyph on `chrome://extensions`. `build.sh` states outright that “‘build’ == ‘make sure a rebranded base bundle exists and is current’”: it fetches the snapshot if missing and runs the rebrand on Darwin only, with no compile step because the base is prebuilt. `nuke.sh` is the heavy reset — it removes the `/Applications` wrapper and the 325 MB base, re-fetches, rebrands, and reinstalls, and its comment is emphatic that “your profile ... is preserved.” `bust.sh` is the macOS-only icon-cache buster and exits early on any other platform.

`test.sh` is a JS syntax gate, not a behavior suite, and its comment says why: “there is no Rust/compile step at the browser level — the base is a prebuilt snapshot.” It runs `node --check` over the repo's own front-end (the new-tab page, the theme, the root scripts), deliberately excluding `node_modules` and the vendored `lib/zgui-core`, and then delegates to each bundled extension's own `pnpm test`. A broken content script would fail silently in Chromium rather than throw, so catching the parse errors cheaply, before a launch, is the point; the behavioral proofs are the CDP probes of Chapter 20, which drive a running browser rather than lint a file.

## What the syntax gate catches, and what it structurally cannot

The test script's own comment explains why it is a syntax gate rather than a behavior suite, and it is worth taking the argument seriously rather than reading it as an apology. In an ordinary application a parse error is loud: the program refuses to start. In a browser extension it is not. A content script with a syntax error does not throw into anything the user sees; Chromium declines to inject it and the feature is simply absent, on every page, with no error and no visible difference from a feature that was never written. That failure mode — silent, total, and invisible — is exactly the one a cheap parse check eliminates completely, which is why the cheapest possible gate is disproportionately valuable here.

The gate's scope is drawn to match that reasoning. It parses the repository's own front-end and the root probe scripts, deliberately excluding vendored trees and the shared toolkit submodule, because a submodule's syntax is the submodule's CI's problem and re-checking it here would only produce failures nobody in this repository can fix. Then it delegates to each bundled extension's own test script, which is the same reuse-not-duplicate discipline the rest of the project follows: the extension knows how to test itself.

What it cannot catch is everything above the parser. A script that parses perfectly but registers a listener on the wrong event, writes a storage key nobody reads, or claims a key it should have left alone will pass the gate and fail in the browser. There is no type checking, no linting of the JavaScript, and no execution. This is a deliberate floor rather than an aspiration: the behavioral proofs live in the verification probes, which drive a running browser, and the gate exists to make sure the code they are driving actually loaded.

## What CI checks, and the one thing it protects that nothing else does

The three CI checks look modest and two of them are: linting every shell script the project ships and parsing every tracked JSON file are hygiene, valuable mostly because this project *is* shell scripts and JSON manifests to an unusual degree. A malformed manifest is another silent failure — an extension that does not load, with the developer banner still appearing because the other two did — so parsing every JSON file is the manifest-level analogue of the syntax gate.

The third check is different in kind. The patch-series consistency check asserts a one-to-one correspondence between the entries in the fork's series file and the patch files on disk, and it is the only automated statement anyone makes about the fork without a hundred-gigabyte checkout. Both directions of the drift it catches are genuinely dangerous and neither is visible by inspection. A patch file on disk that is missing from the series never applies, so a fix you believe is in the build is not; the symptom is a fork that behaves like an older one for no discoverable reason. A series entry with no file fails at apply time, which is at least loud, but it fails after the long fetch step rather than before it. Catching both on every push, on a runner that never touches Chromium, is a large amount of value for a script that compares two lists.

It is worth naming what CI deliberately does not do, since the omissions are principled. It does not check out the power-tool submodule, because nothing it checks needs it — which also means CI runs on a clone shaped exactly like the reduced one a user gets without recursive initialization, incidentally exercising the graceful-skip path. It does not build the browser, because there is nothing to build. And it does not compile the fork, because that requires the full source tree and hours of compute; the fork's CI story stops at “the patches are consistent and were verified apply-clean against the pinned tag,” and the chapter on the fork is explicit that this is a weaker claim than “compiles.”

## The dev loop's real shape

Read as a workflow rather than a list, the scripts form a ladder of increasingly expensive resets, and knowing which rung to reach for is most of the value. Running the browser from the checkout is the inner loop, and its important property is that content script edits hot-reload while a background worker change needs an explicit reload on the extensions page — a distinction that costs people minutes until they know it, because a worker edit that appears not to have taken is indistinguishable from a worker edit that was wrong.

Above that, “build” means only that a rebranded base exists and is current, so it is cheap and safe to run at any time. “Clean” removes the installed wrapper and the staged distribution while leaving the base and the profile alone, which makes it the right first move when the *packaging* looks wrong. The cache-buster addresses the one macOS failure that no amount of correct code fixes — a stale LaunchServices icon — and does nothing anywhere else. And the full reset re-fetches the base from scratch, which is the expensive rung and the one that costs a large download, while still preserving the profile. That preservation is the load-bearing property of the whole ladder: every rung, including the most destructive, leaves the user's bookmarks, history, sessions, and extension state intact, which is what makes reaching for a reset a reasonable troubleshooting step rather than a last resort.

# Chapter 23 — Packaging and Self-Contained Installs

`scripts/install.sh` is the developer install — a PATH symlink to the checkout. But `zwire` also ships three *self-contained* installers that bundle everything the browser needs into one location, so you can delete the repo and the base and the app still runs. This chapter is those three, and the one substitution they all make.

## The developer install vs the self-contained install

`scripts/install.sh` (Chapter 5) is the light path: it fetches the base, symlinks `bin/zwire` into `~/.local/bin`, and rebrands on macOS. The launcher it links still runs *from the checkout* — delete the repo and `zwire` breaks. That is right for development, where the checkout is where you are editing.

The self-contained installers are the opposite: they copy everything into one owned location so the checkout becomes disposable. On macOS, `localinstall.sh` builds `/Applications/zwire.app` containing the Chromium base (`Contents/Resources/browser/`), the three extensions (`Contents/Resources/ext/`), the native host (`Contents/Resources/native/`), and a bundle-relative launcher (`Contents/MacOS/zwire`). Delete the repo and `~/.zwire/base` afterward and the app still runs, with no system dependencies. Only the per-user profile (`~/.zwire/profile`) and the per-user extension copy (`~/.zwire/ext`, staged from the bundle at launch) live outside it.

The Linux and Windows installers mirror this shape:

- `localinstall-linux.sh` assembles everything under `~/.local/opt/zwire/` (browser + extensions + host), drops a `zwire` launcher on PATH and a `.desktop` entry for the app menu and icon, and needs no root — a pure user install. Delete the repo afterward and it still runs.
- `localinstall-windows.ps1` assembles everything under `%LOCALAPPDATA%\zwire` (the `Win_x64` snapshot, the extensions, `zwire-host.exe`), and wires the native host through the *registry* — because Windows, unlike macOS and Linux, does not read native-messaging host manifests from the profile directory. It drops a `zwire.cmd` launcher.

## The native host in the packaged install

The one component that cannot ship as source is the native host, because it is a compiled binary. `scripts/setup-native-host.sh` builds `zwire-host` with `cargo build --release` (its own submodule, `extensions/hud-internal/native/zwire-host`) and writes the native-messaging manifest pointing at it; the launcher calls that on first run. Because the binary is a single static Rust artifact with no runtime dependencies, it drops straight into the app bundle — there is nothing for the deployed machine to provide, no interpreter and no `psutil` to be missing. On the scheme-bridge / sysinfo / PTY path `zwire`'s browser actually drives, the wire format is exactly Chapter 15's: the 4-byte-length framing, the eight scheme names, the atomic tmp-and-rename writes to the HUD scheme/ui files, the base64 `output/exit` PTY frames. On Windows the manifest becomes a `.exe` path and lives in the registry; on macOS and Linux it is the profile-relative JSON of Chapter 15.

## What the host really is: a full local agent

zwire's browser uses only a small corner of `zwire-host`. The scheme bridge, the sysinfo stream, and the PTY relay are three of roughly fifty commands across fifteen modules, reachable over two transports — the host is a local-machine agent that the HUD happens to be one client of.

The extra surface, module by module: a namespaced key-value store (`kv_get/kv_set/kv_merge/kv_del/kv_keys`, each an atomic JSON file under `~/.<app>/kv/`); a filesystem API (`fs_read/fs_write/fs_append/fs_list/fs_walk/fs_stat/fs_mkdir/fs_rm`, with a 64-MiB read cap and a 50k-entry bounded recursive walk); file observers (`fs_watch` and a rotation-aware `fs_tail`); an arbitrary command runner (`exec`, returning base64 stdout/stderr and an exit code); background jobs (`job_start/job_poll/job_list/job_result`, which fire a desktop notification on completion); process tools (`ps/kill/which`); a process- global publish/subscribe bus (`sub/unsub/pub`); OS integration (`open/clipboard_get/clipboard_set/notify/hostinfo`); and a TCP peer mesh (`auth/peer_hello/peer_connect/peers/remotes`) that federates the bus and runs a request on another host. On the shared capabilities it is also richer: PTY sessions are multiplexed by id (one connection can drive many terminals) and take a shell, args, cwd, and env; the sysinfo interval is configurable and there are one-shot (`sysinfo_once`) and explicit `sysinfo_stop` variants, so a client tunes the cadence or halts the stream without closing the port. A `hello` reply advertises exactly which of these capabilities were compiled in, so a client can feature-test rather than guess.

The second transport is the reason the Rust host exists as its own project rather than a script. Besides Chrome native-messaging over stdio, it runs a line-delimited JSON daemon over a Unix-domain socket (a named pipe on Windows), created owner-only under an owner-only directory precisely because it exposes `exec`, the filesystem, and PTYs — a local agent any app on the machine can call, not only the browser. That is the whole point of the README's framing: one self-contained binary means the consuming bundle has “zero runtime dependencies,” and an embedder that only wants the light capabilities can compile it without the system-stats and PTY features entirely. “No runtime dependencies” is about the deployed artifact needing no Python and no `psutil`, not about the crate graph — it builds on `serde_json`, `base64`, and the optional `sysinfo` and `portable-pty`.

## Two sysinfo details worth pinning

Two fields of the sysinfo frame carry more care than the rest, because a naive implementation would drop or mangle them. First, the disk-I/O rate (`io: {r, w}`, read and write bytes per second): the host keeps a `Disks` handle alive across ticks and reads `Disk::usage()` — bytes since the last refresh — summed over every disk and divided by the elapsed interval, so the streaming rate is real rather than a lifetime total. (The one-shot `sysinfo_once` has only a single instant to work with, so it double-refreshes to report a near-zero delta rather than a lifetime total mislabelled as a rate — an honest “no rate from one sample.”) Second, battery, which has no single cross-platform source: the host reads it natively on all three platforms — `pmset` on macOS, `/sys/class/power_supply` on Linux (capacity plus a charging/`Mains`-online check), and `GetSystemPowerStatus` on Windows — so the statusbar renders identically everywhere. The only remaining absence is honest and physical: machines with no battery — desktops and VMs — omit the segment, exactly as they should.

The whole packaging story is one idea: make the app own everything it needs so the checkout is disposable, and the native host is part of that — a single self-contained Rust binary bundled into the app with nothing for the deployed machine to provide. What the user gets is a normal app — an `.app`, a `~/.local` install, a `%LOCALAPPDATA%` install — that happens to be Chromium wearing the HUD, with that host doing the scheme, stats, and terminal work.

## What the self-contained installer actually assembles

The macOS `localinstall.sh` copies four things into `/Applications/zwire.app`: the whole Chromium base bundle into `Contents/Resources/browser/`, the three extensions into `Contents/Resources/ext/` (via `rsync` excluding `node_modules`, `.git`, `tests`, and `target`), the freshly built `zwire-host` into `Contents/Resources/native/`, and a bundle-relative launcher at `Contents/MacOS/zwire`. That launcher is a quoted heredoc, so every `$var` is resolved at *runtime*, not install time; it defaults its state to `~/Library/Application Support/com.menketechologies.zwire`, resolves the browser executable from the bundle's own `CFBundleExecutable` via `PlistBuddy`, and passes the same nine flags `bin/zwire` does. The Info.plist it writes sets the bundle id to `com.menketechologies.zwire`, and the whole app is ad-hoc signed and LaunchServices-registered at the end.

The one subtlety that repays reading is the per-user extension staging, and its comment spells out the failure it avoids: Chromium writes each unpacked extension's compiled declarativeNetRequest ruleset and content hashes into `<ext>/_metadata/`, so the browser needs *write* access to the extension directory, but a shared `/Applications` bundle has a single owner — so a second user would fail to load the extensions with an “Internal error while parsing rules.” The launcher therefore `rsyncs` the bundled extensions into a per-user `$ZWIRE_STATE/ext` (excluding `_metadata`) at every launch and loads from there. The Linux installer mirrors the shape into `~/local/opt/zwire` with a `.desktop` entry, and its launcher adds a tenth flag the macOS one omits — `--no-sandbox`. The Windows installer assembles under `%LOCALAPPDATA%\zwire` and, because Windows does not read host manifests from the profile directory, wires the native host through the registry under both the Chromium and Chrome hives so it resolves regardless of the snapshot's product name.

## The build artifact that is not in the repository

One thing the installer stages is not in the checkout at all, and the way it went missing is the argument for how it is built now. The Hooks, Commands, and Triggers pages each mount a Monaco editor — the stryke code surface of Chapters 49 and 50 — from `extensions/hud-internal/lib/hooks-editor/`, four files (`hooks-editor.bundle.js`, `hooks-editor.bundle.css`, and two workers) that together define `window.HooksEditor`. That directory is a *gitignored* esbuild artifact, produced from `vendor/zpwr-hooks-editor/src`, and the extension's own `build.sh` used to skip producing it “cleanly when the monaco/esbuild devDeps aren't installed.”

Every mount site is written `if (window.HooksEditor)`. So the skip is invisible: the `<script src>` 404s, the global is never defined, the guard is false, and the pane comes up *empty* — the surrounding page chrome renders normally, nothing is logged, and only a devtools network tab distinguishes “the bundle was never built” from “there is no editor on this page.” A release shipped exactly that.

The fix is that there is nothing left to skip. The script `extensions/hud-internal/scripts/build-hooks-editor.sh` is the single implementation: it installs the `monaco-editor` / `monaco-vim` / `monaco-emacs` devDeps itself when they are absent (they resolve against the *consumer's* `node_modules`, which is why the install must happen in that directory rather than at the repo root), runs the esbuild bundler, and then checks all four artifacts are present and non-empty — because a partial build from an aborted esbuild looks identical to a good one once `rsync` has copied it, and a missing worker kills completion without killing the editor. The repo-root `scripts/build-hooks-editor.sh` is a nine-line forwarder to it, so the four packaging paths — `install.sh`, both `localinstall` scripts, and the extension's own `build.sh` — run one build with one set of checks. Every caller treats a nonzero exit as fatal, and the two Unix installers add a second assertion *after* the copy that the bundle survived into the staged tree, since an `--exclude` that starts matching it or a `lib/` symlink would drop it silently. The Windows installer performs the same sequence in PowerShell, calling `node` directly so it needs no bash. The one requirement this pushes onto a fresh clone is node  $\geq 20$  plus pnpm, which is why `install.sh` now runs the build as its step 1b, immediately after fetching the base.

## Stamping the staged extensions so the browser reloads them

The macOS installer's last act before assembling the bundle is to rewrite four things in the *staged* copies of the extensions, and each rewrite exists to defeat a specific cache.

The first is cosmetic and the oldest: `pages/version.js` hardcodes `ZWIRE_VERSION`, because an extension cannot read the app bundle's `CFBundleVersion` at runtime, so the installer stamps the staged copy from `package.json` on every build and the System page can never disagree with the release it shipped in. (The stamp is a `perl -i -pe` substitution. This is not incidental: an in-place `sed` here is a portability trap — BSD `sed` requires `-i ''` and GNU `sed` rejects it, and on a machine where the GNU build shadows the system one the empty argument is parsed as the script, the real expression becomes a filename, and the stamp silently no-ops. That failure has shipped a release whose System page read one version while its manifest read another.)

The second is the extension manifest's own `version`, and the reason is that Chromium keys extension reload on it. But stamping it from `package.json` alone is not enough, because a *submodule* can ship a behavior change without a release bump, and then two installs produce identical manifest versions. So each staged extension gets a content-derived fourth component: `stamp_content_version` hashes every file the browser loads — excluding `manifest.json` itself, so the stamp cannot feed into the next hash, and excluding the generated worker shims below, so a leftover from a previous install cannot churn the version when nothing real changed — and appends the digest modulo 65536, since a manifest version is at most four dot-separated integers of at most 65535 each. Same files, same version, no needless reload; changed files, new version.

The third is the one that took measurement to find, and it is the reason the second is not sufficient either. Measured against this Chromium (150.0.7871.46) under `--load-extension`: after replacing `background.js` and bumping the manifest version, the browser served the *new* file over `chrome-extension://` — 197,490 bytes, the new handlers present — while the worker it had actually parsed was the *old* cached script at 196,642 bytes, handlers absent, and this survived a full browser restart. Every message kind the new page JS sends and the cached worker never learned then fails with “The message port closed before a response was received.” Service-worker registration is keyed on the worker's **script URL**, not on the extension's version, so `stamp_service_worker` points each staged manifest at a content-keyed shim — `background.sw-<hash>.js`, a single line that `imports` or `importScripts` the real `background.js`, following the manifest's own `type` declaration because getting that wrong breaks the extension outright. New code, new filename, new registration. `background.js` keeps its name, so nothing that references it by URL has to change.

The fourth is the trap the third one opens. The launcher `rsyncs` the staged tree into the per-user `$ZWIRE_STATE/ext` with `--delete`, so the *previous* run's shim file is gone — and a profile still holding a registration for that filename holds a **dangling** one. That worker can never start. Silently. Across restarts. The launcher therefore evicts the profile's Service Worker script cache when a marker changes, and the marker is every staged manifest's version *and* its declared `service_worker` filename, so any shim rename clears the stale registration on the very next launch. (The cache lives under `$PROFILE/Default/Service Worker`, not `$PROFILE/Service Worker`; the launcher deletes both to cover either profile layout.)

Both stamps have to cover every extension, and the bug that proved it is worth recording because the symptom pointed nowhere near the cause. `hud-internal` was originally left out of the content-version stamp while `stamp_service_worker` renamed its shim on every content change. Chromium saw the same extension version, kept the registration pointed at the previous shim URL, and `rsync --delete` had already removed that file — a dangling registration, so `hud-internal`'s worker never started at all. Everything the worker owns died with it, most visibly the browser-level `⌘K open-palette` command: the accelerator stays bound in the profile and swallows the keystroke, but nothing is listening. The palette stopped opening on ordinary web pages and from the omnibox while continuing to work on HUD pages, which bind `⌘K` in page JS and never touch the worker. A keyboard shortcut that works on some surfaces and is inert on others is what a dead service worker looks like from the outside.

## Cutting a macOS release: `release-macos.sh`

Past macOS releases were built and uploaded by hand, so each cut risked drifting from the last; `scripts/release-macos.sh` captures the recipe as a repeatable script that turns a `v<version>` tag into published artifacts. It reuses `localinstall.sh`'s self-contained bundle assembly — the same `/Applications/zwire.app` of the section above — but redirects it to a writable staging dir through a `$ZWIRE_DEST` override (default `/Applications`), so the release build needs no `sudo: localinstall.sh` learned to honor `$ZWIRE_DEST` precisely so CI and this script can assemble the self-contained app into a directory they own. It then packages the staged app two ways and uploads both to the GitHub release for the tag: a `zwire-<v>.pkg` (a `pkgbuild` component, identifier `com.menketotechnologies.zwire`, install-location `/Applications`, forced non-relocatable to match prior pkgs) and a `zwire-<v>-macos.zip` (a `ditto --keepParent` archive with `zwire.app` at its root). The guards are what make it safe to run repeatedly: it refuses to overwrite an existing release, requires the `v<version>` tag (pushing it to origin if missing), and warns on a dirty tree; `--dry-run` builds the artifacts without publishing and `--notes-file` overrides the auto-generated changelog (commit subjects since the previous `v*` tag). Signing is unchanged — the bundle stays ad-hoc signed with no notarization, exactly as every prior release, so first launch still needs a right-click → Open. Artifacts land in `dist/` (gitignored), and the whole thing is wired as the `release-macos package.json` script.

# Chapter 24 – Cross-Platform, Updating, and Operations

The operational chapter: what works on macOS (aarch64/x64) versus Linux (x86\_64) versus Windows, how to roll the base forward, and the day-to-day notes — native messaging setup, the cosmetic developer banner, and re-running the rebrand after an upgrade.

## Platform matrix

The `zwire` launcher itself is portable — it is POSIX shell that reads a base path, builds an extension list, ensures a native host, and `execs` Chromium — and it runs on macOS (aarch64 and x64) and Linux (x86\_64), with the PowerShell installer covering Windows. What is shared and what differs, at a glance:

|                    | macOS            | Linux               | Windows               |
|--------------------|------------------|---------------------|-----------------------|
| snapshot platform  | Mac_Arm / Mac    | Linux_x64           | Win_x64               |
| state dir default  | ~/Library/App... | ~/.config/zwire     | %APPDATA%\zwire       |
| identity rebrand   | bundle patch     | launcher + .desktop | launcher name         |
| native host wiring | profile dir      | profile dir         | registry (both hives) |
| extra launch flag  | –                | --no-sandbox        | –                     |
| self-contained to  | /Applications    | ~/.local/opt        | %LOCALAPPDATA%        |

The parts that differ per platform are three:

- **The base download.** `fetch-base.sh` maps the host to the right snapshot: `Mac_Arm` and `Mac` for the two macOS architectures, `Linux_x64` for Linux, and `Win_x64` for Git-Bash/MSYS on Windows. The launcher and the native host are cross-platform; the base is per-platform because Chromium's snapshots are.
- **The rebrand.** As Chapter 5 established, the in-place Dock/icon rebrand is macOS-only, because only macOS carries app identity as bundle metadata. On Linux the launcher name is the brand, and a `.desktop` entry gives it a menu icon; on Windows the installer places a named launcher. `install.sh` only calls `rebrand-macos.sh` under `Darwin`.
- **The native host install.** On macOS and Linux the HUD host is installed *profile-relative* into `<user-data-dir>/NativeMessagingHosts/`, which Chromium reads on every platform, so there is no per-OS `~/Library` vs `~/.config` split — `setup-native-host.sh` writes the same manifest in the same place. On Windows the manifest is wired through the registry instead, because Windows Chromium does not read host manifests from the profile dir. `bin/zwire` invokes the setup on launch when the manifest is missing or dangling, which is what makes a bare `zwire` self-sufficient in dev on macOS and Linux.

The host binary is `zwire-host`, a cross-platform Rust program built with `cargo build --release` on first need (or when `FORCE=1`) — the sole native host, detailed in Chapter 15. On Windows/WSL the manifest points at a `.exe` because the browser wants a Windows path even though the host source is portable Rust.

## Updating the base and re-applying the rebrand

Rolling the base forward is two commands on Linux and three on macOS:

```

scripts/fetch-base.sh           # latest Chromium snapshot (or a pinned revision)
scripts/fetch-base.sh 1656770  # pin an exact revision instead
scripts/rebrand-macos.sh       # macOS only: re-apply the rebrand after the swap

```

The rebrand must be re-run after every base swap for the mechanical reason Chapter 5 gave: `fetch-base.sh` unzips a fresh, stock, Chromium-signed bundle over the patched one, so the name, the icon, the deleted `CFBundleIconName`, and the ad-hoc re-sign are all gone and must be reapplied. `install.sh` chains the two on macOS so a first install is one command; an in-place upgrade is where you run them separately and must not forget the rebrand, or the Dock reverts to “Chromium.” If the Dock icon lingers even after the rebrand, `scripts/bust.sh` forces the LaunchServices cache to refresh (Chapter 5).

## The runbook notes

Two operational notes carry over from earlier chapters and belong in the runbook:

- **Native messaging setup.** `zwire`’s own HUD host (`com.zwire.hud`) is wired automatically on launch. `zpwrcrome`’s native host — needed for its `pass` integration and segmented downloads — is a separate install documented by `zpwrcrome` itself; `zwire` does not install it for you.
- **The developer-mode banner.** Loading unpacked extensions via `--load-extension` raises Chromium’s developer-extensions notice on startup. It is cosmetic; the extensions run fully. It is expected, not a fault.

That is the whole operational surface: fetch (and pin) the base, rebrand on macOS, let the launcher wire the native host, and expect one cosmetic banner. Everything else — the profile, the extension staging, the scheme file, the terminal PTY — the launcher and the host manage on their own, which is the point of a browser that is mostly a shell script wrapped around a stock engine.

## First run, migration, and the degradation ladder

A first launch on a clean machine does more than open a window, and knowing the order helps when something is missing. The launcher resolves its state directory (the OS app-data location unless `$ZWIRE_STATE` says otherwise), performs the one-time migration of any leftover `~/zwire` into it, and then, if no base is recorded, exits with the exact `run: .../scripts/fetch-base.sh` message — so a fresh clone’s first `zwire` tells you the single missing step rather than failing opaquely. Once the base is present, the launcher ensures the native host by building the Rust binary if its manifest is missing or its target is gone, and only then execs the browser.

The value of that sequence is a *degradation ladder* rather than a set of hard requirements. No base is the only fatal condition. A missing native host is not fatal — the browser launches with the terminal and system stats disabled and one stderr line. A missing submodule (`zpwrcrome`, `zgui-core`) is not fatal — its extension directory has no `manifest.json`, so the launcher skips it and loads what remains. A stale LaunchServices icon is a cosmetic annoyance with a dedicated fix (`bust.sh`), not a functional fault. A base upgrade on macOS silently reverts the rebrand, which is why the update recipe ends with `rebrand-macos.sh` — the one manual step the launcher cannot infer. Read as a whole, the operational model is that the browser tries to run with whatever is present and tells you, tersely and on stderr, only about the one gap that actually stops it: no base. Everything above the base is best-effort, and the launcher’s job is to assemble as much of the HUD as the machine currently has.

## Three platforms, one port, and where the asymmetry actually lives

The platform matrix reads like three ports of the same program, and that framing is misleading in a way worth correcting, because it makes the Windows column look like an unfinished version of the macOS one rather

than a structurally different situation.

The launcher is not ported at all. It is one POSIX shell script that resolves a base path, filters a list of directories, ensures a native host, and `execs` a binary, and every one of those operations is platform-neutral. The native host is one Rust program compiled per target. The extensions are JavaScript loaded by a Chromium that behaves identically everywhere. The overwhelming majority of this book describes code that has no platform-specific behavior whatsoever, and that is a direct consequence of the design premise: a browser that is mostly a stock engine plus a shell script plus web code has almost no surface on which platforms can differ.

The asymmetry is concentrated in exactly three places, and each is a place where the *operating system*, not *zwire*, defines the mechanism.

**Where an application's identity lives.** Chapter 5 established this and it bears restating as a platform statement rather than a macOS one. macOS carries identity as metadata inside a bundle, which is why the rebrand is a bundle patch and why it must be re-applied after a base swap that replaces the bundle. Linux carries identity in how a process was invoked plus a desktop-entry file, which is why the launcher's name *is* the brand and no rebrand step exists. Windows carries it in the launcher the installer places. Three mechanisms, three amounts of work, and the amount of work is proportional to how much the OS insists on knowing about an application before it will display its name — which is a property of the OS, not a measure of how well *zwire* supports it.

**Where the browser looks for a native-host manifest.** Chromium reads host manifests from the profile directory on macOS and Linux, which is what lets `setup-native-host.sh` write one file in one place and have it work on both — and, more importantly, is what makes the host *profile-relative*, so it is wired for *zwire* and invisible to system Chrome. Windows Chromium does not read manifests from the profile directory; the wiring goes through the registry. That single difference propagates: the install step is different, the uninstall step is different, and the failure mode is different — Chapter 36 records that a silent host on Windows is a registry problem rather than a file one, which means the whole diagnostic path diverges from the one a macOS or Linux user follows. It also means the isolation property is weaker in shape: a registry key is a machine-or-user-scoped fact rather than a profile-scoped one, so the neat statement that the host is wired to *zwire*'s profile and nowhere else is a statement about the two Unix-like platforms specifically.

**The sandbox.** The matrix carries one extra launch flag on Linux, and it is the one entry in the table with a security consequence rather than a packaging one. Chapter 25's residual risk section already names it: on Linux the renderer sandbox is off, and the protection is the dedicated profile and the fact that the user chose the pages. It belongs in the platform chapter as well, because it is the clearest case of a platform difference that is not merely mechanical. macOS and Windows run with the engine's own sandbox intact; Linux does not, so the profile boundary is carrying more weight there than elsewhere. A reader deciding which machine to run *zwire* on should know that the containment story is not identical across the three.

What falls out of this is a better way to read the matrix. The rows are not a support-tier ranking. `install.sh` chains the rebrand under Darwin because Darwin is the only platform with a bundle to patch; the Windows installer is PowerShell because Windows is the platform where a shell script is not the native idiom; the Linux column has an extra flag because Linux is where the sandbox needs it. In every case the platform-specific code exists at the exact boundary where an OS-defined mechanism begins, and nowhere else — which is the property that keeps a project this size cross-platform at all. The generalizable rule is that portability is not achieved by abstracting over platform differences but by arranging for almost nothing to touch them, and then writing the small amount that does in each platform's own idiom rather than in a lowest-common-denominator one.

# Chapter 25 – The Trust Boundaries and the Threat Model

Every capability the earlier chapters celebrated is also a boundary widened. `zwire` loads unpacked extensions with elevated flags, runs a native host that hands out `exec`, `fs`, and `pty`, injects content scripts into privileged `chrome://` pages, and — on the fork — bypasses `X-Frame-Options` and frame-ancestors CSP so the tmux overlay can pane any site. Each of those is a hole in a wall Chromium built on purpose, and a browser that opens holes without accounting for them is a liability, not a tool. This chapter is the accounting: who can do what inside a running `zwire`, where the widenings actually land, and why the ones that look alarming are contained by the ones that do not.

## The profile is the blast radius

The first and most important boundary is the one Chapter 1 opened with: `zwire` runs against `~/.zwire/profile`, a dedicated `--user-data-dir`, and never the profile that holds your real logins. This is the containment on which everything else rests. The elevated flags, the unpacked extensions, the developer-mode banner, the native host wired into `NativeMessagingHosts/` — all of it is scoped to a profile that started empty and holds only what you did *inside* `zwire`. If every extension in the load set were malicious, the worst it could reach through the browser is the cookies, history, and saved passwords of the `zwire` profile, which is to say the ones you chose to create there. Your system Chrome, its logins, and its saved payment methods live in a different `--user-data-dir` the `zwire` process never names and cannot see. The isolation is not a courtesy to Chrome; it is the design's primary security property, and it is why loading unpacked, elevated-flag extensions is a defensible thing to do at all. You would never point those flags at the profile with your bank in it — so `zwire` never does.

## The launch flags and what they actually relax

Three flags in `bin/zwire` change Chromium's posture, and it is worth being exact about what each one loosens, because the reputational weight of the flag names exceeds what they do. `--load-extension` loads the unpacked extension directories; its real cost is that unpacked extensions are not Web Store-reviewed, so the trust you extend is trust in *this author's* code, staged from a copy you can read on disk. `--extensions-on-chrome-urls` is the one that lets the internal-HUD content scripts run on `chrome://` pages at all — the single relaxation that makes the skinned internal pages possible — and it is narrow: it widens *where a content script may run*, not *what the script may then do*, which stays governed by the manifest's declared permissions. `--test-type` suppresses the automation-restriction warnings Chromium raises when it detects it is being driven, which is cosmetic in effect but carries a real caveat: it also relaxes a handful of automation guardrails, and it is the reason the developer-mode banner is merely annoying rather than blocking. On Linux a fourth flag, `--no-sandbox`, appears where the macOS launcher omits it — a genuine reduction in the renderer sandbox, forced by the way the snapshot ships on that platform, and the one flag in the set the book does not wave away: it is the place where `zwire` on Linux is measurably less defended than `zwire` on macOS, and a reader who runs untrusted pages should know it.

## The native host is the largest boundary, and its gates

The native host is the widest hole zwire opens, by a wide margin: `exec` runs arbitrary programs, `fs_*` reads and writes the filesystem, and `pty_spawn` forks a login shell. Anything that can send that host a command owns the machine to the extent your user account does. So the gates around *who can send it a command* are the load-bearing security in the whole system, and they are worth stating plainly. On the native-messaging path, Chromium itself is the gate: the host's manifest lists `allowed_origins`, and only the extension origins named there can `connectNative` to it — a page you visit cannot, because a page is not an extension origin. On the daemon path, where the same router is reachable over a socket, the gate is the filesystem: the Unix socket is created `0600` inside a `0700` directory, so only your own user account can open it, and a different user on the same machine simply cannot see it. The one path that crosses a machine boundary — TCP peering — is the one path that demands a shared `--token` before anything privileged runs, because it is the only path where the “same user, same machine” assumption the local gates rely on no longer holds. The design is coherent: local callers are trusted because the OS already authenticated them by letting them open an owner-only socket, and remote callers are trusted only after presenting a secret. The place to be careful is exactly the place the host is careful — the moment a capability leaves the machine.

## What the fork widens, and what it deliberately does not

The fork's patches include two that a security reader should look at directly, because both loosen a Chromium invariant. The first is the `is_sandboxed` gate in `extension_function_dispatcher.cc` (patch 0018), which fixes a real SIGTRAP by letting a content script's extension function run in a context Chromium's assertion would otherwise reject. The important detail is the one the chapter on the fork already drew: the patch widens *scripting*, not *privilege*. The store-privileged JavaScript bindings — the APIs Chromium reserves for Web Store extensions — stay gated by `IsWebstoreOrigin` in `permissions_data.cc`, untouched by the patch. zwire's extensions gain the ability to run where the assertion used to fire; they do not gain the ability to call APIs they were never granted. The second is the CSP and `X-Frame-Options` bypass that lets the tmux overlay embed a site whose headers forbid framing. This is a genuine relaxation of a defense-in-depth mechanism, and it is scoped as tightly as the goal allows: it yields for *sub-frame navigation* so a pane can load an otherwise-unframeable site, not for the top-level page, and the sandboxed pane iframe (Chapter 10's exact token set) is the compensating control — the embedded site runs, but in a frame whose sandbox attributes the overlay chose, not with the ambient authority of the host page. The pattern in both patches is the same, and it is the pattern to hold the fork to: widen the narrowest thing that unblocks the feature, and leave the adjacent invariant standing.

## The honest residual risk

No account of a threat model is complete if it only lists the mitigations, so here is what remains. You are running unpacked extensions you did not review line by line; your trust is in the author, backed by the fact that the code is readable on disk and staged into a copy you can inspect. You are running a native host that can own your account; your protection is that only your own extensions and your own user can reach it. On Linux you are running with the renderer sandbox off; your protection is the dedicated profile and the fact that you, not an adversary, chose the pages. And you are, on the fork, running with two Chromium invariants relaxed; your protection is that each relaxation is scoped to the exact surface the feature needs. None of these is zero risk, and the book does not pretend otherwise. The design's claim is narrower and defensible: every widening is contained by the profile boundary, and the profile boundary is the one wall zwire never touches. Keep your real life in your real browser, keep zwire's power scoped to zwire's profile, and the residual risk is the risk you can actually reason about.

# Chapter 26 – The Performance Budget

A browser that injects a suite of content scripts on every page, streams system stats continuously, holds live PTYs, and repaints every `chrome://` page is spending your machine's time whether you asked it to or not. If that spending is not accounted for, the HUD becomes a tax on every navigation and every idle minute, and the neon is not worth a browser that stutters. This chapter is the budget: where the cycles and bytes go, what keeps the always-on machinery cheap, and where the cost could bite if the disciplines slipped.

## The per-page injection cost

The largest recurring expense is the one that runs on literally every page load: the internal-HUD content-script suite injects vim keys, the tmux overlay, the powerline statusbar, the palette, and the fuzzy find into each document. The suite is kept cheap by three habits. First, the scripts run in a fixed order the manifest declares, so shared state (the keybinding registry, the scheme) is resolved once and read by the rest rather than each script recomputing it. Second, every injectable is idempotent and guards its own re-entry — a content script that has already installed its listeners on a document detects that and returns, which matters because same-document navigations and the overlay's own re-injection would otherwise stack duplicate handlers. Third, the heavy surfaces are lazy: the terminal's xterm bundle, the palette's list machinery, and the fuzzy matcher's index are not built at injection time but on first use, so a page you open and immediately leave paid only for the key listeners, not for the UI behind them. The statusbar is the one visible always-on surface, and it reads its data from a storage key the worker fills, so painting it is a read and a layout, not a computation.

## The always-on streams

Two things run when no page asked them to, and both are deliberately throttled. The system-stats stream samples the machine on its own background thread in the native host, at an interval floored at 250 ms — fast enough that the statusbar's CPU and memory readouts feel live, slow enough that the sampling itself never shows up in the number it reports. Running it on a dedicated thread is what keeps a held stream from blocking the connection's other RPCs, so a HUD can carry a stats stream, a PTY, and a file watch on one pipe without any of them starving the others. The rates in the sample — network and disk throughput — are computed as deltas over the elapsed wall-clock interval, which means the host does no work to *maintain* a counter between ticks; it subtracts two readings when a tick fires and is otherwise idle. The second always-on cost is the CI poll, and it is deliberately slow: a `chrome.alarms` heartbeat every ten minutes, tuned to stay under GitHub's anonymous request cap. Ten minutes is not a monitoring interval anyone would choose for urgency; it is the interval chosen so the feature is free — a handful of requests an hour against a public API, invisible in the budget.

## The native host's footprint

The bridge that does the most is the smallest thing in the system. `zwire-host` is a single static Rust binary around half a megabyte, built with `opt-level="z"` for size and `panic = "abort"` to drop the unwinding machinery, with no Python interpreter, no `psutil`, and no runtime dependencies to load. This is not incidental thrift; it is what makes the host affordable to start on demand. Chromium spawns it per connection on the native-messaging path, so its startup time is paid every time the browser opens a port to it, and a half-megabyte static binary starts in the noise. The alternative the book explicitly rejected — a Python reference host — would

have paid an interpreter startup and a dependency import on every spawn, and would have failed outright on a machine without the right Python and packages. The Rust host's budget is: near-zero to ship (it is packaged inside the bundle), near-zero to start, and, while running, the cost of whatever it was asked to do — a sysinfo sample, a PTY read loop, a filesystem crawl — and nothing while idle.

## The base and the staging

The one genuinely large number is the base itself: the extracted Chromium snapshot, or the packaged fork, is around 325 MB on disk. That is the price of a real Blink engine and it is non-negotiable — it is the same disk a Chrome install spends, and it is exactly what a WebView wrapper would have saved by giving up the capabilities Chapter 2 said zwire cannot give up. The one place zwire adds to it is the per-user extension staging: at launch the loaded extensions are copied into a writable `~/.zwire/ext/` tree rather than run in place, because Chromium writes `_metadata/` indexes into the extension directory and every user needs a writable copy. That copy is small — the extensions are JavaScript and CSS, not the engine — and it is paid once per launch, fast enough to be invisible against the browser's own startup. The honest summary of the disk budget is that zwire costs one Chromium plus a rounding error, and the rounding error is the price of never running an extension in place.

## Where it could bite, and the mitigations

The failure modes are the mirror image of the disciplines. If a content script dropped its idempotence guard, the overlay's re-injection would stack handlers and a long-lived tab would slow with every same-document navigation — which is precisely why the guards exist and why the CDP probes assert single-injection. If the stats interval floor were removed, a HUD asking for a tighter stream could spin the host's sample thread hot enough to perturb its own CPU reading — which is why the floor is in the host, not left to the caller. If the terminal built its xterm bundle at injection instead of first use, every page load would pay for a terminal most pages never open — which is why it is lazy. The pattern across all of them is that the budget is enforced at the cheapest layer that can see the whole picture: the host floors the stream, the manifest orders the injection, the guards live in the scripts themselves. A HUD is only worth running if it is nearly free when you are not looking at it, and the whole point of the accounting is that, when you are reading a page and not touching the HUD, almost nothing in this chapter is spending anything at all.

## What the budget does not price

The accounting above covers what the HUD *executes*. Three costs sit outside it because they are not executed by any of zwire's own code — they are paid by the browser on the HUD's behalf, in the compositor, the network stack, and the process model — and they are the ones most likely to be underestimated because no profiler attributes them to a script.

**Compositing the always-on surfaces.** The statusbar is fixed to the bottom of every top-level page with a translucent, blurred background. A translucent blur is not a paint; it is a read-back of what is behind the element followed by a filter, and it is among the most expensive common CSS effects there is. Worse, it is not a one-time cost: an element whose backdrop is blurred must be recomposited whenever the content behind it changes, which on a scrolling page is every frame. A fixed-position element also promotes itself and its neighbourhood into their own compositing layers, which costs GPU memory proportional to the painted area, per tab, for as long as the tab exists. None of this shows up as JavaScript time and none of it appears in the “painting it is a read and a layout” accounting, which is accurate about the script and silent about the pixels.

The mitigations are ones the design already has, and it is worth recognizing them as mitigations rather than as features. The bar is top-frame only, so a page with twenty iframes pays for one bar and not twenty. It is a fixed small height rather than a proportional one, so the recomposited area is bounded regardless of viewport size. And it is toggleable from the palette, which converts an unconditional per-tab GPU cost into an opt-out. The

honest addition to the budget is that of everything the HUD does on an ordinary page, the blurred bar is the item whose cost scales with the *user's* behavior — scrolling — rather than with the HUD's, and it is therefore the one worth measuring on a low-end machine before assuming the accounting above is complete.

**The rule engine.** Chapter 7 records a declarative-net-request rule set carrying the frame-busting rule. Declarative rules are evaluated by the browser's own network stack, per request, on every navigation and subresource load in the browser — which is precisely why the declarative form exists rather than a JavaScript listener, and why it is fast. But fast is not free, and the cost is structurally different from every other cost in this chapter: it is paid per network request rather than per page or per keystroke, it is paid whether or not any HUD surface is open, and it is paid inside a component the extension cannot instrument. A single small rule set is genuinely negligible here. The reason to name it is that this is the one budget line with no natural back-pressure — nothing about adding a second, tenth, or hundredth rule would feel expensive at the moment it was added, and the cost would appear as a uniform slight slowdown of all browsing that no profiler would attribute to the extension.

**Per-tab multiplication.** The per-page injection cost is described above as the price of a page load, which is the right frame for CPU. It is the wrong frame for memory. A content script suite is instantiated per document and lives as long as the document does, so a browser with a hundred open tabs holds a hundred instances of the keybinding table, the scheme variables, the palette's structures, and the statusbar's DOM. The laziness discipline is what keeps this bounded — the third habit above is that the heavy surfaces build on first use rather than at injection, so a tab you opened and never touched holds listeners and a small table rather than a terminal bundle and a fuzzy index — and it is worth seeing that this discipline does double duty. It was introduced as a page-load latency argument, and the larger payoff is a memory one: laziness means the resident cost of a background tab is a floor rather than the full feature set. A long-lived browser with many tabs is the workload where an eager implementation of the same features would become visibly expensive, and it is the workload nobody tests.

The unifying observation is that all three of these costs are invisible to the tools that would measure the costs this chapter already prices. Script execution shows in a profiler; compositing shows in a frame timeline nobody opens; rule evaluation shows nowhere; and per-tab residency shows only in a process-level memory figure that mixes the page's own usage with the extension's. A performance budget that counts only what a profiler attributes to your code is a budget for the costs you would have found anyway. The ones worth writing down are the ones nothing will tell you about.

# Chapter 27 — Extending the HUD

zwire is built to be extended by its author, and the seams it exposes to him are the same seams open to you. This chapter is the practical one: how to add a ninth scheme, a new HUD internal page, a new content script, a palette command, and a native-host capability — each as a walk down the exact files the earlier chapters mapped, so that extending zwire is a matter of adding a row to a table the system already reads, rather than inventing a new mechanism.

## Adding a ninth scheme

A scheme is a set of role colors under a name, and adding one means writing that name into every table that enumerates the eight. There are more of those than there look to be, and forgetting one is the classic way a new scheme half-works. The color table lives in `schemes.js`, and it lives there *twice* — once for the new tab and once for the internal HUD — because the two run in different worlds and each carries its own copy; both must gain the new entry, dark and light. The native host validates incoming scheme names against its list of eight allowed names before it will write the scheme file, so a ninth name the host does not know will be rejected at the bridge and never persist — the allowed-names list in the host is the third place to edit. If you want the scheme reachable from the palette, it wants a row in the palette's scheme commands; if you run the fork, the DevTools Theme dropdown that Chapter 21 wired enumerates the same names and wants the ninth too. The rule to internalize is that the eight is not a constant in one place; it is a consensus across the content scripts, the new tab, the host's validator, and the fork's native mixer, and a new scheme exists only once all of them agree it does.

## Adding a HUD internal page

The internal pages are the most self-similar surface to extend, because they are all built from the same shell. A new page is four moves. First, add the page under `pages/` and boot it with `zg-boot.js`, the shared shell that gives every internal page the scheme, the fonts, the fuzzy-filter behavior, and the HUD chrome for free — you write the page's content, not its frame. If the page is a list of things, build it on `zg-list.js`, the list factory the history, bookmarks, and downloads pages already share, and you inherit the fuzzy filter and the row rendering rather than re-implementing them. Second, if the page is meant to *replace* a `chrome://` page, add its redirect to `redirect.js`, which is what turns a navigation to the native page into a load of your HUD replacement. Third, if the page draws on a privileged Chrome API — the settings page's `settingsPrivate`, the extensions page's `developerPrivate` — declare that permission in the manifest, because the redirect gives you the surface but not the authority. Fourth, if the page should be reachable by keyword, seed a palette command for it. The whole design of the internal pages is that they are the same page with different content, so the cost of a new one is the content and the wiring, never the chrome.

## Adding a content script

A new always-on behavior — the class vim, the overlay, and the statusbar belong to — is a content script, and the two things that make it a good citizen are the two the suite already enforces. First, place it in the manifest's injection order relative to the scripts it depends on: if it reads the keybinding registry or the scheme, it runs after the scripts that establish them, because the order is the contract. Second, make it idempotent — guard the moment it installs its listeners so that a second injection into the same document detects the first and returns

— because the overlay’s re-injection across same-document navigations *will* run it twice and a script that stacks its handlers is a slow leak. If the behavior needs a key, register it in the `zkeys.js` keybinding registry rather than attaching a raw listener, so it appears in the keybinding editor, respects the user’s remaps in `zb_keys`, and does not collide silently with an existing binding. A content script that reads the registry, honors the injection order, and guards its own re-entry is indistinguishable from the ones that shipped — which is the point of having a suite rather than a pile.

## Adding a palette command

The command palette is a registry, not a switch statement, so a new command is a new entry, not a new branch. Seed it in `cmd-defaults.js`, which is where the built-in keyword commands live, and give it a command *type* the palette already knows how to execute — a URL to open, a `shell` one-liner to run in the terminal, an internal page to navigate to. The type is what matters: because the palette dispatches on type, a new command that reuses an existing type needs no new execution code at all, and a command that genuinely needs a new behavior adds one type handler that every future command of that kind reuses. Commands that must reach across into the service worker — anything touching tabs or windows — go through the `zb_cmd` storage bus rather than calling into the worker directly, because a content script cannot call the worker’s privileged APIs and the bus is the seam that wakes the worker to do the privileged work. User-defined commands land in `zb_custom_cmds` through the command editor; the seeded ones in `cmd-defaults.js` are the same shape, which is why the editor can show and edit both.

## Adding a host capability

The native host is the most disciplined thing to extend, because its whole architecture is one router in front of many small modules. A new capability is a new module under `src/` and a new `cmd` the router in `session.rs` dispatches to — and because all three transports sit behind that one router, the capability is immediately reachable over native messaging, over the local socket, and from the in-process library, without touching any of them. The module’s shape is set by the fifteen that precede it: it takes the parsed request, does its work, and returns a JSON reply, echoing the request’s `id` if it carries one so it can be multiplexed against other in-flight work. If the capability streams — like `sysinfo_start` or `fs_watch` — it runs on its own thread and emits keyed frames the caller cancels by `id`, following the pattern the stats stream and the file watcher already set. And if the capability is privileged in a way the socket’s owner-only gate does not cover, it inherits the token requirement on the TCP path for free, because that gate lives in the transport, not in the module. The router is the extension point; the modules are leaves; and a new leaf is a new file and a new line in the dispatch, which is exactly as much ceremony as a capability of the machine deserves.

# Chapter 28 – The Keyboard Arbitration Model

Every feature chapter so far has claimed a key. Vim claims `j`, the overlay claims `Ctrl-b` and then a second key after it, the palette claims `⌘K`, the find bar claims `⌘F`, the terminal claims `Ctrl+`` and then every `Ctrl+<letter>` while it has focus, and the pages underneath all of them claim whatever they like. Read one chapter at a time this looks like a set of independent features; read together it is a contention problem, and the HUD only works because the contention was designed rather than discovered. This chapter is the arbitration model: who can claim a keystroke, in what order, with what mechanism, and what the invariant is that lets six claimants share one keyboard without a table of special cases.

## Seven claimants, in dispatch order

A key pressed inside `zwire` passes through seven potential owners before anything happens, and only the last four are `zwire`'s code:

|   |                                        |                                                                              |
|---|----------------------------------------|------------------------------------------------------------------------------|
| 1 | the OS / window manager                | ⌘-Tab, ⌘-Space, screenshot chords                                            |
| 2 | Chromium's own accelerators            | ⌘T, ⌘W, ⌘Q, ⌘L, ⌘K on the new tab                                            |
| 3 | <code>chrome.commands</code> shortcuts | the worker's open-palette ( <code>⌘K</code> / <code>Ctrl+K</code> )          |
| 4 | the top frame's capture list           | <code>zvim</code> , <code>zpalette</code> , <code>zfind</code> , the overlay |
| 5 | the pane forwarder                     | <code>ztmux-pane.js</code> , inside a framed document                        |
| 6 | the page's own listeners               | a site's single-key <code>j</code> , its own / shortcut                      |
| 7 | the focused field / xterm              | text entry, readline, control codes                                          |

The first three are not negotiable from JavaScript. The OS wins outright. Chromium's accelerators win over any page listener on the surfaces Chromium chooses to guard, which is exactly the situation the new-tab page runs into: Chrome reserves `⌘K` at the browser level there, and no amount of capture-phase cleverness in the page will see it. And `chrome.commands` — layer three — is the only mechanism that fires *regardless of focus*, which is why the background worker, not any content script, is the browser-wide owner of `⌘K`.

That ownership has a price the worker's own comment names: registering `⌘K` as a browser command consumes it on every tab, including ones where a page listener would have been perfectly capable of catching it. `zwire` pays that price deliberately, because the alternative — a page listener on ordinary pages and nothing on the new tab — would make the palette's most-used chord work in most places and mysteriously fail in one. A key that works everywhere is worth more than a key that is cheap to bind. The worker then routes the press by inspecting the active tab: the content-script palette on an ordinary page, the new-tab extension's own palette through a cross-extension message, a HUD page's palette through a runtime broadcast. Three destinations, one owner, one chord.

## The four arbitration devices

Below layer three, `zwire`'s scripts arbitrate among themselves with exactly four mechanisms, and every keyboard feature in the book is built from some combination of them.

**Capture-phase listening** is the first. Every HUD key listener registers with the capture flag, on `document`, which puts it ahead of the page's own handlers in the dispatch order. This is what lets vim's `j` beat a site's own

`j` without knowing anything about that site. Capture is the cheap half of the mechanism; the expensive half is knowing when *not* to use it.

`stopImmediatePropagation` is the second, and it is the one that distinguishes a claim from a suggestion. `preventDefault` alone stops the browser's default action but lets other listeners — including the page's, and including other HUD scripts — still run. When vim decides it owns a key it calls both, so the site's single-key shortcut never also fires. The comment in `zvim.js` on that call reads “win over site single-key shortcuts,” and it is the entire reason vim on a page feels like vim rather than like vim plus whatever the site thought that letter meant.

The **prefix state machine** is the third, and it is how the overlay gets a whole command vocabulary out of keys that are otherwise unowned. Arming `Ctrl-b` puts the top frame into a state where the *next* key is interpreted against the tmux table and then the state clears; a 900-millisecond timeout clears it anyway if nothing follows. The same device appears at smaller scale inside vim, where `g`, `z`, `y`, `m`, and ``` each arm a one-key prefix under the same timeout. A prefix is the standard answer to “I need forty commands and I have twenty-six unclaimed letters,” and it is also an *ownership* device: while a prefix is armed, one claimant owns the next key unconditionally, which resolves the contention for that keystroke by fiat instead of by rule.

**Delegation across frames** is the fourth. A pane iframe is a different document with a different listener set, and the top frame cannot listen inside it. So the pane runs a thin forwarder that catches only the keys the overlay needs — the prefix, the key that follows it, the palette chord, a copy-mode yank, a synchronized keystroke — and `postMessages` them up with a tagged envelope. The overlay's logic stays in one place; only the *sensing* is duplicated. This is the difference between a distributed feature and a duplicated one, and it is why the tmux prefix works while your cursor is inside a framed site.

## The invariant: claim only what is mapped

The rule that makes all of this compose is stated in one sentence and enforced in several places: **a HUD listener calls `preventDefault` only when the key maps to a real action**. An unknown key reaches the page untouched.

This sounds obvious and is not. The natural way to write a keyboard layer is a switch statement that swallows everything and falls through to a default that also swallows, because that guarantees your feature works. It also guarantees that every site's own keyboard interface breaks, and — worse — that the *next* HUD feature to want a key finds that key already consumed by a script that was not using it. The claim-only-what-is-mapped rule keeps the unclaimed keyspace genuinely unclaimed, which is what allows six independent scripts to be added over time without an arbitration table.

The rule has three corollaries visible in the code. First, the editable guard: inside an `input`, `textarea`, `select`, or `contentEditable`, vim claims nothing except `Escape` — because in a text field every letter is already mapped, by the field. Second, the modifier bailout: a key held with `Alt`, `Meta`, or `Ctrl` is left alone by the motion layer, because those chords belong to the palette, the find bar, and the vim toggle, and a motion handler that grabbed them would silently break the features above it. Third, the ordering of guards inside a single listener matters as much as the ordering of listeners: `zvim.js` checks the per-tab disable flag, then hint mode, then the editable guard, then the modifier bailout, and only then consults the keymap. Hint mode has to come *before* the editable guard, because typing a hint label is not typing into a field even when a field happens to be focused; getting that order wrong would make link hints unusable on any page with a search box.

## Focus as a claim modifier

Two features change what the keyboard means depending on where focus sits, and both are worth naming because they are the cases the layered model does not handle by itself.

The terminal is the sharper one. When the terminal overlay has focus, `Ctrl+<letter>` must reach the PTY as a control code — `Ctrl-A` to the start of the line, `Ctrl-W` to kill a word, `Ctrl-U` to kill to the start — because

that is what those keys mean in a shell. But on macOS the web view would otherwise swallow several of them for its own Emacs-style text bindings, which would leave the browser terminal missing exactly the editing keys that make a terminal usable. So the terminal installs a capture-phase handler that, while it has focus, forwards those combinations straight through as control bytes. This is a focus-scoped claim: the same chord is claimed by the terminal when the terminal is focused and by nobody in particular otherwise.

The second is the tmux prefix's alternate. The prefix is `Ctrl-b`, the real tmux one, and it is also `⌘-b`. The alternate exists for a historical reason the fork's series file records: an earlier native split patch consumed `Ctrl-b` and the key after it before the page ever saw them — the “C-b %%” double-tap — and while that patch is gone, the alternate chord it forced remains, because a second prefix that nothing else intercepts is a genuinely useful escape hatch on a page or a platform where the first one is contested.

## Remapping makes the model dynamic

The last complication is that none of the bindings above are constants. Every default key lives in `zkeys.js` as data, and a user's remaps live in `chrome.storage.local` under `zb_keys` as `{action: key}` pairs merged over the defaults. That merge happens at load *and* again whenever the key changes, because the settings page writes it live.

This has a real consequence for how the listeners are written. A keyboard layer that bound a handler per key would need to unbind and rebind on every remap, and would race with a page that was already open. So the HUD listeners are written the other way around: one listener per concern, which consults a `key → action` table built from the registry at dispatch time. Rebinding `scrollDown` from `j` to `s` rewrites a table entry, and the very next keystroke on every open page dispatches through the new table with no reload and no re-registration. The same registry and the same storage key drive the overlay's post-prefix keys, which is why one settings page remaps both layers and why the overlay's help card — built from the same registry — can never drift from what the keys actually do.

The whole model reduces to four rules a new feature must follow to be a good citizen: register in the shared registry rather than binding a raw key; listen once, in capture, and dispatch through the table; claim only what is mapped and claim it completely with both `preventDefault` and `stopImmediatePropagation`; and if the feature spans frames, forward the sensing upward rather than duplicating the logic. A keyboard layer built that way can be added to the five that already exist without anyone auditing the other five — which is the only property that matters when the layers were written years apart.

# Chapter 29 – The Six Boundaries and Their Transports

zwire is not one program. It is a content script in an isolated world, a service worker with its own lifecycle, two other extensions with their own storage, a set of cross-origin iframes, a native process, and — on the fork path — a C++ color mixer inside the browser itself. Every feature in this book that feels like one coherent behavior is actually a message crossing at least one boundary between those, and the choice of transport at each crossing is the single most consequential design decision in the runtime path. This chapter is the catalog: six boundaries, the transport each one uses, what that transport can and cannot carry, and the rule that decides.

## The six crossings

**Page and content script.** A content script runs in an isolated world: it shares the DOM with the page and shares nothing else — not the page's JavaScript globals, not its prototypes, not its variables. The transport across this boundary is the DOM itself, plus synthesized events. zwire uses it deliberately little. The find bar registers highlight ranges rather than wrapping matches in elements; the synchronize-panes receiver splices text into a field and then dispatches a real `input` event so the page's own handlers see the change. Both are the same discipline: touch the DOM the way a user would, not the way a script would, so the page cannot tell the difference and cannot be broken by the difference. The one thing this boundary is *good* for is that it is free and synchronous, and the one thing it cannot do is reach anything outside the document.

**Content script and service worker.** This is the most-crossed boundary in the system and the one with the most careful transport, because the receiver may not be running. The obvious mechanism, `runtime.sendMessage`, is not reliable against a suspended MV3 worker. The mechanism zwire actually uses for anything that must happen is the storage command bus: a content script writes a `zb_cmd` object into `chrome.storage.local`, and `storage.onChanged` is a reliable wake. The worker's own comment states the contract — storage wakes it where a message to a sleeping worker would not, “so palette navigation / tab-switching always executes.” In the other direction the worker does not wait to be asked at all: it publishes the tab list, the extension list, the shortcut table, the frequency list, and the system-stats sample eagerly, so a page reads a value rather than issuing a query that might never be answered.

**Extension and extension.** Three extensions run in a zwire window and they share nothing. `chrome.storage.local` is per-extension. There are exactly two channels. The first is messaging with explicit consent on both ends: the worker pushes the scheme to the power-tool over `runtime.sendMessage`, and the new-tab page is reachable because its manifest lists the HUD worker in `externally_connectable`. That channel is asymmetric — it works when the sender knows the receiver's id and the receiver has opted in — and it is push-only, so it cannot serve as a source of truth. The second is the native host's files, which every extension can reach through its own native-messaging port, and which is the only *symmetric* channel the three share. That is the entire reason the scheme lives in a file rather than in storage.

**Top frame and pane iframe.** A pane holds an arbitrary cross-origin site, so the two documents share nothing at all, not even the DOM. The transport is `postMessage` with a tagged envelope and a deliberately small vocabulary: upward, the prefix, a post-prefix command key, a palette request, a yank, a synchronized keystroke; downward, a sync-group change, a copy-mode toggle, paste text, and a keystroke to apply. The vocabulary is small on purpose — every message shape is a piece of API surface between two documents that cannot be updated atomically, since one of them is a page from the internet running the forwarder that was injected into it.

**Browser and native host.** Length-prefixed JSON over stdio: a four-byte little-endian length, then that many

bytes of UTF-8 body. It is the only transport in the system that crosses a process boundary, and it is the only one with a real multiplexing story — a request may carry an `id` that is echoed on its reply, so one connection can hold a system-stats stream, several PTY sessions, and a file watch at once, each keyed and each cancellable independently. The cost of this boundary is that only an extension origin listed in the host manifest's `allowed_origins` may open it at all, which is also, from the threat-model chapter's point of view, its best property.

**Browser and the compiled fork.** On the fork path there is a consumer of the scheme that is not a web context at all: the native color mixer, compiled into Chromium's Views layer. There is no message channel between a content script and a C++ mixer, and there will never be one. The transport is a file — and specifically, a *plain-text projection* of a file. The host owns `global.toml` as the real store, and writes a trivial `hud-scheme` (and, for light mode, `hud-light`) beside it purely so a C++ process that cannot parse TOML at paint time has something it can read in a line. The projection is not redundancy; it is a deliberate narrowing of a format to fit the weakest consumer.

## The durability ladder

Laid out together, the six transports sort into a ladder by how much they require of the receiver, and the ladder is the design rule:

| transport            | receiver must be   | survives             | notifies?       |
|----------------------|--------------------|----------------------|-----------------|
| file (+ projection)  | nothing            | everything           | no (poll/pub)   |
| chrome.storage.local | the same extension | worker eviction      | yes (onChanged) |
| long-lived port      | alive now          | navigation (if held) | yes             |
| postMessage          | a live frame       | nothing              | yes             |
| one-shot sendMessage | awake now          | nothing              | yes             |

The rule that falls out is: **pick the transport with the weakest liveness requirement that can carry the payload.** Every hard-won behavior in the runtime path is an application of it. The scheme is a file because one of its consumers is a C++ process and another is a different extension, and neither can be assumed to be running. The command bus is storage because the worker may be asleep. The PTY is a long-lived port because it is a stream and a stream has no meaning without an attached receiver. The pane protocol is `postMessage` because there is no weaker option across an origin boundary. And one-shot `sendMessage` is used only where the receiver's liveness is already established — the worker pushing to the power-tool, which is either loaded or irrelevant.

The corollary is equally practical: when a path stops working, the first question is whether it used a transport stronger than its receiver could guarantee. That was exactly the automation bus's original failure. A one-shot native message issued from a page's content script lost its reply, because the worker was torn down during the native round trip, and external pages saw a zero-percent success rate. The fix was not to add a keepalive timer; it was to move the reply onto a channel that was already open for another reason — the persistent sysinfo port — and correlate it with a request id. The path did not need more liveness; it needed a transport whose liveness was already paid for.

## What a file costs

The bottom rung deserves its own accounting, because “just write a file” hides real engineering. A file has no lifecycle, no receiver, and no version skew, and it can be read by a content script through the host, by a separate extension through its own host connection, by a C++ mixer at paint time, and by a shell script. That is an enormous amount of reach for a transport with no protocol.

It buys that reach by giving up three things. It does not notify — which is why the new tab polls the host every 1.5 seconds and why the newer design adds a publish/subscribe topic on the host so subscribers get a push instead. It has no atomicity of its own — which is why every write is a write to a temp file followed by a rename, so a concurrent reader sees the old contents or the new ones and never a half-written file. And it has no serialization across writers — which is the failure that actually bit: two concurrent writes, one carrying a

scheme and one carrying a UI preference, could each read the old state, modify their own field, and write back, so the second write silently reverted the first. The fix is a cross-process lock file around the read-modify-write of `global.toml`. That sequence — reach for the file because it is the only universal channel, then re-add atomicity, notification, and serialization one at a time — is the honest cost of the bottom rung, and it is worth remembering before reaching for a file as though it were free.

## Why the boundaries are not consolidated

A reasonable reader asks why six. Could the HUD not run one transport everywhere and be simpler? It could not, and the reason is that the boundaries are not *zwire*'s; they are Chromium's, and each exists to enforce something. The isolated world exists so a page cannot tamper with an extension. The per-extension storage partition exists so one extension cannot read another's data. The service worker's eviction exists so background extensions cannot hold memory forever. The cross-origin frame boundary exists so a framed site cannot read its framer. The native-messaging origin allowlist exists so an arbitrary page cannot spawn a local process. Every one of these is a security property the threat-model chapter relies on, and consolidating the transports would mean defeating them.

So the design is the opposite of consolidation: accept all six boundaries as given, choose the correct transport at each, and make the *feature* the thing that spans them. The scheme picker looks like one control and is several renderers reconciling to one file. The palette looks like one list and is four implementations sharing one item source. The terminal looks like one window and is a page, a worker, a process, and a PTY. In each case the coherence the user sees is the sum of correct transport choices, and none of it survives a single wrong one.

# Chapter 30 – Living Inside MV3

Half the non-obvious engineering in the internal HUD exists because of one fact: the background service worker can be destroyed at any moment, and there is nothing the extension can do about it. Manifest V3 replaced MV2's persistent background page — a document with a stable heap, long-lived timers, real DOM, and a lifetime equal to the browser's — with an event-driven worker the browser wakes on demand and suspends when idle. Everything that used to be a variable is now a question about where state lives. This chapter takes the MV3 constraint seriously as a design input and walks the five patterns zwire uses to live inside it, because those patterns generalize well past this browser.

## What was actually taken away

It is worth being precise about the loss, because the popular framing (“MV3 constrained blocking `webRequest`”) is not the part that shaped this codebase. Three things went.

The **heap** went. A persistent page could hold a map of tab id to session, a cached list, a debounce timer, an open connection — and rely on all of it still being there the next time an event fired. A worker cannot. Every in-memory structure is a cache of something that must exist elsewhere, or it is data you have agreed to lose.

The **guaranteed reachability** went. `runtime.sendMessage` to a persistent page always landed because the page was always there. To a suspended worker it is a race: the browser may wake the worker and deliver, or the send may fail in a way the sender is not well positioned to detect or retry. Any feature whose correctness depends on the message arriving needs a different mechanism.

The **timer** went, in the sense that mattered. A `setInterval` in a persistent page is a heartbeat for the browser's lifetime; a `setInterval` in a worker lasts until the worker is suspended, which may be seconds. Periodic work has to move to `chrome.alarms`, which survives suspension by waking the worker — and which, by design, is unsuitable for anything fine-grained. The HUD's only alarm is the CI poll, on a ten-minute heartbeat, which is precisely the kind of coarse periodic work alarms are for.

## Pattern 1: storage as the wake mechanism

The single most load-bearing MV3 pattern in the HUD is that `storage.onChange` is a reliable wake and a message is not. So anything a page needs the worker to actually do is written into `chrome.storage.local` as a `zb_cmd` object rather than sent as a message. The worker's `onChange` handler dispatches on the action field across the whole tab-and-window vocabulary — open, activate, new tab, new window, duplicate, reopen, pin, mute, close, close others, next, previous, the tiling engine, and a bare ping that re-runs every mirror.

There is a subtlety in the bus that is easy to miss and painful to debug: a storage event fires on a *change*, and writing the same value twice is not a change. A user who presses `x` to close a tab and then presses `x` again would write an identical `zb_cmd` and the second press would silently do nothing. So the bus carries a monotonic tag alongside the action, and every write bumps it. The tag has no meaning to the handler; its only job is to guarantee the write is a change. Any storage-as-a-bus design needs this, and every one that omits it has an intermittent bug that reproduces only on repeated identical commands.

## Pattern 2: publish eagerly, never query

The second pattern inverts the usual client-server shape. A content-script palette cannot call `chrome.tabs` or `chrome.management`; those APIs are the worker's. The naive design is a request-response: the palette asks the worker for the tab list when it opens. Under MV3 that request may be issued to a sleeping worker at the exact moment the user is waiting on a UI, which is the worst possible time to lose a message.

So the worker does not wait to be asked. It keeps the tab list, the enabled-extension list, the extension shortcut table, and the frequency list continuously fresh, each wired to the Chromium event that would change it — tab updates, extension installs, new visits — and the palette simply *reads storage*. The read always succeeds and never depends on the worker being awake. The cost is that the mirrors are maintained even when nothing is reading them, which is a small, bounded, event-driven cost; the benefit is that the most latency-sensitive surface in the HUD never waits on a process that might not exist.

The same reasoning explains why the scheme is mirrored into `zb_scheme` rather than fetched on demand. The worker's comment is explicit that a `sendResponse` across the native host is unreliable because the worker can suspend mid-round-trip, while a storage write always lands. Publish the answer before the question is asked, and the question never has to survive the worker's lifecycle.

## Pattern 3: keepalive by useful work

The third pattern is the one most likely to be got wrong elsewhere. A worker that must stay alive across a slow operation needs a keepalive, and the tempting implementation is a timer or a dummy port whose only purpose is to prevent suspension. `zwire` does not have one. What it has is a port that was already open for a real reason: the persistent native connection that streams system stats. The worker's own comment names the second motive for holding it open — it keeps the worker itself alive.

That port keeps the worker alive as a *side effect* of doing something the HUD wanted anyway, and the automation bus deliberately reuses it. The bus's original failure was precisely a lifecycle failure — a one-shot native message from a page lost its reply because the worker was torn down during the native round trip, and external pages saw a zero-percent success rate. The fix routes the reply over the already-open `sysinfo` port, correlated by a request id so the browser action fires exactly once. The general principle: if you need the worker alive, find the work that legitimately keeps it alive and put your dependency on that, rather than adding a lifecycle hack whose only justification is itself.

## Pattern 4: idempotent seeding with tombstones

A worker's startup handlers run more often than a persistent page's ever did — on browser start, on extension install, on extension reload, and effectively any time the browser felt like evicting and re-waking it. Anything those handlers do must therefore be safe to do repeatedly.

The keyword-command seeder is the clean example. The worker seeds a default set of palette commands on first run, guarded by a seeded flag so it does not re-run, and merges defaults with any existing user entries so edits are never clobbered. That covers the first-run case. It does not cover the *deletion* case: a user who deletes a seeded command would find it resurrected the next time the seeder decided it had not run. So deletions are recorded in a tombstone list the seeder consults. Guard plus merge plus tombstone is the full shape of an idempotent seed, and any two of the three is a bug that only shows up after a browser restart.

The native seed follows the same discipline from the other direction: on startup the worker asks the host for the current scheme and mirrors whatever the file already held, because — as its comment says — the file may already carry a scheme picked in a prior session or written before launch. Startup is not initialization; it is reconciliation with state that outlived the process.

## Pattern 5: state outside the heap, with a reconcile step

The last pattern is what happens to structures that genuinely cannot be recomputed. The worker's window-tiling engine holds a session of named window groups and their panes — a real data structure with no source of truth anywhere else. It survives eviction by mirroring itself to `zb_tmux_state`, and it rebuilds from that mirror when the worker comes back, defaulting to “the current window is the sole pane” if there is nothing to rebuild from.

Mirroring alone is not enough, though, and the engine's second half is the interesting part. The world it describes — real browser windows — can change while the worker is not running. Windows get closed, focus moves, groups empty out, and none of those events reached a process that was asleep. So the engine carries a `reconcile` step that drops panes whose windows no longer exist and prunes empty groups, wired to `windows.onRemoved` and `onFocusChanged`. The mirror restores what the worker believed; the reconcile step corrects it against what is actually true. Any persisted model of external state needs both, and the reconcile step is the one people forget, because it only matters after an eviction they did not observe.

The per-tab PTY relay shows the same shape with different tradeoffs. Sessions are keyed by tab id so a shell survives page navigation; output produced while no page is attached is buffered and flushed on reconnect, capped at 500 messages; a spawn is honored once per tab so a re-injected content script re-attaches instead of forking a second shell; and a content-script disconnect nulls the attachment rather than killing the native port, while an actual tab removal tears it down. Here the state is genuinely unrecoverable — a live process — so the design does not try to persist it; it tries to keep it attached to the one thing with a comparable lifetime, the tab.

## What is deliberately allowed to be lost

The honest end of this chapter is the list of state `zwire` does *not* try to save, because deciding what to lose is as much a part of the pattern as deciding what to persist. Vim's marks are per-page scroll positions in memory, and they are gone on reload — they are bookmarks for the page you are reading, not durable locations, and persisting them would create a synchronization problem far larger than the feature. The per-tab vim disable flag is in memory too, deliberately transient, so a page where you turned vim off gets its raw keys back and the next page starts clean. The PTY's detached-output buffer is capped and will drop the oldest messages rather than grow without bound. Each of these is a decision that the state's value does not justify a persistence mechanism, and stating them as decisions is what distinguishes a design from an oversight.

The rule that ties the chapter together: treat the worker as a pure function from events plus persistent storage to effects. Anything it remembers between two events is either mirrored, reconstructible, or expendable — and you must be able to say, for every piece of state, which of the three it is.

# Chapter 31 – The Storage Schema and Its Ownership Rules

Appendix I lists the `zb_*` keys. A list is not a schema. What makes the storage layer work is not the set of keys but the discipline around them: for each key, exactly one writer, a stated set of readers, a defined lifetime, and a rule for what happens when its shape changes under a page that is already running. A shared key-value store with no ownership rules is a mutable global namespace shared by a dozen scripts across three execution contexts, which is the fastest way known to make a feature that works in isolation fail in combination. This chapter is the discipline.

## Five roles, not two dozen keys

Every key in the store falls into one of five roles, and the role determines the rules.

**Mirrors** are worker-written and page-read: the active scheme, the system-stats sample, the open-tab list, the enabled extensions, the frequency list, the extension shortcut table, the CI tally. The worker maintains each against the Chromium event that would invalidate it, and a reader treats the value as a fact about the world. The rule is absolute: **a reader of a mirror never writes it**. A page that wants a different scheme does not write `zb_scheme`; it asks the owner, which relays to the host, which writes the file, which comes back as a new mirror value. Writing a mirror directly would produce a value that disagrees with its source and would be silently overwritten at the owner's next update — the worst kind of bug, because it works exactly once.

**Buses** are page-written and worker-read, and consumed rather than held: the command bus and the session-load trigger. A bus value is an *edge*, not a state. It carries a monotonic tag so a repeated identical command still registers as a change, and nothing reads it for its value after the handler has run. The rule here is that a bus key must never be treated as a source of truth for anything — its content is meaningful for exactly one dispatch.

**Preferences** are written by a settings surface and read everywhere: the keybinding remaps, the keyword commands, the statusbar visibility, the tmux prefix chord and its timeout, the CI configuration. These have a real multi-writer story — the Keyboard page, the Commands page, and the palette can all touch them — and the rule that keeps it safe is that preferences are *user intent*, so a later write legitimately supersedes an earlier one, and every reader re-reads on change rather than caching at load. That last half is why rebinding a key changes behavior on already-open pages with no reload: the readers subscribed rather than sampled.

**Session state** is written by either side and exists to survive an eviction or a navigation: the window-tiling engine's state, the named tmux layouts, the terminal's open flag. These are the keys where lifetime is the hard part, and the rule is that anything describing transient UI must have a defined reset. The terminal's flag is the clearest case: it re-opens the overlay only when the value is the boolean `true` — not merely truthy — and the worker resets it to false on browser startup and on extension reload, so a stale flag from a previous session cannot make the terminal pop up on every page of a fresh one. A session key without a reset owner is a haunting.

**Guards and tombstones** are one-shot markers: the seeded flag and the removed-defaults list. Their only rule is that they are written before the thing they guard is considered done, and never cleared as a shortcut — clearing a tombstone list to “start fresh” resurrects every command the user deliberately deleted.

## The rules, stated

Five roles produce six rules, and they are worth stating flatly because every one of them corresponds to a class of bug:

```
1 single writer      each mirror has exactly one owner; readers never write
2 nonce on buses     a bus write carries a monotonic tag so repeats fire
3 subscribe, don't cache readers re-read on change rather than sampling at load
4 real booleans      a flag is compared to true, never merely for truthiness
5 reset owner        transient UI state names who clears it and when
6 tombstone before seed a seeder checks deletions before it merges defaults
```

Rule three is the one that most shapes how the scripts are written. The HUD's content scripts do not read their configuration once at injection; they read it and then listen for changes to it. That is why a scheme change repaints an open page, why a remap takes effect on a tab that was already loaded, why the statusbar re-renders when the stats sample updates without any polling of its own, and why the tmux overlay notices a session-load trigger written by a page in a different tab. Subscription is what turns a key-value store into a coordination mechanism; sampling would turn it into a configuration file that everyone reads at slightly different times.

## Schema evolution under a running page

The awkward case in any shared store is a key whose shape changes while old readers are still alive. In a browser extension this is not hypothetical: a user updates the extension, and pages loaded before the update keep running their old content scripts against a store the new worker is writing.

The overlay's own history supplies the worked example. Its per-tab session snapshot originally recorded a per-window sync boolean — is this window broadcasting? — and later needed a list of which panes were members of the sync group. The snapshot restore path migrates the old boolean to the new member-id array on the way *in*. That direction matters. Migrating on read means an old value is understood by new code, and the store keeps whatever the newest writer put there. Migrating on write — rewriting the store to the new shape when the new code loads — would break the older content script still running in another tab, which would read a shape it does not understand. The rule generalizes: **migrate on read, never on write**, and accept that the store may contain several generations of a value at once.

The corollary is that a reader must tolerate a missing key as well as an old one. The tiling engine defaults to “the current window is the sole pane” when there is nothing to rebuild from; the palette treats an absent mirror as an empty list rather than an error. A store that is read by code older and newer than itself has to be read defensively at every site, and the cheapest way to make that consistent is to have exactly one reader per key inside each script rather than scattered direct reads.

## The two escapes from the schema

Two categories of state deliberately do not live in this store, and knowing why is knowing the boundary the schema draws.

The first escape is per-origin `localStorage`: the new tab's quick-launch tiles and its search-engine template, and the visual-effect toggles the shared effects module persists under its own keys. These are page-local by nature — they belong to one document's own presentation — and using extension storage for them would mean the worker mediating a preference no other context needs. The cost is exactly the isolation that makes them cheap: `localStorage` does not cross origins, so a preference stored there is invisible to every other surface. This is precisely why the effects toggles exist in *two* places. The page keeps its own local copy so it can apply the effect without a round-trip, and the Settings page pushes the same four values through to the native prefs so the separate new-tab extension follows. The duplication is not sloppiness; it is a local cache plus a shared source, and the settings page is the bridge that keeps them agreeing.

The second escape is the native store — the host's `global.toml` and its plain-text projections. Anything that must be seen by a second extension, or by a C++ mixer, or by another application in the suite, cannot live in `chrome.storage` at all, because that store is partitioned per extension by design. The scheme, the light-mode flag, the effect preferences, and any user-defined custom schemes are therefore native-side, and `zb_scheme` is a *mirror* of the native value rather than the value itself. The schema's top-level rule follows from that: for any piece of state, ask how many extensions and how many processes must see it, and let the answer choose the store. One script, `localStorage`. One extension, `chrome.storage.local`. More than one extension, or any non-web consumer, the native file — with a storage mirror in front of it for the scripts that need a fast local read.

## Why this is a schema and not a convention

The reason to write these rules down rather than leave them as habits is that the store is the one part of the system every context touches. A content script, the worker, four palette implementations, twenty-three HUD pages, and the settings surfaces all read and write it, and they were written at different times for different features. There is no compiler enforcing any of the six rules; there is no type on any key. The only thing standing between this design and a mutable global namespace is that each key has a stated owner, a stated lifetime, and a stated read discipline — and that a new feature adds a *row*, with all three declared, rather than a new access pattern to an existing row. A schema, in a system with no schema enforcement, is exactly the set of invariants everyone agreed to maintain by hand, and the value of writing it down is that the next person to add a key knows which five roles they are choosing among.

# Chapter 32 — When a Hop Fails

Chapter 36 is the field guide for the symptoms a user sees. This chapter is the layer underneath it: what happens *inside* a multi-hop path when one hop does not do its job, which failures the design absorbs silently, which ones surface, and why the ones that surface were chosen to surface. Every headline feature in this book is a chain of crossings — surface to worker to host to file to renderers, or page to worker to process to PTY — and a chain is only as trustworthy as its account of partial execution. Appendix AA traces three of these paths when everything works. This chapter traces the same shapes when something does not.

## Path A: a scheme change with a hop missing

The scheme fan-out is a write followed by an uncoordinated set of reads: a surface relays the pick, the worker writes the mirror and relays to the host, the host takes the lock and rewrites the store and its projections, and the renderers reconcile on their own schedules — content scripts through the observed mirror, the new tab through its own poll of the host, the power-tool through a push, the fork's mixer through the projection file at paint time. There is no transaction anywhere in that sentence, and the interesting question is what each missing hop actually costs.

If the **host is not running**, the mirror still updates and the file does not. The symptom is the most-reported desync in the book: the internal pages and the ordinary page skins recolor immediately, and the new tab does not, because the new tab's only channel is the file. This is a good failure in one specific sense — it fails toward *visible* disagreement rather than silent partial persistence, and the disagreement points directly at the missing component. It is a bad failure in that nothing announces it; the surfaces that worked give no hint that a consumer was left behind. The compensating design is that the file is the source of truth, so as soon as the host returns and any surface writes a scheme again, every renderer converges. Nothing needs repair; the system only needs one more correct write.

If the **write lands but a renderer has not read yet**, there is a window in which two surfaces disagree. For the new tab that window is bounded by its poll interval; for a content script it is bounded by the storage change event, which is prompt; for the fork's mixer it is bounded by a repaint. Transient disagreement across a fan-out with per-consumer refresh is not a bug and should not be engineered away — the alternative is a coordination protocol whose failure modes would be far worse than a fraction of a second of mismatched neon.

If **two writers race**, the failure is real and was fixed rather than tolerated. Two concurrent writes — one carrying a scheme, one carrying a UI preference — could each read the current state, modify their own field, and write the whole thing back, so the second write silently reverted the first and a scheme change appeared to “not take.” This is the classic lost update, and it is the one failure in the fan-out that the design refuses to absorb, because its symptom is indistinguishable from the feature not working. The host serializes the read-modify-write behind a cross-process lock.

If the **push loops**, the guard catches it. The worker pushes the scheme to the power-tool, and the power-tool can push a user's pick back; without a guard, each push would trigger the other. A record of the last pushed value breaks the cycle. This is worth naming as a category: any bidirectional sync between two stores that both emit change notifications needs an echo guard, and the guard belongs on the side that can distinguish “this value came from me” — which is the pusher.

## Path B: a browser action that must run exactly once

The automation bus has a stricter correctness requirement than the scheme fan-out, because its effects are not idempotent. Recoloring twice is recoloring; closing a tab twice is closing two tabs. The path is a trigger running stryke code, the host stamping the action into a file-backed key-value entry, and the service worker's single executor reading it and issuing the Chrome API call.

The failure that shaped the design was a **lifecycle failure**: a one-shot native message issued from a page's content script lost its reply because the worker was torn down during the native round trip. The symptom was total — a zero-percent success rate from external pages — which is, perversely, the easiest kind of failure to diagnose, because nothing worked at all. The fix delivers the reply over the persistent port that already streams system stats and correlates it with a request id so the action fires exactly once. Two properties are doing the work there: the port is alive because it was alive for another reason, and the id makes a duplicated delivery detectable rather than merely unlikely.

The failure the design deliberately does *not* guard is an **unknown verb**. The host strips the `browser.` prefix and stamps whatever verb it was given; it never consults a list, so discovery is advisory and execution is open. A verb the executor has no branch for is a no-op. That is a real gap and it is a chosen one: the alternative is that the host carries a copy of the verb list and every new capability requires changing two components in lockstep, with a version-skew failure mode of its own — a host one release behind would reject verbs the executor could handle perfectly well. The trade is that a typo fails silently instead of loudly, which is the right side of the trade for a surface where the executor is the single place a browser action is ever run.

The concentration of execution in one function is itself a failure-containment decision. Four surfaces can ask for a browser action — a page's palette, the new-tab palette, a HUD page, a script from a terminal — and the new-tab page in particular is a different extension with isolated storage, bridging through `externally_connectable`. Because all four converge on one executor, there is exactly one place where an action can fire, one place to add a capability, and one place a bug in dispatch can live.

## Path C: a keystroke broadcast to eight panes

Synchronize-panes is the least reliable path in the system and is the one where that is the correct design. A key typed in a pane is forwarded up as a semantic token, relayed down to every other pane in the group, and applied by each pane against its own focused field, dispatching a real `input` event so the framed page's handlers see the change. There is no acknowledgement anywhere in that chain.

Consider what the alternative would cost. To make the broadcast reliable you would need each pane to confirm application, the top frame to track outstanding confirmations, and some policy for a pane that never answers — a cross-origin iframe running an arbitrary site, which may be slow, may have navigated, may have lost its focused field, or may have been closed between the keydown and the relay. The policy for a non-answering pane would have to be either “block the typist,” which makes the feature unusable, or “proceed anyway,” which is what fire-and-forget already does with none of the machinery.

The design instead makes each hop *locally correct* so that a failure is confined to one pane. The most important instance is why **tokens travel instead of results**. When you press `Ctrl-w`, the top frame does not compute the resulting text and send it; it sends the kill-word token, and each receiving pane walks back over trailing whitespace and a run of non-whitespace *in its own field* and deletes that span. If the panes' fields hold different text — which they will, the moment one site autocompleted and another did not — a broadcast of resulting text would corrupt seven fields to match one. A broadcast of intent leaves each field correct with respect to its own contents. The same reasoning explains the last-focused fallback: a pane whose field lost focus applies the edit to the field it most recently had, because the alternative is dropping the keystroke in exactly the pane the user was not looking at.

The residual failure is honest and small: a pane that is genuinely unable to apply an edit silently does not, and the typist finds out by looking. For a feature whose purpose is typing one command into eight terminals at

once, that is the right failure — the same one you would get typing into eight real terminals where one had scrolled away.

## The one path that does not tolerate partial execution

Three paths above are safe under partial execution because their writes are idempotent, their truth is single-sourced, or their failures are confined. The terminal is none of those. A PTY is a stateful byte stream, and a dropped write is not a missing repaint; it is a corrupted command line, possibly a half-typed destructive command. There is no reconciliation step that fixes it after the fact.

So the terminal is the one place the design pays for ordering and delivery instead of routing around them. The bytes travel over a long-lived port rather than a bus, because a stream needs an attached receiver by definition. Output produced while no page is attached is buffered and flushed on reconnect rather than discarded, so navigating does not lose the shell's output. The spawn is honored once per tab, so a re-injected content script re-attaches to the running shell instead of forking a second one behind the same UI — which would produce the genuinely confusing failure of two shells sharing a screen. A content-script disconnect leaves the native port alive, because the page's lifetime is not the session's lifetime; only the tab's removal ends it. And the palette's shell one-liner waits — polling for up to ten seconds until the PTY is actually alive — before writing the command, rather than writing optimistically into a session that may not exist yet.

Every one of those is a mechanism the other three paths do without. That is the chapter's point: partial-failure tolerance is not a property you apply uniformly, it is a property you buy where the cost of failure justifies it. The scheme fan-out gets idempotence and a single source of truth and nothing else, because a stale color for a second costs nothing. The automation bus gets exactly-once delivery and an open verb surface, because double execution is unacceptable and an unknown verb is merely useless. The broadcast gets local correctness and no delivery guarantee, because confirmation would cost more than the failure. The terminal gets ordering, buffering, a spawn guard, and a readiness poll, because a stream cannot be repaired. Reading the four together is reading the actual engineering judgment in the system, which is not “make everything reliable” but “know what each path costs when it half-runs.”

# Chapter 33 — The Cost of Reuse

The book has said “extracted verbatim,” “consumed as a submodule,” and “promoted into the shared toolkit” in a dozen places, always approvingly. That approval is earned but it is not free, and a chapter that only celebrated reuse would be leaving out the half of the story a maintainer actually lives with. `zwire` consumes shared code three different ways, and the differences between them are not stylistic — each is the correct answer to a different constraint, and choosing the wrong one produces a specific, predictable class of failure. This chapter is the taxonomy and the tax.

## Three consumption models

**The submodule** is the default: `zgui-core`, `zpwchrome`, and the native `zwire-host` are all git submodules with their own repositories, their own CI, and their own release cadence. The condition that makes a submodule right is that the shared thing is a *project* — it has consumers other than this one, tests of its own, and a version history worth tracking separately. The payoff is that an improvement upstream reaches `zwire` with a pointer bump instead of a hand-merge, and that there is exactly one canonical copy rather than a frozen fork drifting quietly. `zgui-core`’s own guidance is emphatic on this point: consume it as a submodule and load modules directly from the submodule path, never copy the component tree, because copies go stale and partial-sync.

**The verbatim copy** is used exactly where a submodule cannot reach, and the constraint is an extension boundary. The palette’s backend-agnostic item source is vendored into each extension that carries a palette, and the scheme table exists in more than one place — once for the new tab and once for the internal HUD — because the two run in different worlds. This is not a lapse in discipline; it is forced. A Chrome extension cannot load a script file out of a *different* extension’s directory at content-script injection time. The new-tab extension and the internal HUD are separate extensions with separate packages, so a file both need is a file both must contain. The discipline that makes it survivable is that the copies are *verbatim* and declared as such — the point of vendoring the palette’s item source into each extension is precisely so the four palette implementations cannot drift, and the value of saying so in the file is that the next maintainer edits the source rather than the copy.

**The shim-adapted shared component** is the most interesting of the three, because it is what happens when a shared component’s assumptions do not match the consumer’s environment. The terminal front end is the suite’s shared embedded-terminal component, and its own header records that the only `zwire`-specific change is a transport shim that picks a transport at load. The tiling overlay is the shared window manager plus a provider that supplies iframe panes and a pane-side forwarder. The statusbar is the shared powerline bar plus a thin adapter that feeds it from storage keys. In each case the *component* is shared unchanged and the *difference* is isolated into a named adapter, which is the pattern that lets one implementation serve a browser extension and a desktop application at the same time.

## The promotion migration, as a case study

The two largest content scripts in the HUD — the tiling overlay and the powerline statusbar — both began as bespoke `zwire` code and were later promoted into the shared toolkit. That migration is worth walking, because “move the file into the shared repo” describes almost none of the actual work.

The overlay’s original form assumed it owned its panes. Once it became a shared window manager consumed by desktop applications as well, that assumption no longer held for `zwire` specifically: `zwire`’s panes are

cross-origin iframes, so the shared manager cannot listen for keys inside a pane, cannot read a pane's selection, and cannot apply a keystroke to a pane's field. The shared component therefore had to grow an *external-feed* surface — entry points through which a consumer outside the component can inject a prefix, a command key, a relayed sync keystroke, and a yank. That API exists in the toolkit at a specific version precisely so a cross-origin-iframe consumer can feed the manager from outside. The migration was not a move; it was an API design exercise driven by the one consumer whose environment was hardest.

On *zwire*'s side the single script split into two. The top frame loads the shared manager, its widget dependencies, and a provider that initializes it with iframe-backed panes — an address bar, the pane creation and render hooks, and bridges for applying keys, entering copy mode, and pasting, all over `postMessage`. Every frame loads the pane-side forwarder extracted from the old script, which relays the prefix, synced keystrokes, and yanks upward and runs copy mode locally over the pane's own document. The model, the command set, the copy-mode behavior, and the persistence are all unchanged; what moved is where the code lives. The retired original stays in the tree for reference and is no longer injected — which is itself a small honest decision worth naming, because deleting it would erase the record of what the shared component was derived from.

## The tax

Three costs come with all of this, and they are real.

The first is **version coupling**. Before the promotion, the overlay's behavior was determined entirely by a file in this repository. After it, the overlay's behavior is determined by the toolkit submodule's pointer, and the external-feed API the provider depends on exists only from a particular toolkit version onward. A submodule bump is therefore no longer a routine hygiene commit; it is a semantic event that can change how the browser's tiling behaves, and a bump *backward* — or a checkout that resolves the submodule to an older pointer — produces a provider calling an API that is not there. The failure signature is a feature that is present in the manifest and dead at runtime, which is a distinctly worse debugging experience than a file that is simply missing.

The second is **the absent-submodule case**, which *zwire* handles well precisely because it planned for it. A clone without recursive submodule initialization leaves the *power-tool*'s directory without a manifest, and the launcher's rule — include an extension directory only if it contains a manifest — silently skips it. The browser runs with what remains. This turns a missing dependency into a missing feature, which is the right conversion for an additive component, and it is why a fresh clone runs at all. The cost is that the failure is *quiet*: nothing errors, the capability is just not there, and a user who does not know the submodule exists has no way to discover that they are running a reduced browser. The failure-mode chapter has to name this explicitly for exactly that reason.

The third is **the copy that gets edited**. The verbatim-copy model works only as long as every copy is genuinely verbatim, and there is no mechanism enforcing that — no generator, no checksum gate, nothing in CI comparing the vendored item source across the extensions that carry it. It survives on discipline and on the files saying what they are. This is the weakest link in the reuse story and the honest thing to say about it is that its safety is a convention rather than a property. A divergence would show up as a palette that behaves subtly differently on the new tab than on an ordinary page, which is precisely the kind of bug that gets attributed to the surrounding surface rather than to the shared file.

## The rule

The three models sort cleanly by constraint, and the decision is nearly mechanical:

| constraint                            | model          | zwire example               |
|---------------------------------------|----------------|-----------------------------|
| own repo, own tests, other consumers  | submodule      | zgui-core, zwire-host       |
| crosses an extension package boundary | verbatim copy  | palette items, scheme table |
| shared component, other environment   | shim + adapter | terminal, tmux, powerline   |

Never copy what can be submoduled. Never submodule across a boundary that cannot load across it. When you must copy, copy verbatim, say so in the file, and edit the source. And when a shared component nearly fits, add an adapter rather than a fork — because a fork is a copy that has stopped announcing itself, and every cost above gets worse when the divergence is invisible.

The net verdict on the reuse discipline is favorable and should be stated as a measurement rather than a virtue: promoting the overlay and the statusbar into the shared toolkit made the HUD's own code smaller and its dependency graph *sharper* — fewer lines here, more explicit versions there. The adapters are the price, and they are cheap. But the price is paid in a currency the original bespoke scripts did not use at all: version management. That is the trade, and it is worth making, as long as everyone reading the manifest understands that a submodule pointer is now part of the browser's behavior.

# Chapter 34 — The Lineage: The Cyber-punk Idiom and the zpwr Fleet

zwire did not arrive from nowhere, and it is not a browser that happened to be painted with neon. It is the browser-shaped member of a larger family of tools that share an idiom — a keyboard grammar borrowed from tmux and vim, a powerline aesthetic, a neon palette — and, more concretely, share code. Understanding the lineage explains things the feature chapters can only assert: why the terminal is “extracted verbatim from Audio-Haxor,” why the fuzzy matcher is the same one in six other apps, and why a browser was, in a sense, the family’s missing member.

## The idiom: tmux, vim, and powerline as a grammar

The HUD’s controls are not invented from scratch; they are a dialect of a grammar the author already spoke everywhere else. The tmux overlay uses tmux’s prefix and tmux’s split, navigate, and zoom verbs, because a person who lives in tmux should not have to learn a second tiling vocabulary to tile browser panes. Vim on every page uses vim’s motions for the same reason: the cost of the feature is near zero for someone who already has the motions in their fingers, and the whole value of putting them in the browser is that the browser stops being the one place the grammar does not work. The powerline statusbar is the same aesthetic that terminal prompts and editor status lines in this world already wear — the segmented, arrow-jointed bar that encodes state as colored runs. None of these is a novelty; each is a *continuation*, and that is the point. The HUD is legible on first sight to anyone who came from tmux, vim, and a powerline prompt, because it is those three grammars wearing a browser instead of a terminal.

## The fleet: shared submodules across the house

The lineage is not only aesthetic; it is literal shared code, tracked as git submodules. `zgui-core` is the shared toolkit — the fuzzy matcher, the palette, the visual effects, the fonts — consumed by Audio-Haxor, `zmax`, `zpwr-daw`, `traderview`, `ztranslator`, `zpdf`, `zemail`, `zoffice`, and more, and by `zwire`’s internal HUD among them. `zpwr-i18n` is a shared translation catalog. The native `zwire-host` is itself a submodule with its own CI and README, written to be embedded by sibling hosts. Because these are submodules rather than copied snapshots, a fix to the fuzzy matcher or a new palette behavior propagates to every consumer through the same pointer bump, which is why the book can describe a feature as behaving “the way it does in the other apps” and mean it exactly: it is the same code. The fleet is not a set of apps that resemble each other; it is a set of apps built from a shared kit, and `zwire` draws on that kit the way the others do.

## Verbatim extraction as a design value

The most distinctive habit in the lineage is the refusal to re-implement. The terminal front end is not `zwire`-original — `terminal.js` is the shared `zpwr-embed-terminal` component, extracted verbatim from Audio-Haxor, with its header recording that the only `zwire`-specific change is a transport shim. The palette and the find bar are not `zwire`-original UI. The fuzzy matcher is the shared one. This is a value, not an accident: when a component is *extracted verbatim* rather than forked and edited, every app that carries it benefits from every fix, and no app accumulates a private divergence that has to be maintained alone. The transport shim pattern is

how this survives real differences between hosts — the terminal component auto-detects whether it is talking to a Tauri backend, a JUCE backend, or, in *zwire*, a native messaging port through a service worker, and the same component runs over all of them because the *difference* is isolated to a shim while the *component* stays shared. The discipline is the same one the native host follows with its library crate: write the capability once, embed it many times, and let the seam absorb the platform.

## Why a browser was the missing member

Seen against the fleet, *zwire* fills an obvious gap. The family had an audio workstation, an editor, a translator, a PDF tool, a mail client, an office suite — tools that each put the *tmux/vim/powerline* grammar on a specific task. The one application everyone uses all day and nobody in the family had reshaped was the browser, and the browser is the hardest one, because you cannot reshape it with a Tauri window: as Chapter 2 argued at length, the power the fleet's idiom wants — the extensions, the native chrome — exists only inside a real Blink engine. So the browser waited until the approach was clear: not a from-scratch engine and not a WebView shell, but a real Chromium owned at the seams a rebrand and a patch series can reach. *zwire* is the fleet's grammar finally applied to the application that resisted it longest, which is why it reads as continuous with the others despite being, mechanically, the least like them.

## The neon palette as house style

The eight schemes are house style, not *zwire* style. The neon-on-near-black palette, the rationing of high chroma to edges and signal, the Share Tech Mono and Orbitron display faces — these are the fleet's look, and *zwire* wears them so that a person moving from the audio workstation to the browser to the editor sees one continuous world rather than three apps that happen to be dark. This is why the palette is a closed color *system* (Chapter 8) rather than a set of pretty colors: a shared look across many apps has to be defined as roles and derivations, so that a new app can adopt it correctly and an old one can gain a light mode without every surface being hand-tuned. The cyberpunk idiom is the family's signature, and *zwire*'s contribution to it is the hardest surface the family had ever had to paint — a browser's chrome, reached on the runtime path through content scripts and on the fork path through C++ — proving the house style survives all the way down to the native frame.

# Chapter 35 – Positioning: zwire Among the Custom Browsers

“Custom browser” is a crowded and imprecise category, and zwire is easy to misfile in it. This chapter draws the map of the neighborhood — qutebrowser, Vivaldi, Arc, plain theme extensions, Electron wrappers, and the from-scratch engine — and places zwire precisely against each, because the value of the project is clearest when you can see exactly which trade every neighbor made that zwire refused, and which one it made that they did not.

## The five other answers

There are five well-trodden ways to get a browser that is not stock Chrome, and each is a coherent answer to a different question. **qutebrowser** is keyboard-first and vim-modal like zwire’s page layer, but it is built on Qt WebEngine and its own Python application, so it is a different application wearing a Chromium *render* engine rather than a Chromium *browser* — the extension ecosystem is not there. **Vivaldi** is a heavily customized Chromium with tiling, sessions, and deep UI configuration, shipped as a closed product on top of the engine; it is the closest in *features* and the furthest in *model*, because it is a company’s fork you consume, not a rebrand you own. **Arc** reshaped the browser’s information architecture — spaces, a sidebar, a different notion of a tab — on a Chromium base, as a designed product. **Theme extensions** are the floor: they repaint the frame and nothing else, and Chapter 1 spent its length on exactly why that floor is where a rebrand that only themed would stop. **Electron and Tauri wrappers** are the other pole: they give you total control of a window’s contents at the cost of not being a browser at all — no address bar you did not build, no extension host, and, for Tauri, not even a Blink engine.

## What zwire shares with each, and where it diverges

zwire shares qutebrowser’s conviction that the browser should be driven from the keyboard, and diverges by keeping the full Chrome extension host — zwire’s vim is a content script layered onto a real Chromium, not a modal application that replaced it, so **zpwirchrome** and its MV3 permissions still run. It shares Vivaldi’s and Arc’s ambition to reshape the whole browser, and diverges on ownership: Vivaldi and Arc are products you receive, while zwire is a rebrand and a patch series you apply to a snapshot Google builds, so nothing sits between you and the upstream engine except a shell script and, optionally, twenty-seven readable patches. It shares the theme extension’s mechanism on the runtime path — it is an extension-and-flags rebrand — but pushes that mechanism far past where a theme stops, into content, behavior, and the internal pages. And it shares nothing important with the Electron wrapper, because the wrapper gave up the one thing zwire’s whole justification rests on: it is a real browser, with a real address bar, a real extension host, and a real Blink engine, not a single web view in a native window.

## The runtime rebrand versus everyone who forks

The sharpest divergence is structural, and it is the two-tier design itself. Every neighbor that reshapes the native chrome — Vivaldi, Arc — does it by *maintaining a fork*, which means they own a divergence from upstream permanently and pay to carry it forward forever. zwire’s insight is that most of what people want from

a custom browser does not actually require the fork: colors, content, behavior, the new tab, the internal pages, the vim layer, the tiling, the terminal — all of it lives on the runtime path, on top of a stock snapshot that tracks upstream for free. The fork exists only for the residue that genuinely needs C++ — tab shape, the UI font, the native frame, the audio engine — and it is offered as an *option*, not imposed as the price of entry. No neighbor draws this line. The theme extension stops short of it; Vivaldi and Arc pay past it for everyone. *zwire* is the only one that put the line exactly where the capability boundary is and let you choose which side to stand on.

## The cost *zwire* chose not to pay

Positioning is as much about refusals as features, and *zwire*'s defining refusal is the one Chapter 1 named: it does not write a browser. A from-scratch engine is a decade of work nobody finishes, and it buys independence from Google's engine at a price no small project can afford. *zwire* refuses that cost and inherits Blink's engine, security model, and update cadence instead — which is why it exists in kilobytes of shell and JavaScript rather than millions of lines of C++. It also refuses the Electron cost, which is the opposite trade: total window control in exchange for not being a browser. And on the daily-driver path it refuses the fork's cost — the recurring rebase tax of Chapter 21 — by making the fork optional. The through-line is that *zwire* is unusually clear-eyed about which costs buy something and which do not: it pays for a dedicated profile, an extension set, a native host, and (optionally) a patch series, and it declines to pay for an engine, a runtime, or a permanent fork it does not need.

## Who *zwire* is for

The honest answer to “who is this for” falls out of the positioning. It is for someone who already lives in *tmux*, *vim*, and a powerline prompt and wants the browser to stop being the exception — who values the keyboard grammar enough to want it on every page. It is for someone who wants a real Chromium with real extensions, not a web view in a native shell, so the *WebView* wrappers are out. It is for someone who would rather own a rebrand of an upstream snapshot than consume a company's fork, so the update cadence stays Google's and the customization stays readable. And it is for someone willing to run a separate profile for their power tools, keeping their real logins in their real browser. It is not for someone who wants a supported product with a company behind it, or who needs the native-chrome HUD without being willing to build and rebase, or who wants their everyday and their experimental browser to be the same profile. The category “custom browser” holds all of these people; *zwire* is the answer for the specific one who wants the fleet's grammar on a Chromium they own.

# Chapter 36 — Failure Modes and Recovery

Chapter 24 gave the degradation ladder — the principled account of what is fatal and what is best-effort. This chapter is the field guide beside it: the symptoms you will actually see, the cause behind each, and the fix, ordered roughly by how often you hit them. It is the chapter you turn to when `zwire` is misbehaving and you want the one command that sets it right, and it closes with the recovery kit — the small set of scripts that resolve almost everything.

## The base is gone or wrong

The one fatal condition is a missing base, and it announces itself clearly: a fresh `zwire` on a machine with no base recorded exits with the exact `run: ../scripts/fetch-base.sh` message rather than failing opaquely. The fix is the command it printed. The subtler version is a *wrong* base — a snapshot that is too old for the `minimum_chrome_version: 127` the power-tool declares, or one that lost the `--load-extension` switch. The tell is that the browser launches but the extensions do not load or the power-tool refuses to; the fix is `fetch-base.sh` to a current snapshot, or `fetch-base.sh <revision>` to pin a known-good one. The base is the floor of the whole stack, so almost any “nothing works” symptom is worth checking against the base first: `base.path` and `base.version` at the top of the state directory are the entire contract, and if `base.path` points at a binary that is gone, everything above it fails in confusing ways until you re-fetch.

## The rebrand reverted (macOS)

The most common cosmetic failure, and the one Chapter 24 warned would happen, is the Dock reverting to “Chromium” after a base upgrade. The cause is mechanical: `fetch-base.sh` unzips a fresh stock bundle over the patched one, so the name, the icon, the deleted `CFBundleIconName`, and the ad-hoc re-sign are all gone. The fix is `rebrand-macos.sh`, the one manual step an in-place upgrade must not forget — which is why `install.sh` chains it on a first install but leaves it to you on an upgrade. If the rebrand ran but the old icon lingers, the cause is one level further out: macOS’s LaunchServices cache is still serving the previous icon, and `bust.sh` forces it to refresh. The order to try is: re-run the rebrand first, and only reach for `bust.sh` if the name is right but the icon is stale.

## The native host is silent

If the terminal will not open and the statusbar’s system stats are blank, the native host is not answering. This is not fatal — the browser runs fine without it, with one `stderr` line — but it disables the two features that depend on it. The launcher builds the host on launch when its manifest is missing or its target binary is gone, so the first fix is to relaunch and let it rebuild, or to force it with `FORCE=1`. If it still will not connect, the manifest is the thing to check: `com.zwire.hud.json` under the profile’s `NativeMessagingHosts/` must point `path` at a binary that exists and must list the extension’s origin in `allowed_origins`. On Windows the wiring is the registry rather than the profile dir, so a Windows host that is silent is a registry problem, not a file one. The daemon path has its own failure — a socket that will not open — and the cause there is almost always

permissions: the socket is `0600` inside a `0700` dir, so a socket owned by another user, or a stale socket file from a crashed daemon, blocks a new one until it is removed.

## A submodule is missing

If a feature is simply absent — no power-tool, no shared toolkit behaviors — the likely cause is an uninitialized submodule. `zpwchrome` and `zgui-core` are submodules, and a clone without `--recurse-submodules` leaves their directories without a `manifest.json`, so the launcher skips them silently and loads what remains. The symptom is a browser that works but is missing a whole capability; the fix is `git submodule update --init --recursive`. This is best-effort by design — a missing submodule degrades rather than crashes — which is exactly why it can be easy to miss: nothing errors, the feature is just not there. If a submodule is present but at the wrong commit, the tell is subtler still, a feature that half-works or behaves like an older version; the fix is to sync the submodule to the pointer the superproject records.

## Extensions did not load, or the wrong ones did

If the developer-mode banner appears but a specific extension is not running, the staging is the place to look. The launcher copies the loaded extensions into `~/.zwire/ext/` at launch rather than running them in place, so a stale copy there can shadow a change you made to the source. The fix is to clear the staged copy and let the launcher restage — the dev-loop scripts (Chapter 22) do this as part of a rebuild. If the *wrong* extensions loaded, or system Chrome's extensions somehow appeared, the cause is a profile confusion: `zwire` must be running against `~/.zwire/profile`, and if `$ZWIRE_STATE` or the profile path were overridden to point elsewhere, the browser is loading a different world. The `state-dir.sh` resolution and the `base.path` contract are the two things to verify when the browser is running but is not the browser you expected.

## Schemes desynced

If the new tab is one scheme and the internal pages are another, the sync bus has a stale reader. The scheme lives in a native file the host owns, written atomically, and every surface reads it — the new-tab poller, the internal HUD, the fork's color mixer. A desync means one reader did not see the last write: usually a page that was open across the change and did not re-read, which a reload fixes, or a host that is not running so the pub/sub push never fired, which folds back into the silent-host case above. The mirrored `zb_scheme` storage key is the cross-script copy; if it and the native file disagree, the native file is the source of truth and the fix is to re-trigger a scheme write (set the scheme again) so every subscriber gets the push.

## The fork will not rebase

On the fork path, the recurring failure is a patch that no longer applies after upstream moved, and Chapter 21's economics section is the frame for it. The `check-patch-series.sh` gate catches the structural drift — a patch on disk missing from `series`, or a `series` entry with no file — before anything applies. A patch that applies but is *wrong* after a rebase is the dangerous case the CI cannot catch, and the audio patches are the ones to re-verify by hand. The pragmatic recovery is the one the series file already models: if a patch has become fragile, back it out with a commented-out `series` line and a note, as `0008` was, rather than carrying a broken one forward. And the ultimate fallback is the two-tier design itself — if the fork will not build, the runtime rebrand still runs on a stock snapshot, so a broken rebase costs you the native-chrome HUD, not a working browser.

## The recovery kit

Almost everything above resolves to one of a small set of scripts, and it is worth holding them as a kit. `fetch-base.sh` re-establishes the base, pinned or latest, and fixes the fatal case and the wrong-base case. `rebrand-macos.sh` re-applies the identity after any base swap, and `bust.sh` clears the LaunchServices cache when the icon is the only thing stale. `setup-native-host.sh` (or a relaunch, or `FORCE=1`) rebuilds and rewires the host when the terminal and stats go quiet. The dev-loop rebuild restages the extensions when a stale copy shadows a change. `git submodule update --init --recursive` restores a missing capability. And `nuke.sh` is the bottom of the ladder — a full reset of the state directory back to nothing, after which a single `install.sh` (or `fetch-base.sh` plus `rebrand-macos.sh`) rebuilds the world from the base up. The reason the kit is this small is the reason the whole browser is this small: zwire is mostly a shell script wrapped around a stock engine, so almost every failure is a missing file, a stale cache, or a reverted patch — and each has one command that puts it back.

# Chapter 37 – Text Input, Composition, and the Keyboard That Is Not Yours

Chapter 28 settled who owns a keystroke. It did not settle what a keystroke *is*, and that turns out to be the harder question. A keyboard layer built on single-character bindings — `j` scrolls, `x` closes, `f` starts link hints — is making an assumption about the relationship between a physical key, a character, and an intention, and that assumption holds cleanly for exactly one configuration: a US QWERTY layout, no input method engine, no dead keys, no modifier that composes. Every other configuration in the world stresses it somewhere. This chapter is about those stresses: which of them the model the book has described already absorbs, which ones it absorbs by accident, and which are genuinely open.

It is worth saying plainly at the start that this chapter reasons *forward* from the design the earlier chapters established rather than backward from a file. Where the book has stated a mechanism — the capture-phase listener, the editable guard, the key-to-action table rebuilt from the registry at dispatch time, the prefix state machine and its timeout — this chapter takes it as given and works out what follows. Where the consequence depends on an implementation detail the book has not stated, it says so rather than guessing.

## The character, the key, and the physical position

A browser hands a keyboard event three different descriptions of the same press, and they answer three different questions. One is the *character produced*, which is what the user believes they typed and which already accounts for the active layout, the modifiers held, and any transformation the OS applied. One is the *physical position*, which identifies the key by where it sits on a notional US keyboard regardless of what the user's layout prints on it. And one is the legacy numeric code, which is neither and which exists because it predates the other two.

The choice between the first two is not a style preference; it is a decision about what a binding *means*. A binding expressed as a character means “the key that types this letter,” which is stable across layouts in the sense the user cares about: a Dvorak typist who has spent a decade reaching for `j` with the index finger of the right hand will find `j` wherever their layout puts it, and the vim binding follows their fingers because their fingers follow the letter. A binding expressed as a physical position means “this key,” which is stable in a different sense: the same piece of plastic in the same place, whatever it prints. Games choose position. Text-adjacent tools choose character, because their users learned the binding as a letter in a mnemonic — `f` for follow, `x` for close, `d` for down.

The registry model the book describes is a character model. Chapter 28 states the mechanism directly: defaults live in `zkeys.js` as data, user remaps live in storage as `{action: key}` pairs merged over the defaults, and rebinding an action from `j` to `s` rewrites a table entry that the next keystroke on every open page dispatches through. A table whose keys are single characters and whose remap surface lets a user type the character they want is a character table, and the mnemonic vocabulary the HUD inherits from vim and tmux is a mnemonic vocabulary, so this is the correct choice. It is worth naming as a choice, though, because it has two consequences that only appear on layouts the author does not use.

The first is that a layout where the mnemonic letter is unreachable makes the binding unreachable. On a layout where a character requires a modifier that the motion layer bails out on — Chapter 28's modifier bailout leaves anything held with `Alt`, `Meta`, or `Ctrl` alone — a binding on that character cannot fire at all, because the layer that would claim it has already decided the press belongs to someone else. This is not a bug in the bailout; the bailout is what keeps the motion layer from eating the palette's and the find bar's chords. It is a genuine

interaction between two correct rules, and the remedy is the remedy the design already provides: the binding is data, so a user on such a layout rebinds it to a character their layout produces unmodified, and the shared registry means the keybinding editor and the overlay's help card both immediately describe the new reality.

The second consequence is subtler. A character table is only as stable as the layout is, and the layout can change *while a page is open*. Switching input source is an OS gesture, not a browser one, and nothing in the page is notified. The design absorbs this without doing anything, because the table is consulted at dispatch time rather than bound at registration time — the same property Chapter 28 introduced for a quite different reason. A listener that had bound one handler per key at injection would hold a set of handlers keyed to the layout that was active when the page loaded; a listener that consults a table on every press simply looks up whatever character the new layout produced. Dispatch-time lookup was chosen so remaps take effect without a reload; it pays for layout switching as a side effect. That is the pleasant kind of design accident, and it is worth recording precisely because nobody planned it.

## Composition: the case where a keystroke is not an intention

The genuinely hard case is an input method engine. When a user is composing text in Japanese, Chinese, Korean, or any of the other scripts that require an IME, the keystrokes they type are not characters they intend to insert. They are input to a composition session that the IME owns, and which will eventually produce a completely different string. During that session, a press of `j` is not the letter `j` in any sense the user would recognize; it is a syllable fragment, a candidate selection, or a navigation within a candidate list.

A capture-phase listener on `document` sees these presses. That is the whole point of capture-phase listening — it runs ahead of the page's handlers — and the IME's composition is happening in a field the page owns. So the arrangement that makes vim beat a site's single-key shortcut also puts the HUD's listener in front of an in-flight composition, and a layer that claimed `j` there would break text entry in text entry for every script that requires one.

What saves the common case is the guard that was written for a different reason. Chapter 28's editable guard states that inside an `input`, `textarea`, `select`, or `contentEditable`, vim claims nothing except `Escape`, with the justification that in a text field every letter is already mapped by the field. Composition almost always happens inside exactly such an element, so the guard that exists to stop vim eating your search query also stops it eating your composition. This is the same shape as the layout-switch case: a rule written for the obvious reason covering a non-obvious one, and it is worth being explicit that the coverage is *incidental*, because incidental coverage is fragile in a specific way. The guard is scoped to a node type. Composition is scoped to a composition session. Those two scopes coincide in ordinary use and diverge in the cases that are hardest to test.

Where they diverge is worth enumerating honestly, because the book has been clear elsewhere that naming a gap is more useful than implying completeness. The clearest divergence is hint mode. Chapter 28 records that the guard ordering inside the vim listener puts hint mode *before* the editable guard, and gives the reason: typing a hint label is not typing into a field even when a field happens to be focused, and reversing the order would make link hints unusable on any page with a search box. That ordering is correct for its stated purpose and it is exactly the ordering that takes hint mode outside the guard that would have protected a composition. A user who arms link hints while an IME session is live is in a state where two systems both believe they own the next keystroke, and the model as described has no arbitration for it. The practical impact is small — arming hints mid-composition is an unusual thing to do — but “unusual” is not “impossible,” and the honest statement is that this is an unarbitrated corner rather than a solved one.

The second divergence is the platform's own signal for the condition. Browsers do expose whether a key event belongs to an in-flight composition, and they emit explicit start and end events around the session. A layer that consulted that signal directly would be scoped to the actual condition rather than to a proxy for it, and would cover the hint-mode corner and any future feature that claims keys outside a field. The book does not claim the HUD does this, because the book has not read a file that says so. What the book can say is what the

correct shape would be if it does not: a composition check belongs at the very top of the guard ladder, above hint mode, because composition is the one state in which the user's keystrokes provably do not mean what they appear to mean.

## Dead keys and the two-press character

A dead key is a smaller version of the same problem, and it stresses a different mechanism. On layouts with dead keys, a character like `é` is produced by pressing an accent key that produces nothing visible and then a letter key that combines with it. Two physical presses, one resulting character, and an intermediate state the user is in for the duration.

The interesting collision is with the prefix state machine. Chapter 28 describes the overlay's `Ctrl-b` prefix and vim's one-key prefixes — `g`, `z`, `y`, `m`, and the backtick — as a state in which “one claimant owns the next key unconditionally, which resolves the contention for that keystroke by fiat instead of by rule.” A dead key is structurally identical: the OS also owns the next key unconditionally, for its own reasons, and it armed first because it sits below the browser entirely. Two prefix machines stacked, one in the OS and one in the page, both consuming the same next press.

The resolution is layering, and it favors the OS: a dead key's combination happens before the browser produces a character event at all, so the page sees the composed result and never the intermediate. That means a page-level prefix armed *before* a dead-key sequence receives the composed character as its post-prefix key — an accented letter that almost certainly maps to nothing in the tmux table, which clears the prefix harmlessly. It also means the reverse ordering is invisible: a dead key armed while nothing else is pending simply produces a composed character the HUD sees as one press. The stacking is benign, but only because the timeout exists. Without the 900-millisecond prefix timeout Chapter 28 records, a prefix armed and then followed by a sequence the table does not recognize would leave the overlay waiting indefinitely for a command key that is never coming. The timeout was written as a courtesy to a user who pressed the prefix by accident; it is also the thing that keeps a prefix from being stranded by an input path it knows nothing about.

## The macOS Option key, and why the alternate prefix is a hazard as well as a help

Chapter 28 records the tmux prefix as `Ctrl-b` with `⌘-b` as an alternate, and gives the historical reason the alternate exists: an earlier native split patch consumed `Ctrl-b` before the page saw it, and the alternate outlived the patch because a second prefix that nothing else intercepts is a useful escape hatch.

It is worth naming the cost of that particular alternate, because it lands exactly on this chapter's subject. On macOS the Option key is not only a modifier; it is a character-producing modifier. Option plus a letter yields a different character, and Option plus certain letters is itself a dead key that arms an accent. So `⌘-b` on a US layout is a chord that also produces a character, and on some layouts it is a chord that also arms a dead key. A binding on that chord is therefore doing something the `Ctrl-b` binding is not: it is claiming a keystroke that the text-input system would otherwise have turned into content.

Chapter 28's invariant handles this correctly and it is worth tracing why. The rule is claim only what is mapped, claimed completely with both `preventDefault` and `stopImmediatePropagation`. Because `⌘-b` is mapped, the claim is legitimate and the suppression of the character it would have produced is intentional rather than collateral. And because the editable guard fires first, the claim does not apply inside a text field, which is the only place the character would have been wanted. The chord is claimed where it means “arm the prefix” and released where it means “type a character,” which is the correct resolution and is produced by rules that were written before anyone thought about Option-as-a-character-modifier. That is the third instance in this chapter of a general rule covering a case it was not designed for, and the pattern is worth extracting: a keyboard layer that arbitrates by *condition* — is this mapped, is this field editable, is a prefix armed — degrades gracefully on inputs its author never saw, while one that arbitrates by *enumeration* fails on the first input not in

the list.

## The terminal is the exception, and it is exempt for the right reason

Everything above is about a layer that must share the keyboard with text entry. The terminal shares nothing: while it has focus, Chapter 28 records that it installs a capture-phase handler forwarding `Ctrl+<letter>` combinations straight through as control bytes, because those chords mean line editing in a shell and the web view would otherwise consume several of them for its own text bindings.

That is a total claim over a range of chords, and it is safe precisely because it is focus-scoped. When the terminal has focus, the user is not composing text into a web page; they are typing into a shell, and the shell's own line editor is the thing that should interpret their keystrokes. The exemption is not “the terminal is special”; it is “the terminal is a different text-entry system, so the browser's text-entry rules do not apply inside it.”

The consequence worth stating is that this makes the terminal the one surface where the IME story is genuinely out of the HUD's hands. Composition inside a terminal emulator is a hard problem in every terminal emulator, native ones included, because the composition UI wants to render inline over a grid the shell owns. Whatever the embedded terminal component does about it is what zwire does about it, and that is a correct place for the responsibility to sit: the terminal is a shared component consumed from the toolkit, and the right fix for a text-input gap in a shared component is a fix in the shared component, where every consumer in the fleet gets it. Chapter 33 made the general version of this argument about the cost of reuse; this is the case where the reuse discipline pays, because a text-input problem solved once in the toolkit is solved for the browser, the editor, and the audio workstation at once.

## What a reader should take from this

Three things generalize past zwire.

Bind by character when the vocabulary is mnemonic, by position when it is spatial, and never mix the two in one table — a table with some entries meaning “the letter” and some meaning “the key” is unexplainable to a user on any layout but the author's.

Guard by condition rather than by enumeration. Every case in this chapter that the design absorbed, it absorbed because a rule tested a property of the current state — is this element editable, is this key mapped, is a prefix armed — rather than checking membership in a list. Every case that remains open is one where the tested property is a proxy for the real condition rather than the condition itself, which is exactly the relationship between “the focus is in a text field” and “a composition is in flight.”

And put the timeout in. A prefix machine without one is a state that some input path you have never heard of will eventually strand, and the input paths you have never heard of outnumber the ones you have.

# Chapter 38 — Living on the Open Web

Every chapter so far has looked outward from the HUD: what it does, how it is wired, what it costs zwire to run. This one looks the other way. The internal-HUD suite injects into `<all_urls>`, which means every design decision in this book is also a decision imposed on every document on the web, written by people who have never heard of it. A browser extension that runs everywhere is a guest in every page, and the difference between a good guest and a bad one is not politeness — it is a small set of concrete disciplines about namespace, layout, event synthesis, and the platform defenses it chooses to relax. This chapter is those disciplines, the hazards they address, and the ones they do not.

## The namespace problem, and why it is worse than it looks

The HUD styles itself entirely in terms of CSS custom properties. Chapter 8 names them explicitly — the accent, the cyan, the magenta, the green, the red, the text weights, the background depths, the borders — as an ordered vocabulary that every HUD surface reads, so that swapping which scheme's values are written to `:root` recolors the entire browser. Chapter 11 records that the statusbar adapter injects the scheme onto `:root` so the shared powerline bar inherits the live theme.

Writing to `:root` is writing to the document's own root element, in the document's own cascade. Custom properties inherit, and they inherit into shadow trees, which is precisely why they are the standard mechanism for theming web components. So a value written at `:root` by a content script is visible to every element in the page, including elements inside shadow roots the script cannot otherwise touch. That reach is what makes the mechanism work for the HUD's own surfaces. It is also what makes it a namespace collision waiting to happen, because the names in question are exactly the names a site would pick. A design system that defines an accent color and a border color and a primary background is not being unimaginative when it calls them accent, border, and primary background; those are the words. If a site defines its own property with a colliding name at a lower-specificity origin, and the HUD writes the same name at `:root`, the site's components silently render in cyberpunk colors.

The mitigation available to any injecting extension is a prefix, and the tension is that a prefix costs exactly the reach that made the mechanism attractive. Chapter 8's color-science section names the HUD's roles in prefixed form when it describes the skinning layer painting roles rather than pixels — the page asks for the accent role, and the scheme table decides what the accent is this minute. The general rule for anyone building this is: a custom property written into a document you do not own must carry a namespace nobody else would choose, and the HUD surfaces that consume it must read the namespaced name. The cost is that a shared component promoted into the toolkit — the powerline bar, the tiling manager — has to be fed the namespaced values by its adapter, which is precisely the job Chapter 33 assigns to adapters. The cost is real and it is the correct place to pay it.

The same reasoning applies one layer up, to element identifiers and class names. Chapter 20 records the terminal content script injecting a `#terminalPane` into ordinary web pages, and Chapter 11 records the statusbar's segments carrying `data-*` hooks. An id is a document-global name. A page that already has an element with that id has a duplicate-id document, and every selector either party writes becomes ambiguous. This class of bug does not announce itself: the page keeps working, one of the two parties occasionally addresses the other's element, and the failure surfaces as an inexplicable misbehavior on one specific site. The discipline is the same as for properties — a namespace, applied consistently — and the reason to state it

as a rule rather than a habit is that ids are added casually, one at a time, by whoever is building a feature, which is exactly how a namespace erodes.

## Layout: the browser's viewport is not the HUD's

The statusbar is fixed to the bottom of every top-level page at a defined height, with a translucent blurred background. That is a deliberate, permanent occupation of the bottom strip of every document in the browser, and it collides with the open web in three distinct ways.

The first is a site's own fixed bottom furniture. Cookie banners, chat launchers, audio players, sticky action bars, and mobile-style bottom navigation all live in the same strip. Two fixed elements at `bottom: 0` do not negotiate; they stack, and which one is on top is decided by the z-index ladder. Chapter 10 records that the HUD has a z-index ladder for exactly this reason on the overlay side, and the statusbar's presence in the same contested band means the ladder has to account for a value the HUD does not control — sites routinely use very large z-index values, and a HUD element that wanted to be reliably on top would need to be above all of them. Being above all of them has its own cost: the HUD is then also above the site's modal dialogs, its video controls, and its own overlays, which is the wrong answer for a bar that is decorative most of the time.

The second is viewport-relative sizing. A layout built on full-viewport-height units computes against the viewport, not against the space the HUD left. A page that positions its own footer at the bottom of a full-height column now has that footer under a statusbar. There is no mechanism by which a content script can shrink the viewport a page computes against; the only options are to overlay and accept the occlusion, or to displace the document by offsetting the body, which breaks any element positioned relative to the viewport rather than the document. Overlaying is the lesser evil and it is the choice the design makes, and the honest statement is that the bottom strip of every page is a place where a site's content can be hidden. That the bar can be toggled from the palette — which Chapter 11 records — is not a workaround; it is the actual remedy, and it is why a toggle for an always-on decorative surface is not optional polish.

The third is scroll containers. A site with its own internal scrolling regions, or with a scroll-jacking implementation, interacts with vim's motions in ways neither party anticipated: the motion layer scrolls a document that the site believes it is managing. Chapter 28's invariant limits the damage — the motion layer claims only what is mapped, and a site that handled the key itself would have received it if the HUD had not — but it does not eliminate it, because the HUD's claim on `j` is precisely a claim to scroll in place of whatever the site would have done. This is the one place where the HUD's whole value proposition is in direct tension with web compatibility, and the resolution is the per-tab disable flag Chapter 30 lists among the state deliberately allowed to be lost: on a page where the site's own scrolling is the right behavior, you turn vim off for that tab and the raw keys go back to the page.

## Touching the DOM the way a user would

Chapter 29 states a discipline in one sentence that deserves more than one: the find bar registers highlight ranges rather than wrapping matches in elements, and the synchronize-panes receiver splices text into a field and then dispatches a real input event so the page's own handlers see the change — “touch the DOM the way a user would, not the way a script would, so the page cannot tell the difference and cannot be broken by the difference.”

Both halves of that are load-bearing and they fail in opposite directions.

Wrapping matches in elements is the classic implementation of an in-page find, and it is a structural edit to a document the extension does not own. It changes the node tree under the site's own scripts, which may be holding references, observing mutations, running a diffing renderer that will overwrite the wrappers on its next pass, or computing text content that now contains element boundaries where it expected a text run. On a modern application framework, injecting wrapper elements into rendered output is an invitation for the framework to either erase them or crash trying to reconcile them. Registering ranges with the platform's own highlight mechanism — which Chapter 13 covers — sidesteps all of it: the document is not modified, the

highlight is a presentation-layer concern, and a framework re-render simply invalidates ranges rather than fighting with foreign nodes.

Splicing text into a field is the opposite situation: here the extension *must* modify the document, because the whole point is to change the field's value. The hazard is that setting a value programmatically does not, by itself, tell the page anything happened. A controlled input in a modern framework holds its value in application state and re-renders from it; a programmatic write that produces no event leaves the DOM and the application state disagreeing, and the next render silently reverts the write. Hence the synthesized input event, which is not a formality but the actual mechanism by which the page learns. Chapter 32 adds the second half of the discipline for the broadcast case: tokens travel instead of results, so each pane applies the *intent* against its own field rather than receiving a computed string, because the panes' fields hold different text and a broadcast of results would corrupt seven fields to match one.

The generalizable rule is: read without modifying wherever the platform provides a non-destructive mechanism, and when you must modify, modify through the same events the user's own input would have generated. An extension that follows both is nearly invisible to the pages it runs on. An extension that follows neither is the reason users blame websites for extension bugs.

## The defenses the HUD relaxes, and the blast radius of relaxing them

The frame-busting rule is the sharpest thing in this chapter, because it is the one place the HUD does not merely coexist with a site's intentions but overrides them. Chapter 7 records the `frame_bust` declarative-net-request rule that lets the tmux overlay iframe any site, and Chapter 25 records the fork's corresponding relaxation of `X-Frame-Options` and frame-ancestors CSP.

Those headers exist for a specific reason: they are a site's declaration that it must not be embedded, and the attack they prevent is clickjacking — an attacker frames a sensitive page, overlays invisible or misleading UI, and induces the user to click a control they cannot see. Every bank, every payment flow, and every account-settings page that sets those headers is defending against exactly that.

The HUD's justification is real and it is worth stating precisely rather than waving at. The threat model those headers address is *an attacker's page framing a victim's page*. The overlay's situation is *the user's own browser, at the user's own instruction, framing a page the user navigated to*. The party doing the framing is the user, and the party being defended against is the user, which is not a coherent threat. This is the same reasoning that lets a browser's own reader mode, print preview, or developer tools show a page in ways the page did not sanction: the user's agent acts for the user.

What keeps that argument honest is scope, and Chapter 25 names the compensating controls: the relaxation yields for sub-frame navigation so a pane can load an otherwise-unframeable site, not for the top-level page, and the pane iframe carries a sandbox attribute set the overlay chose, so the embedded site runs with the overlay's chosen authority rather than ambient authority. The residual risk is not zero and the shape of it is specific: any page that runs in a zwire tab can, in principle, frame any other site, because the rule that strips the header is not scoped to the overlay's own frames in a way a page cannot reach. The containment for that is the containment for everything else in Chapter 25 — the dedicated profile — and the honest summary is that this relaxation is defensible because of the profile boundary, not independently of it.

## The re-injection problem, which is the web's fault and the HUD's problem

A single-page application navigates without a document load. The URL changes, the view changes, and from the extension's point of view nothing happened — or, depending on the navigation event, something happened that looks enough like a load to trigger another injection. Chapter 7 records that every script carries a load

guard “because content scripts can be injected more than once across an SPA’s frame and navigation events, and a second injection must be a no-op,” and Chapter 26 records that dropping the guard would stack handlers and make a long-lived tab slow with every same-document navigation.

The part worth adding is what a *correct* guard has to do beyond returning early. Returning early is right for a script whose only effect is to install listeners on a document that still exists. It is not sufficient for a script that installed UI into a subtree the application has since replaced, which is the ordinary outcome of an SPA route change: the guard says “already loaded,” the script returns, and the UI it installed is gone because the framework unmounted the container it lived in. The full shape of an idempotent injectable on the modern web is therefore a guard plus a re-attach path — detect that a prior installation exists, verify its DOM is still present, and re-mount if it is not — and the reason the HUD mostly does not need the second half is that its surfaces attach to the document body or to fixed-position containers of its own creation rather than into the application’s subtree. That is a design property worth naming as a defense rather than an accident: injecting *beside* the application rather than *into* it is what makes the guard sufficient.

## What good citizenship actually costs

Collected, the disciplines are five, and they are cheap individually and invisible collectively:

```
namespace    every property, id, and class carries a prefix nobody else picks
overlay      occupy fixed strips, never displace the document; offer a toggle
read-only    prefer non-destructive platform mechanisms over structural edits
synthesize   when you must write, write through the events a user would generate
attach-beside inject next to the application, not inside its subtree
```

The rule underneath all five is that the page is a hostile-by-default environment in the specific sense that it was written by someone who did not know you would be there, will change without telling you, and is entitled to every name and every pixel you did not explicitly claim. That framing produces the right instinct at every decision point, and it is the same instinct Chapter 28’s keyboard invariant encodes for keys: claim only what you are actually using, claim it completely, and leave the rest of the space genuinely unclaimed for whoever needs it next.

# Chapter 39 – Accessibility and the Neon Surface

A HUD built on color, motion, glyph density, and keyboard chords is not obviously an accessible interface, and a book that spent a chapter on the color science owes a chapter on who the colors do not work for. This one is deliberately uncomfortable in places. Some of what follows is a description of properties the design already has, some is an argument for why certain choices are better than they look, and some is a plain statement that a capability is absent. Distinguishing the three is the whole value of writing it down.

## Two accessibility stories, and only one is usually told

The word covers two quite different populations, and a keyboard-driven HUD sits at opposite ends of the spectrum for each.

For a user with a motor impairment, or repetitive strain injury, or anyone for whom the mouse is a cost, *zwire* is *unusually* accessible, and the feature that makes it so is the one the earlier chapters treated as a power-user convenience. Link hints — Chapter 9’s **f** — convert every actionable element on a page into a short typed label. That is the single most effective mouse-elimination technique that exists on the web, and it works on pages whose authors did nothing to support keyboard navigation, because it does not depend on the page’s tab order being sane. The vim motion layer means scrolling, paging, and jumping to the top or bottom of a document never require a pointing device. The palette means navigation, tab management, and window operations are reachable by typing a few characters. A browser where every operation has a keyboard path is a browser a person can use without a mouse at all, and that is a genuine accessibility property that *zwire* has by construction rather than by retrofit.

For a user with a visual impairment, the story is harder, and the honest version has both good and bad in it.

## Contrast: the work that was already done, and the gap that remains

Chapter 8 records a real piece of accessibility engineering without calling it one. The light schemes, it says, are the harder problem: inverting a dark cyberpunk palette does not produce a usable light one, so each role is re-derived against a light ground, and the dimmed foreground — “the one role most likely to fail a contrast check” — is pinned to a value that clears the readable threshold rather than being computed as a fixed alpha over the background, “because a fixed alpha is exactly what fails when the background changes from black to white.”

That is a contrast-first derivation, and it is the right method. Computing a dim foreground as a translucency over an unknown background is the single most common way a theme fails a contrast requirement, because the resulting contrast depends on a value the theme author did not control. Pinning the value instead means the ratio is a property of the palette rather than a property of the page.

The discipline Chapter 8 describes for the accent colors is also, incidentally, an accessibility discipline: the accent is rationed to borders, the active tab, the statusbar segments, the cursor, and the focus ring, and never used for body text, with the body foreground held as a high-value near-white well below full saturation. Saturated neon as body text on near-black is unreadable for everyone and disproportionately unreadable for

users with low vision or astigmatism, where high-chroma text on a dark ground produces visible halation. A palette that never sets prose in the accent has avoided its worst failure mode as a side effect of a design preference.

The gap is that none of this is verified anywhere in the build. Chapter 22 covers the dev loop and CI; the checks it describes are a JavaScript syntax gate, shell linting, JSON validation, and a patch-series gate. A contrast audit — computing the ratio of every foreground role against every background role in all sixteen palette derivations, and failing the build on a regression — is a mechanical check that the palette’s own structure makes easy, because Chapter 8 established that the palette is a small closed set of roles rather than a pile of colors. The table exists, the roles are named, the pairings that matter are enumerable. That such a check does not exist is a real gap, and it is the kind of gap that a book grounded in the tree should name rather than imply away.

## Motion, and the preference the platform already knows

The HUD ships visual effects — scanlines, a vignette, neon glow, and animation — and Chapter 8 records that they are toggled through the shared preferences, the four toggles travelling through the fleet-wide store so every surface follows. Chapter 17 records the notable inversion in how `ZGui.fx` applies them: an effect is *on* when its class is *absent*.

That toggle set is, functionally, an accessibility control, and it is a better one than most applications ship because it is granular: a user who is fine with glow but for whom animation is a problem can turn off animation alone. Vestibular disorders, migraine triggers, and photosensitivity are distinct conditions with distinct triggers, and a single “reduce effects” switch conflates them.

What the design has that it does not obviously use is that the platform already knows the answer for one of them. Operating systems expose a reduced-motion preference, browsers surface it to CSS, and a user who has set it has already told every application on their machine that animation is a problem. The correct default for the animation toggle is therefore not “on” but “on unless the platform says otherwise,” which costs a media query and removes the need for the user to discover a setting in a browser they just installed. The same argument applies to the other three effects with less force — scanlines and vignette are not motion — but a photosensitivity-driven reduced-motion preference is a reasonable signal for damping a full-screen glow as well.

The book cannot claim the HUD does this, because the book has not read a file that says so. It can say that the preference plumbing already exists, that Chapter 8’s store is the correct place for a platform-derived default to land, and that the fleet-wide nature of the store means solving it once solves it for every application in the family — which is the strongest argument for putting the work there rather than in the browser.

## Screen readers and the overlay problem

This is where the honest answer is least comfortable.

A screen reader presents a document as a linear traversal of an accessibility tree derived from the DOM. Anything a content script adds to the page is part of that tree unless it is explicitly excluded, and anything the script overlays visually is *not* thereby overlaid in the traversal — a screen reader user encounters the statusbar’s segments as content in the document, in whatever position the elements occupy, regardless of the fact that they are painted as a fixed bar.

That produces a specific, predictable annoyance: a system monitor’s worth of continuously changing numbers, inserted into every page in the browser, traversable as text. The correct handling is that decorative chrome is marked as hidden from assistive technology, so it paints for sighted users and does not exist for the traversal. The statusbar is the clearest candidate — it is a status display, not content, and its numbers change on the stream’s cadence, which if announced would be intolerable. Chapter 11’s `data-*` hooks mean the values are

already written into spans by a refresh function, so the exclusion would be a property of the container rather than something to maintain per segment.

The palette and the find bar are the opposite case: they are genuinely interactive, they take focus, and they must be *reachable*. A modal overlay that takes focus needs to announce itself as such, needs its input labeled, needs its results list to expose the active item so a screen reader can announce the selection as the user moves through it, and needs focus to return where it came from on dismissal. These are not exotic requirements; they are the standard obligations of a combobox pattern, and the palette is exactly a combobox. Chapter 17 records that the palette shell and the readline behavior are shared from the toolkit, which means — again — that this is work that belongs in `zgui-core` and that doing it once fixes it across the whole fleet, including the palettes Chapter 33 enumerates as verbatim copies in the other extensions.

The tiling overlay is the hardest surface and there is no comfortable answer for it. Panes are cross-origin iframes; a screen reader traverses each frame's document, and the visual arrangement — which pane is where, which is zoomed, which is active — is conveyed entirely by geometry and border color, neither of which reaches the traversal. Making a tiling window manager legible to a linear traversal is a genuinely unsolved interface problem and not one a browser extension is going to solve in passing. The defensible position is that the overlay is an *optional* surface, not part of the browsing baseline: a user for whom it is unusable does not turn it on, and everything else in the browser continues to work. That is a real answer, and it is only a real answer because the overlay genuinely is optional.

## Color as the only channel

Three signals in the HUD are carried by color alone, and each is worth checking against a user with a color vision deficiency — a population large enough that any interface shipping a hue-only signal will be used by someone who cannot read it.

The prefix lamp is the clearest case. Chapter 11 records that pressing the tmux prefix flips the block's background from dim to the scheme accent for a period and then fades back. That is a brightness change as well as a hue change, which is what makes it survive: a luminance difference is perceptible regardless of hue discrimination, and a signal built on “dim becomes bright” works where one built on “green becomes red” would not. The design got this right, and it got it right by accident of the palette's structure — Chapter 8's roles include dimmed and full-strength variants of the same hue, so the natural way to express “active” in this palette is already a luminance step.

The CI indicator is the case that needs the most care. Chapter 8 records the worker tallying failing, running, and passing counts into a status object “the statusbar can show as a dot.” A three-state indicator distinguished only by hue is the textbook color-only signal, and the three states in question are conventionally red, yellow, and green — the exact pairing that the most common deficiency makes hardest. The remedy is cheap and standard: carry a glyph or a count alongside the hue, so the state is legible without color discrimination. The statusbar already does this elsewhere — Chapter 11 records the battery segment prefixing a charging glyph before the percent rather than relying on color to distinguish charging from discharging, and the network and IO segments using directional arrows rather than colored runs. The convention exists in the bar; the CI dot is where it would need to be applied.

The third is the active-pane indication in the overlay, which is a border in the accent color. Here the mitigating factor is that the accent is the *only* high-chroma element in a near-monochrome field, so it is distinguished from its surroundings by chroma and luminance rather than by hue against another hue — which is the discrimination that survives. A user who cannot distinguish the accent's hue can still distinguish “the saturated one.”

## The summary a maintainer can act on

The honest ledger has three columns.

Already good, by construction: complete keyboard reachability, link hints as a mouse eliminator, a contrast-derived light palette, accent rationed away from body text, granular effect toggles, and signals that mostly use luminance rather than hue.

Cheap and missing: a contrast check in CI over the palette's role pairings, a platform-reduced-motion default for the animation toggle, the CI dot carrying a glyph, and marking the statusbar as decorative so it leaves the accessibility tree.

Hard and honestly unsolved: the combobox obligations of the palette and find bar, which belong in the shared toolkit rather than here; and the tiling overlay, which is a genuinely unsolved interface problem and whose defense is that it is optional.

The reason to lay it out this way rather than as a general statement of intent is that two of the three columns are actionable lists, and the third is a boundary. An accessibility chapter that only described the good column would be marketing; one that only described the gaps would be flagellation. The useful artifact is the one that says which is which.

# Chapter 40 – Two Browsers, One Host, One Theme File

Chapter 19 ends with a sentence that opens a whole subject and then closes the chapter: point `ZWIRE_STATE` at a different directory and you get a second, independent zwire — a different profile, a different base, a different native host manifest — without touching the first. That is true, and it is a genuinely useful property, and it is also only half of the picture, because the same chapter records that the theme deliberately does *not* live in that tree. The scheme, the light flag, the effect preferences, and any custom schemes live in the fleet-wide store the host owns, read and written by every zwire-host client on the machine.

So the two-instance case is not “two isolated browsers.” It is two isolated browsers sharing one theme, and possibly sharing it with an audio workstation, an editor, and a PDF tool at the same time. This chapter is about what is shared, what is not, what the sharing costs, and what the design does about it.

## The partition, stated precisely

The state a running zwire touches divides into three tiers, and the tier determines the concurrency story.

**Per-install state** lives under the state directory and is isolated by construction. The base and its two contract files, the profile with its bookmarks and history and sessions, and the staged extension copy are all rooted there, so two state directories are two independent worlds. Chapter 19’s `ext/` note makes the multi-user version of this explicit: the staged copy exists because Chromium writes extension metadata indexes into the extension directory, so every user needs their own writable copy. The same reasoning applies to two instances of one user — two state directories, two staged copies, no contention.

**Per-extension state** lives in `chrome.storage.local` inside a profile, so it inherits the profile’s isolation completely. Every `zb_*` key Chapter 31 catalogs — the mirrors, the buses, the preferences, the session state, the guards — exists once per profile. Two instances have two independent command buses, two independent tiling engine states, two independent keybinding remap sets. This is worth stating because it is the surprise in the other direction: a user who sets up their keybindings in one instance does *not* get them in the second, because keybindings are extension storage and extension storage follows the profile. The theme travels; the keymap does not.

**Fleet state** lives in the host’s store, outside every state directory, shared by every client on the machine. Chapter 19 names it as a real distinct directory rather than shorthand, and Chapter 8 enumerates what it holds: the scheme, the light flag, the effect preferences, and user-defined custom schemes. This is the only tier with a genuine concurrency problem, and it is the only tier that has a lock.

The partition is defensible in each direction. Theme is fleet state because Chapter 8’s whole argument is that a scheme picked in any application repaints all of them, and an isolated theme would defeat that for the second browser instance as thoroughly as for the audio workstation. Keybindings are per-profile because a remap is a property of how you drive *this* browser, and the fleet’s other applications do not have a vim layer to remap. What is worth noticing is that the partition was not chosen by asking “should this be shared” per key; it falls out of which store each piece of state was already in, for reasons that had nothing to do with multi-instance. That it lands somewhere sensible is fortunate rather than designed, and it is the kind of thing worth checking rather than assuming the next time a piece of state is added.

## The lock, and what it is actually protecting

Chapter 8 records the failure that motivated the lock and Chapter 32 records its shape: two concurrent writes, one carrying a scheme and one carrying a UI preference, could each read the current state, modify their own field, and write the whole thing back, so the second write silently reverted the first. The fix serializes the read-modify-write behind a cross-process lock file.

The detail worth drawing out is *why the write is a read-modify-write in the first place*, because that is what makes the lock necessary and it is not inevitable. The store is a single document holding several independent values. A writer that wants to change one value has to read the document, change its field, and write the whole document back, because the format has no notion of a partial update. That is the classic lost-update shape, and it appears the moment two writers exist.

There were two other available designs and it is worth seeing why the lock wins. One is to split the document into one file per value, so a scheme write and a light-mode write touch different files and cannot conflict. That eliminates the lost update entirely and it is why the plain-text projections Chapter 29 describes — the ones the fork’s C++ mixer reads — are immune to it: a projection holds exactly one value, so a write is a whole-file replace with no read step. The cost is that a consumer wanting a consistent view of several values has to read several files with no guarantee they were written together, which trades a lost update for a torn read. The other design is a last-writer-wins document with a version counter, where a writer that finds the version has changed re-reads and retries. That is a lock in everything but name, with the retry loop exposed to the caller.

The lock is the right answer here for a reason specific to this workload: the writes are human-initiated and rare. A user picks a scheme, toggles light mode, or changes an effect — these are events measured in seconds apart at their fastest, and the contention window is the duration of a small file rewrite. A lock whose contention is effectively never is a lock with no cost, and it buys a consistent multi-value document that every consumer can read in one operation. The projections exist alongside it precisely because one consumer — a C++ paint path — cannot afford even that, and Chapter 29 is right to call the projection “a deliberate narrowing of a format to fit the weakest consumer.”

## How many host processes are there, actually

The native-messaging model spawns a host process per connection. Chapter 26 makes this explicit as a performance argument: Chromium spawns the host per connection on the native-messaging path, so its startup time is paid every time the browser opens a port to it, and a half-megabyte static binary starts in the noise.

Follow that through the multi-instance case. Each browser instance has its own service worker. Chapter 8 records the worker holding a persistent native port for the system-stats stream, and Chapter 30 records that the same port is the worker’s keepalive. So each running instance holds at least one long-lived host process. Chapter 6 records the new-tab extension polling the host on its own interval through its own native-messaging connection, which is a second origin with its own port. Chapter 8 records the power-tool subscribing the host bus directly. Multiply by instances and the machine is running several host processes at once, each with its own sysinfo sampling thread on the interval Chapter 26 floors.

This is fine, and it is worth explaining why rather than treating it as obviously fine. The host is stateless with respect to everything except the files it owns. A sysinfo sample is a read of the machine, and N processes reading the machine produce N identical answers at slightly different instants — wasteful but not incorrect. A PTY session belongs to the connection that spawned it, and Chapter 8’s per-tab keying means sessions are already partitioned by tab within an instance, so partitioning further by instance changes nothing. The file store is the only shared mutable resource, and it is the one thing behind a lock.

The daemon path is where this stops being free, and Chapter 25 already names its gate: a Unix socket created `0600` inside a `0700` directory, so only the owning user can open it. A socket is a single rendezvous point, which means the daemon path is the one where two instances genuinely share a process rather than each having their own — and where a stale socket file from a crashed daemon blocks a new one until it is removed, which

Chapter 36 lists as a real failure mode. The tradeoff between the two transports is therefore also a concurrency tradeoff: per-connection spawn gives every client its own process and no rendezvous problem at the cost of N processes, and the daemon gives one process at the cost of a rendezvous that can go stale.

## The shared log is the interesting case

Chapter 8 records that the host keeps a shared command log recording every message it sends and receives, the sysinfo stream included, which the HOST internal page reads back through a command.

A shared append-only log written by several concurrent processes is the one piece of fleet state that is not behind the lock and does not need to be, provided one property holds: that each record is written in a single append and is small enough that the operating system does not interleave two writers' bytes. Line-delimited records are the standard way to get this, and the log's format is line-delimited by construction. The consequence is that the log is *interleaved* rather than corrupted — records from several processes in timestamp-ish order, each intact.

That interleaving is a feature for the purpose the log serves. A user debugging why a scheme change did not reach the new tab wants to see, in one file, the write from one client and the absence of a read by another. A per-process log would put those two facts in two files and make the correlation manual. Chapter 41 takes this up as the central observability argument; the concurrency point here is narrower: the log is the one shared mutable resource whose semantics *improve* under concurrency, and it is the one that correctly has no lock.

The open question the book cannot answer from the text is growth. A log that records every message including a continuously streaming sysinfo feed, written by several processes, grows at a rate proportional to the number of running clients and never shrinks on its own unless something rotates it. Whether something does is not stated anywhere the book has read, and the honest thing is to flag it as the question to ask rather than to assert an answer in either direction.

## The rule that falls out

The multi-instance story reduces to a single question to ask of every new piece of state, and it is the question Chapter 31 already asks in a different form: how many processes must see this?

One profile, and it belongs in extension storage, where it is isolated for free and needs no concurrency thought at all. One machine, and it belongs in the host's store, where it needs a lock the moment there are two writers and a projection the moment there is a consumer that cannot parse the format. One process, and it belongs in memory, with Chapter 30's discipline about whether it is mirrored, reconstructible, or expendable.

The failure mode to watch for is a piece of state that answers “one profile” today and “one machine” after a feature is added — a preference that starts in extension storage and later needs to be seen by the new-tab extension, which Chapter 29 establishes cannot read it. That migration is the one that arrives without warning, because the code that needs the value is in a different extension and simply finds nothing there. Knowing the three tiers in advance is what makes that a five-minute decision rather than a surprise.

# Chapter 41 – Debugging a System With Six Boundaries

Chapter 36 tells you what to do when a symptom matches a known failure. Chapter 32 tells you what happens inside a path when one hop fails. Neither tells you how to find out which hop failed when the symptom is new, and that is the everyday work. A feature in this browser is a message crossing several of Chapter 29's six boundaries, each with a different transport, a different observability surface, and a different failure signature — and the single most useful skill for working on it is knowing, for any symptom, which boundary to interrogate first and what tool sees across it.

This chapter is that method. It is deliberately concrete: the instruments available at each boundary, what each can and cannot see, and three worked bisections against the paths Chapter 32 already described.

## The instruments, and what each one can see

There are five ways to observe a running zwire, and they have almost no overlap.

**The service worker's own inspector.** A Chromium extensions page exposes an inspect link for a running service worker, opening devtools attached to it. This is the only view of the worker's console, its exceptions, and its heap. Its defining limitation is the one Chapter 30 is entirely about: the worker may not be running when you go looking, and attaching an inspector to a worker is itself an event that affects its lifecycle. Anything you learn from the worker's console is about the worker *now*, not about the worker at the moment your bug happened.

**A page's devtools, on the content script's context.** Devtools opened on an ordinary page can switch its console context to the extension's isolated world, which is where every content script in Chapter 7's injection stack runs. This sees the content script's globals, its listeners, and the DOM as the script sees it. It does not see the worker, does not see another extension's scripts, and — because Chapter 29's first boundary is real — does not see the page's own JavaScript globals.

**The storage viewer.** Every `zb_*` key Chapter 31 catalogs is inspectable, and this is the most underrated instrument in the set, because Chapter 30's second pattern means the store holds the worker's answers rather than the worker's questions. If the tab list mirror is stale, the worker has not run its update; if the command bus key holds an action with a tag that never incremented, the page never wrote; if the scheme mirror disagrees with the host's file, the fan-out lost a hop. The store is where most of the system's state is *already* published for reasons that have nothing to do with debugging, which makes reading it the cheapest possible first move.

**The host's shared log.** Chapter 8 records the host recording every message it sends and receives, the sysinfo stream included, readable back through the HOST internal page. This is the only instrument that sees across the process boundary, and — per Chapter 40 — the only one that sees several clients at once. It answers the question no browser-side tool can: did the request leave the browser, and did a reply come back?

**The CDP probes.** Chapter 20's `v*.mjs` scripts attach to a running browser over the debugging protocol and assert on real DOM in a real page. Their designed use is verification, but their debugging use is different and arguably more valuable: each probe is a pre-written, known-good traversal of one multi-hop path, so running one tells you whether that entire path is intact without you constructing the test. Chapter 20 is also explicit about their limits — they cannot see module-scoped state, cannot assert absence, cannot distinguish slow from broken, and cannot reach the fork's native chrome at all.

## The bisection order

The instruments sort into a natural order, and the order is a bisection: start at the end of the path with the cheapest observation and walk backward.

|                 |                     |                     |                     |
|-----------------|---------------------|---------------------|---------------------|
| symptom         | first look          | then                | then                |
| UI wrong        | the store           | content script ctx  | worker inspector    |
| nothing happens | the store's bus tag | worker inspector    | host log            |
| stale value     | the store vs host   | host log            | the writing surface |
| cross-extension | host log            | the other ext's ctx | the manifest wiring |
| process silent  | host log            | the manifest        | the binary path     |

The reason to start at the store rather than at the code is that the store is a *published answer*. Chapter 30's publish-eagerly pattern means the worker writes its state into storage as a matter of course, so the store already contains a record of whether the worker did its job, with no instrumentation required and no dependence on the worker being awake when you look. That is an enormous debugging advantage that fell out of a design decision made for latency reasons, and it is worth naming because the general lesson transfers: a system that publishes its intermediate state for correctness reasons is a system that is observable for free.

## Bisection one: the scheme did not follow

The symptom Chapter 32 calls the most-reported desync: internal pages and page skins recolor, the new tab does not.

Chapter 29 gives the path shape — a surface relays the pick, the worker writes the mirror and relays to the host, the host takes the lock and rewrites the store and its projections, and the renderers reconcile on their own schedules. Four hops, four places to look, and the bisection takes two observations.

Read the scheme mirror in storage. If it holds the new scheme, the surface and the worker both did their jobs and the failure is downstream — the host did not write, or a renderer did not read. If it holds the old scheme, the failure is upstream and the worker never ran the relay, which means either the surface did not ask or the worker was asleep and the ask used a transport that does not wake it.

Then read the host's log. A relay that reached the host appears there as a message in; a write that succeeded appears as its reply. Between those two observations every hop in the path is localized, and the two most common outcomes have distinct signatures. If the mirror updated and the log shows nothing, the host is not running — which Chapter 36 tells you produces exactly this symptom, because the new tab's only channel is the file. If the mirror updated and the log shows the write, the failure is a renderer that has not re-read, and Chapter 32's second case applies: for the new tab that window is bounded by its poll interval, so waiting one interval distinguishes a slow renderer from a stuck one.

The instructive part is what you did *not* have to do. You did not attach an inspector to the worker, did not open devtools on the new tab, and did not read any code. Two published records — one in extension storage because the worker publishes eagerly, one in a log because the host records everything — localized a four-hop failure to one hop.

## Bisection two: a browser action did not fire

Chapter 32's Path B: a script asks for a browser action and nothing happens.

This path has a property the scheme fan-out does not, and it is the one that makes it debuggable: the action carries a correlation id. Chapter 7 records the host stamping the action with a nonce into the file-backed key-value entry, and Chapter 32 records the reply travelling over the persistent sysinfo port correlated by a request id so the action fires exactly once. A correlation id is a debugging affordance as much as a correctness one — it means every record of this action, in every log and every store, is findable by one string.

The bisection follows the id. Does the host log show the stamp? If not, the trigger never reached the host and the problem is upstream of the browser entirely — the script, the subprocess, the invocation. If the stamp is there, does the store hold the action entry? If not, the write failed. If it is there and nothing happened, the executor did not run it, and there are exactly two reasons: the worker never woke, or the worker woke and had no branch for the verb.

Those two are distinguishable, and Chapter 32 explains why the second is silent by design: the host accepts any verb and never consults a list, so discovery is advisory and execution is open, and a verb the executor has no branch for is a no-op. That is a chosen trade with a chosen cost, and the cost is paid precisely here, in a debugging session, by a person who has to rule out a typo. The mitigation is not a change to the design — Chapter 32 defends the openness convincingly — but a debugging habit: check the verb against the discovery list before checking anything else, because the failure mode that produces silence at the executor with a clean record everywhere upstream is overwhelmingly a name that does not match.

## Bisection three: a key does nothing

This is the hardest symptom in the system to localize, because Chapter 28 establishes seven potential owners of a keystroke and the first three are outside JavaScript entirely.

The bisection is a ladder and it runs top-down rather than bottom-up, because the upper layers are the ones that consume a key invisibly. Does the key produce an event in the page at all? A listener in the content script's console context answers this, and a *no* means one of the first three layers took it — the OS, a Chromium accelerator, or a registered browser command. Chapter 28 names the specific case that trips people: the new-tab page reserves the palette chord at the browser level, so a page listener there will never see it, which is exactly why the worker owns that chord as a browser command rather than a page listener.

If the event arrives, the next question is whether the HUD claimed it. Chapter 28's invariant says a listener calls `preventDefault` only when the key maps to a real action, so an unmapped key reaching the page untouched is *correct behavior* and the symptom “my binding does nothing” most often means the binding is not in the table. Read the merged keymap — defaults from the registry, overlaid with the stored remaps — rather than reading the defaults file, because Chapter 28's whole point is that the table is dynamic and a remap the user forgot about is indistinguishable from a missing binding.

If the key is in the table and the event arrives and nothing happens, the remaining candidates are the guard ladder and the transport. The guards are enumerable — Chapter 28 lists them in order: the per-tab disable flag, hint mode, the editable guard, the modifier bailout — and each has a visible state. And if the action is one that crosses to the worker, the failure may not be in the keyboard layer at all but in the bus, which puts you back in bisection one's territory: read the command bus key and see whether the write happened and whether its tag incremented.

That last point is the most useful thing in this chapter for a new maintainer. A large fraction of “the keyboard is broken” reports are not keyboard failures; they are bus failures wearing a keyboard symptom, because the keystroke's whole job was to write a bus entry. Chapter 30's monotonic tag is the tell: an entry whose tag did not move on the second press means the write path is dead, and an entry whose tag moved while nothing happened means the write path is fine and the worker is the problem. One field distinguishes two entirely different investigations.

## What is genuinely unobservable

Four things resist every instrument above, and knowing them prevents hours spent looking for a view that does not exist.

**The worker's past.** Chapter 30 establishes that the worker can be destroyed at any moment, and a destroyed worker's console history goes with it. There is no retrospective view of what the worker did before an eviction.

The compensating design is, again, the store: anything the worker *published* survives the worker, which is why a debugging session reads state rather than logs on the browser side.

**A pane's interior.** Chapter 29's fourth boundary is a cross-origin frame, and it is opaque by design. The top frame cannot read what happened inside a pane, and neither can you from the top frame's context. The forwarder protocol's deliberately small vocabulary — which Chapter 29 justifies as API-surface minimization — is also the complete list of what is observable about a pane from outside it. Debugging inside a pane means attaching to the pane's own context, which is a different devtools target.

**The fork's native chrome.** Chapter 20 says this outright: the fork's contributions are native chrome, none of which is reachable from a page context at all, and its verification story is a patch-application check rather than a behavioral one. There is no CDP view of a color mixer.

**The moment of a race.** The lost-update failure Chapter 8 records — two concurrent writes each reverting the other — is invisible after the fact by construction, because the store holds one consistent-looking value and nothing records that it was written twice. The shared log is the only instrument that would show both writes, which is a strong argument for the log recording everything rather than only what seemed interesting at the time.

## The general principle

Every instrument in this chapter exists because a design decision made for another reason left a record behind. The store is readable because the worker publishes eagerly for latency. The correlation ids exist because exactly-once delivery needed them. The shared log exists because a host with several clients needed one place to see them. The CDP probes exist because the runtime path needed a verification story.

That is not a coincidence; it is the strongest available argument for a design habit. A system whose components communicate through *durable, inspectable intermediates* — a key in a store, a line in a log, a file on disk — is debuggable with a text editor. A system whose components communicate through ephemeral in-memory calls is debuggable only with a debugger attached at the moment of failure, which for a service worker that may not exist when you look is not a workable proposition at all. Chapter 29's durability ladder was presented as a reliability argument. It is an observability argument in exactly the same order, and for exactly the same reason: a transport that survives its receiver also survives your investigation.

# Chapter 42 – Incognito, and the Extension That Is Not There

Appendix E ends with the observation that `zwire --incognito` “just works,” which is true in the narrow sense the appendix means: the launcher passes user arguments straight through, Chromium receives the flag, and an incognito window opens. What opens, though, is not a `zwire` window with private browsing turned on. In the default configuration it is a stock Chromium window with none of this book in it, and that is worth understanding precisely, because the two ways to react to it — leave it as it is, or turn the HUD on in incognito — have consequences that run through every chapter of the runtime path.

## The default is a browser with no HUD at all

Two independent facts decide what runs in an incognito window, and neither of them is under `zwire`'s control.

The first is a per-extension user setting. Chrome's own documentation for the `incognito` manifest key lists three values — `"spanning"`, `"split"`, and `"not_allowed"` — with `"spanning"` as the default. What the key does *not* do is enable the extension in incognito; that is the user-facing “Allow in incognito” toggle, which is off by default and which the extension author cannot set. The reasoning behind that default is exactly the reasoning this chapter is about: an extension enabled in incognito can write incognito-derived data to disk, and the browser has no way to verify that it does not, so the decision is handed to the person who can.

The second fact is that Appendix O's three manifest summaries record no `incognito` declaration at all — not on the new-tab extension, not on `hud-internal`, not on the power-tool. No declaration means the default, and the default is `"spanning"` *if the user has allowed the extension at all*.

Put together: on a fresh profile, an incognito window in `zwire` has no statusbar, no vim motions, no `%K` palette on ordinary pages, no tmux overlay, no fuzzy find bar, no terminal, and no HUD replacements for the five redirected `chrome://` pages. The new tab is Chromium's, not the HUD's, because `chrome_url_overrides.newtab` is a capability of an extension that is not running. The neon is gone. Every keystroke goes to the page.

That is a startling amount of absence, and it is worth naming the one place it is genuinely useful. Chapter 41 lays out an instrument set and a bisection order for a system with six boundaries, and the instruments it names are all *additive* — read the store, read the log, attach a probe. Incognito is the one subtractive instrument available, and it is the fastest possible answer to the question “is this the site or is this us?” A page that misbehaves in a normal window and behaves in an incognito window has told you, in one keystroke and with no extension reload, that the entire injected suite is implicated. A page that misbehaves in both has exonerated it. No other single action in the system removes all three extensions at once while leaving the browser, the profile, and the base identical.

## If you turn it on: spanning means one store, not two

The HUD's own extensions page can flip the incognito toggle — Chapter 14 lists it among the per-extension controls that page exposes through `chrome.developerPrivate` — so this is a decision a `zwire` user can make from inside `zwire`. If you make it, the default `"spanning"` mode has one architectural consequence that dominates all the others: `chrome.storage.local` is a single store, shared by the incognito and non-incognito contexts, and Chapter 31's five roles all become channels between them.

Walk the roles with that substitution in place.

The **mirrors** now describe both contexts at once. `zb_tabs` is the worker's live list of open tabs, read by the palette; with the HUD spanning, an incognito tab is a tab, and its title and URL are visible in the `%%K` palette of a normal window. `zb_frecent` is frequency-scored history, top thirty, read by both the palette and the new tab — so an incognito visit competes for a slot in a list that renders on a durable surface.

The **buses** cross contexts too. `zb_cmd` is written by a content script and executed by the worker; in spanning mode there is one worker, so a command written from an incognito page is executed by the same executor that serves normal windows, against a tab id that may belong to either.

**Session state** is the sharpest case, because the role is *defined* by durability. `zb_tmux_sessions` holds named layouts written by the Sessions page, and Chapter 14 records that a layout is a set of windows and panes with a rendered preview — which is to say a list of URLs. Saving a session from an incognito window writes those URLs into a durable key that the Sessions page will happily export as JSON. `zb_term_open` is a flag that re-opens the terminal overlay on the next page; set in an incognito window, it re-opens the terminal in a normal one, because the worker resets it on browser startup and extension reload and nothing about incognito is a reset event.

The alternative, "`split`", gives incognito its own store — and pays for it in exactly the places Chapter 31 says to look. The one-shot guards are the clearest: `zb_cmds_seeded` is a first-run flag and `zb_cmds_removed` is a tombstone list, and in split mode each exists twice. The seeder runs again against the incognito store, and it runs without the tombstones the user accumulated in the normal one, so every default keyword command the user deliberately deleted comes back in incognito. Split mode does not remove the coupling; it converts a leakage problem into a divergence problem.

## The host has never heard of incognito

The deeper issue is below the extension entirely. `chrome.storage` at least has a mode distinction to argue about; the native host has none.

A native-messaging port opened from an incognito context reaches the same `com.zwire.hud` manifest, spawns the same binary, and talks the same protocol as one opened from a normal window. Appendix G's legacy path carries a scheme read, a scheme write, a UI-prefs merge, a sysinfo stream, and the PTY relay, and not one of those messages has a field that could carry "this request came from a private context" even if the host wanted to honor it. The host writes `hud-scheme` and `hud-ui.json`, appends to the shared command log Chapter 40 describes, and — through `pty_spawn` — forks a login shell.

That last one deserves to be stated without any softening. The terminal in an incognito window is a login shell on your machine. It reads your shell configuration, it writes your shell's history file, and every command you run from it is recorded by the shell exactly as if you had opened a terminal application. The browser's incognito contract covers cookies, history, and cache in a profile; it does not and cannot cover a process the browser spawned on your behalf. This is not a defect in the host — the host is doing precisely what Chapter 15 says it does — but it is the concrete form of the hazard that the "Allow in incognito" default-off setting exists to prevent, and in zwire it is not hypothetical.

## The rule, and why zwire needs incognito less than most browsers

Chapter 41 closes on a design principle: a system whose components communicate through durable, inspectable intermediates is debuggable with a text editor. That principle is the whole reason the runtime path works — the store survives worker eviction, the file survives everything, the log survives the process. It is also, read from this chapter's angle, a statement that the system is built out of things that persist by construction. An architecture whose central virtue is durability cannot be made ephemeral by a toggle. The toggle governs

whether the extensions run; it does not and could not reach into a storage key, a TOML file, an append-only log, and a shell history file and make them forget.

So the honest posture is a choice between two coherent configurations, not a slider:

```
toggle off (default)  incognito is stock Chromium; the HUD is absent;
                      the incognito contract holds exactly as Chromium
                      defines it; you also get the best subtractive
                      debugging instrument in the system for free
toggle on (spanning)  the HUD is present and incognito means "no profile
                      cookies," not "no record" - mirrors, buses, session
                      keys, the native files, the log, and the shell's own
                      history all see the session
```

What makes this less painful in zwire than it would be in a general-purpose browser is Chapter 25's containment. The reason a person reaches for incognito in their daily browser is usually that the daily browser is signed into everything and every navigation lands in a profile full of their real life. zwire's normal windows already run against a dedicated `--user-data-dir` that started empty and holds only what you did inside zwire. The distance between "a zwire window" and "an incognito window" is therefore much smaller than the distance between those two things in a browser with your real profile behind it, and the correct move for most zwire users is to leave the toggle off, keep the HUD out of private windows entirely, and get a clean bisection tool in the bargain.

# Chapter 43 – The Dead Zones

Appendix P's table is usually read as a description of where the HUD runs. Read the other way — as a description of where it does not — it is a map of the surfaces on which every keyboard model, every visual affordance, and every assumption in this book quietly stops applying. Those surfaces are not rare. A user hits several of them in an ordinary hour, and because the HUD's absence is silent rather than an error, the experience is “the keybinding didn't work” rather than “you are somewhere the keybinding does not exist.” This chapter is the map, and the design consequences of each region on it.

## The regions

**The new-tab URLs.** Every block in Appendix P that matches broadly excludes the new-tab URLs, and the reason is ownership: the new-tab extension serves that page. The consequence is that on the HUD's own front door, the statusbar does not exist, the tmux overlay is not loaded, the fuzzy find bar is not installed, the terminal boot script has not run, and the vim motion layer is absent. The palette is present, but it is the new tab's own — which is why the extension declares `externally_connectable.ids: [ hud-internal ]`, so the HUD worker can reach in and open it. Pressing the tmux prefix on the new tab does nothing at all, and nothing indicates why.

**The five redirected `chrome://` pages.** Block 1 runs `redirect.js` and only `redirect.js`. It navigates you to an extension-origin document under `pages/`, and every other block's match set — `http`, `https`, `file`, `chrome://` — excludes `chrome-extension://` by construction. So the destination of the redirect is a dead zone for the entire content-script suite. This is the structural reason `zg-boot.js` exists in the shape Chapter 14 describes: the HUD pages do not *inherit* the palette, the effects, and the scheme bridge from the injected stack, because the injected stack does not reach them. They mount their own. Every HUD page is a re-implementation of the ambient environment, and the reason it is not duplication is that it is the same shared modules mounted from a different entry point.

**Another extension's pages.** Appendix O records the power-tool's `options_ui` as `scripts-manager/manager.html`. That page lives on the power-tool's own extension origin, and no host permission zwire can declare will let `hud-internal` inject into it. Chromium forbids one extension from scripting another, and this is one of the boundaries Chapter 29 correctly says exists to enforce something. The design consequence is direct: the cross-extension scheme bridge is not an optimization. It is the *only* mechanism by which the power-tool's own surfaces can follow the active scheme, because the mechanism used everywhere else — injecting a stylesheet — is unavailable on principle.

**DevTools.** A DevTools window is not a document any block matches. When focus is in DevTools, there is no HUD code running in that surface at all: no vim, no palette handler, no prefix machine. This is the largest routinely-visited dead zone for a developer, and it has a pleasant property — the keyboard model reverts wholesale rather than partially, so there is no ambiguity about who owns a key. It also means that the one place a developer is most likely to want the fuzzy find bar is the one place it is guaranteed absent.

**Subframes on ordinary pages.** Block 6 is the only `all_frames` block, and it carries `ztmux-pane.js` alone. Every other block is top-frame only. That is correct — one statusbar per page, not one per iframe — but it produces a behavior that looks like a bug from the outside: click into an embedded video player, a comment widget, or a framed editor on an ordinary page, and the vim motions stop responding, the palette hotkey stops working, and the find bar does not open, because the focused document has none of those scripts in it. The pane forwarder is loaded, but Chapter 10's forwarder protocol is a conversation with the *overlay's* top frame,

which is not present on an ordinary page. The user's model is "I am on this page"; the platform's model is "you are in that document," and the keyboard follows the platform.

**view-source: and the viewer surfaces.** The match patterns are scheme-based. A **view-source:** URL is its own scheme and appears in no block, so it is a dead zone by construction rather than by policy. The same reasoning applies to any surface where the top-level document is not an ordinary HTML page the site authored — a built-in viewer renders content, but the assumptions the suite makes about a document (a body to attach a fixed bar beside, a scrollable page for motions to move) are assumptions about ordinary documents.

**The browser's own chrome.** The omnibox, the tab strip, native menus and dialogs, and the download surfaces are not documents at all. Chapter 1 draws this line as the boundary between what a runtime rebrand can reach and what only the compiled fork can, and Chapter 3 lays out the seven layers. The point to add here is a *keyboard* one, and it is the reason this chapter matters more than a list of URLs would suggest.

## Why the palette needs two doors

Chapter 28 enumerates seven claimants for a keystroke, in dispatch order, all of them running inside a document. When focus is in the omnibox, none of them is running. Nothing in the content-script stack can see that keystroke, arbitrate it, or claim it, because the stack is not there.

That is why Appendix O records a **commands** entry on **hud-internal** — **open-palette** bound to **%K / Ctrl+K** — *in addition to* the palette's own key handling inside the page. The two are not redundant paths to the same thing. A manifest command is registered with the browser and fires regardless of which surface has focus, including surfaces where no content script exists; a page-level handler is precise, arbitrable against the other six claimants, and able to consider what the page is doing. Neither can do the other's job. The command entry cannot participate in Chapter 28's dispatch order because it fires above it; the page handler cannot fire in the omnibox because it does not exist there.

The generalizable rule is the one worth taking out of this chapter: **an affordance that must work everywhere needs a browser-level door as well as a page-level one, and the two doors have different arbitration models.** The palette has both. The vim layer, the find bar, the terminal hotkey, and the tmux prefix have only the page-level door, which is a deliberate and correct choice — they are all page-directed operations, and an operation that acts on a page has nothing to do when there is no page. The affordance that breaks this rule is the one that feels global to the user but is implemented page-side only, and the test for it is simple: put focus in the omnibox and press the key.

## The map, collected

|                             |                |                                       |
|-----------------------------|----------------|---------------------------------------|
| surface                     | suite present? | what fills the gap                    |
| new-tab URLs                | no             | the new-tab extension's own page      |
| redirected chrome:// pages  | redirect only  | zg-boot.js re-mounts on the HUD page  |
| other chrome:// pages       | skin only      | theme.js token override               |
| another extension's pages   | never possible | the cross-extension scheme bridge     |
| DevTools                    | no             | nothing; the model reverts wholesale  |
| subframes on ordinary pages | pane forwarder | nothing, unless it is an overlay pane |
| view-source / viewers       | no             | nothing                               |
| omnibox, tabs, native menus | no             | the manifest `commands` entry (%K)    |

Read down the right column and the architecture of the HUD's edges becomes visible. There are exactly three mechanisms for covering a dead zone — mount the same shared modules from a different entry point, push state to a surface you cannot inject into, or register at the browser level instead of the page level — and every place the HUD feels seamless across a boundary is one of the three. Everywhere else, the correct behavior is the honest one: nothing happens, the key belongs to whoever is there, and the user is briefly in a plain browser.

# Chapter 44 – Zoom, Density, and the Geometry That Is Not Yours

The HUD draws in three coordinate systems and mostly does not say which one it is in. Chapter 11 gives the statusbar a 22-pixel height with 11-pixel transparent borders forming the chevrons; Chapter 10 gives the overlay a z-index ladder from 2 to 2147483647; Chapter 8's window tiler computes pixel rectangles from a display work area. Those are three different kinds of pixel, and the differences surface only under conditions a developer's own machine usually does not reproduce: a non-default page zoom, a fractional device pixel ratio, a second monitor with a different scale factor, and a fullscreen element. This chapter is those four conditions and what each one does to geometry the rest of the book states as fixed.

## Three coordinate systems

| space              | set by                          | affected by page zoom?    |
|--------------------|---------------------------------|---------------------------|
| CSS px in a page   | the document's own layout       | yes – everything scales   |
| device px          | the display's scale factor      | no – physical pixels      |
| DIPs in window API | chrome.windows / system.display | no – browser-level coords |

Every content-script surface — the statusbar, the find bar, the palette, the overlay's panes and dividers — lives in the first space. Every measurement the window-tiling engine makes lives in the third. Nothing in the HUD lives in the second, which is exactly why the second is where the visual artifacts appear: it is the space nobody wrote code against.

## Page zoom scales the HUD, including the parts that were meant to be fixed

A content script's stylesheet is part of the document's cascade, so a 22-pixel bar is 22 CSS pixels, and page zoom multiplies CSS pixels. At 150% zoom the statusbar is half again as tall and occludes half again as much of the page; at 200% it is twice as tall. There is no mechanism by which a stylesheet injected into a document can opt out of the document's zoom factor, because zoom is applied below the level the stylesheet describes.

This interacts with two things the book states elsewhere as bounded. Chapter 26's addendum argues the recomposited area of the blurred bar is bounded because the bar is a fixed small height rather than a proportional one. That argument holds at 100% and degrades linearly with zoom: the bar's area scales, and so does the cost of recompositing it on every scroll frame. And Chapter 38's argument that the bar occupies a strip of the page which the toggle exists to reclaim becomes more pressing at high zoom, because a user who zooms is usually a user who needs the pixels.

The design options are worth naming honestly rather than pretending the issue is settled. A surface can be left in the document's zoom space, which is what happens today and which has the virtue that the bar scales with the text a user chose to make bigger — an accessibility argument, and not a weak one, given Chapter 39's subject. Or a surface can counter-scale, which requires knowing the current zoom factor and re-deriving it on every change, and which produces a bar rendered at 22 physical-ish pixels next to text the user deliberately enlarged. The first is defensible; the second is a decision to override a user preference. What is not defensible is having neither answer written down, because then the behavior is whatever the cascade happened to do.

## Fractional zoom and the chevrons

Chapter 11's chevrons are geometry, not glyphs: an 11-pixel transparent top border and an 11-pixel transparent bottom border on a 22-pixel bar, with a solid side border drawing the arrow. That arithmetic is exact — 11 and 11 sum to 22 — and it stays exact only while both numbers stay integers.

At a zoom of 110%, 11 becomes 12.1 and 22 becomes 24.2. At a device pixel ratio of 1.5, integers in CSS space land on half-pixels in device space. In both cases the engine rounds, and the two halves of a triangle can round in different directions, which puts the arrow's tip a device pixel off the vertical center and leaves a hairline seam between two interlocking chevrons that are supposed to tessellate. The two shades that Chapter 11 uses to make consecutive blocks read as distinct steps are what make the seam visible: a seam between two identical colors is nothing, and a seam between `--bg-card` and `--bg-secondary` is a line.

This is the class of visual bug that never reproduces on the machine it was written on and always reproduces on someone's external monitor. The structural fix is to make the height even in the target space — which is why 22 with 11-pixel halves was a good choice to begin with — and the structural hazard is that any future change to the bar height must preserve divisibility by two, in a codebase where nothing enforces that and nothing documents it. Naming it is the enforcement mechanism available.

## The window tiler computes in a third space, on one display

Chapter 8's window-tiling engine takes its tiling area from `chrome.system.display`'s *primary* work area, with a documented fallback of 1440×820 when the display API is unavailable, and `rectsFor(n, layout)` produces pixel rectangles — a 60% main pane for `main-v`, a  $\lceil\sqrt{n}\rceil$ -column grid otherwise.

Two things follow that a single-monitor developer never sees.

The first is that the tiling area is the primary display's. On a multi-monitor desktop, the rectangles are computed against the primary work area regardless of which display the browser windows are currently on, so `retile` gathers the panes onto the primary monitor. That is a coherent behavior — a tmux-style tiler that owns a screen — but it is a decision about *which* screen that the vocabulary does not express: tmux's `select-layout` has no notion of a second monitor because a terminal has one. A window manager that tiles real OS windows inherits a question a terminal multiplexer never had to answer, and the current answer is “the primary one.”

The second is the fallback. 1440×820 is a plausible-looking rectangle, which is what makes it dangerous. When the display API is unavailable the engine does not fail; it tiles against a fictional screen, and on a large display the result is a set of windows crowded into the upper-left region with a large empty area to the right and below. The failure presents as a layout the user considers ugly rather than as an error they would report. The general lesson is one worth applying to every fallback constant in a geometry path: a fallback that produces a *wrong* result silently is worse than one that produces an obviously broken result, and a fallback whose value looks like a real device is the worst of both.

A third, subtler hazard belongs to the same space. Display work areas are reported in device-independent pixels, and a mixed-DPI desktop — a high-density laptop panel plus a standard external monitor — has different scale factors per display in one global coordinate space. A rectangle computed from one display's work area and applied to a window that ends up on the other is correct in units and wrong in appearance. Nothing in the tiler's model distinguishes the two, because the model is a single work area and a grid inside it.

## Fullscreen outranks the entire ladder

Chapter 10 records that the copy-mode indicator inside a pane climbs to z-index 2147483647 “so it is never hidden by page content.” That is true of page content and it is not true in general, because there is a rendering surface above every z-index there is.

MDN's definition of the top layer is explicit: it “sits on top of all other layers in a web document,” it “renders above all other content regardless of z-index and stacking context,” and it cannot be manipulated from code. Three things are promoted into it — an element made fullscreen through `requestFullscreen()`, a `<dialog>` shown with `showModal()`, and an element shown with `showPopover()`.

All three are reachable from an ordinary page, and one of them is reachable from inside a pane on purpose: Chapter 10 records that pane iframes carry `allow='clipboard-read; clipboard-write; fullscreen'` precisely so fullscreen works inside them. So a video player in a pane, put into fullscreen by the user, renders above the copy-mode indicator, above the pane badges, above the drag dividers, and above the overlay itself at 2147483640. The maximum integer is not the top of the stack; the top layer is, and no z-index reaches it.

The same mechanism explains a smaller behavior on ordinary pages: a fullscreen video hides the statusbar, which is the desired outcome, but the HUD does not learn that it happened. `zb_status` records whether the bar is *shown*, and a bar that is shown-but-covered is a state the key cannot represent. That is a benign disagreement today. It stops being benign the moment any logic conditions on `zb_status` to decide whether a surface is visible to the user, which is the kind of inference that gets added later by someone who reads the key's name and not this paragraph.

## The rule

Each of the four conditions in this chapter is a case of code written in one coordinate system being read as though it were in another. The discipline that prevents all four is a documentation discipline rather than a code one: **every geometric constant should record the space it is in**. A 22-pixel bar in document space, a 60% main pane in display space, an 11-pixel border that must stay an integer after zoom and DPR, a fallback rectangle in DIPs that is a fiction rather than a device. None of those facts is expensive to write down, and each of them is a bug that otherwise reproduces only on hardware the author does not own.

# Chapter 45 – The Long Session

Chapter 26 prices what the HUD spends: per page load, per sample interval, per network request, per open tab. It is an accounting of rates, and a rate can be small forever. This chapter asks a different question of the same system — which quantities never go down — and it asks it because the workload `zwire` is built for is not a browsing session. It is a browser left open across a week of work, with the terminal attached, the stats stream running, the CI poll firing every ten minutes, and a tab count that goes up on Monday and comes down on Friday.

For every durable thing in the system, there is exactly one interesting property: what bounds it. Chapter 31's schema gives each key an owner, a reader set, and a lifetime in principle. The column it does not have is the bound in practice, and the absence of that column is where long-session problems live.

## The quantities and their bounds

**Bounded by the platform: the service worker.** The one component that cannot leak across days is the one most people would suspect first. MV3 evicts the worker, and Chapter 30's patterns are all written around that fact. An eviction frees the worker's entire heap, so whatever the worker accumulated between wakeups is discarded on a cadence the platform enforces and the extension cannot defeat. The worker is structurally incapable of a slow leak.

**Bounded by nothing: a content script in a tab you left open.** The inverse case, and the one the intuition gets backwards. A tab open since Monday is running the same content-script instances since Monday. Every listener registered at injection is still registered; every structure the palette or the find bar built on first use is still resident; every subscription Chapter 31's rule three requires — readers re-read on change rather than caching at load — is still live and still firing on every storage change from every other tab. The “background” process has a platform-enforced lifetime and the “foreground” script does not, which is the opposite of how the two are usually described.

**Bounded by the process model: the native host on the messaging path.** Chapter 26 records that Chromium spawns the host per connection on the native-messaging path, which makes the host's lifetime a function of the port's. Close the port, the process goes.

**Bounded by nothing: the daemon.** The same binary run as a socket daemon is the longest-lived thing in the system. It holds the PTYs, the file observers Appendix G lists under `fs_watch` and `watch_list`, the pub/sub subscriptions, and the peer connections — all of them keyed and individually cancellable, which is the right design, and all of them held until something cancels them. `watch_list` exists so you can see what accumulated; that it exists is a sign the accumulation was anticipated.

**Bounded by user intent: the durable overlay keys.** `zb_tmux_sessions` grows when the user creates or duplicates a session and shrinks when they delete one. This is the healthy case: growth is proportional to deliberate action, and the Sessions page is both the thing that grows it and the thing that prunes it. The one entry that grows without an explicit create is the reserved `__auto_last__` session detach commits an unattached layout into, and it is bounded by construction: one stable id, overwritten rather than appended, so repeated detaches leave exactly one row.

**Bounded by an explicit cap, or not bounded at all: the mirrors.** Appendix I is unusually informative here, because it records a cap on exactly one mirror: `zb_frecent` is “frequency-scored history, top 30.” That is the pattern every mirror needs, and it is the question to ask of each of the others. `zb_tabs` is bounded by the tab count, which is bounded by the user. `zb_shortcuts` is bounded by the extension count. `zb_ci_cache` is

described as “cached CI run data,” and a cache of run data for a repo list whose size is set by `repolimit` in `zb_ci` is the entry on the list whose bound is a configuration value rather than a structural one — which is fine, and which is worth knowing when someone raises `repolimit`.

**Already flagged, not re-litigated here: the shared log.** Chapter 40 states the growth question and correctly declines to assert an answer. It stays there.

## The quota failure nobody expects

The reason the bounds matter more in this system than in most is a property of `chrome.storage.local` that the manifest makes checkable. Chapter 7’s walk through `hud-internal`’s permissions names fifteen: `storage`, `nativeMessaging`, `management`, `tabs`, `webNavigation`, `history`, `downloads`, `bookmarks`, `sessions`, `alarms`, `notifications`, `developerPrivate`, `settingsPrivate`, `system.display`, and `declarativeNetRequest`. Appendix O carries the full declaration, twenty, adding `scripting` plus the four the automation bus needs — `browsingData`, `tabGroups`, `readingList`, and `power`, backing its data-clearing, tab-grouping, reading-list, and keep-awake verbs. The chapter’s fifteen is the summary; the appendix is the manifest. What matters here survives either reading: `unlimitedStorage` appears in neither.

Without it, `storage.local` has a fixed quota, and the failure mode when a store approaches its quota is the one that makes unbounded growth genuinely dangerous rather than merely untidy: **the write that fails is not the write that grew**. A store filled by an ever-growing cache does not break the cache. It breaks whichever key is written next, which in this system is whichever feature the user touched most recently — a remap they just saved, a session they just named, a command the palette just put on the bus. Chapter 32’s hop-failure analysis assumes a hop that does not arrive; this is a hop that arrives and cannot land, and it lands on an innocent key.

That converts an abstract question (“is `zb_ci_cache` bounded?”) into a concrete one with a blast radius that has nothing to do with CI. It is also why the single-writer rule from Chapter 31 is load-bearing for capacity and not only for correctness: with one writer per key, there is one place to add a cap, and the cap is enforceable.

## The maintainer’s column

Everything above reduces to one addition to the storage schema — a sixth column recording, for each durable key, what bounds it:

| bound type    | example                                             | what to check                          |
|---------------|-----------------------------------------------------|----------------------------------------|
| platform      | worker heap                                         | nothing; eviction handles it           |
| user action   | <code>zb_tmux_sessions</code>                       | that a delete path exists              |
| structural    | <code>zb_tabs</code> , <code>zb_shortcuts</code>    | that the underlying set is bounded     |
| explicit cap  | <code>zb_frecent</code> (top 30)                    | that the cap is enforced on write      |
| configuration | <code>zb_ci_cache</code> ( <code>repolimit</code> ) | what happens when the config is raised |
| unbounded     | a new key with none of these                        | add one before shipping it             |

The rule is that a durable key with no entry in the last column is a key whose growth is someone’s future incident, and the cheapest moment to give it a bound is the moment it is added — the same moment Chapter 31 says to declare its owner, its readers, and its lifetime. Three of those four are already the convention. The fourth is the one that only matters after the software has been running longer than anyone tested it, which is precisely the workload this browser is built for.

# Chapter 46 — What Leaves the Machine

Chapter 25 answers “what can reach my data” and answers it with the profile boundary. This chapter answers a different question with a different answer: what does this browser talk to, and what can talk to it. The two are not the same question, and the profile boundary — the strongest containment in the system — has nothing to say about the second one. A dedicated `--user-data-dir` contains *data*. It does not contain *packets*.

An owner of a browser should be able to enumerate its network surface on one page. Here is that page, split into what zwire’s own code initiates, what it deliberately does not do, what listens, and what is inherited rather than chosen.

## Egress that zwire’s own code initiates

**The base download.** Chapter 2 and Chapter 4 name the host directly: `storage.googleapis.com/chromium-browser-snapshots`, from which `fetch-base.sh` reads a sentinel to learn the current revision for the platform and then downloads the archive. Chapter 26 puts the archive at around 325 MB. This is unauthenticated, it happens on install and on each deliberate base upgrade, and it is the single largest transfer the project makes. It is also the one with the clearest integrity question, which Chapter 4’s discussion of pinning is the right place to reason about.

**The CI poll.** The worker’s `chrome.alarms` heartbeat fires every ten minutes and queries the GitHub REST API. Chapter 26 records the interval as chosen to stay under GitHub’s anonymous request cap — which tells you the default posture is anonymous — and Chapter 14 records what changes when it is not: a token in `zb_ci` “raises the repo budget and switches the repo query from the public account to your full affiliations.”

That sentence deserves to be read as a privacy statement, because it is one. Configuring a token converts a periodic anonymous request into a periodic authenticated one that identifies an account and enumerates private affiliations, every ten minutes, for as long as the browser runs. That may be exactly what the user wants; the point is that it is a different network posture, entered through a configuration field rather than a prompt.

It also has a storage consequence that belongs to Chapter 25’s accounting and is not in it. Appendix I records `zb_ci` as `{ user, token, repolimit }` — the token lives in `chrome.storage.local`, in cleartext, in the same store as the keybinding remaps. That does not change the blast radius, which is still the profile, but it changes what is *in* the blast radius: a credential to a service outside the profile. The threat model’s claim is that the worst an attacker reaches through the browser is what you created inside zwire, and a token you pasted into the CI config is now one of those things.

**Search.** The new tab’s `zb.engine` is a search-engine template with `%s`, and Chapter 6 records that anything typed into the omnibox that is not a URL goes to the search engine. This is ordinary browser behavior and it is listed here for completeness, with one distinction worth drawing: the palette’s own matching, the fuzzy find bar, and the frequency ranking are entirely local computations over local data. The palette is not a search box that queries anything. The fallthrough to a search engine is, and it is the only part of the typing experience that leaves the machine.

**The power-tool’s own surfaces.** Chapter 18 and Chapter 24 both draw the line: `zpwchrome` has its own native host, installed separately, documented by `zpwchrome` itself, and zwire does not install it. Whatever

egress its features carry is outside zwire's inventory and should be read from its own documentation rather than inferred from this one.

## Egress that is absent by construction

Three absences are worth stating as positive properties, because each one was a decision and each one is checkable.

**The new tab makes no network requests of its own.** Chapter 6 records that its fonts are vendored locally under `newtab/fonts/` rather than linked from a CDN, and Chapter 3 states the consequence plainly: it renders offline. A new-tab page with no remote font, no remote stylesheet, and no analytics is a page that reveals nothing by being opened — and a new tab is the most frequently opened page in any browser.

**There is no extension-update channel.** The three extensions are loaded unpacked from disk by `--load-extension`, and Appendix O's manifest summaries record no `update_url` on any of them. Nothing periodically checks a server on their behalf.

**The host has no network path of its own on the browser's route.** Appendix G's legacy path — the only path zwire's front end invokes — is a scheme read, a scheme write, a UI-prefs merge, a sysinfo stream, and the PTY relay. None of them is a network operation. The extended agent includes a TCP peer mesh, and it is off unless a daemon is running with peering configured. The one field in the sysinfo sample that involves the network is `pip`, the public IP, which Appendix G notes is “refreshed sparingly” — an admission that determining a public IP requires asking someone, and a design choice to ask rarely.

## Ingress: two local channels with very different gates

Two things in this system listen, and comparing their gates is instructive.

Chapter 25 describes the daemon's socket carefully: a Unix socket created `0600` inside a `0700` directory, so the operating system's own permission check is the authentication, and a different user on the machine cannot see it. Remote callers over the peer mesh must present a shared `--token`. That is a considered design with a gate at each tier.

The CDP endpoint is the other one. Chapter 20 records that `--remote-debugging-port=9222` opens the DevTools protocol on `127.0.0.1:9222`, and Appendix L records that the endpoint is “opened only when you pass `--remote-debugging-port=9222`.” What the chapters do not say, and what a reader should hold alongside them, is that CDP is a full-control channel — a client that connects can read pages, inject script, and drive the browser — and that a loopback listener has no per-user gate of the kind the daemon socket has. The entire security of this channel is the fact that the flag is opt-in and short-lived.

The two channels sit at opposite ends of a spectrum in the same system:

| channel         | gate                               | lifetime                     |
|-----------------|------------------------------------|------------------------------|
| daemon socket   | filesystem mode 0600 in a 0700 dir | as long as the daemon runs   |
| peer mesh (TCP) | shared --token                     | as long as peering is on     |
| CDP :9222       | the flag was passed                | as long as that browser runs |

The practical guidance follows from the table rather than from any claim about relative risk: the flag is a verification tool, Chapter 20's probes are what it is for, and a browser left running all week with the port open is a decision to leave a full-control channel open all week. Nothing about the profile boundary constrains it, because the profile constrains what an attacker *reaches* and this channel is about who is *driving*.

## What is inherited rather than chosen

The honest limit of this inventory is the base. `zwire`'s contribution is nine flags in Appendix E, and none of them changes the engine's own network behavior. A stock unbranded Chromium snapshot arrives with whatever component-updater, safe-browsing, and field-trial behavior that build carries, and the launcher neither enables nor disables any of it.

So the accurate summary is two-part, and the second part is the one usually left out of documents like this: **`zwire` adds a small, enumerable egress surface — a base download, a ten-minute API poll, and the search fallback — and inherits an unenumerated one from the engine underneath it.** An owner who cares about the second part has the same tools any Chromium user has, and Chapter 14 notes the settings replacement renders every preference `chrome.settingsPrivate` exposes, which is the surface where those behaviors are configured. What this book can say with confidence is limited to `zwire`'s own code, and pretending otherwise would be the kind of rounding-up the introduction promised not to do.

# Chapter 47 — Teardown, Portability, and the Second Machine

Appendix B ends with a sentence that is precise about what it describes and easy to over-read: delete the state tree and “there is no hidden state anywhere else.” That is a statement about where *zwire writes at runtime*, and as such it is exactly right — the appendix itself names the one runtime file that lives outside the tree, and the tree accounts for everything else the browser produces while running.

It is not a statement about *zwire’s footprint* on a machine, and the difference matters twice: when removing *zwire*, and when trying to reproduce a working *zwire* somewhere else. Installation artifacts, platform registrations, and the fleet-wide theme file all live outside the tree by design. This chapter is the complete footprint, the order in which to take it apart, and the considerably more interesting question of what actually has to travel to a second machine.

## The footprint, inside and out

```
inside $ZWIRE_STATE
base/           the extracted snapshot or packaged fork
base.path       absolute path to the resolved binary
base.version    "r<REV>" or "fork-<TAG>"
profile/        the dedicated --user-data-dir
  NativeMessagingHosts/com.zwire.hud.json (path + allowed_origins)
  Default/NativeMessagingHosts/          (second location)
ext/           the per-launch writable extension staging

outside $ZWIRE_STATE
~/.zwire/global.toml    the fleet-wide theme store
~/.zwire/hud-scheme     plain-text projection for the fork's mixer
~/.zwire/hud-light      light-mode projection
~/.zwire/hud-ui.json    shared visual-effect + light-mode prefs
the PATH symlink        placed by install.sh
the .desktop entry      Linux
the registry entries    Windows, both hives
the self-contained target /Applications, ~/.local/opt, %LOCALAPPDATA%
the checkout            including submodules and the host build output
legacy dirs             ~/.zwire and the bare-zwire macOS folder
zpwrcrome's native host installed separately, removed separately
```

Three of those deserve comment.

`~/.zwire/global.toml` and its projections are outside the per-install tree on purpose: Appendix B calls them the one channel through which the new-tab extension, the internal HUD, the power-tool, the wider app suite, and — on the fork — the native chrome agree on one look. A file shared across a fleet of applications cannot live inside one application’s state directory. The consequence for teardown is that removing *zwire* leaves the theme behind, which is correct behavior if the other applications in the suite are staying and surprising if the user believed the tree was everything.

The native-host manifest inside `profile/` holds an absolute path to the host binary plus the `allowed_origins` list. That is what makes the host profile-relative on macOS and Linux, and it is also what makes the

profile directory non-relocatable: move the tree and the manifest's path is stale. On Windows the same wiring is registry state, which is why Chapter 24 says the uninstall step differs there — a file that vanishes with a directory versus a key that does not.

The legacy locations are a teardown detail with a lifespan. Appendix B records that the old `~/.zwire` dotdir and a brief bare-`zwire` macOS folder are auto-migrated once. A migration that runs once leaves the question of whether the source was removed or merely copied, and a thorough teardown checks both paths rather than assuming.

## Order matters, because the launcher rebuilds

Teardown is not “delete the tree,” because the tree is partly self-healing. Chapter 24’s first-run sequence has the launcher ensuring the native host — building the Rust binary if the manifest is missing or its target is gone — and Chapter 26 records that extensions are staged into a writable `ext/` copy at launch. Both of those run every time. So deleting the tree while a launcher can still be invoked produces a partial rebuild on the next accidental `zwire`, and deleting `profile/` while a `zwire` is *running* leaves a browser writing into a directory that no longer exists.

The correct order is the reverse of install:

1. stop everything (browser and any host daemon)
2. remove the platform registration:  
the `PATH` symlink, the `.desktop` entry, or the registry keys
3. remove the state tree (`$ZWIRE_STATE`) and any legacy dirs
4. decide about `~/.zwire/global.toml` and its projections –  
keep them if any other suite application remains
5. remove the self-contained install target, if one was used
6. remove `zpwchrome`'s native host per its own documentation

Step two before step three is the load-bearing ordering. Once nothing on `PATH` can invoke the launcher, nothing rebuilds what step three removes.

## What cannot travel, and why

The instinct for a second machine is to copy the state tree. Most of it cannot go.

`base/` is per-platform by construction — Chapter 24’s matrix maps the host to `Mac_Arm`, `Mac`, `Linux_x64`, or `Win_x64`, and a snapshot is a binary for one of them. `base.path` is an absolute path. The native-host manifest holds another absolute path, to a binary compiled for the local target. On Windows the wiring is not in the tree at all. Copying any of these produces a configuration that is wrong in a way that presents as “the host is silent,” which Chapter 36 catalogs as a symptom with several causes and which now has one more.

What is portable is short: `global.toml` and its projections. A theme is not machine-specific, and the fleet file is a plain text store outside the profile. Copy it and the second machine looks right immediately, on every application in the suite.

## The portability problem the design did not choose

Everything else the user authored is in the wrong place to move, and it is worth being exact about why, because this is a structural constraint rather than an oversight.

The HUD’s user-authored configuration is a set of `chrome.storage.local` keys. Appendix I lists them, and the ones a user actually creates by hand are the remaps in `zb_keys`, the keyword commands in `zb_custom_cmds` with their `zb_cmds_removed` tombstones, the tmux prefix in `zb_tmux_prefix` and its options

in `zb_tmux_opts`, the named layouts in `zb_tmux_sessions`, the CI configuration in `zb_ci`, and — in page `localStorage` rather than extension storage — the new tab's `zb.tiles` and `zb.engine`.

`chrome.storage.local` lives inside the Chromium profile, in Chromium's own on-disk format. There is no supported way to copy one key out of it, and copying the whole profile means copying cookies, history, cache, and sessions along with a format that has no cross-version or cross-platform portability guarantee. The `localStorage` keys are worse still: Chapter 31 records that they are per-origin by nature, which is the property that made them the right choice for a page's own presentation and the property that makes them invisible to any exporter that is not that page.

So the state a user would most want to move is the state stored in the least portable place in the system, and the remedy is not a better backup — it is an export path per key. The book records exactly one: Chapter 14 notes the Sessions page “exports and imports the whole set as JSON” for `zb_tmux_sessions`. That is the model. Every other user-authored key group listed above has an editor — the Keyboard page, the Commands page, the CI configuration, the new tab itself — and an editor that can write a key can serialize it.

## The smallest set that reconstitutes a workstation

The useful inversion of Chapter 24's degradation ladder is a reconstitution ladder: rather than asking what still works when a piece is missing, ask what is the minimum that has to arrive for everything else to rebuild itself.

|                 |                                                                                                                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| must be carried | the checkout with its submodule pointers<br>~/.zwire/global.toml (+ hud-ui.json)<br>a JSON export of the tmux sessions<br>whatever the other editors can be made to export                               |
| rebuilds itself | base/ + base.path + base.version (fetch-base.sh)<br>the host binary (built on first launch)<br>the native-host manifest (written on launch)<br>ext/ (staged on launch)<br>profile/ (created by Chromium) |

The left column is small, and every item on it is a plain file. The right column is everything the launcher and the browser reconstruct from nothing, which is the same property that makes the degradation ladder work — a system that assembles as much of itself as the machine currently has is also a system that rebuilds most of itself on a new machine.

The rule to carry away is the one the storage schema does not currently state: **any state a user authored by hand needs an export path owned by the surface that edits it**. Not a backup of the store — an export of the value, in a format the user can read, from the page that created it. One page in the HUD already does this. The gap between that page and the others is the whole distance between “my configuration is portable” and “my configuration is in a profile.”

# Chapter 48 – Version Numbers as a Contract

There are a lot of numbers in this book. `v0.6.59` on the cover, `1.0.13` and `0.6.29` and `0.22.39` on three manifests, `minimum_chrome_version: 127`, `r<REV>` or `fork-<TAG>` in `base.version`, `v1.0.18` and `v1.0.61` on the shared toolkit, a pinned Chromium tag under twenty-seven patches. They look like one kind of thing and they are three kinds, and confusing them is how a system acquires a failure that nothing reports.

Every version number in a system is one of three things. It is a **gate**, enforced by something that refuses to proceed on a mismatch. It is a **resolution input**, consumed by tooling to decide what to fetch or load. Or it is a **label**, displayed to humans and read by nothing. The dangerous ones are the labels that look like gates and the gates that are enforced somewhere the operator cannot see.

## The inventory, classified

| number                                   | kind                             | enforced by                                                       |
|------------------------------------------|----------------------------------|-------------------------------------------------------------------|
| <code>minimum_chrome_version 127</code>  | gate                             | Chromium's extension loader                                       |
| <code>base.version / base.path</code>    | resolution input                 | the launcher                                                      |
| <code>zgui-core submodule pointer</code> | de-facto gate                    | nothing at load time                                              |
| Chromium tag (fork)                      | resolution input                 | the patch series                                                  |
| <code>package.json v0.6.59</code>        | label                            | the cover; the System page                                        |
| extension manifest versions              | resolution input<br>(was: label) | Chromium's extension reload +<br>the worker-cache eviction marker |
| <code>service_worker filename</code>     | resolution input                 | Chromium's SW registration                                        |
| native protocol (legacy)                 | no version at all                | –                                                                 |
| storage keys                             | no version at all                | per-key shape sniffing                                            |

Four rows on that table are worth the rest of the chapter, and one of them is a correction this edition makes to itself.

## A label that turned out to be a gate

Earlier printings classified the extension manifest versions as **labels** — numbers displayed on the System page and read by nothing. That was wrong, and the way it was wrong is the chapter's own thesis turned back on it: the manifest version is consumed by Chromium to decide whether to reload an unpacked extension, so it is a *resolution input*, and treating it as decoration produced exactly the failure this chapter warns about — one that nothing reports.

Chapter 23 records the mechanism in full. The short form: the installer now appends a content-derived fourth component to every staged manifest's version, so the number follows the code rather than the release; and because service-worker registration is keyed on the worker's **script URL** rather than on the extension version at all, a second stamp points each manifest at a content-keyed `background.sw-<hash>.js` shim. The filename is therefore a version number too — the row above — and it is the one with the sharpest failure mode, because the launcher's `rsync --delete` removes the previous shim while a profile may still hold a registration for it. A registration pointing at a file that no longer exists is a worker that can never start, and it

reports nothing at all: the extension shows as loaded, its pages work, and only the surfaces the worker owns go quiet.

Which is the general lesson restated: a number that nothing enforces is safe to misclassify, and a number something *does* enforce is only safe once you have written down where the mismatch appears. For this one the answer is “a browser-level keyboard shortcut that swallows the keystroke and does nothing,” which is nowhere near the version number it comes from.

## The gate nobody local can check

`minimum_chrome_version: 127` is a real gate. Chapter 2 states what it means without hedging: a base older than Chromium 127 refuses to load the power-tool, and the browser enforces it.

Now put that next to how a base is chosen. `fetch-base.sh` reads a sentinel from the snapshot bucket to learn the current revision for the platform, or accepts a pinned revision directly, and Appendix B records the result as `base.version`, in the form `r<REV>`. A Chromium revision is a monotonically increasing build number. It is not a milestone. Nothing in the launcher, the fetch script, or the state directory holds a milestone number for the installed base at all.

So the gate is enforced by the browser and unverifiable by the tooling around it. Pin an old enough revision and the failure appears as Chapter 36 describes it — an extension that did not load — with the diagnosis living in the browser’s own extensions surface rather than in anything zwire wrote. This is not a defect to be fixed by inventing a version check; it is a property to know about, because it is the one constraint in the system that the resolution input cannot express. The practical form of knowing it: a base chosen by “latest snapshot” satisfies the gate trivially and a base chosen by pinning an exact revision is the case where the constraint is live.

## The de-facto gate with nothing behind it

The submodule story is the sharpest item in this chapter, because the two submodules fail in completely different ways and the project’s stated safety net covers only one of them.

Two chapters make the same statement. Chapter 24’s degradation ladder says “a missing submodule (`zpwchrome`, `zgui-core`) is not fatal — its extension directory has no `manifest.json`, so the launcher skips it and loads what remains,” and Chapter 36 repeats it: a clone without `--recurse-submodules` “leaves their directories without a `manifest.json`, so the launcher skips them silently and loads what remains.” That is precisely true of `zpwchrome`, which *is* an extension directory in the launcher’s load list. It is not true of `zgui-core`, which Chapter 7’s map places at `lib/zgui-core/` — inside `hud-internal`, as a library. The launcher’s `manifest.json` filter never examines it, because it is not a candidate for loading; it is a set of files a content script expects to have been injected before it runs.

The failure modes diverge accordingly. A missing `zpwchrome` is one absent extension and a browser that otherwise works — the ladder’s promise, kept. A missing `zgui-core` is a set of content scripts that load in the order Appendix P fixes and then reference a global that was never defined. Chapter 7 is explicit that the order in block 4 is load-bearing because later scripts read globals earlier ones set, and `ZGui` is one of those globals.

Version skew is the same story one degree subtler, and here the book supplies the exact case. Chapter 10 records that `zgui-core` v1.0.18 “added the external-feed API (`prefix/key/relaySync/yank`) precisely so a cross-origin-iframe consumer like this one can feed the shared WM from outside,” and that `ztmux-config.js` is the provider that uses it. A checkout whose `zgui-core` pointer is behind v1.0.18 therefore has an overlay whose top frame loads, whose panes load, and whose forwarder relays prefix and keystrokes into a shared window manager that does not expose the entry points to receive them.

Chapter 36 already names the symptom class — “if a submodule is present but at the wrong commit, the tell is subtler still, a feature that half-works or behaves like an older version” — and this is the mechanism under that sentence. The submodule pointer is functioning as a version gate, and nothing enforces it at load time. The enforcement is git’s, at checkout time, and only if someone runs the update.

## The channel with a version handshake is the one nobody uses

Appendix G splits the native protocol into two paths, and the split is the interesting part.

The legacy path — “what the browser uses,” in the appendix’s own words — is the scheme read, the scheme write, the UI-prefs merge, the sysinfo stream, and the PTY relay. Not one of those messages carries a protocol version, on either side. The browser sends `{cmd:"get"}` and the host answers `{scheme, ui}`, and if the two were built from different generations of the source there is no field in which they could discover it.

The extended agent, which the appendix says zwire’s front end does not call, opens with `hello`, and `hello` returns `{ok,host,version,os,arch,pid,caps:[...]}` — a version *and* a capability list, which is the strongest form of the handshake, because a caller can ask what is supported rather than infer it from a number.

That asymmetry is worth stating plainly: the one channel in the system with a real version negotiation is the one the browser never opens, and the channel the browser depends on for the scheme, the statusbar, and the terminal has none. What makes it work anyway is that the host binary is built from the same checkout by the launcher on first need, so the two sides are versioned together by construction. That is a legitimate answer — co-deployment is a real compatibility strategy — and its boundary is exactly where co-deployment stops holding: a packaged install (Chapter 23) ships a host inside the bundle, and a self-contained install and a developer checkout on the same machine are two hosts built at two times. The `caps` list already exists in the binary. Nothing about the legacy path is stopping a future scheme read from learning what it is talking to.

## Migration without a schema version

The last row is the storage schema. Chapter 31 establishes the discipline — migrate on read, never on write, because an old content script in another tab may still be running against the same store — and the worked example is the overlay’s per-window sync boolean becoming a list of member ids.

What the schema deliberately does not have is a version number. Migration is therefore *shape sniffing*: the restore path looks at the value it found and decides what generation it belongs to. That is the correct choice for the stated constraint, because a store-wide version number would have to be written by someone, and the only writer available is exactly the new code whose write would break the old readers.

The limit is worth naming, because it is where this strategy stops. Shape sniffing scales per-key and does not compose: it works when a value’s generations are structurally distinguishable (a boolean and an array are not confusable), and it has no answer when two generations of a key are the same shape with different meaning, or when a change spans several keys that must move together. Neither has happened. When one does, the migration cannot be per-key and per-read, and that is the moment the schema needs a version — introduced, per the chapter’s own rule, in a way that old readers tolerate rather than one they must understand.

## The rule

Write the classification down. For every version number in a system, record which of the three it is, and for every gate, record what enforces it and where the failure appears.

Applied here, that exercise produces four sentences that are more useful than any of the numbers individually. The power-tool’s minimum is enforced by the browser and invisible to zwire’s tooling, so pinning a base is the operation that can violate it. The toolkit submodule pointer is a version dependency with no runtime check, so a stale pointer presents as a half-working feature rather than an error. The native protocol’s compatibility rests on co-deployment, which holds for a developer checkout and is worth re-examining for a packaged install. And the storage schema has migration without versioning, which is correct today and has a known boundary. Every one of those is a sentence about where a mismatch would *appear*, which is the only thing anyone actually needs from a version number at three in the morning.

# Chapter 49 – Lifecycle Hooks: stryke on Browser Events

Every chapter so far describes a browser that responds to a person. This one describes the browser responding to itself. A **hook** binds a lifecycle event — a tab opened, a download finished, the scheme changed, the audio engine reshaped — to a **stryke** script that runs outside the browser and can act back on it. It is the seam where the HUD stops being an interface and becomes an automation substrate.

The interesting property is that neither side can do the job alone. The MV3 service worker sees every browser event and cannot run a scripting language: it is JavaScript in a sandbox with no process spawn, no filesystem, and a lifetime measured in seconds. **zwire-host** can run anything the machine can run and sees no browser events at all. The hook is the wire between them, and the whole design falls out of that split.

## The firing side

**background.js** defines **fireHook** at the top of the worker, above every listener, so any handler in the file can call it:

```
chrome.runtime.sendMessage(HOST, { cmd: 'hook_fire', event: event, payload: payload
  || {} },
```

It is deliberately best-effort. There is no reply to wait for, no error surfaced to the user, and no check for whether a hook exists — the host no-ops when nothing is bound to the event. A browser event must never be slowed down, or worse blocked, by an automation layer nobody has configured.

Forty-nine **fireHook** call sites are registered as listeners in one IIFE near the end of the worker, each wrapped so a missing API or permission on one platform cannot break the rest:

```
var on = function (obj, ev, fn) { try { if (obj && obj[ev] && obj[ev].addListener)
  obj[ev].addListener(fn); } catch (e) {} };
```

The registration is at top level for an MV3 reason rather than a stylistic one. A listener attached inside a callback does not wake a torn-down worker; a listener attached during the worker's initial evaluation does. Every hookable event is therefore also a wake reason, which is what makes hooks fire at all in a browser that keeps its extension worker dead most of the time.

Alongside the specific events there is **act()**, the catch-all:

```
var act = function (command, extra) { var p = { command: command }; if (extra) for (var k
  in extra) p[k] = extra[k]; fireHook('action', p); };
```

**action** fires for every command, menu item, palette entry, and toolbar invocation, carrying the command name in **\$.command**. One hook bound to **action** sees everything the HUD does; the specific events still fire independently, so a script can be as narrow or as broad as its author wants. The pattern is carried over from Audio-Haxor, where the same catch-all lets one script observe an entire application.

## The event catalogue

The authoritative list is `hooks::events()` in the host — 53 named events in twelve groups: the `action` catch-all, runtime and browser lifecycle, tabs, windows, top-frame navigation, downloads, bookmarks, history, sessions, management of other extensions, input and system, and HUD lifecycle. Each entry carries three fields:

```
{ "name": "tab-updated", "desc": "A tab finished loading", "sample": { "tabId": 12, "url":  
  "https://example.com", "status": "complete" } },
```

The `desc` is what the page's searchable picker filters on. The `sample` is the more useful field: it is the shape of the payload a script will actually receive, so the editor can show an author what `$_` looks like before the event has ever fired. A catalogue with samples is a documentation surface that cannot drift from the code, because the code is the catalogue.

The worker's listeners and the host's list are two halves of one contract, and the source says so — “*Event names match `hooks::events()` in the native host (keep in sync).*” Four catalogued events do not come from a worker listener at all: `palette-command`, `session-saved`, `pane-split`, and `audio-eq-changed` are fired by HUD pages through a `zbFireHook` relay message, so that every hook firing still passes through the one native-host seam rather than opening a second path. The contract has one loose end in the other direction: the worker's global error handler fires `zerror` with the failing site, message, and stack, and that name is not in the catalogue — a hook can be bound to it by hand, but the picker will not offer it.

## The running side

Hooks live in `~/.zwire/hooks`: a `hooks.json` manifest of `{ id, name, event, enabled, timeout_ms }` records, and one `<id>.st` stryke file per hook next to it. The host reads the manifest per operation rather than caching it in memory. That is a deliberate departure from the Audio-Haxor original, noted in the source: `zwire-host` has no long-lived per-app state object to hang a cache on, and a manifest this small is cheaper to re-read than to invalidate.

When an event arrives, `fire` matches it against the enabled hooks, spawns each one through `stryke_runner::run_script` with the hook's timeout (default 10 seconds), and pipes the envelope in on stdin:

```
{ "event": "download-completed", "payload": { ... } } → stdin  
{ "actions": [ { "notify": { ... } }, ... ] } ← stdout
```

The script's stdout is where the loop closes. `dispatch_stdout` scans the output for an `{"actions": [...]}` object — tolerating log noise around it, which the unit tests pin explicitly — and `dispatch_action` validates and runs each recognized action against the host's own capabilities:

| action              | dispatches to                             |
|---------------------|-------------------------------------------|
| <code>notify</code> | <code>osops::handle("notify", ...)</code> |
| <code>open</code>   | <code>osops::handle("open", ...)</code>   |
| <code>exec</code>   | <code>exec::run(...)</code>               |
| <code>pub</code>    | <code>bus::publish(...)</code>            |

Unknown actions are ignored rather than rejected, so a script written against a newer host degrades to doing less on an older one instead of failing outright. This is the same four-verb set the Audio-Haxor port arrived with, minus that application's database-bound verbs (tag, favorite, trash) and re-hosted onto the modules `zwire-host` already had — the port's stated method throughout: carry the model verbatim, replace the seams.

The result of every run — success, failure, timeout, and the action count — is published on the host's own pub/sub bus under the `hook-result` topic. That is what makes the Hooks page a live view rather than a form:

it subscribes to the topic over a `connectNative` port, so a hook that fires because a tab opened in some other window shows its result in the editor immediately.

## The editing side

`pages/hooks.js` is the CRUD surface, and it reaches the host through ten commands — `hooks_list`, `hooks_events`, `hooks_save`, `hooks_delete`, `hooks_set_enabled`, `hooks_get_script`, `hooks_script_path`, `hooks_set_script`, `hooks_test_run`, and `hook_fire`. The editor is Monaco with the stryke language server behind it (`stryke_lsp_start` / `stryke_lsp_send` / `stryke_lsp_stop` on the host side), carrying vim and emacs modes, which is the same editing surface the rest of the MenkeTechnologies stack uses rather than a bespoke textarea.

Two affordances matter more than the rest. The **event picker** is searchable over the 53-entry catalogue with its descriptions, so binding a hook does not require knowing the event's name in advance. The **test-run** button sends `hooks_test_run`, which executes the script against the event's `sample` payload without waiting for the real event — the difference between authoring a download hook in ten seconds and authoring it by downloading things.

The failure semantics are worth stating plainly, because an automation layer that hides them is worse than none. A hook that hangs is killed at its timeout. A hook whose script is missing, or whose stryke binary is absent, fails on the host and reports on the bus rather than anywhere the browser can see. And a hook is not a transaction: the browser event has already happened by the time the script runs, so a hook observes and reacts — it never vetoes.

# Chapter 50 – Output Triggers: Regex on What the Page Renders

A terminal emulator can watch its own output. iTerm2 calls them triggers: a regex over the text scrolling past, and an action when it matches. The reason no browser ships the equivalent is not that nobody wants it — it is that a tab is not a text stream. A tab multiplexer never sees rendered text; it sees tab objects and URLs.

zwire sees rendered text, because the HUD is already a content script on every page. `ztriggers.js` is the terminal-trigger idea moved into the browser: a regex matched against page text *as it renders and streams*, and, on a match, a chain of typed steps.

## The scan loop

The engine runs on real web pages only — `http`, `https`, `file`; the manifest's injection group excludes the HUD's own extension pages. It attaches one `MutationObserver` to `document.documentElement` with `subtree`, `childList`, and `characterData` set, plus a one-shot scan of what is already rendered when it starts:

```
try { enqueue((document.body || document.documentElement).innerText || ''); } catch (e) {}
```

Mutations do not scan directly; they enqueue text and schedule a flush. Three constants bound the work, and each one exists because the unbounded version is a page-stalling bug:

|                             |                   |                                                    |
|-----------------------------|-------------------|----------------------------------------------------|
| <code>FLUSH_MS</code>       | <code>350</code>  | coalesce a mutation burst into one scan            |
| <code>MAX_SCAN_LINES</code> | <code>2000</code> | cap the lines examined per flush                   |
| <code>MAX_LINE_LEN</code>   | <code>4000</code> | ignore minified blobs and base64 – not real output |

A page that rewrites its DOM sixty times a second produces at most three scans a second, each bounded in both dimensions. That is the difference between a trigger engine and a jank generator.

## Not scanning yourself

The single most important selector in the file is the exclusion list:

```
var ZWIRE_UI_SEL = '.palette-overlay,.zs-toast-container,.zs-toast,[data-zwire-ui]';
```

zwire injects its own UI into the page it is watching. Without the exclusion, two failures follow immediately. The ⌘K palette's command names are page text, so opening the palette to run a command can match a trigger looking for that command's own words. Worse, a trigger's result toast is page text too — a trigger that fires and reports would match itself on the next flush and re-fire forever. `isZwireUi` walks up from the mutated node with `closest()` and drops anything inside zwire's own surface, and the observer skips both the mutation target and each added node.

## Compilation and the gate

Each stored trigger compiles to a small record: the regex, an optional URL-filter regex, a cooldown, the once flag, and the step chain. Two compilation details are load-bearing. Flags are normalized before use:

```
function normFlags(f) { var out = (f || 'i').replace(/[gy]/g, ''); return out || ''; }
```

`.test()` on a global or sticky regex advances `lastIndex`, so the same pattern would skip every other line on a multi-line scan. Stripping `g` and `y` removes a bug class that would otherwise present as “my trigger only matches sometimes.” The default is case-insensitive, because a user typing a pattern into a form is not thinking about flags at all.

An invalid pattern, or an invalid URL filter, drops the trigger rather than throwing into the reactive loop — fail-closed, in a loop that runs on every page.

The URL filter is applied at compile time, not match time: `recompile` keeps only the triggers whose filter matches the current page, and if none survive, the observer is never attached. A page that can match nothing pays nothing.

Firing is gated twice. Each trigger carries its own `cooldownMs` (default 1500), so a burst of matching output cannot spawn a process storm; and a `once` trigger records its id in a module-level `firedOnce` map, so it fires at most once per page load. The map is module-level for a subtle reason the source calls out — it must survive `recompile()`, because an unrelated storage edit (enabling some other trigger) must not re-arm a trigger that has already fired. It resets naturally on a full navigation, when the content script is reloaded. Within a flush, a matching trigger fires once and breaks out of the line loop, so one burst of output is one action.

## The action

A trigger’s action is not a command string; it is a **chain of typed steps** — `shell`, `stryke`, `js`, `applescript`, `batch`, `action`, `scheme`, `host`, `url` — the identical step set a `⌘K` keyword command runs, edited by the same `ZwireStepWizard` and executed through the same exported runner:

```
window.ZWIRE_CMD_EXEC.runCustom({ steps: c.steps }, line);
```

The matched line is passed as the argument, so `{q}` in any step expands to it. This is the chapter’s real design point: triggers did not need an execution model, because the palette already had one. Adding a new *cause* to an existing *effect* system is a different amount of work from adding both, and it is why a trigger can do anything a command can — including running a stryke script, which puts Chapter 49’s automation surface behind a regex on page text.

`pages/triggers.js` is the CRUD surface over `chrome.storage.local`’s `zb_triggers` array, with a per-trigger enable toggle; the content script watches the same key through `chrome.storage.onChanged` and recompiles in place, so editing a trigger takes effect on already-open tabs without a reload.

# Chapter 51 — Pane Pipelines: A Dataflow Edge Between Two Panes

Chapter 10 describes ztmux as a tiling window manager in the browser: recursive splits, a prefix, a status bar, and a live webview in every pane. This chapter describes what happens when the panes stop being independent.

A **pane pipeline** is a persisted, reactive dataflow *edge* between two tiled webviews:

```
source pane --[ extract ]--> [ filter ] --> sink pane
```

It is the browser-native analog of `curl ... | jq | xargs`, except the stages are live rendered pages rather than processes. The tiling overlay stops being a layout manager and becomes a graph.

## A pure engine with three consumers

`zpipes-core.js` holds every decision and touches nothing:

THIS FILE IS PURE: no DOM, no `chrome.*`, no window side effects beyond hanging the API off a `window`-like global.

What a source emits, how a filter transforms it, whether an emission fires, the sink message to post, and whether an edge would close a cycle — all computed here. Three runtimes consume it: `ztmux-pane.js` (the pane-side forwarder that extracts and applies), `ztmux-config.js` (the top-frame relay that runs the edge), and `pages/pipes.js` (the CRUD page that validates one before saving). A fourth consumer is `tests/pipes.mjs`, which is the reason for the purity: a rule about cycles is worth far more when it can be asserted headlessly than when it can only be clicked.

Three call sites sharing one engine also means the page cannot accept an edge the runtime would reject, and the runtime cannot behave in a way the page's preview did not show.

## Source, filter, sink

A **source** names a URL-filter regex and one of four extraction kinds — a CSS selector's text, a regex over rendered text (capture group 1 when present), the current selection, or the pane's live URL. Extraction re-runs reactively when the source pane's content changes, through the same throttled `MutationObserver` discipline Chapter 50 describes; `linesOf` is deliberately identical in semantics to the triggers engine's, so a pipeline and a trigger see page text the same way.

A **filter** is one of three kinds. `none` passes values through. `js` evaluates an expression with `lines` and `text` in scope. `ops` is a small stryke-flavoured chain written with `|>`:

```
trim |> nonempty |> uniq |> grep ^https? |> take 5 |> join ,
```

The verb table is closed and split by arity — `trim`, `upper`, `lower`, `uniq`, `sort`, `reverse`, `nonempty`, `first`, `last`, `count`, `collapse` take no argument; `nth`, `take`, `drop`, `join`, `grep`, `reject`, `replace`, `prepend`, `append` take one. `parseOps` rejects an unknown verb at *validation* time, so a typo surfaces in the editor rather than as an edge that silently never fires.

A **sink** names its own URL filter and one of five deliveries: **navigate** the pane, **fill** a selector-addressed field, **replace** or **append** a node's content, or **batch**-open every line. Sink writes are posted as messages through the pane forwarder — never a direct cross-origin DOM write from the top frame, which the browser would refuse and which would be the wrong trust boundary even if it did not.

## The gate and the graph

Every emission passes `gate(edge, state, now, value)`: the per-edge cooldown (default 1500 ms), the **once** flag, and value **dedupe**, which is on by default so a re-render that produces the same text does not re-fire the edge. Dedupe is the flag that makes a reactive edge usable on a page that repaints constantly.

The structural risk is a loop. An edge from A to B and an edge from B to A is a livelock: each sink write mutates a page that is some other edge's source. `zwire` refuses it before it can exist. Edges are nodes-by-URL-pattern — `nodeId` maps an empty pattern to `*`, the match-anything node — and `wouldCycle(existing, sourcePattern, sinkPattern)` runs a reachability check over the enabled edges' adjacency before the page saves a new one. `graphCycle(edges)` answers the same question for a whole set, which is what validates an imported configuration.

Because a cycle check is a *structural* guarantee and pages are not structures, there is a second, dumber defense underneath it:

```
var HOP_BUDGET = 8;
```

A single emission may traverse at most eight hops before the runtime drops it. The source comment is explicit that this is the belt to the cycle check's suspenders: even with  $A \rightarrow B \rightarrow A$  rejected at creation, a long fan-out or a self-referential DOM write can re-arm an observer, and a budget bounds what that can cost.

## Living with it

An edge is started from the overlay with the `tmux` prefix then `|`, which seeds the active pane as the source and opens the editor; `pages/pipes.js` is full CRUD over `chrome.storage.local`'s `zb_pipes` array with per-edge enable toggles. Every edge is persisted, so a saved session's panes reconnect with their dataflow intact.

The positioning claim in the project's README is narrow and worth restating in the book's own terms: tiling browsers tile, and command-chain features chain commands, but a persisted data relationship *between two tiled views* is not something the neighbouring tools model at all. Vivaldi's Command Chains are sequences of actions; Zen and Arc split without cross-split dataflow; qutebrowser has no tiling to connect. The pairing is the novel part, not either half.

# Chapter 52 — The Audio Dashboard

Chapter 21 describes patches 0022 through 0024 — the browser-wide DSP engine compiled into Chromium's audio service, its live configuration path, and the meters back-channel. This chapter is the surface on top of them: [pages/audio.html](#), the page that makes an engine buried in a sandboxed utility process feel like a piece of studio hardware.

The page has two jobs, and the file says so up front: **control** the real browser-wide engine, and **preview and analyze** locally.

## Control: a spec string on disk

The engine's entire configuration is one string. The EQ bands serialize first, followed by two-field directives — [gain](#), [pan](#), [mono](#), [drive](#), [thresh](#), [ratio](#), [width](#), [delay](#), [feedback](#), [delaymix](#), [reverb](#), [room](#), [damp](#), [ceiling](#) — and the whole thing is written to `$ZWIRE_STATE/audio-eq`. The browser process polls that file and pushes it into the audio service over the `mojom` interface patch 0024 adds; the engine rebuilds its cascade when the generation changes. Nothing in the path requires the page to be open, which is why an EQ configured once keeps shaping audio in a browser whose HUD is closed.

The interesting engineering is in how fast that write is. The first implementation sent a native message per knob move, which spawned a host process, which ran `/bin/sh`, which wrote the file — 30 to 80 milliseconds of process setup per event, on a control the user is *turning*. The current path writes over the port that is already open:

```
meterPort.postMessage({ id: 'eqw', cmd: 'fs_write', path: eqPath(), text: spec });
```

The persistent meter port is already connected for the meters feed, so a knob move costs one message on an existing pipe. A one-shot `sendNativeMessage` remains as the fallback for the window where the port is down or reconnecting.

On top of that sits a **leading-and-trailing throttle** rather than a debounce:

fire immediately on the first move, then at most every 30 ms DURING a continuous turn (plus a trailing write)

A pure debounce would be silent for the whole gesture and correct only at the end, which is exactly wrong for a control you are listening to. Thirty milliseconds of write cadence against the engine's 25 ms poll puts the audible change inside the same perceptual moment as the knob movement — the measured round trip dropped from roughly 160 ms to 40–50 ms.

One more detail closes a paint gap: the authoritative spec lives on disk and is read back asynchronously, so the page mirrors every persist into `localStorage` and paints from the mirror on load. Without it the controls would show defaults for the length of a host round-trip and then jump.

Saving explicitly — as opposed to turning a knob — also fires the `audio-eq-changed` lifecycle event, so Chapter 49's hooks can react to an EQ change like any other browser event.

## Analyze: the real output, not a capture

The meters are the part that is easy to get wrong. The obvious implementation taps the tab with `tabCapture` and analyzes that stream; patch 0023 even makes `tabCapture` available to the HUD without a picker or prompt. But a capture mutes the source while it runs and drops audio when it is released, which makes “watch the meters” a destructive operation.

Instead the audio service computes the meter frame itself — a Goertzel spectrum, peak and RMS per channel, phase correlation, and a decimated stereo scope — from the **real post-DSP output**, the browser process pumps it to `$ZWIRE_STATE/meters` at roughly 30 fps, and the page reads that file over the native host. Closing the page touches nothing; the audio never notices it was being watched.

That pump is also the chapter’s best example of a bug that only exists at the seam. The first version ran on the UI thread, where the compositor’s gesture scheduler starved it: during a trackpad scroll the meter file froze, so the charts appeared to hang until the gesture ended. Moving the pump to a dedicated ThreadPools sequence took the file from 0.7 writes/second under gesture to 10+, and the page’s feed from 2/second to 27.5. The lesson generalizes past audio: a data path that is correct in isolation can still be starved by the thread it borrowed.

## The local graph

Alongside the real engine, the page runs its own Web Audio chain so a curve can be auditioned against a file, the microphone, pink noise, or a tone:

```
source → inputGain → [5-band biquad EQ] → compressor → pan → master → analyser →  
~ out
```

That local graph drives the instrument wall — spectrum, spectrogram, oscilloscope, goniometer, correlation, LUFS, peak, VU, gain reduction — with every widget a `ZGui.*` component from the shared toolkit, per the consume-never-copy discipline Chapter 17 sets out. The same rule is why the audio page looks like the rest of the HUD despite being, in function, a mixing console.

## Correctness

DSP is the one part of the fork where “it looks right” is not evidence. `fork/tests/run_dsp_tests.sh` extracts the engine’s source **straight out of patch 0022** and asserts per-effect invariants against it, so the tested code is literally the patched code rather than a reimplement that agrees with it today. That is the same port-verification instinct the rest of this book documents — pin the thing itself, not a copy of it.

# Chapter 53 — The Later HUD Pages

Chapter 14 covers the pages that *replace* a `chrome://` surface. The pages in this chapter replace nothing: they are features the browser does not have, added as HUD pages because a HUD page is the cheapest surface in the system — an extension page, the shared shell, the shared palette, the shared fuzzy filter, and the scheme applied for free.

One pattern runs through all of them and is worth naming before the list. Each page exposes its pure logic on a `window.ZB*` global — `ZBNotes`, `ZBFeeds`, `ZBTranslate`, `ZBReadingList`, and `ZWIRE_PIPES` for Chapter 51's engine — so the parsing, sorting, and tree logic can be exercised headlessly by the `tests/*.mjs` suite without a browser. A page whose logic is only reachable through a click is a page whose logic is untested.

## Dashboard

`dashboard.html` is the launcher: one searchable, categorized tile grid that opens every zwire HUD page *and* every `chrome://` internal page. The chrome is `ZGui.tiles` — the launcher component the toolkit already carried from traderview and Audio-Haxor — and per-category tile order is drag-reorderable and persisted through `ZGui.drag`. The shell header's search box drives the filter and the tiles' own search is switched off, so the page has exactly one filter box rather than two that disagree.

Its `chrome://` catalogue is transcribed verbatim from this build's own `chrome://chrome-urls`, in six sections: browser destinations, curated `settings` sub-pages (which `chrome-urls` does not enumerate), WebUI and dialogs, side panels and top-chrome, internals and diagnostics, and debug internals. Two classes are deliberately excluded, and the source says why: the “Command URLs for Debug” — `chrome://crash`, `kill`, `hang`, `gpubcrash`, `memory-exhaust`, `quit` — crash or hang the renderer on purpose, and `chrome-untrusted://` origins are component and iframe origins rather than navigable top-level pages. The maintenance rule is stated in the file rather than left implicit: when the browser gains or loses a WebUI, re-dump `chrome://chrome-urls` and reconcile.

The zwire section carries 23 tiles: one per HUD page, plus the two native destinations (`chrome://downloads`, `chrome://newtab`) the HUD intentionally does not replace with a page of its own.

## Notes

`notes.html` ports Vivaldi's Notes: a folder tree of Markdown notes over a flat array of `{id, parentId, type, title, content, url, ts}` nodes in `chrome.storage`'s `zb_notes`. Left pane is folders and notes with search; right pane is a Markdown editor with live preview through `ZGui.markdown`, plus attach-current-page, move-to-folder, and delete. It autosaves as you type — there is no save button to forget. The store helpers are pure functions over the node array, which is what `tests/notes.mjs` drives.

## Feeds

`feeds.html` ports Vivaldi's built-in feed reader. Feed URLs persist in `zb_feeds`; each is fetched directly from the extension page under `<all_urls>` host access — no proxy, no service — and parsed by a regex-based RSS and Atom parser that handles CDATA and entity decoding. The parser is regex-based specifically so it runs outside a browser: `window.ZBFeeds` is what `tests/feeds.mjs` exercises, with no DOM parser to stub.

## Translate

`translate.html` ports Vivaldi's Translate panel: type text, choose source (or auto-detect) and target, get the translation live. It calls the keyless Google `translate_a/single` endpoint directly from the extension page — no API key to store, no external command to shell out to — and `parseTranslation` folds that endpoint's nested array response into text plus detected language. Swap languages, copy the result; nothing is stored.

## Reading list

`readinglist.html` surfaces `chrome.readingList`, which has no first-class UI of its own worth the name: add the current tab, mark read or unread, remove, open, with two filters (Unread, All). The ordering helper is the page's whole logic — newest first, unread before read when both are shown — and it is exported for tests like the rest.

## App Store

`store.html` is the HUD's shop window: the MenkeTechnologies product catalogue rendered as `ZGui.productCard` tiles from a static `store-catalog.js`, grouped by category and filterable, each linking to its live product page. Cover images are vendored under `pages/store-assets/` so the tab renders with no network at all, with a glyph fallback when an image is missing.

It is also the one page zwire opens by itself. On `onInstalled`, `background.js` opens `store.html?welcome=1` once, with a welcome modal — the entire first-run experience, deliberately placed on a page the user can close rather than on the new-tab page, which stays exactly as configured. zwire itself is free and open source; the catalogue is the rest of the stack.

## Host console

`host.html` stands on the protocol Chapter 15 specifies and reads back the shared command log Chapter 8 describes, but it belongs to the same family as the pages here: a HUD page whose only job is to make an invisible subsystem inspectable — every command of the one JSON protocol, issued by hand, with the reply rendered.

# Chapter 54 — The Fourth Transport

Chapter 15 counts the ways to reach the host's dispatcher and arrives at three: native messaging on stdio, the NDJSON socket daemon, and the in-process library. That count is now short by one. `src/zbus.rs` — 871 lines, the largest single module in a `src/` tree that is 6,518 lines across 25 files — adds a fourth endpoint that is not the socket daemon, does not speak the host's own request shape, and is owned by a process whose only job is to own it.

The module's own first line says what it is for: it “makes `zwire-host` reachable as `App::open("zwire")` from a stryke script.” The caller is not the browser. It is a script, running outside everything this book has described so far, that wants to drive the machine *and* the browser through one handle.

## Why a fourth endpoint rather than a fifth command

The obvious objection is that the host already has a socket daemon speaking newline- delimited JSON, and a script could simply connect to it. The reason it does not is a protocol boundary, not a transport one. The bus speaks the `zgui-bridge` NDJSON wire protocol — a different frame vocabulary, with a `t` discriminator, request ids, and transaction control — while the socket daemon speaks the host's `{"cmd":...}` requests. The module header states the constraint that forced the split and the one that forced the hand-implementation:

We speak the `zgui-bridge` NDJSON wire protocol **NATIVELY** here rather than depending on the `zgui-bridge` crate: `zwire` is MIT and public, `zgui-bridge` is a private UNLICENSED crate, so a dependency would break public builds and mix licenses.

So the fourth transport exists because a second *protocol* had to be served, and it is hand-written because the crate that defines that protocol cannot be linked into an MIT binary. Chapter 59 is what that hand-mirroring costs and what keeps it honest. This chapter is the endpoint itself.

## The address, on three platforms

The bus address is chosen to match a client that was written independently, so the two meet with no registry, no discovery service, and no configuration file.

On Unix the directory is `$XDG_RUNTIME_DIR/zgui`, falling back to `$TMPDIR/zgui`, falling back to `/tmp/zgui`, and the leaf is `zwire.sock`. The directory is created and forced to mode `0700`; the socket itself is bound at mode `0600`. That is the same exposure reasoning the socket daemon uses and for a stronger reason — this endpoint reaches `exec`, `fs_write`, and `pty_spawn` through a session constructed as *trusted*.

On Windows there is no directory at all. The leaf name `zwire.sock` is mapped through `to_ns_name::<GenericNamespaced>()` onto the named pipe `\\.\pipe\zwire.sock`, and the module comment states the consequence plainly: “There is no filesystem directory on Windows — the pipe is a named kernel object, not a path.”

There is a third platform arm, `#[cfg(not(any(unix, windows)))]`, whose `ensure_daemon` is an empty function and whose `run_daemon` exits zero. The bus is absent rather than broken on anything that is neither.

The naming rule that makes client and server meet is one line of the header: the address “matches the bus client's per-app path (`<app>.sock` → same leaf → `\\.\pipe\<app>.sock`), so client and host meet with no registry.” An app named `zwire` is reachable at a path derived from its name on both families of platform, and the derivation is the whole protocol for finding it.

## The bug that dictated the ownership model

The interesting part of this module is not the socket. It is the answer to a question that looks trivial and is not: *which process owns the bus?*

The first answer was “whichever host runs first,” and the crate-level documentation records what that produced:

The bus is NOT owned by whichever host happens to run first — that was the intermittent `Connection refused (os error 61)` bug: Chrome spawns a short-lived host per `sendNativeMessage`, and one of those would bind the bus, get adopted by the persistent `connectNative` hosts, then exit — leaving a stale socket that nobody re-bound.

Every element of that failure is a property of the surrounding system rather than of the bus. Chrome’s native-messaging API has two shapes — a one-shot `sendNativeMessage` and a persistent `connectNative` port — and the one-shot shape spawns a fresh host process per message and reaps it when the reply lands. A design that lets any host bind the bus therefore hands ownership, at random, to the shortest-lived process in the system. The long-lived hosts then *adopt* the live socket rather than rebinding it, which is correct behaviour and exactly what makes the failure intermittent: the bus works, sometimes for hours, until the process that happened to own it exits for reasons that have nothing to do with the bus.

The fix is not a retry loop. It is a dedicated owner:

Instead, ownership belongs to a DEDICATED, detached `bus-daemon` singleton (the `ssh-agent/tmux` model): its only job is to hold the socket, so it never exits early.

`bus-daemon` is an internal subcommand of the same binary. `lib.rs`’s `run` matches it *before* anything else — before `ensure_strike_app`, with the comment “it has no need for the App package and should stay minimal” — so the daemon arm allocates nothing the other run modes allocate. Every other long-lived arm (`serve`, and the stdio native-messaging default) calls `zbus::ensure_daemon()` on the way up, which is what makes the arrangement self-healing: any host, of any lifetime, will notice an absent bus and start one, and none of them will own it.

The detachment is explicit on both platforms. On Unix the spawn sets `process_group(0)`, “so it survives this host’s exit and any signal sent to the host’s group,” with all three standard streams set to `Stdio::null()`. On Windows the equivalent is two `CreateProcess` creation flags the module defines by value because it does not link a Win32 crate: `DETACHED_PROCESS (0x0000_0008)` so the daemon gets no console, and `CREATE_NEW_PROCESS_GROUP (0x0000_0200)` so it ignores a `CTRL_C` aimed at the host’s group.

## Two singleton guards, one guarantee

Spawning a daemon whenever the bus looks dead is only safe if a second daemon cannot also bind. The module solves that twice, using whatever primitive the platform actually provides, and both solutions end in the same behaviour: the loser exits immediately, so exactly one daemon lingers.

On Unix the guard is an advisory `flock` on a sibling `zwire.sock.lock`, and the sequence under the lock is the interesting part. `bind_bus` first re-checks liveness by attempting a connect — a check the comment calls “authoritative” *because* it is held under the lock, “nobody can rebind between here and our bind.” Then it binds a private path `zwire.sock.<pid>.tmp`, sets it `0600`, and `renames` that over `zwire.sock`.

Two failure modes are being closed at once, and both are stated in the source. The lock serializes check-and-bind “so two starters can’t both bind (the loser’s `remove_file` would otherwise unlink the winner’s live socket — last writer wins the path).” The `rename` makes the replacement atomic: “no window where the path is missing (ENOENT) or points at a dead socket, and the listener survives the rename on macOS/Linux (verified).” That parenthetical is the honest form of the claim — a bound Unix listener continuing to accept after its path is renamed is a property of the platform, not of POSIX, and the comment records it as observed rather than assumed.

On Windows no lock is needed, because the kernel already offers the primitive: `interprocess` creates the first pipe instance with `FILE_FLAG_FIRST_PIPE_INSTANCE`, so a second daemon's `create_sync()` fails with `ERROR_ACCESS_DENIED` while the first owner is live. The comment names it “the direct analog of the Unix `flock`,” which is the right framing — one platform makes you build mutual exclusion out of a lock file, the other hands it to you as a flag on the object you were creating anyway.

The Windows liveness check is also structurally simpler for a reason worth noting: a successful connect means an owner exists, “the pipe object is refcounted by the kernel — no stale path to mistake for a live one.” The entire `flock`/temp-bind/rename dance on Unix exists to compensate for the fact that a Unix socket is a *file* that outlives its listener, and a named pipe is not.

## Synchronous by design

`ensure_daemon` does not return as soon as it has spawned. It spins — up to 200 iterations of a 10 ms sleep, a bounded two seconds — until a connect succeeds. The reason is given at the crate level:

SYNCHRONOUS: after spawning we spin (bounded ~2s) until the daemon has bound, so a script that `App::opens` immediately afterward (e.g. a `stryke -E` hook this host spawns) never races an unbound endpoint.

That is the case the asynchronous version gets wrong. A lifecycle hook (Chapter 49) is a `stryke` script the host itself spawns; if the host returns from `ensure_daemon` before the daemon has bound, the very first thing that script does can fail against an endpoint its own parent just started. Bounding the wait means the worst case is a two-second stall on a cold bus rather than an unbounded one on a bus that will never come up.

## The seam nobody in production sees

One environment variable exists solely to keep this machinery out of a developer's real system. `ZWIRE_BUS_NO_DAEMON`, when set, makes `ensure_daemon` perform the liveness check and then return without spawning. Its comment scopes it precisely: “Production never sets it; the `bus-daemon` arm ignores it (it binds directly, not via here).”

The distinction in that parenthetical is the design. The variable suppresses *auto-spawn*, not *serving*. A test that wants a bus can still run the daemon arm explicitly; what it cannot do is leave a detached, process-group-orphaned singleton behind on the developer's machine because `cargo test` happened to construct a session. Chapter 59's conformance suite sets it in its `hermetic()` fixture for exactly that reason, alongside redirected `HOME` and `ZWIRE_STATE`.

That is the whole endpoint: an address derived from a name, an owner that is nobody's side effect, a singleton guard drawn from whatever the platform actually guarantees, and a bounded synchronous handshake so the first caller after startup is not the one who discovers the race. What it *carries* is the next three chapters.

# Chapter 55 – The Verb Surface

A transport is not a capability. Having established, in Chapter 54, that a stryke script can open a socket to the host, the question is what it may say — and the answer is a list the host publishes about itself, on request, with a shape that tells a caller more than the names alone.

## One hundred and sixty-five verbs, in two families

`SURFACE_VERBS` in `src/zbus.rs` is a flat `&[&str]`. Counted from the source it holds **165** entries, which divide cleanly:

```
family      count  executed by
host verbs   65     zwire-host's own session dispatcher
browser.* verbs 100    the HUD service worker, via a forwarded action
```

The 65 host verbs are the capability surface Chapter 15 enumerates, reached under their own names: `fs_walk`, `exec`, `kv_merge`, `pty_spawn`, `sysinfo_start`, `job_start`, `hooks_save`, `stryke_lsp_start`, `peers`, `hostlog`, `meter_stream`, and the rest, plus the three transaction-control commands of Chapter 56. The list's own comment says it is “Comprehensive — every host command `session::handle_cmd` (+ the `fs/jobs/os/procs` sub-handlers) accepts.”

The 100 `browser.*` verbs are a different animal entirely: they are not host commands and the host cannot execute a single one of them. They are grouped in the source by what they touch, and the grouping is the browser's own surface area laid out flat —

```
group (as the source comments it)      verbs
tabs                                   17
tab navigation / position               10
page navigation (active tab)           6
zoom                                    3
windows                                10
window tiling / positioning (system.display work area) 10
bulk tab ops / capture / language      7
tab groups                             4
downloads                              9
browsing data                          5
reading list                           2
power                                   3
extension / app management (id param)  5
history / bookmarks / notifications    7
terminal (browser.tmux)                1
transactional compensation (browser.undo) 1
---
100
```

A stryke script therefore addresses two machines through one handle: the workstation and the browser running on it, in one flat namespace where the only marker of which is which is a `browser.` prefix.

## Discovery is not dispatch

The single most important property of that list is that it does not gate anything. Its own comment says so twice: “Discovery only; `call` accepts any command,” and for the browser half, “`call` accepts any `browser.<verb>`; the HUD worker’s `execZbCmd` is the executor.”

This is a deliberate asymmetry and it is worth being precise about which direction it cuts. A verb absent from `SURFACE_VERBS` still runs — the list is advertising, not an allowlist, so a host command added to `session.rs` and forgotten here works perfectly and is merely invisible to a script that asked what was available. The reverse is also true: a verb listed here whose implementation does not exist is advertised and fails at call time. The list is a claim about the host, kept true by hand.

That choice makes the surface cheap to extend and impossible to trust as a security boundary — which is consistent, because the bus is not one. Chapter 54 established that the socket is `0600` under a `0700` directory; that file mode *is* the boundary. Inside it, every caller is the owner.

## Running a real session against an in-memory connection

The mechanism by which a bus frame becomes a host reply is the neatest thing in the module, and it exists to avoid a duplicate dispatcher.

The naive implementation of `call` would be a second `match` over verb names, translating each to the corresponding host operation. `zbus.rs` instead constructs the host’s own request object — `{"cmd": verb, ..args, "id": 1}` — and runs it through the real `session::Session::handle`, against a writer that is not a connection at all:

```
let buf = Arc::new(Mutex::new(Vec::<u8>::new()));
let out = Peer::ndjson(Box::new(Capture(buf.clone())));
let mut sess = Session::new();
sess.handle(&out, &msg);
```

`Capture` is a short `std::io::Write` implementation that appends to a shared `Vec<u8>` and flushes to nothing. The session writes its NDJSON reply frames into that buffer exactly as it would to a socket, and `run_command` reads the first parseable line back out. The function’s doc line states the payoff: “every host command works with zero duplication.”

Two consequences follow from the specific choice of *first line*, and the source names both. “Streaming commands (sysinfo/pty/job) return only their first frame” — so `sysinfo_start` over the bus yields its acknowledgement and nothing else, because there is no connection for the subsequent frames to arrive on. And `Session::new()` is documented in `session.rs` as the “Fresh, trusted session (local socket / stdio)” constructor, as distinct from `Session::guarded(require_auth)`, which is what inbound TCP gets and which refuses everything but `auth`, `peer_hello`, `hello`, and `ping` until a token is presented. The bus takes the trusted constructor. A caller that reached the socket has already passed the only check there is.

## The frames, and the two that are almost the same

`serve_conn` dispatches on the frame’s `t` field. Eight kinds are understood, and everything else takes the fallthrough:

| t       | argument          | behaviour                                         |
|---------|-------------------|---------------------------------------------------|
| call    | verb + args       | run_command(verb, args); transaction gate applies |
| get     | state             | run_command(state, {})                            |
| verbs   | –                 | surface()                                         |
| sub     | event             | replies ok/null; not bridged                      |
| begin   | txn (or args.txn) | txn_begin                                         |
| commit  | txn               | txn_commit                                        |
| abort   | txn               | txn_abort – the one that does work                |
| undo    | args              | browser.undo                                      |
| (other) | –                 | ok:false, error "unknown request kind"            |

`call` and `get` are almost the same function, and the difference is entirely in the caller's intent rather than in the host's behaviour: `get` passes an empty argument object and skips the transaction gate. There is no separate registry of readable state — any host command is a legal `state`, which means `{"t":"get","state":"exec"}` is a well-formed frame that runs a subprocess. The `surface()` reply advertises exactly two states, `hostinfo` and `scheme`, and that advertisement is again a claim rather than a constraint.

`sub` is the one frame that is acknowledged without being implemented, and the source is explicit about why rather than leaving a silent stub: “Event subscriptions are not bridged yet (host pub/sub is process-global and not request/reply). Acknowledge so the client doesn't hang.” The `surface()` reply's `events` array is correspondingly empty. A client that subscribes gets a successful reply and no events — which is a worse failure than an error would be, and is the honest consequence of the alternative being a hang. The underlying obstacle is real: `bus::publish` fans out to subscribers registered on *this process's* broker, and the bus daemon is a different process from the one holding the browser's native port. Chapter 58 is what the system does about that for the one topic it cannot do without.

## The reply that tells you what you may not do

`surface()` returns each verb as `{id, label, rev}`, and the third field is the one that makes the discovery list worth requesting. `rev` is the verb's *reversibility class*, and the doc comment states what it is for: “so a script can tell, before it builds a chain, which steps are safe to put inside `App::txn`.”

That is a genuinely unusual thing for a command surface to publish. Most automation APIs tell a caller what exists; this one tells a caller what can be taken back. The next chapter is the classification, its default, and the point in the request path where it is enforced.

# Chapter 56 – Reversibility as a Declared Property

`App::txn { ... }` wraps a chain of bus calls so a failure anywhere in the chain undoes everything before it. Implementing that across a browser and a workstation raises a question transaction systems normally answer with storage: what, exactly, gets rolled back when the thing you did was *close a tab*?

zwire's answer has two halves in two repositories. The host owns order and admission control; the browser owns pre-state and replay. This chapter is the host half — `src/txn.rs` at 165 lines, plus the classification table in `zbus.rs`. Chapter 57 is the other.

## Three classes, and which one is the default

`REV` is a `&[(&str, &str)]` mapping verb to class. It holds **65** entries: **34** classed `pure` and **31** classed `inverse`. `SURFACE_VERBS` holds 165 verbs. The 100 verbs with no entry are not unclassified — they have the class `rev()` returns when the lookup misses:

```
pub fn rev(verb: &str) -> &'static str {
    REV.iter()
        .find(|(id, _)| *id == verb)
        .map(|(_, class)| *class)
        .unwrap_or("irreversible")
}
```

The three classes, in the source's own words:

- `inverse` — “a compensation exists that restores the prior observable state. Journalled on call, replayed in reverse order on abort.”
- `pure` — “reads only. Runs inside a transaction, is not journalled, is not compensated.”
- `irreversible` — “the DEFAULT for anything absent from `REV`. Rejected at call time while a transaction is open.”

Counted against the surface, the distribution is:

| class        | verbs | browser.* | host |
|--------------|-------|-----------|------|
| inverse      | 31    | 31        | 0    |
| pure         | 34    | 6         | 28   |
| irreversible | 100   | 63        | 37   |

Two facts in that table are design statements rather than arithmetic. First, **no host verb is ever `inverse`**, and the comment says why without hedging: “Only `browser.*` verbs can be “`inverse`”... No host-side command has a compensation, so every host WRITE is “`irreversible`” here rather than optimistically classed.” There is no inverse for `fs_write` that restores the previous bytes, no inverse for `exec`, no inverse for a killed process — so rather than invent optimistic ones, the table declines to classify them and lets the default catch them. The six `browser.*` verbs classed `pure` are the ones that change nothing observable: `reload`, `reloadHard`, `reloadAll`, `screenshot`, `detectLanguage`, and `undo` itself.

Second, the default is the strict one, which is the only defensible direction. A missing entry that defaulted to `pure` would let an unclassified write run inside a transaction and silently not be compensated; a missing entry that defaulted to `inverse` would journal something with no inverse and produce a failed unwind at

abort time. Defaulting to `irreversible` makes a forgotten entry fail loudly at the first call inside a transaction — the safest of the three ways to be wrong. The conformance suite pins it with a literal: `assert_eq!(zbus::rev("browser.nothing-like-this"), "irreversible")`.

## Refusal at the top of the chain, not at the bottom

The class is enforced in exactly one place, in `serve_conn`'s `call` arm, and the position is the whole point:

```
if crate::txn::any_open() {
    let txn = req.get("txn").and_then(Value::as_u64);
    if txn.is_none_or(crate::txn::is_open) {
        match rev(verb) {
            "irreversible" => { /* reply error, continue */ }
            "inverse" => { crate::txn::record(txn, verb, &args); }
            _ => {}
        }
    }
}
```

An `irreversible` verb inside an open transaction never runs. The comment states the alternative that was rejected: “an `irreversible` verb is refused HERE rather than at abort time — the whole point of the class is that a chain never gets stranded half-undone.”

The distinction matters because the two designs fail at opposite ends of the chain. Refusing at abort time means step seven of a nine-step chain discovers that step three cannot be undone, at which point the transaction is already in a state nobody designed. Refusing at call time means step three fails immediately, with the error text `verb not reversible: browser.clearHistory`, and the chain has done exactly what a chain that stops at step three should have done.

`any_open()` exists so this costs nothing when no transaction is in flight. Its doc says: “Cheap gate: when false, `call` dispatch takes exactly the path it took before transactions existed — no journaling, no reversibility check.” That is a real property and the conformance suite pins it too — `irreversible_verb_runs_normally_outside_a_transaction` asserts that `browser.clearHistory`, refused inside a transaction, is untouched outside one. Transactions gate transactions; they never become a general permission system that quietly narrows ordinary automation.

## One clock for every transaction

`txn.rs` holds three pieces of state, all process-global: a `HashMap<u64, Vec<JournalEntry>>` of open transactions under a `Mutex`, an `AtomicU64 seq`, and an `AtomicU64 next_txn`.

The single shared `seq` is the design decision worth extracting. The header explains it: “One monotonic `seq` clock is shared by ALL of them so interleaved steps from different connections still unwind in a defined global order.” Two scripts, on two connections, each inside its own transaction, produce steps that interleave in real time — and if each transaction numbered its own steps from one, an abort of both would have no defined order between them. A single clock gives every step in the process a total order, which is the only thing an unwind actually needs.

`record` has a second subtlety in its signature: `txn: Option<u64>`. A `None` records the step “against every open transaction,” which the doc explains is “what an un-tagged `call` frame from a client that only sent `begin` does.” A client that opened a transaction and then made plain calls without echoing the id is not making an error; it is using the API the obvious way, and the host attributes its steps to whatever is open.

## Take before you compensate

`take_reversed` removes a transaction from the map, sorts its entries by descending `seq`, and returns them. The removal happens *before* anything is compensated, and both `txn.rs` and `zbus.rs` flag the reason in comments — “so a concurrent second abort on the same id gets an empty list rather than compensating twice.”

This is the same idempotence discipline the storage layer uses elsewhere in the system, and it is load-bearing here because compensation is destructive in the ordinary sense: re-running an unwind reopens tabs that were already reopened and closes tabs that were already closed. `double_abort_compensates_once` in the conformance suite asserts the observable form — the first abort reports `steps: 1`, the second reports `steps: 0` and a null `undo`, because “nothing is forwarded when there is nothing to compensate.”

`commit` is the mirror image and is deliberately not a no-op: it removes the transaction and reports how many forward steps it dropped un-compensated. A caller therefore learns, from a successful commit, how much it just made permanent.

## What `txn_abort` actually sends

The abort path is the one place the two halves of the system meet, and its shape is dictated by a limitation stated in `txn.rs`’s header: the host “does NOT own the pre-state of a `browser.*` verb. Those verbs are fire-and-forget across the native port — the forward reply carries a delivery count, not a browser result, so there is no undo handle to journal here.”

So the abort does not send N undo commands. It sends one:

```
let entries = crate::txn::take_reversed(txn);
let steps: Vec<Value> = entries.iter()
    .map(|e| json!({ "seq": e.seq, "verb": e.verb, "args": e.args }))
    .collect();
let undo = if steps.is_empty() { Value::Null }
    else { run_command("browser.undo", &json!({ "txn": txn, "steps": steps })); };
crate::hooks::fire("txn-aborted", json!({ "txn": txn, "steps": steps.len() }));
```

One `browser.undo` action, carrying the whole reversed step list. The comment on `execZbCmd` in the browser gives the reason from the other side: “an N-step unwind is a single native-messaging round trip, not N.” Given that each round trip is the ~200 ms this system measures elsewhere, a nine-step unwind that cost nine trips would be visibly slow at exactly the moment the user is watching something go wrong.

The `txn-aborted` lifecycle hook fires afterwards, which means the compensation itself is observable by the automation layer of Chapter 49 — a script can react to a rollback.

## The seam: a sequence number that is allocated and never sent

The two halves agree on a contract that the current tree does not quite implement, and the gap is worth stating precisely because everything around it works.

The browser half keys its journal on a `_seq` field carried by the forwarded action. `background.js:815` reads `if (!c._seq) { J.arm(c, ...); return; }`, and `zjournal.js`’s `arm` opens with `var seq = c._seq; if (seq == null) { run(); return; }`. The comment in `zjournal.js` describes the intended source: the host “stamps `_txn/_seq` only onto `inverse` verbs, and this module journals exactly the actions that carry a `_seq`.”

In the tree as it stands, nothing writes that field. `txn::record` returns `Option<u64>` — the `seq` it allocated — and the call site at `zbus.rs:563` is `crate::txn::record(txn, verb, &args)`, discarding it. The payload `run_command` builds for a `browser.*` verb is `{ "a": action }` plus the caller’s args plus the `_n` nonce (`zbus.rs:364-390`); a search for `_seq` and `_txn` across `zwire-host/src` returns only `txn.rs`’s

own `next_txn` field. The `seq` exists, orders the journal, and rides out in the abort's `steps` list — but it never reaches the executor that would key a capture on it.

The observable consequence is specific and quiet. Forward calls run normally. The host journal fills, orders correctly, and aborts in the right order — `abort_unwinds_in_reverse_call_order` passes because it asserts on the host's forwarded payload, not on a browser. But the browser journal is never armed, so `zjournal.undo` looks up `steps[String(f.seq)]`, finds nothing for every frame, and returns `{ok: true, steps: N, ops: 0, compensated: 0, failed: []}`. An abort reports success, reports the right number of steps, and compensates nothing — and the `ops: 0` counter in that report is the only place the difference shows.

That is a one-field gap in a two-repository contract, and it is the shape this system produces when the two halves are pinned by tests that each exercise one side. Chapter 59's conformance corpus pins the frames; nothing pins the *stamp*, because the stamp crosses the one boundary neither test suite spans.

# Chapter 57 — The Journal the Host Cannot Keep

`zjournal.js` is 314 lines in `extensions/hud-internal/`, and its opening comment states its remit as a negative: it is “the half of transactional compensation that `zwire-host` explicitly does NOT own.” Chapter 56 established what the host keeps — order, classes, admission control — and why it cannot keep the rest: a `browser.*` forward returns a delivery count, not a browser result, so the pre-state of a browser operation does not exist anywhere the host can see it.

The pre-state is captured in the service worker, at execution time. How it is captured is the interesting part.

## Observe the effects; do not re-derive them

There is an obvious implementation of pre-state capture, and the file rejects it in its header before writing a line of code:

The naive design mirrors each verb’s selection logic in a capture function (“`closeRight` closes the tabs after the active one, so snapshot those”). That duplicates every branch of `execZbCmd` and drifts the moment one is edited.

The alternative is to let the browser tell you what happened:

Instead this module snapshots the tab set, runs the action, and listens to the real `chrome.tabs` / `chrome.windows` events it produces. Whatever the browser actually did is what gets journaled, so the inverse is correct by construction for `closeDuplicates`, `sortTabs`, a future verb, or a branch someone rewrites — none of which this file knows anything about.

The structural consequence is stated in the next paragraph and is the file’s strongest property: “Consequently there is no verb table here.” A module implementing undo for 31 reversible verbs contains zero verb names. The host’s `REV` table is the only place a verb is named, and this module journals whatever arrives carrying a sequence number.

That is worth contrasting with the alternative one more time, because the difference is not stylistic. `browser.closeDuplicates` closes a set of tabs computed by comparing URLs across every window. A capture function would have to reimplement that comparison, and would silently produce wrong inverses the day someone changed how duplicates are detected. An effect listener journals a `reopen` step per `chrome.tabs.onRemoved` it actually saw, and cannot be wrong about which tabs closed, because it is reading the browser’s own report.

## What a snapshot is for

Effect events are not sufficient on their own, and `snapshot` captures precisely the two things they cannot supply. Its comment: “Every tab, plus which tab is active per window — the two things an effect event cannot tell us after the fact (a removed tab’s descriptor is gone; the previously-active tab is not in `onActivated`).”

So the snapshot is not a general “before” image used to diff. It is a targeted cache of the facts the event stream is missing. Per tab it holds `{id, url, index, windowId, pinned, muted}`, and per window it holds the id of the active tab. Everything else the inverses need arrives in the events themselves — and where an event

does carry authoritative pre-state, the module prefers it: `onMoved` supplies `fromIndex` and `onDetached` supplies `oldPosition`, and the comment notes these are “authoritative even when the tab moved more than once,” which a single pre-action snapshot would not be.

## Eight inverse operations

The replay vocabulary is eight operations, and it is short because it is expressed in terms of browser state rather than in terms of verbs:

| op                       | fields                                           | inverse of                     |
|--------------------------|--------------------------------------------------|--------------------------------|
| <code>reopen</code>      | <code>url, index, windowId, pinned, muted</code> | a tab that was closed          |
| <code>close</code>       | <code>id, windowId</code>                        | a tab that was created         |
| <code>closeWindow</code> | <code>id</code>                                  | a window that was created      |
| <code>move</code>        | <code>id, index, windowId</code>                 | a tab that moved               |
| <code>flags</code>       | <code>id, pinned?, muted?</code>                 | a pinned/muted flag that       |
| <code>~ flipped</code>   |                                                  |                                |
| <code>activate</code>    | <code>id</code>                                  | the tab that was active before |
| <code>navigate</code>    | <code>id, url</code>                             | the url the tab was showing    |
| <code>zoom</code>        | <code>id, factor</code>                          | the zoom factor it had         |

Thirty-one `inverse` verbs collapse into eight operations because most of them differ only in *which* tabs they select, and selection is exactly the part the effect listeners make irrelevant. `closeTab`, `closeOthers`, `closeRight`, `closeLeft`, and `closeDuplicates` all journal the same `reopen` step, once per tab the browser actually removed.

`runOp` maps each to a single `chrome.*` call — `tabs.create`, `tabs.remove`, `windows.remove`, `tabs.move`, `tabs.update`, `tabs.setZoom` — with one wrinkle: `reopen` cannot restore a muted tab in one call, because `tabs.create` has no `muted` property, so it creates the tab and then issues a follow-up `tabs.update({muted: true})` when the journaled tab was muted. Every `reopen` is created with `active: false`, so an unwind that restores six tabs does not steal focus six times.

## The settle window

`arm` snapshots, dispatches the action, and then holds the capture window open for `SETTLE_MS` — 400 ms — before disarming. The reason is a property of the executor rather than of the journal: “`execZbCmd`’s writes are callbacks on top of `chrome.tabs.query`, so a same-tick window would miss all of them.”

This is the one place the design pays for a timer rather than a completion signal, and the cost is honest: an effect arriving at 401 ms is not journaled, and an unrelated effect arriving at 399 ms is. What makes it tolerable is the MV3 lifecycle note attached to it — “The persistent native port keeps the MV3 worker alive across that window” — so the window is not racing a worker teardown, only racing the browser’s own asynchrony.

`record`’s dedupe key is the second half of the correctness argument, and the comment states the invariant exactly: “the FIRST observation of an effect holds the true pre-state, so a second event for the same tab+aspect (two updates in one action) is dropped rather than overwriting the pre-state with an already-changed value.” The keys are shaped `'reopen:'+tabId`, `'move:'+tabId`, `'pinned:'+tabId`, `'url:'+tabId`, `'activate:'+windowId` — one slot per tab per aspect. An action that navigates a tab twice journals the URL it started from, not the intermediate one.

## Four ceilings

The module carries four bounds, all so “a runaway script cannot grow the journal without bound”:

|                   |      |                                                         |
|-------------------|------|---------------------------------------------------------|
| SETTLE_MS         | 400  | capture window per armed step (ms)                      |
| MAX_STEPS_PER_SEQ | 500  | inverse steps retained for one forward step             |
| MAX_SEQS          | 2000 | forward steps retained, FIFO-evicted by insertion order |
| MAX_NONCES        | 4096 | action stamps remembered for de-duplication             |

`MAX_SEQS` evicts through an `order` array shifted from the front, so the journal forgets its oldest forward step rather than refusing a new one — the right direction for a feature whose failure mode is “an old transaction cannot be undone” rather than “a new action cannot run.”

## Surviving a worker teardown

The journal is written through to `chrome.storage.local` under `zb_journal` after every settle, and `hydrate` reads it back. The merge rule on hydrate is one line and the right one: `if (steps[k]) return;` — “a live entry always beats a persisted one.” A worker that was torn down mid-transaction and revived comes back with its journal; a worker that has already observed effects for a sequence does not have them replaced by a stale snapshot of the same sequence.

This is the same MV3 constraint every chapter about the service worker has had to answer, arriving here in its sharpest form: a transaction is precisely the kind of long-lived state MV3 promises not to preserve.

## Replaying, and the two orders

`undo` receives one `{a:'undo', txn, steps:[{seq, verb, args}]}` frame. The host has already sorted `steps` by descending `seq`, so the outer loop walks the forward chain newest-first; within each forward step, the module walks its own observed effects backwards (`for (var i = got.length - 1; i >= 0; i--)`). Two nested reversals, and the comment names both: “the forward chain unwinds newest-first; within one forward step the observed effects unwind in reverse observation order too.”

One filter runs before replay, and it encodes a real interaction between two ops:

```
var closedWins = Object.create(null);
ops.forEach(function (o) { if (o.op === 'closeWindow') closedWins[o.id] = 1; });
ops = ops.filter(function (o) { return !(o.op === 'close' && closedWins[o.windowId]); });
```

A `browser.newWindow` creates a window *and* its first tab, so the journal holds both a `closeWindow` and a `close`. Closing the window takes the tab with it, and the tab’s own `close` would then fail on an id that no longer exists. The comment gets the framing right: “drop it here rather than report a failure that is really a success.”

Every remaining op is attempted even after one fails — “the same per-step isolation the host and `zgui-core`’s `txnAbort` give” — and the report distinguishes the outcomes: `{ok, txn, steps, ops, compensated, failed}`, where `failed` is a list of `{op, id, error}`. An unwind that half-succeeds says so, per operation, rather than reporting a single boolean that would have to be false.

The report is then handed to `fireHook('txn-undone', report)`, which puts the compensation’s own outcome on the lifecycle bus — so a rollback that partly failed is something an automation script can act on, not just something a log records.

# Chapter 58 – One Action, Three Deliveries

Chapter 55 established that 100 of the bus's 165 verbs are executed by a service worker in a browser the host cannot call into. Chapter 56 established that the abort path depends on one of those forwards arriving. This chapter is how a `browser.*` verb actually crosses, which turns out to require three delivery attempts and one de-duplicator, for a reason that is a straightforward consequence of Chrome's process model.

## Why one path is not enough

`run_command`'s `browser.` branch builds a payload — `{"a": <action>}` plus every caller argument except `cmd` and `a` — and then pushes it three ways:

```
let delivered = crate::bus::publish("zbus.action", &payload);
let forwarded = crate::peer::broadcast("zbus.action", &payload);
// ...
kv.insert("_n".into(), json!(nonce));
crate::store::kv_set("zwire", "__zbus_action", &Value::Object(kv));
return json!({ "ok": true, "action": action, "delivered": delivered,
               "forwarded": forwarded, "queued": true });
```

The first is the same-process fan-out. `bus::publish` collects every subscriber whose topic set contains `zbus.action`, under the broker lock, and writes the `{"ev": "pub", "topic", "data"}` frame to each outside it — “so IO never blocks the broker or deadlocks a publisher that is also a subscriber” — returning the count that lands in `delivered`.

The second is federation. `peer::broadcast` sends `{"cmd": "peer_pub", "topic", "data"}` to every linked peer host, so a browser action published on one machine reaches the subscribers of another. It returns the count in `forwarded`.

The third exists because the first two frequently deliver to nobody, and the comment states the reason precisely:

Cross-process delivery: the HUD's host process is usually NOT the one that owns the zgui socket (a separate `serve` daemon does), so pub/sub alone reaches no subscriber there.

This is Chapter 54's ownership model coming back around. The bus daemon is a dedicated singleton; the browser's native port is held by a *different* host process; a one-shot `sendNativeMessage` is a *third*. `bus::publish` is process-global and therefore process-bounded. A publish inside the bus daemon reaches the subscribers of the bus daemon, which on a typical machine is nobody at all.

So the third delivery is a file. `store::kv_set("zwire", "__zbus_action", ...)` writes `$ZWIRE_STATE/kv/__zbus_action.json` through `write_atomic`, which writes a sibling `.tmp` and renames over the target “so a reader never observes a half-written file.” The key name survives `sanitize` intact because that filter keeps ASCII alphanumerics plus `-`, `_`, and `..`. Where pub/sub is bounded by a process, a file is bounded by a filesystem — and every host process on the machine, whatever spawned it and however long it lives, can see it.

The stamp that makes the file usable is `_n`: milliseconds since the epoch, distinct per action. It is what turns a single-slot file into a delivery channel rather than a race.

## The four ways an action arrives at `execZbCmd`

On the browser side the paths converge on one function. `execZbCmd` in `background.js` is described in its own comment as “the single browser-action executor,” and four routes feed it.

**The persistent port’s subscription.** `startSysStream` opens a `connectNative` port and immediately subscribes to four topics — `zbus.action`, `scheme`, `ui`, `palette` — alongside `sysinfo_start`. Its message handler routes `m.ev === 'pub' && m.topic === 'zbus.action'` straight into `execZbCmd(m.data)`. This is the path that works when the publishing process is the one holding this port.

**The `stryke_run` reply.** A stryke script that calls `browser.*` leaves its action in the kv file, and `session.rs` rides it back on the reply of the run that produced it. The handler snapshots the nonce before running and compares after:

```
let before = crate::store::kv_get("zwire", "__zbus_action").get("_n").and_then(|n|
    ~ n.as_u64());
// ... run the script ...
let a = crate::store::kv_get("zwire", "__zbus_action");
let n = a.get("_n").and_then(|x| x.as_u64());
if a.get("a").is_some() && n != before {
    body["zbAction"] = a;
    crate::store::kv_del("zwire", "__zbus_action");
}
```

The before/after comparison is what stops a stale action from riding back: only an action whose nonce changed during the run is attached, and the entry is deleted once claimed. The comment gives the motivation — “delivery is as reliable as the run” — which is the strongest guarantee available, because the reply is a frame the caller is already waiting for.

**The relayed content-script run.** A content script’s `stryke_run` cannot use its own one-shot channel, and `background.js` explains why: “a one-shot `sendNativeMessage` from a page’s palette loses its callback because the MV3 worker is torn down during the ~200ms native round-trip (external = 0%).” So content-script runs are relayed over the persistent port with a unique `id`, the host copies the `id` back alongside any `zbAction` it stamped, and the worker matches reply to run and executes exactly once.

**The storage bus.** A HUD page that receives a `zbAction` on its own `stryke_run` reply hands it to the worker through `chrome.storage.local`. `zg-boot.js`’s `runZbAction` copies the action, adds `_zbn = (a._n || 0) + ':' + window.__zbSeq`, and writes `zb_cmd`; `background.js` listens on `storage.onChange` and calls `execZbCmd(changes.zb_cmd.newValue)`. The `_zbn` field exists for a reason unrelated to identity: `onChange` fires only on a real value change, so a repeated action written twice with identical contents would not wake the worker the second time. The comment records the symptom that produced the fix — “worked 1x then stopped.”

## Exactly once, and where the guarantee stops

Three deliveries of one action require de-duplication, and it lives in the journal module rather than in the executor. `freshAction(n)` answers true the first time a stamp is seen and false forever after, backed by a 4,096-entry FIFO. `execZbCmd`’s first substantive line is the gate:

```
if (c._n != null && !J.freshAction(c._n)) return;
```

`zjournal.js` explains why the de-duplicator lives beside the journal rather than beside the dispatcher, and the reason is stronger than tidiness: “a double-executed step would journal a second, already-changed pre-state — the undo would then restore the wrong thing, not merely act twice.” Duplicate execution is bad; duplicate capture is corrupting.

The gate is conditional on `c._n != null`, and one path deliberately arrives without it. `runZbAction` copies every field except `_n` (`for (var k in a) { if (k !== '_n') q[k] = a[k]; }`) and substitutes `_zbn`.

That action therefore bypasses `freshAction` entirely and always executes. The choice is defensible on its own terms — the page cannot know whether the worker already ran it, and the host deleted the kv entry when it attached it to the reply — but it is worth stating as what it is: exactly-once is enforced for the two paths that carry `_n`, and the HUD-page drain is at-least-once by construction.

## What the reply tells the caller, and what it does not

The forward reply is `{ok: true, action, delivered, forwarded, queued: true}`. Every field describes the *host's* effort. `delivered` counts local subscribers written to; `forwarded` counts peers; `queued` is a constant. None of them reports whether a tab was closed, and none of them can, because the browser never answers.

`txn.rs` names this as the reason the pre-state journal had to move into the worker, and the conformance suite works around it in the same way — `abort_unwinds_in_reverse_call_order` asserts step order by reading the payload the host *stamped into the kv file*, because “a `browser.*` forward is fire-and-forget (it returns a delivery count, not a browser result), so the ORDER is asserted on the payload it stamped for the HUD worker, not on the reply.”

Which is the honest summary of this chapter: the bus's browser half is a one-way channel with three redundant carriers and a nonce, and everything downstream of it — transactions included — is designed around the fact that no reply is coming back.

# Chapter 59 — A Protocol Mirrored by Hand

Chapter 54 gave the reason the bus protocol is reimplemented rather than imported: `zwire` is MIT and public, `zgui-bridge` — the crate that defines the wire format — is private and UNLICENSED, so linking it “would break public builds and mix licenses.” This chapter is what that decision costs and the single test file that keeps it from costing more.

## The failure mode a hand-mirror produces

The module header does not describe the risk in general terms. It names it, in capitals, and describes exactly how it presents:

PROTOCOL DRIFT IS THE STANDING HAZARD HERE. Because the frames above are hand-mirrored rather than shared with the `zgui-bridge` crate, a frame added there and forgotten here degrades to the “unknown request kind” fallback in `serve_conn` — a clean error, never a hang, but still a silent capability gap.

Two things in that sentence are doing work. The fallback is a *feature*: a client sending a frame this host predates gets a well-formed error reply rather than a socket that never answers, which is the difference between a script that fails and a script that hangs. And the fallback is also the hazard, because a clean error at the wire level is indistinguishable from a capability that was never implemented. Nothing fails in either repository. The feature is simply absent on one side, and the only symptom is a script that does not do what its author expected.

The mitigation is stated as a rule in the same paragraph: “Every frame added to `zgui-bridge/src/proto.rs` must be added to that corpus and to the match below in the same change.”

## The corpus

`tests/wire_conformance.rs` — 260 lines — opens with a **CORPUS** of 10 frame strings, kept as literal text rather than as constructed `Values`, and the reason is given: “so a shape change (a renamed field, a field moved out of `args`) shows up as a diff in this corpus and not just in the code.”

```
{ "t": "verbs", "id": 1 }
{ "t": "get", "id": 2, "state": "hostinfo" }
{ "t": "call", "id": 3, "verb": "ping", "args": {} }
{ "t": "sub", "id": 4, "event": "scheme" }
{ "t": "begin", "id": 5, "txn": 9001 }
{ "t": "call", "id": 6, "verb": "browser.pinTab", "args": {}, "txn": 9001 }
{ "t": "commit", "id": 7, "txn": 9001 }
{ "t": "begin", "id": 8, "args": { "txn": 9002 } }
{ "t": "abort", "id": 9, "args": { "txn": 9002 } }
{ "t": "undo", "id": 10, "args": { "seq": 1 } }
```

Entries five and eight are the same frame kind written two different ways, which is not redundancy. It is the corpus pinning a tolerance the code had to grow, and `args_of` is where that tolerance lives:

```
fn args_of(req: &Value) -> Value {
  let mut o = req.get("args").and_then(Value::as_object).cloned().unwrap_or_default();
  for k in ["txn", "verb", "seq", "result"] {
    if let Some(v) = req.get(k) { o.entry(k).or_insert_with(|| v.clone()); }
  }
  Value::Object(o)
}
```

Its doc explains the asymmetry it absorbs: “`zgui-bridge` carries `txn` (and an `undo`’s `verb/args/result`) at the TOP level of the frame, while a `call` nests everything under `args`. Accept either shape so a client written against the crate’s frames works unchanged here — the hand-mirroring is exactly where the two drift.” Note the merge direction — `entry().or_insert_with()` — so a nested `args.txn` wins over a top-level one rather than being overwritten by it. Four field names are lifted, and `undo`’s `result` is among them even though the current `undo` path does not read it, which is what forward tolerance looks like when it is written deliberately.

## The assertion that had to be inverted

The most instructive line in the file is the one explaining what *not* to assert:

Every corpus frame must be UNDERSTOOD — not merely answered. An unmirrored frame still gets a reply (the fallback), so asserting “a reply came back” would pass against the exact bug this test exists to find; the assertion is therefore on the fallback’s error text being absent.

So `every_mirrored_frame_kind_is_understood` asserts `reply["error"] != Some("unknown request kind")`, and a companion test, `unmirrored_frame_kind_gets_a_clean_error`, asserts the positive case against a deliberately invented `{"t": "no-such-kind"}` — pinning the fallback itself, because “removing the fallback would strand every such client.”

The pair is the right shape for testing a graceful degradation path: one test proves the degradation does not apply to anything real, the other proves the degradation still exists for everything else.

Both sit on top of `assert_reply_shape`, which pins the invariants a client can rely on regardless of frame kind: exactly one `reply` per request, ids preserved and in order, a boolean `ok`, and `value` XOR `error` — never both, never neither.

## Eight tests, and what each one is preventing

The file holds eight `#[test]` functions, and every one carries a doc comment naming the bug it exists to catch rather than the behaviour it exercises:

|                                                            |                                         |
|------------------------------------------------------------|-----------------------------------------|
| <code>test</code>                                          | <code>pins</code>                       |
| <code>every_mirrored_frame_kind_is_understood</code>       | a frame added upstream and forgotten    |
| <code>↳ here</code>                                        |                                         |
| <code>unmirrored_frame_kind_gets_a_clean_error</code>      | removal of the fallback                 |
| <code>abort_unwinds_in_reverse_call_order</code>           | compensation order (asserted as a       |
| <code>↳ sequence)</code>                                   |                                         |
| <code>irreversible_verb_is_refused_while_txn_open</code>   | call-time refusal, not abort-time       |
| <code>↳ discovery</code>                                   |                                         |
| <code>irreversible_verb_runs_normally_outside_a_txn</code> | the class system not leaking into plain |
| <code>↳ calls</code>                                       |                                         |
| <code>double_abort_compensates_once</code>                 | take-before-compensate                  |
| <code>commit_discards_without_compensating</code>          | commit closing the gate as well as the  |
| <code>↳ journal</code>                                     |                                         |
| <code>surface_publishes_a_rev_class_for_every_verb</code>  | an unclassified verb defaulting to      |
| <code>↳ inverse</code>                                     |                                         |

The ordering test states its own reason for asserting a sequence rather than a set: “Compensation order is the whole point of the feature and it fails SILENTLY when wrong.” That is the same category of hazard as the protocol drift the file was written for — a wrong unwind order produces a browser in a plausible-looking state, not an error.

The last test is a cross-check between two representations of the same fact: `assert_eq!(class, zbus::rev(id))` for every verb the surface advertises. `surface()` computes `rev` per verb by calling `rev()`, so today the assertion cannot fail — which is precisely what makes it worth having, because the moment someone caches the classes or hand-maintains a second table, it becomes the thing that notices.

## The counterpart that cannot run here

The design’s most unusual feature is that half the test is in another repository and cannot be executed alongside this half. The header is explicit:

The counterpart test lives in `zgui-bridge/tests/roundtrip.rs` and feeds the SAME corpus to `zgui-bridge’s serve_conn` — it cannot run here, because linking the crate is the exact thing the license split forbids.

So conformance is asserted twice against one shared corpus, in two repositories that never link, and the corpus is duplicated by hand along with everything else. That is not a weakness anyone hid; it is the only arrangement the license split allows, and writing it down — as the header does — is what makes it maintainable rather than merely true.

## The fixture that keeps the test off the real machine

`hermetic()` runs once and redirects three things before anything touches them:

```
std::env::set_var("ZWIRE_STATE", &dir);
std::env::set_var("HOME", &dir);
std::env::set_var("ZWIRE_BUS_NO_DAEMON", "1");
```

The first two exist because a `browser.*` forward writes `__zbus_action` into the file-backed kv — Chapter 58’s third delivery path — so a test that exercises the forward would otherwise stamp actions into the developer’s live `~/zwire`. The third is Chapter 54’s seam, and its comment is the sharpest statement of why it exists: “Never let a test reach for the developer’s live bus daemon.”

`txn_guard` closes the last gap. The transaction journal is process-global by design — one `seq` clock for every transaction — which means a test that leaves one open changes how the *next* test’s `call` frames dispatch, since `any_open()` is the gate on the whole class check. So every test that opens a transaction takes a static mutex and calls `txn::reset()` first. The same property that makes cross-connection aborts orderable makes the test suite order-dependent unless something serializes it, and `reset()` exists in `txn.rs` labelled “Test seam only” for that single purpose.

# Chapter 60 – The Identity Chain

Chapter 21 records patch 0016 as allowlisting “the `hud-internal` extension (by hash)” for `developerPrivate` and `settingsPrivate`, and patch 0023 as exposing `tabCapture` to it. Those are two of the most consequential grants in the fork: they are what let the HUD’s own Extensions and Settings pages drive the same private APIs `chrome://extensions` and `chrome://settings` drive, and what lets the Audio dashboard capture a tab’s output with no picker and no prompt.

A grant to “the `hud-internal` extension” raises a question the phrase glosses over. Chromium has no idea what `hud-internal` is. It knows extension ids, and it knows a hash. So: what exactly is being granted, what determines whether the running extension matches it, and what breaks the match?

The chain is four links long and every one of them is computable from files in the tree.

## From a manifest field to an allowlist entry

`extensions/hud-internal/manifest.json` carries a `key` — a 392-character base64 string, one of 17 top-level manifest fields. Decoded it is **294 bytes** of DER: a `SubjectPublicKeyInfo` structure holding an RSA public key.

Chromium derives an unpacked extension’s id from that key by hashing it and mapping the first 16 bytes into the alphabet `a–p`. Run against the three manifests in the tree, the derivation reproduces the ids the rest of the system hardcodes:

```
manifest key (DER, 294 bytes)
-> SHA-256
-> first 16 bytes (32 hex nibbles)
-> each nibble mapped 0-f => a-p
= extension id
-> SHA-1 of the id string, uppercase hex
= the allowlist entry Chromium's feature files match on
```

| extension                                  | derived id                      | SHA-1 of that id |
|--------------------------------------------|---------------------------------|------------------|
| newtab                                     | gpoepnekoiplhkegjpcnpeiiefgieb  |                  |
| ↪ EC891FD5E456677F762788B0D2A6AAA106588787 |                                 |                  |
| hud-internal                               | omcggnjfbmpdlofkfbpddkhnfibfhgg |                  |
| ↪ DCE846DF72B461D411C8ADA00F7794BFD13F38A6 |                                 |                  |
| zpwchrome                                  | hpppdchpnphmijdeanibpcadgknmaja |                  |
| ↪ 720AAA5B2306BEA5605BAB986E0F93FD04E8E9B  |                                 |                  |

The middle row’s hash is the string that appears, verbatim, three times in the patch series — twice in `0016-ext-private-api-allowlist.patch` and once in `0023-audio-tab-capture.patch`:

```
- "5F57A9AE8DFF5D6BB09DF8606270402612E871E5" // http://crbug.com/40388767
+ "5F57A9AE8DFF5D6BB09DF8606270402612E871E5", // http://crbug.com/40388767
+ "DCE846DF72B461D411C8ADA00F7794BFD13F38A6" // zwire HUD internal (extensions
↪ page)
```

So the grant is not to a name or a path. It is to a public key, addressed through two hash functions, sitting in a JSON feature file that ships inside the browser binary. Chapter 21’s “by hash” is exact; this is the hash, and the four steps that produce it.

## Why the key is there at all

An extension loaded with `--load-extension` from a directory has no packaging and no signature, and so nothing about it is stable except what the manifest declares. The `key` field is the declaration. It fixes the id independently of where the directory happens to be on disk — which matters here more than in most projects, because `hud-internal` is loaded from at least three different absolute paths across the install shapes this book describes: a developer checkout under the repository, a staged tree inside a packaged macOS `.app`, and `%LOCALAPPDATA%\zwire\ext` on Windows.

Without the key there is nothing in the manifest from which an id could be derived at all, and a hash baked into a C++ build would have nothing stable to match. With it, `bin/zwire`'s `--load-extension` list — which is assembled from whatever directories happen to exist — yields the same three ids on every machine and in every install shape. The `key` is what makes a compiled-in allowlist a workable mechanism.

## The second grant, addressed differently

Patch 0023 does the same job twice with two different addressing schemes, and the difference is instructive. Its `_api_features.json` hunk restructures the `tabCapture` feature into a two-element array — the original entry marked `default_parent: true`, plus a second variant gated on the allowlist — and that variant uses the SHA-1, like 0016.

Its C++ hunk does not. `GetAllowlistedExtensionID` in `tab_capture_api.cc` normally returns whatever `--allowlisted-extension-id` was passed on the command line; the patch supplies a default when it is empty, and the default is the `id` string itself:

```
if (id.empty()) {  
    id = "omcggnjfbmpdlofkfbpddkhnfibfhgg";  
}
```

The comment scopes it: applied “only when `--allowlisted-extension-id` is otherwise unset, and only grants this one id, so no other extension’s capture behavior changes.” And `bin/zwire` passes no such switch — its ten flags are `--user-data-dir`, `--load-extension`, `--extensions-on-chrome-urls`, `--test-type`, `--no-first-run`, `--no-default-browser-check`, `--homepage`, `--disable-features`, `--enable-features`, and `--autoplay-policy`, plus whatever the caller appends. So the default is live on every normal launch, and a user who passes the switch explicitly takes it back.

The same extension is therefore identified two ways in one patch: by SHA-1 in a JSON feature file, by literal id in C++. Both are derived from the same key, and a key rotation invalidates both — but only one of them is greppable for the id, which is worth knowing when auditing what a fork has granted itself.

## What breaks, and what it looks like when it does

The chain has three distinct break points, and none of them produces an error message naming the cause.

**A rotated or removed key.** The extension gets a different id — or, with no key, an id not derivable from the manifest at all. Every allowlist entry in the fork then matches nothing. `chrome.developerPrivate` and `chrome.settingsPrivate` are simply not defined in the HUD’s pages, so the Extensions and Settings pages load, render their chrome, and have no engine behind them; `tabCapture.getMediaStreamId` fails, so the Audio dashboard’s capture path stops. Chapter 21 already notes that without 0016 “the reimplemented pages have no engine to talk to”; a key change produces that state with the patch still applied.

**A stock base instead of the fork.** The runtime-rebrand path (Chapter 5) runs an unmodified Chromium snapshot, whose `_permission_features.json` and `_api_features.json` are upstream’s. The `key` still fixes the id, the manifest still requests `developerPrivate` and `settingsPrivate` among its 20 permissions — and the allowlist those permissions are gated on does not contain it. The identity chain is intact and lands

on a browser that has never heard of it. This is the concrete content of the book’s repeated statement that certain HUD pages are fork-only.

**A Chromium version bump.** Here the two grants have different risk profiles, and the patch data makes the difference legible. 0016 is 3 hunks across 1 file, all in `_permission_features.json`; 0023 is 2 hunks across 2 files, one of which is `_api_features.json`. JSON feature files are edited by upstream constantly — the allowlists 0016 appends to are lists of other people’s extensions, growing and shrinking release to release — so the *context* around these hunks moves often while the *semantics* barely move at all. A conflict there is mechanical: find the list, re-append the entry.

0023’s C++ hunk is the opposite case, and it is the pattern Chapter 21 identifies as the expensive tier. `GetAllowlistedExtensionID` is a small helper whose behaviour the patch changes by adding a fallback; if upstream reworks how the allowlisted id is plumbed — moves the check, adds a caller, changes when an empty value means “none” versus “everyone” — the patch can still apply cleanly and grant something other than what it granted before. The JSON patch fails loudly when it fails. The C++ patch is the one that can succeed and be wrong.

## The graph the ids actually form

The same derivation makes one more thing checkable: the cross-extension messaging graph is declared by id, so it can be read off the manifests rather than described. `externally_connectable.ids` in the three manifests, resolved through the derivation above:

|                           |                                         |    |                                                 |
|---------------------------|-----------------------------------------|----|-------------------------------------------------|
| manifest                  | <code>externally_connectable.ids</code> | => | accepts messages from                           |
| <code>newtab</code>       | <code>[omcggnjf...]</code>              |    | <code>hud-internal</code>                       |
| <code>hud-internal</code> | <code>[hpppdchp..., gpoepnek...]</code> |    | <code>zpwirchrome</code> , <code>newtab</code>  |
| <code>zpwirchrome</code>  | <code>[omcggnjf..., gpoepnek...]</code> |    | <code>hud-internal</code> , <code>newtab</code> |

Two entries there are wider than earlier printings of Appendix O record: `hud-internal` accepts from the new-tab extension as well as from the power-tool, and `zpwirchrome` accepts from the new-tab extension as well as from the HUD. Read as a directed graph, `newtab` can now initiate to both of the others while accepting only from `hud-internal` — which is the shape you would expect from a page that is the browser’s startup surface and therefore the first thing in a position to talk to anything.

Two version figures on that same appendix have also moved with the code: `zpwirchrome`’s manifest now reads `0.22.40`, and `package.json` at the repository root reads `0.6.59`. `hud-internal` remains `0.6.29` and `newtab` remains `1.0.13`. Chapter 48’s classification applies unchanged — these are resolution inputs consumed by Chromium’s extension reload and by the installer’s content-derived stamp, not labels — and the reason to re-derive them here rather than carry them forward is the same reason to re-derive the hashes: a number in this book that was true when it was written and is not re-checked is indistinguishable from one that was never checked.

## The general shape

The identity chain is four transformations long, spans a manifest field, a browser-internal id format, a JSON feature file, and a C++ default, and is not written down in any single place in the tree. Each link is individually obvious to whoever wrote it; the composition is what nobody can see, and the composition is what a key rotation, a stock base, or a bad rebase silently breaks.

Which is the same lesson Chapter 48 draws about version numbers, arriving from a different direction. The useful artifact is not the hash. It is the sentence that says where the mismatch appears: a HUD Settings page that renders and does nothing, an Audio dashboard whose capture path fails, and an extension that looks, in every surface the browser offers, perfectly healthy.

# Chapter 61 – The Mesh, and the Boundary at Its Edge

Chapter 58 listed three ways a `browser.*` action leaves the host, and the second one — `peer::broadcast` — was passed over in a sentence. It deserves more, because it is the only line in this system where an action crosses a machine boundary. `src/peer.rs` is 241 lines and it turns a per-machine host into a mesh: the theme, the scheme, and browser automation itself become fleet-wide rather than local.

It is also where the trust model changes shape. Every transport this book has described so far is guarded by a file mode, and every caller inside it is the owner. TCP has no file mode.

## Two things a link carries

The module header names both, and the distinction runs through everything below:

Peer links carry two things: **bus federation** — a `pub` (or a `scheme/ui` change) on one host is forwarded to every peer, whose local subscribers then receive it, so the event bus spans all your machines; and **remote requests** — `{"cmd": "remote", "peer": "host:port", "request": {...}}` runs a request on another host and returns its reply.

Federation is push and fire-and-forget; `remote` is request-and-reply. They use different connections for a reason given in `remote`'s own doc: it “uses a fresh connection (authenticating if a token is set), so it is independent of the long-lived federation link. Intended for RPC commands, not streams.” A long-lived link that also had to carry correlated replies would have to multiplex them against the events already flowing over it; a fresh socket per remote request does not, at the cost of a connect. The 15-second read timeout on that socket is the only deadline anywhere in the peering code.

Four flags on `serve` configure the whole thing, and the help screen's own example is the canonical shape:

```
--tcp <addr>      (serve) also listen for peers/remote clients on TCP
--token <tok>     (serve) shared secret required of inbound TCP
--name <name>     (serve) advertised peer name (default: hostname)
--peer <addr>     (serve) dial a peer and keep it linked; repeatable

zwire-host serve --tcp 0.0.0.0:7420 --token SECRET --peer other.local:7420
```

`--peer` is repeatable and `peer_connect` adds one at runtime, so a mesh can be built from the command line, from a config, or from a script that discovered a machine.

## Federation is single-hop, and that is the whole loop pre-vention

`broadcast` sends `{"cmd": "peer_pub", "topic", "data"}` to every registered link. The receiving end is four lines in `session.rs`, and the comment carries the entire topology argument:

```
// A federated event from a peer: deliver locally only (never
// re-forward) so events can't loop around the mesh.
"peer_pub" => {
  if let Some(topic) = msg["topic"].as_str() {
    bus::publish(topic, &msg["data"]);
  }
}
```

The header states the consequence: federation “is single-hop: a forwarded event is delivered locally but not re-forwarded, which covers star and fully-meshed topologies without loops.”

This is the cheapest correct answer to event looping, and its limits are exactly the ones that follow from the rule. A star (every machine linked to a hub) works because every event is one hop from anywhere. A full mesh works because every pair is directly linked. A chain does not:  $A \rightarrow B \rightarrow C$  delivers A's events to B and stops. There is no message id, no seen-set, no TTL — because with those two topologies covered, none is needed, and every one of them would have to be carried in every frame forever.

`peer_pub` is also the one command in the dispatcher that sends no reply at all. There is no `respond` call in its arm, which is correct for a push — but it means the frame is invisible to any client that assumes request-and-reply, and it is the reason `try_dial`'s read loop is a bare `while let Some(msg) = read_ndjson(...)` rather than a correlation table.

## What actually crosses

Four topics are broadcast from the theme-write path in `session.rs`, each written, published locally, and pushed to peers in the same breath — `ui`, `scheme`, `palette`, and `schemes` — with the comment “so all apps on all machines stay in sync live.” That is the fleet-wide theme of Chapter 40, and the mechanism is three lines per field.

The fifth topic is the one from Chapter 58. `run_command`'s `browser.` branch calls `peer::broadcast("zbus.action", &payload)` alongside the local publish, which means a `browser.*` verb called on one machine is delivered to the local subscribers of every linked peer — and on a peer running the HUD, the subscriber holding `zbus.action` is a service worker in a browser. A stryke script on the workstation can therefore close a tab on the laptop, and the reply field that reports it is `forwarded`, the count of links that took the frame.

That capability is not advertised anywhere in the verb surface. It is a consequence of two independent decisions — that browser actions publish on a topic, and that topics federate — meeting.

## The handshake, and the asymmetry underneath it

Inbound and outbound links are constructed differently, and the difference is the trust model.

An inbound TCP connection is handled by `handle_inbound`, which builds `Session::guarded(auth_required())` — untrusted whenever a token is configured. Until it authenticates, `handle_cmd`'s first statement refuses everything outside a four-command allowlist:

```
if !self.authed && !matches!(cmd, "auth" | "peer_hello" | "hello" | "ping") {
  respond(out, msg, json!({"ok": false, "err": "unauthorized"}));
  return;
}
```

`hello` and `ping` are open because they are the feature-test and liveness probes, and both are harmless: `hello` returns the host's version, os, arch, pid, and capability list, which is information a caller needs before it can decide whether authenticating is worthwhile.

An outbound dial is the reverse. `try_dial` sends `{"cmd": "peer_hello", "token", "name"}`, reads one reply, takes the peer's advertised name from it, registers the link — and then serves the rest of the connection with `Session::guarded(false)`, which the comment justifies: “We initiated a trusted dial, so this session is pre-authenticated; it handles the peer's forwarded events (`peer_pub`) for the life of the link.”

Stated plainly: on a link we dialed, we authenticate to them and they do not authenticate to us. Everything that arrives on that socket for the life of the link is dispatched through a trusted session — `peer_pub`, and also `exec`, `fs_write`, and `pty_spawn`, which are the same capabilities the `0600` socket exists to protect locally. The reasoning is that dialing is a deliberate act naming a specific address, so the operator has already made the trust decision; the property worth knowing is that the decision covers the whole link and not just the events it was opened for.

The token check itself has a default that matters more than the check: `token_ok` returns `true` when no token is configured. So `serve --tcp` without `--token` yields `auth_required() == false`, every inbound session constructed pre-authenticated, and a host that hands `exec` to whoever connects. The flag pair is documented together in the help screen and the example uses both. It is worth reading them as one flag.

`register_link` closes the last edge case: it removes any existing link with the same name before inserting, “a reconnect, or the reverse direction of a mutual peering.” Two machines that each `--peer` the other end up with one link per name rather than two, and a dial loop that reconnects every five seconds after a failure does not accumulate dead entries.

## The command that is reachable but not advertised

Chapter 55 established that `SURFACE_VERBS` is advertising rather than an allowlist, and the peering commands are the sharpest illustration available. `peer`, `peers`, and `peer_connect` are all in the list, the first two classed `pure` and `peer_connect` — which starts a dial loop — left to the `irreversible` default. `remote` is not in the list at all.

It is still dispatchable. `call` accepts any command, so a bus client can send this frame:

```
{"t": "call", "verb": "remote", "args": {"peer": "other.local:7420", "request": {"cmd": "exec", ...}}}
```

The host will run it, through `run_command`'s capture sink, against a machine on the other side of the network — and because `remote` has no `REV` entry, it is `irreversible` and refused inside a transaction, which is the one place the surface's silence about it does not matter.

That is the composition worth writing down. The bus has no authentication because a file mode is its boundary; `remote` reaches past that boundary to a different machine; and the surface list, which is the only place a reader would look for an inventory of what the bus can do, does not mention it. None of the three decisions is wrong on its own. The inventory being incomplete is the part that is worth fixing, because an inventory is the artifact people audit.

# Chapter 62 – Two Info.plists and a Signature

Chapter 5 describes the runtime rebrand as four operations on one bundle: set the name, install the icon under a new filename, delete `CFBundleIconName`, re-sign ad-hoc. That is the *development* rebrand — it edits the base browser sitting in `$ZWIRE_STATE/base` so that running it from the checkout presents as `zwire`. It is not the rebrand that ships. The packaged app does the same job twice, on two different bundles, for two different reasons, and then does an entirely different job on Linux and a third one on Windows. This chapter follows the rebrand from `package.json` to a Start Menu shortcut.

## Why the packaged app has two bundles

`scripts/localinstall.sh` builds `/Applications/zwire.app` as a self-contained bundle, and the browser goes *inside* it: the whole base `.app` is copied to `Contents/Resources/browser/` (`localinstall.sh:180`). So the installed product contains a nested application bundle, and the outer bundle's `Contents/MacOS/zwire` is a shell launcher that `execs` the nested browser's binary (`localinstall.sh:521`).

That nesting splits the app's identity across two `Info.plist` files, and macOS reads different things from each. The script's own comment at `localinstall.sh:183` states the split: the nested bundle “— not the outer wrapper — is the process macOS actually runs (the launcher `exec`'s its binary), so the Dock, ⌘-Tab switcher, and notification icon come from ITS name/icon.” The outer wrapper is what the user double-clicks and what Finder lists; the nested one is what ends up as the running process. Get either wrong and the app is half-branded in a way that is obvious on screen and confusing to diagnose, because the two failures look nothing alike: a wrong outer plist gives you a correctly-named process under a wrong Finder icon, and a wrong nested plist gives you a correct Finder icon under a Dock tile that says Chromium.

So the nested bundle gets the Chapter 5 treatment applied a second time, inline: `CFBundleName` and `CFBundleDisplayName` set to `zwire`, `branding/zwire.icns` copied in as `Resources/zwire.icns`, `CFBundleIconFile` pointed at `zwire`, and `CFBundleIconName` deleted (`localinstall.sh:190-196`) — the same four PlistBuddy calls with the same set-or-add fallbacks, because the base bundle being copied may have come from either the snapshot path or the fork build and only one of those two already carries the keys. The comment at `localinstall.sh:186` records what the fork build produces and why the rename is still needed: it “ships as ‘zbrowser’ with the stock Chromium icon.” It also records the limit of the technique — “the About/Quit MENU ITEMS come from the compiled product name and still need a branded rebuild.” That is patch 0006's job, and it is the one part of the app's name that a plist cannot reach.

The outer wrapper's plist is not patched but *written*, as a heredoc (`localinstall.sh:539-556`): `CFBundleName` and `CFBundleDisplayName` `zwire`, `CFBundleExecutable` `zwire`, `CFBundleIdentifier` `com.menketotechnologies.zwire`, `CFBundleIconFile` `zwire`, `CFBundlePackageType` `APPL`, and both `CFBundleShortVersionString` and `CFBundleVersion` set to `${VERSION}`.

## Where the version number comes from, and why not with python3

`VERSION` is read from `package.json` at `localinstall.sh:32`, and the comment above it explains a tooling choice that looks arbitrary until you know what it cost: the JSON “here is a flat manifest, and a perl one-liner has no interpreter-flavor dependency (a non-CPython python3 on PATH broke the release script mid-cut).” The extraction is `perl -ne 'if (!$seen && /"version"\s*:\s*"([^\"]+)/) { print "$1\n"; $seen = 1 }' package.json` — first match wins, guarded by `$seen` so a nested “version” key later in the file cannot win.

The same number is then stamped into two places inside the staged tree, because two consumers cannot read it any other way. `pages/version.js` carries a hardcoded `ZWIRE_VERSION` and gets rewritten with `perl -i -pe (localinstall.sh:245)` — an extension has no API for its containing app bundle’s `CFBundleVersion`. And the staged `hud-internal/manifest.json` gets its `version` field rewritten (`localinstall.sh:255`), with the substitution guarded by `if $. < 12` so it can only touch a “version” key in the manifest’s first eleven lines and cannot walk into a nested object further down. Chapter 23 covers what that stamp is *for*; the point here is that the version is one value with three destinations and one extraction, so the System page, the manifest, and the bundle cannot disagree.

## The keys that are not about branding at all

Between the icon swap and the re-sign sits a block that has nothing to do with how the app looks and everything to do with whether it stays running. Seven `NS*UsageDescription` keys are written into the nested plist (`localinstall.sh:211-217`): Bluetooth (always and peripheral), camera, microphone, location (when-in-use and the legacy key), and local network.

The comment at `localinstall.sh:197` gives the reason, and it is sharper than the usual privacy-prompt story: “macOS does not merely DENY a privacy-gated capability with no usage description — it SIGABRTs the whole process (TCC namespace abort).” The plain Chromium snapshot from `fetch-base.sh` “ships ZERO `NS*UsageDescription` keys, so the first page that calls Web Bluetooth / getUserMedia / Geolocation kills the browser.” A real Chrome uses entitlements instead; an ad-hoc-signed zwire cannot.

And the keys have to be in *both* plists. The comment continues: macOS “attributes TCC to the LAUNCHED (outer) app bundle: a Gmail passkey sign-in via Web Bluetooth SIGABRT’d even with the nested keys present until the outer had them too.” So the same seven strings appear again in the outer wrapper’s heredoc, and the comment at `localinstall.sh:556` marks them there. This is the clearest case in the project of the two-bundle structure demanding a duplicated fact rather than a shared one: there is no inheritance between a wrapper’s `Info.plist` and a nested bundle’s, so the strings are written twice on purpose.

`NSBonjourServices` is handled differently from the rest because it is an array and PlistBuddy has no set-or-add idiom for one. The script deletes the key and rebuilds it — delete, add array, add index 0 as `_googlecast._tcp` (`localinstall.sh:220-222`) — so a re-run over an already-installed bundle produces exactly one entry rather than appending a second. The comment names the constraint it satisfies: macOS 15+ local-network privacy, mirroring what Chrome declares for casting and mDNS.

The nested bundle is then re-signed (`codesign --force --sign - "$NESTED", localinstall.sh:223`), and the outer bundle is re-signed at the end of assembly (`localinstall.sh:590`) — after a permissions pass, with the comment at `localinstall.sh:585` recording the ordering constraint: the mode fixes run “BEFORE codesign so the signature covers the final modes.” Same rule as Chapter 5’s, one level out: edit, then seal, then register.

## Linux: the identity is the launcher and a text file

There is no bundle to patch on Linux, and `scripts/localinstall-linux.sh` does not pretend otherwise. The install lands under `~/.local/opt/zwire` with a `zwire` launcher on `PATH`, and the app’s identity is a 12-line `.desktop` file written at `localinstall-linux.sh:190-203`:

```
Type=Application           Name=zwire
GenericName=Web Browser    Comment=Chromium superset with the zwire cyberpunk HUD
Exec=$DEST/zwire %U       Icon=$ICON_DEST
Terminal=false            StartupNotify=true
StartupWMClass=chrome
Categories=Network;WebBrowser;
MimeType=text/html;text/xml;application/xhtml+xml;x-scheme-handler/http;
                          x-scheme-handler/https;
```

One line there is the Linux analogue of the `CFBundleIconName` trap. `StartupWMClass` is set to `chrome`, not to `zwire`, because the value has to match the WM class the *running process* reports — and the running process is Chromium, which reports `chrome` regardless of what the launcher script is called. A desktop environment matches a window to its launcher by that string; set it to the pretty name and the window detaches from its own icon in the taskbar. The two cache-refresh calls that follow (`update-desktop-database`, `gtk-update-icon-cache`, both `|| true`) are the LaunchServices `lsregister` of Chapter 5 by another name, and they are non-fatal for the same reason: a stale menu icon is not a broken browser.

## Windows: a registry key and a COM-built shortcut

`scripts/localinstall-windows.ps1` is PowerShell rather than bash, and `localinstall.sh:47-52` explains the dispatch rather than attempting it: “Windows native-messaging is registry-based + shortcuts are `.lnk`”, so the bash installer detects `MINGW*|MSYS*|CYGWIN*`, prints the PowerShell command to run, and exits 0. Linux, by contrast, is `exec`’d straight into (`localinstall.sh:46`).

Everything is per-user — the header at `localinstall-windows.ps1:16` states “No admin rights needed — everything is per-user (HKCU + %LOCALAPPDATA%)” — and the install root is `$env:LOCALAPPDATA\zwire (:39)`. The native hosts are registered by writing registry keys rather than dropping JSON into the profile, and each host is registered under *two* hives: `HKCU:\Software\Chromium\NativeMessagingHosts\...` and `HKCU:\Software\Google\Chrome\NativeMessagingHosts\...`, for `com.zwire.hud (:181-182)` and for `com.menketotechnologies.zpwchrome (:201-202)`. Both hives, because the base may be a Chromium snapshot or a Chrome-branded build and the browser only reads its own.

The brand itself is a Start Menu `.lnk` built through COM (`localinstall-windows.ps1:233-243`): `WScript.Shell.CreateShortcut`, `TargetPath` pointed at `$Dest\browser\chrome.exe`, `Description` set to “Chromium superset with the zwire cyberpunk HUD”, and `IconLocation` pointed at a copy of `branding/zwire.ico` placed next to the install. The executable is still named `chrome.exe`; only the shortcut says `zwire`. That is the whole of the Windows rebrand, and it is honest about being less than the macOS one.

## The flags, which are not the same anywhere

The rebrand ends with the launcher, and the launchers are where the three platforms quietly diverge. Four surfaces spawn the browser, and no two pass the same switches:

```
bin/zwire:90-101          --user-data-dir --load-extension
                          --extensions-on-chrome-urls --test-type
                          --no-first-run --no-default-browser-check
                          --homepage --disable-features=NtpFooter
                          --enable-features=SplitView...
localinstall.sh:521-532   ...the same, but --restore-last-session
                          instead of --autoplay-policy
localinstall-linux.sh:165-175 ...the same, plus --no-sandbox, and neither
                          --autoplay-policy nor --restore-last-session
localinstall-windows.ps1:236 ...nine switches in the .lnk Arguments string:
                          none of the three extras
```

The three differences each have a defensible local cause — `--no-sandbox` is the usual Linux unpacked-install concession, `--restore-last-session` duplicates in a switch what patch 0013 does in C++ for the fork base, and the developer launcher's `--autoplay-policy=no-user-gesture-required` is a convenience for the audio work of Chapter 52. What they do not have is a single definition. A user who develops against `bin/zwire` and then installs the packaged app gets a browser that restores its tabs and refuses to autoplay, and a user on Windows gets neither behaviour. The launcher is generated four times from four sources, so the flag set is a fact that exists in four places and is asserted in none.

## The three identity surfaces, side by side

The rebrand touches a different set of artifacts on each platform, and the useful comparison is not “how much work” but “how many independent places the name lives,” because that is the count of places it can drift:

|               | macOS                                                                                        | Linux                                             | Windows                                                   |
|---------------|----------------------------------------------------------------------------------------------|---------------------------------------------------|-----------------------------------------------------------|
| name          | CFBundleName + Display on TWO plists (nested + outer wrapper)                                | .desktop Name= (one file)                         | .lnk basename (one shortcut)                              |
| icon          | zwire.icns copied into the nested Resources/, and CFBundleIconName DELETED so the .icns wins | zwire icon copied to \$PREFIX/share/icons/hicolor | zwire.ico copied next to the install, set as IconLocation |
| identity key  | CFBundleIdentifier                                                                           | StartupWMClass= chrome                            | TargetPath                                                |
| seal          | codesign --force --sign - on BOTH bundles                                                    | (none)                                            | browser\chrome.exe (none)                                 |
| cache refresh | lsregister -f + touch                                                                        | update-desktop-database + gtk-update-icon-cache   | (none)                                                    |
| privacy       | 7 NS*UsageDescription keys on BOTH plists + Bonjour                                          | (none needed)                                     | (none needed)                                             |

Three properties of that table are worth stating. First, macOS is the only platform where the same fact must be written twice — the name, the icon, and all seven privacy strings appear in both the nested and the outer `Info.plist`, and the book’s earlier chapters do not distinguish them because the runtime rebrand of Chapter 5 only ever sees one bundle.

Second, the *identity key* row is the one that is counter-intuitive on every platform. On macOS the icon does not follow `CFBundleIconFile`; it follows the *absence* of `CFBundleIconName`. On Linux the window does not follow the launcher’s name; it follows `StartupWMClass=chrome`, which must stay wrong-looking to be right. On Windows the shortcut points at `chrome.exe` because there is nothing else to point at. In all three cases the branded surface is a layer over an executable that still calls itself Chromium, and the only thing that changes that is patch 0006 — which requires the compile.

Third, macOS is the only platform that needs the privacy block at all, and it needs it not for a prompt but to avoid a `SIGABRT`. That is the one line of this table that is about liveness rather than appearance, which is why it sits in the middle of a rebrand script instead of somewhere more obvious.

# Chapter 63 – Reading the Series

Chapter 21 explains what the fork's patches *do* and what rebasing them costs. This chapter reads the series as an artifact: how the twenty-seven diffs are distributed, which of them are load-bearing for others, what the file that orders them encodes beyond order, and the one test in the repository that compiles code out of a diff.

## The shape of 3,250 lines

`fork/patches/` is 3,340 lines in total; 3,250 of those are the patches themselves and the remaining 90 are the directory's `README.md` (54) and `series` (36). The twenty-seven patches carry 67 hunks across 32 distinct Chromium source paths — the same figures Chapter 21 quotes — but they are not distributed anything like evenly:

| patch                          | files | hunks | lines |
|--------------------------------|-------|-------|-------|
| 0022-audio-eq-output           | 3     | 9     | 1472  |
| 0024-audio-live-config         | 4     | 10    | 370   |
| 0002-ui-colors-hud             | 3     | 6     | 350   |
| 0001-tab-shape-hud             | 1     | 4     | 164   |
| 0025-hud-native-custom-palette | 1     | 4     | 133   |
| 0007-devtools-hud-theme        | 3     | 4     | 111   |
| ...                            |       |       |       |
| 0005-omnibox-hud               | 1     | 1     | 17    |
| 0003-ui-font-mono-hud          | 1     | 1     | 18    |
| 0012-allow-frame-src           | 1     | 1     | 18    |

One patch is 45% of the series. `0022-audio-eq-output.patch` at 1,472 lines is larger than the other twenty-six put together minus `0024`, and together the three audio patches (`0022`, `0023`, `0024`) are 1,891 lines — 58% of all patch content for three of twenty-seven entries. Read by line count the fork is an audio project with a colour scheme attached. Read by hunk count it is the opposite: fifteen of the twenty-seven patches are a single hunk, and nine of those are under 25 lines. The typical HUD patch is a handful of lines in one function; the atypical one is a DSP engine.

That distribution is the real predictor of rebase pain, and it is not the same as the risk tiering Chapter 21 gives. A one-hunk, 17-line patch in `location_bar_view.cc` fails loudly or not at all. A nine-hunk patch spread across `output_controller.cc`, `output_controller.h`, and `chrome_content_browser_client.cc` has nine independent opportunities to half-apply.

## The two files that more than one patch owns

Thirty of the thirty-two target paths are touched by exactly one patch. The other two are the ones to watch:

```
5 chrome/browser/ui/color/chrome_color_mixer.cc -> 0002 0020 0021 0025 0027
2 chrome/browser/ui/views/toolbar/toolbar_view.cc -> 0004 0011
```

`chrome_color_mixer.cc` is the fork's centre of gravity. `0002` adds the HUD palette and, per its own series note, "ends with `AddZwireHudFinalMixer` (dead-last) re-asserting tab colors so the Material/native tab-strip mixers can't leave the active tab a scheme-independent GM3 default." Every colour patch after it extends that one function: `0020` binds the `ui::kColorMenu*` family into it, `0021` generalises `0020` to the six native primitives, `0025` teaches it to read resolved base colours from a `hud-palette` file instead of a baked name

table, and 0027 binds the tab-strip control buttons. Four of the five carry an explicit dependency in their own series line — 0020 “depends on 0002’s Sys-token block,” 0025 “Depends on 0002,” 0027 “Depends on 0002/0025 (uses the `ActiveHudScheme()` locals).”

So the series is not a flat list; it is one long patch delivered in five instalments plus twenty-two independent ones. Reordering it is not a preference. The five colour patches must apply in exactly the order listed or the later ones’ context does not exist yet, and `apply-patches.sh` is what enforces that: it reads `fork/patches/series` in order and applies each with `git apply --3way (apply-patches.sh:33)`, and when reversing it walks the array backwards (`apply-patches.sh:40`) so the last-applied is the first-removed. A `--reverse` that went forwards would try to unapply 0002 while 0027’s additions were still sitting in the function.

## What the series file encodes besides order

`series` is 36 lines for 27 entries, and the difference is where the interesting content lives. The parser is deliberately forgiving so that it can be: `apply-patches.sh:23` strips `#`-to-end-of-line comments and surrounding whitespace before treating each remaining line as a bare filename, which means every entry can carry an unbounded trailing note and most do. Those notes are the fork’s verification record — the recurring string is (`VERIFIED git-apply @150.0.7871.46`), and the stronger variants are `git-apply+reverse` (the patch was checked to apply *and* cleanly unapply, which is what makes the rebase procedure trustworthy) and, for 0022, 0023 and 0024, `compiled + runtime` with the measured evidence inline.

Two facts about numbering are visible only here. The first is the tombstone: an `0008-tmux-split-prefix.patch` is commented out across four lines with the reason kept — the native two-pane split “consumed the prefix + the next key before the page saw them (the ‘C-b %%’ double-tap)” and was superseded by the JS overlay that owns `Ctrl-b`. Its number was then reused by `0008-allow-framing.patch`. The second is that numeric order and apply order are not the same: the series lists 0022, then 0024, then 0023. 0024 has to precede 0023 because of what the file it edits became — 0024’s note records that it “ALSO carries the fork-patch-0025 output METERS back-channel (folded into these same files),” so an earlier 0025 was absorbed into 0024 and the number 0025 was subsequently reused for the custom palette patch. Two numbers in a 27-entry series have been recycled, which is exactly why the series file rather than the filename ordering is authoritative.

That is also why the series file has a dedicated gate. `scripts/check-patch-series.sh` asserts a strict one-to-one correspondence in both directions: it walks every non-comment line and fails if the named file is absent (`check-patch-series.sh:22-30`), then walks every `*.patch` on disk and fails if it is not listed (`check-patch-series.sh:32-43`). Its header states the asymmetric failure modes plainly — “a patch on disk but absent from series is silently never applied, and a series entry with no file breaks the apply run” — and one of those is silent, which is the whole argument for the check. It reports success as `check-patch-series: OK - ${#listed[@]} patches, series and disk match`, deriving the count rather than printing a constant.

## The test that compiles code out of a diff

`fork/tests/` is 640 lines, and what it does is unusual enough to be worth stating carefully: it is a C++ unit test whose subject is not a source file. The engine under test lives inside `0022-audio-eq-output.patch`, as added (+-prefixed) lines of a diff that is only ever applied to a Chromium checkout the test does not have.

`run_dsp_tests.sh` extracts it. Two `awk` programs read the patch and print bounded regions, which are then stripped of their leading + (`run_dsp_tests.sh:33-47`): the first starts at `+struct ZwireEqBand {` and runs to the `};` that follows `generation = 0;`, producing `dsp_cfg.inc`; the second starts at `+class ZwireAudioEq {` and stops at the first `};` at column zero, producing `dsp_class.inc`. The column-zero anchor is the whole trick and the script says so — inner struct closes are indented as `+ };`, so `^\+};` matches only the class’s own terminator.

Four `grep` assertions then check the extraction is not silently truncated (:50-53): `struct ZwireEqConfig` and `uint64_t generation` must appear in the config include, `class ZwireAudioEq` and `void Process`

in the class include. Each failure exits 2 with **FATAL**. Without those, a patch reshuffle that moved `generation = 0`; would yield a short but syntactically valid extraction, and the test would compile and pass against half an engine.

The compile is `clang++ -std=c++20 -O2 -Wall` against `dsp_shim.h` (`run_dsp_tests.sh:57`), a 73-line file that supplies just enough of `media::AudioBus` for the engine to link outside Chromium. Two symbols the engine calls in production — `LoadZwireEqConfig()` and `SeedZwireEqFromLaunchArgs()` — are supplied by the test as a controlled slot (`dsp_test.cpp:30-32`), which is what lets each case install a fresh config and force a cascade rebuild by bumping `generation` (`dsp_test.cpp:47`).

The result is nineteen `check()` assertions over the real, current DSP code:

```
unity: default config is bit-identical passthrough
limiter: no sample exceeds the linear ceiling
width=0: L and R collapse to identical mono
width=2: L/R match mid+/-side*w exactly
delay: impulse reappears at delay_ms*sr/1000
reverb: recirculated tail energy appears after input stops
compressor: output above threshold is reduced vs bypass
full chain: no NaN/Inf across 8 blocks with everything engaged
lowpass: 12kHz tone attenuated below 25% of bypass
notch: on-frequency tone attenuated below 10% of input
gate: sub-threshold signal ducked, supra-threshold passes
bit-crush: alters the signal and stays bounded
haas: mono input decorrelates into distinct L/R
modulation: chorus+flanger+phaser finite, non-silent, alters signal
waveshaper: hard-clip bounds to +/-1 and alters signal
ring-mod: alters signal, finite, non-silent
tremolo: bounded by dry peak, dips to a near-silent trough
auto-pan: mono input sweeps into distinct L/R
auto-wah: envelope band-pass alters signal, finite
```

The first one is the load-bearing one. “Default config is bit-identical passthrough” is the invariant that makes an always-on engine in every audio stream acceptable at all: 0022’s series note says the engine “is ALWAYS-ON by default: empty/no-config => unity (audio still routed through it), only ‘off’/‘0’ bypasses,” and this test is what keeps that claim true after an edit. Most of the rest are shaped to be robust against tuning rather than pinned to constants — “alters the signal and stays bounded,” “finite, non-silent” — so that changing a coefficient does not fail the suite while breaking the DSP does.

`dsp_test.cpp:14` is careful about the one place it restates engine arithmetic: the four-line mid/side formula in the width test, described as “a specification check, not a reimplement of the engine.” Everywhere else the file only observes outputs. That distinction is what keeps the test from becoming a second copy of the DSP that can drift from the first.

The property this buys is stated in the script’s header: “Any DSP change in the patch recompiles here and must still satisfy the invariants, or this fails.” It is the only part of the fork that can be verified without a 100 GB checkout, and the only patch in the series whose *behaviour*, not merely its context, is under test.

## Three documents describe the series, and only one of them is gated

The series exists in three written forms, and they do not agree — not because any of them is wrong, but because only one of them has a check.

`fork/patches/series` is authoritative by construction: `apply-patches.sh` reads it, and `check-patch-series.sh` asserts it matches the directory in both directions. It lists all twenty-seven entries.

`fork/patches/README.md` carries the target-file map — the table giving each patch's HUD element, its exact Chromium anchor, the approach, and its status. That table documents twenty-one of the twenty-seven: `0001` through `0015`, then `0019`, `0020`, `0021`, `0022`, `0026`, `0027`. Six patches have no row — `0016` (the `developerPrivate/settingsPrivate` allowlist), `0017` (hold-to-quit), `0018` (the null-policy-container crash fix), `0023` (tab capture), `0024` (live audio config and the meter back-channel), and `0025` (the custom-palette read). Five of those six are among the most behaviourally significant patches in the fork, and `0024` is the second-largest by line count.

`fork/README.md` carries a second status table, and it is further behind still: seven rows, `0001` through `0007`, sitting directly beneath prose that states “HUD patches (`patches/series`): **all 27 authored** against the pinned tag `150.0.7871.46`.” The sentence is current; the table under it stopped being updated at `0007`.

None of that is a defect in the patches, and both documents are honest about the series count in their prose. The point is structural: `scripts/check-patch-series.sh` exists precisely because a drift between the series file and the directory is silent, and the same argument applies one document out. A patch that lands in `series` and on disk is applied and shipped whether or not any prose mentions it, so the target-file map degrades from “the map” to “a map of the first fifteen plus whichever later ones someone remembered.” There is no `check-patch-readme.sh`, and the reason there is not is that the failure it would catch is cosmetic rather than functional — which is exactly the reasoning that lets it accumulate.

The practical consequence falls on the rebase. The README's table is where an anchor lives: `TabStyleViewsImpl::GetPath`, `AddChromeColorMixer`, `before the high-contrast early-return`, `ResourceBundle::InitDefaultFontList`, `the non-SetDefaultFontDescription #else branch`. For the twenty-one documented patches, a rebase starts by reading that anchor and checking whether it still exists at the new tag — the procedure the same README recommends, using `raw.githubusercontent.com/chromium/chromium/<TAG>/<path>`. For the six undocumented ones the anchor exists only inside the diff's context lines, so re-anchoring them means reading the patch itself first to work out what it was attached to. The series file's inline notes partly cover the gap — `0016`'s entry names `_permission_features.json` and the `DCE846DF...` hash, `0018`'s names `extension_function_dispatcher.cc` and `crbug.com/455908853` — which is why those notes are as long as they are. The series file has quietly become the more complete of the two documents, and it is also the only one anything checks.

# Chapter 64 – The HUD’s Own UI Layer

Appendix Y lists the HUD’s replacement pages and Chapter 14 walks several of them. Underneath all of them is a layer this book has only referred to in passing: the component library the pages are built from, the boot script that turns a nearly-empty HTML file into a working page, and the small amount of glue that exists because a browser extension cannot always behave like a browser.

## Twenty-three pages, seven to thirty-one scripts each

`extensions/hud-internal/pages/` holds 62 files, 23 of which are HTML. Each page is a shell: a `<head>` of stylesheet links, a `<body>` containing `<div id="app">`, and a run of `<script src>` tags. `history.html` is representative at 21 scripts and 4 stylesheets, and reading it top to bottom gives the layer’s whole structure:

```
all.css · donut.css · ext-page.css · history.css
util.js fzf.js search-box.js colorscheme.js crt.js fx.js toast.js
command-palette.js ../palette-cmds.js widgets.js card.js product-card.js
field.js table.js data-table.js modal.js popconfirm.js chart.js donut.js
hud-accel.js zg-boot.js history.js
```

Everything in that list is library except four files: `../palette-cmds.js`, the extension’s own `⌘K` command registry; `hud-accel.js` and `zg-boot.js`, the two page-agnostic scripts described below; and `history.js`, the page itself, always last. The spread across the 23 pages runs from `terminal.html` at 7 scripts to `audio.html` at 31, and the CSS count runs 2 to 4. Across all 23 pages, 56 distinct `zgui-core/webui/*` modules are referenced — a slice, not the library.

## The library, and the file that keeps it from drifting

`extensions/hud-internal/lib/zgui-core/webui/` holds 260 JavaScript modules (not counting tests), 205 stylesheets, and 22 `.test.cjs` files — 33,820 lines of JS and CSS in one directory. Chapter 17 and Appendix X cover what zwire consumes from it; what matters here is how a consumer loads it without maintaining a list.

`webui/all.css` is a generated file — its first line says “GENERATED by scripts/build-all-css.mjs. Do not hand-edit” — containing 204 `@import` rules and nothing else. `build-all-css.mjs:3` states the bug it exists to prevent: a consumer loads one file and gets the complete styling, so there is “No hand-maintained per-file `<link>` list to drift out of sync (the bug that broke zterm’s settings).” The generator sorts alphabetically with two exceptions pinned by name (`build-all-css.mjs:11-13`): `cyberpunk.css`, `fonts.css`, `base.css` first, in that order, because they define the tokens everything else consumes; and `print.css` last, because “it must `@import LAST` so its `@media print` rules win the cascade.” A cascade-ordering requirement that would otherwise live in a comment somewhere is instead encoded in the tool that writes the file.

So a HUD page’s four stylesheets are four layers with four owners. `all.css` is the whole library. `donut.css` is a per-component addition for a component the page uses. `ext-page.css` (158 lines) is the app-specific layer, and its header states its boundary exactly: the card grid and cards “come from zgui-core (product-card.css / ZGui.productCard); this only adds the app-specific bits product-card.css doesn’t define,” with “Tokens from zgui-core base.css + colorscheme.js.” `history.css` (79 lines) is the page. Nothing in the app layer redefines a token; it adds classes (`.card-chip`, `.xt-navlink`, `.info-row`) that consume them.

The one file that does declare tokens is `pages/hud-pages.css` (72 lines), which opens `:root` with thirteen custom properties — `--bg-primary:#05050a` through `--border:#1a1a3e`, plus the three glow values — and sets the page's grid background as two `linear-gradients` at `background-size:26px 26px`. It is the standalone skin for pages that do not load the library, and its first line labels it: “zwire HUD internal-page skin — our own pages, fully styled (no Chrome WebUI).”

## zg-boot.js, the 652 lines that make a page a page

Every HUD page's second-to-last script is `zg-boot.js`, and its header describes the contract: it “Mounts ZGui.appShell (brand · filter · ⌘K palette · settings w/ colorscheme + CRT/neon · shortcuts), wires the cross-page nav into the shell, and bridges the zgui colorscheme picker to the native host so a pick repaints the whole browser (`~/.zwire/hud-scheme`) + mirrors to storage for the content-script theme.” The last clause of the header is the rule the whole layer follows: “All UI comes from ZGui.\* per the zgui-core-only rule.”

The navigation is a literal table (`zg-boot.js:15-22`), and two of its nineteen entries are not HUD pages at all. `DOWNLOADS` points at `chrome://downloads` and `NEW TAB` at `chrome://newtab`. The comment above the table explains the first, and it is a real architectural constraint rather than an omission: “zp-wrchrome owns the download manager (its takeover cancels + reissues Chrome downloads), so a HUD downloads page built on `chrome.downloads.search` would only show the cancelled stubs.” A HUD page that queried the obvious API would render a list of the corpses left behind by the segmented downloader. Rather than build a page that is wrong, the nav sends you to the native one. A second table, `NATIVE_PAGES`, holds `chrome://flags`, and a third, `MORE` (`zg-boot.js:26-30`), holds nine palette-only destinations — `chrome://password-manager`, `chrome://inspect`, `chrome://net-export`, `chrome://policy`, `chrome://components`, `chrome://about`, `chrome://settings/content`, and the two web stores — reachable by ⌘K but not occupying a nav button.

`zg-boot.js` also carries the shell-execution path for custom ⌘K commands, and its per-OS dispatch is worth quoting because it is where a HUD page reaches the machine. `osKind()` classifies the platform from `navigator.userAgentData.platform` with two fallbacks, and `shellReq()` builds the native-host request accordingly (`zg-boot.js:44-51`): `cmd.exe /d /s /c` on Windows, `/bin/sh -c` elsewhere, with an explicitly widened `PATH` — `/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:...` on macOS, `/usr/local/bin:/usr/local/sbin:...` on Linux. A browser-spawned process inherits a minimal environment, so a command the user can run in their terminal would otherwise fail to find its own binary. The reply's `stdout/stderr` arrive base64 and are decoded through a two-stage `decodeURIComponent(escape(atob(s)))` with a bare `atob` fallback (`zg-boot.js:52`), then truncated to 160 characters for a toast.

## One keystroke that had to be reimplemented

`pages/hud-accel.js` is 58 lines and exists because of a collision between two correct behaviours. Chrome's clear-browsing-data shortcut — `⌘⇧⌘` on macOS, `Ctrl+Shift+Del` elsewhere — is not in Chromium's reserved shortcut table (`global_keyboard_shortcuts_mac.mm`), so the focused renderer sees the key first and the browser only runs `IDC_CLEAR_BROWSING_DATA` if the page leaves it unhandled. And every HUD page auto-focuses its filter box at `zg-boot` mount. The consequence, in the file's own words: “on a HUD page the keystroke lands in a text field and the browser command never fires — it worked on the new-tab page and on web pages but not here.”

The fix binds the same keys to the same destination in page JS. `isClearCombo` (`hud-accel.js:29-35`) accepts either `Backspace` or `Delete` on both platforms — the comment notes Apple keyboards report the top-right key as `Backspace` while full keyboards send `Delete` — requires `shift`, rejects `alt`, and demands `meta && !ctrl` on macOS versus `ctrl && !meta` elsewhere. The platform flag is a parameter with a default rather than a closure read, and the comment says why: “isMac is a parameter so the platform split is testable headless.”

Two details make it behave like the command it replaces. The listener is registered in the capture phase (`hud-accel.js:55`) — “the filter box (or any focused field) must not eat it first” — which is the same precedence Chrome’s own accelerator would have had. And on the Settings page itself it calls `window._zb0openClearData` if `settings.js` has published it, jumping to the wizard in place instead of opening a second copy of the page you are already on (`hud-accel.js:39-43`). It depends on nothing but the DOM and `chrome.tabs`, stated as a deliberate constraint: “Pure DOM + chrome.tabs — no ZGui dependency, so it works even if a page’s widget stack fails to load.” It is loaded before `zg-boot.js` on every page for that reason.

## The gate that renders rather than greps

The library ships its own CI gate, and its design is the interesting part. `lib/zgui-core/scripts/baseline-gate.mjs` “RENDERS each target’s built frontend in headless Chrome and inspects the RESULTING DOM — it never greps source or a magic comment, so a stamp cannot be faked by typing it into static HTML.” An app passes only if three runtime artifacts are present: the `<html data-zgui-baseline=“...”>` stamp containing every required token (written by `ZGui.baseline`), a live filter/search input in the shell bar, and the `.zg-crt-h` scanline overlay element created at runtime by `ZGui.crt`.

The last line of its header is the part worth carrying to any other checker: the required-token list “is parsed from `webui/baseline.js` so the gate and the contract can never drift apart.” The gate does not hold its own copy of what it requires. It reads the definition, then renders the page and looks for the consequences of that definition having been honoured — which is a different and much harder thing to satisfy than a string in a file.

## Twenty-two tests, and the harness that makes them possible

The component library ships 25 `.test.cjs` files totalling 3,162 lines — 203 `node:test` cases, all passing in 590 ms at the submodule pointer this superproject records — and they are unusual in a way that is worth explaining before the contents: they test browser UI modules with `node:test` and no jsdom. (Earlier printings said 22 files and 2,691 lines, which was true when they were set. Chapter 74 records the counts as measured, and the fact that nothing in zwire runs any of them.)

`webui/harness.cjs` is how. Its header states the problem — “Each `webui/*.js` module is a classic IIFE that registers on `window.ZGui.*` and reaches for browser globals (window/document/navigator/localStorage/performance/Event/MutationObserver)” — and the solution: “We load the REAL source into a fresh `vm` context with hand-written stubs, then return the sandbox so a test can read `sandbox.window.ZGui.<ns>` and assert against the live functions.” The DOM stub is deliberately minimal and its limits are documented in place: it is “good enough for the `escapeHtml.textContent -> innerHTML` trick and for the trivial DOM the modules touch at load time,” with `innerHTML` escaping only `&`, `<`, `>`.

One property of `vm.runInContext` is load-bearing and the header names it: each load “gives each load its own module-local state (e.g. `fzf`’s mutable scoring weights, shortcuts’ `DEFAULTS/init._done`), so tests don’t bleed into one another.” A library of IIFEs that mutate module-scope state is exactly the shape that produces order-dependent test suites; a fresh context per load removes the failure mode rather than working around it.

What the tests pin is more interesting than how many there are. Reading four of the headers gives the pattern:

- `crt-layering.test.cjs` (42 lines) — “The CRT banding must never paint over what an app renders.” The regression it encodes is a stacking-context bug: the overlay sat at `z-index: 1` and depended on every renderer opting out with `.zg-crt-exempt` at `z-index: 2`, and “That contract silently fails the moment ANY ancestor of the renderer forms a stacking context — a transform, a filter, `contain`, `isolation`, or a positioned element with a z-index — because the renderer’s z-index is then scoped inside that ancestor.”

- `toolbar-wrap.test.cjs` (39 lines) — “A toolbar with more tools than fit must WRAP, not run off the window.” The cause was structural rather than stylistic: every control goes into a single `.zg-toolbar-group` div, so a host setting `flex-wrap: wrap` on `.zg-toolbar` achieved nothing because “the toolbar has only three children (left group, spacer, right group), so there was nothing for it to break onto a second line.”
- `host-prefs.test.cjs` (123 lines) — `host-prefs.js` “REPLACES `window.prefs` outright, so it must satisfy the whole zgui prefs contract.” It had shipped with only `localStorage`’s surface, no `getObject`, “while 13 call sites across the toolkit (shortcuts’ registry, colorscheme’s custom vars/presets, table’s column state, fzf) call `prefs.getObject(key, default)`.”
- `app-shell-version.test.cjs` (138 lines) — “The settings panel must state the running build’s version in EVERY app,” after the identity row had been gated on an `opts.version` that only one consumer passed.

Every one of those is a *contract between the library and a consumer* rather than a property of a function in isolation, and every one describes a failure that was silent in production: an overlay that covers a canvas, tools that vanish off an edge, a settings panel with no version, a `getObject` that is simply absent. That is the category of bug a shared toolkit produces — not incorrect output, but a host whose assumptions the library stopped honouring — and it is why the suite reads as a list of past incidents rather than a list of features.

The relationship between these tests and `baseline-gate.mjs` is a division of labour worth naming. The tests assert against the module’s live functions in a stub context; the gate renders the built page in headless Chrome and inspects the resulting DOM. The first can prove a function behaves; only the second can prove it *ran*.

# Chapter 65 – The Extension Behind the Clock

Chapter 6 walks the new-tab page: the neon clock, the omnibox, the tiles, the palette module, the scheme sync. It describes a *page*. What is installed is an extension — twelve top-level source files (nine JS, one CSS, two HTML) totalling 2,415 lines, its own service worker, its own sandbox origin, its own copy of a command registry, and one page that has nothing to do with new tabs at all. This chapter is the rest of it.

## The smallest manifest of the three

`newtab/manifest.json` declares three permissions: `nativeMessaging`, `tabs`, `history`. Against `hud-internal`'s twenty and `zpwchrome`'s nineteen, that is the whole point of the extension being separate. It has no `host_permissions`, no content scripts, no `declarativeNetRequest`, and one job in the manifest that no other extension can do: `chrome_url_overrides.newtab -> newtab.html`.

It does declare two things that read as oversized for a clock page, and both are load-bearing. `web_accessible_resources` exposes eleven entries to `<all_urls>` — `newtab.html`, `tmux.html`, `newtab.css`, `newtab.js`, `palette.js`, `tmux-pane.js`, `tmux-host.js`, `scheme-sync.js`, `schemes.js`, `fonts/*`, `lib/*` — because the tmux overlay frames these pages from arbitrary origins. And `content_security_policy.sandbox` plus `sandbox.pages: ["sandbox/js-run.html"]` carve out a second, weaker CSP for exactly one file.

## A twenty-five-line service worker with two jobs

`newtab/background.js` is 25 lines, and neither of its two handlers is what you would guess from the file's existence.

The first is a toolbar action listener that opens a new tab. Its comment gives the reason: the worker's "only job is to give the extension a real toolbar action so Chrome draws a live, COLORED button (a newtab-override extension with no action shows a dead grayed-out icon)." The behaviour is incidental; the *existence* of an action handler is what makes the icon render normally.

The second is a cross-extension relay, and it is the seam Chapter 10's tiling overlay needs to cross. `chrome.runtime.onMessageExternal` accepts a `zbOpenTmuxCarrier` message from exactly one sender id — `omcgnnjfbmpdlofklbpddkhnfibfhgg`, checked explicitly at `background.js:18` — and responds by opening `chrome.runtime.getURL('tmux.html') + '#zbtmux=' + encodeURIComponent(JSON.stringify(msg.session))`. The comment states the two-part constraint: "Chrome won't inject its overlay onto a `chrome-extension://` page, and a cross-extension tab-open of our page is not guaranteed, so WE open our own carrier." The HUD cannot open this extension's page reliably, so it asks this extension to open it, and the session travels in the URL fragment because `chrome.storage` is per-extension and the saved layouts live in the HUD's storage, not here.

## The carrier page

`tmux.html` (38 lines), `tmux-host.js` (143) and `tmux-pane.js` (106) are the other half of that relay, and they exist for one scenario: a saved tmux layout in which every pane is a new tab. The header of `tmux-host.js` sets it out — with no `http/https/file` page in the layout there is no page for the HUD's content script to attach the overlay to, so “This page (`tmux.html`) IS that carrier: it ships its own copy of the zgui-core tmux overlay and drives it exactly like hud-internal's `ztmux-config.js`, but seeds the session from its URL hash.”

Three details make it behave identically to the content-script path rather than merely similarly. It bails immediately if it is not the top frame (`tmux-host.js:16`) — panes are iframes of this same page, and only the carrier hosts the overlay. Panes are same-origin new-tab iframes named `zbtmux`, so `tmux-pane.js` runs inside each and “forwards the prefix + synced keystrokes up here — same protocol as the http path.” And the per-pane chrome (address bar plus framed page) is injected as four CSS rules, annotated as “the same 12 lines `ztmux-config.css` carries on the http path” (`tmux-host.js:32-40`), because zgui-core's `tmux.css` self-injects only the tiling geometry.

`normalizeUrl` (`tmux-host.js:22-27`) is where an empty pane's behaviour is decided: blank or `about:blank` becomes this extension's own `newtab.html`, a string with a scheme is passed through, a bare host-looking token gets `https://`, and anything else becomes a Google search. `hostLabel` strips `www.` and falls back to the literal string `newtab`.

## The page itself is 116 lines

`newtab.js` is small, and every constant in it is overridable. `DEFAULT_TILES` is six entries and `DEFAULT_ENGINE` is `https://duckduckgo.com/?q=%s`, both read through `readJSON` from `localStorage` keys `zb.tiles` and `zb.engine` (`newtab.js:5-25`) with a `try/catch` that returns the fallback on any parse failure. The clock is a `tick()` that formats `HH:MM:SS` and an uppercased locale date.

The one piece of real logic is `looksLikeUrl` (`newtab.js:39-43`), which decides whether typed input is a destination or a query, in three tests and in this order: anything matching `^[a-z][a-z0-9+.-]*:\|/\\|/i` is a URL because it has a scheme; then anything containing whitespace is a search; then a single dotted token with a last segment of two or more characters is a URL, as is the literal `localhost`. The ordering is what makes it correct — checking for a dot first would send `what is a .com domain` to a nonexistent host.

## Two files that are byte-identical to the HUD's

`newtab/palette-cmds.js` (866 lines) and `newtab/cmd-defaults.js` (96 lines) are not similar to `extensions/hud-internal/palette-cmds.js` and `extensions/hud-internal/cmd-defaults.js`. They are the same bytes — `diff` reports no difference for either pair.

That duplication is forced by the boundary, not chosen. The two extensions are distinct origins with distinct `chrome.storage` areas and no shared script loading; `web_accessible_resources` could expose the file but a cross-extension `<script src>` would still execute in the loading extension's context with the loading extension's CSP, and the file's `chrome.storage` calls would then address the wrong store. The copy is the only way both palettes can offer the same commands. Appendix X notes the same pattern one level down for the toolkit — “the new-tab extension, a distinct origin, bundles its own copy of the util/fzf/command-palette trio under `lib/zgui/`” — and the vendored count is exactly ten files: three in `lib/zgui/` (`command-palette.js`, `fzf.js`, `util.js`) and seven in `lib/zgui-core/webui/` (`button-bar.js`, `modal.js`, `powerline.js`, `split-pane.js`, `tmux.js`, `toast.js`, `util.js`).

`cmd-defaults.js` holds 65 default ⌘ commands — 62 URL templates and 3 actions (`reload`, `copyUrl`, `cycleScheme`) — built by two three-line constructors `u()` and `a()` so that adding one is a single line. The seeding rule is more careful than a first-run write. `zwireSeedCmds` (`cmd-defaults.js:81-95`) reads both `zb_custom_cmds` and `zb_cmds_removed`, and adds only defaults whose id is neither already present nor in the removed list. The comment states the property that buys: it “Runs on every load, so EXPANDING the defaults above makes the new rules appear automatically, without re-adding ones the user removed.” A one-shot seeder would strand every default added after a user's first launch; a naive re-seeder would resurrect

deletions. The file's own header records the reason the seeder lives here rather than in a worker: this is “a page/content-script context where storage writes are reliable (the MV3 worker's startup write wasn't landing).”

## The theme path that skips the browser entirely

`scheme-sync.js` is 92 lines and is the new-tab page's most direct connection to the rest of the fleet. It does not read `chrome.storage`, does not poll a file, and does not message another extension. It opens one persistent `connectNative` port to `com.zwire.hud` and `subs` to three topics — `scheme`, `ui`, and `palette` — where “the host pushes the CURRENT value on subscribe (snapshot) and every change after — from ANY app in the fleet.”

The third topic is the one that makes custom schemes work. Its header explains: `palette` “carries a custom/edited colorscheme's resolved var->hex map (no name in the baked SCHEMES table); without it the new-tab page cannot follow a custom scheme.” The same gap on the native side is what patch `0025` fixes for the browser chrome.

Two guards in `applyUi` are worth noting because they are the kind of thing a snapshot-on-subscribe design needs. An empty object is ignored outright (`scheme-sync.js:24`) — “An EMPTY reply (missing/fresh native file) carries no state — ignore it so it can't clobber an optimistically-applied or pushed value back to dark.” And each of the five preferences is applied only when explicitly present as a boolean (`scheme-sync.js:31-35`), “so a partial reply never silently resets an unrelated pref (e.g. a light-only write flipping fx).” The effect classes follow `ZGui.fx`'s inverted convention: each effect ON means the class is *absent*.

## The one page with a weaker CSP

`sandbox/js-run.js` is 29 lines and exists because of a single MV3 rule. Its header: every other zwire surface — content scripts, HUD pages, New Tab — “is bound by the MV3 default CSP `script-src 'self' 'wasm-unsafe-eval'`, which forbids `eval` / `new Function`, so `(new Function('q', code))()` throws `EvalError` there and the user's JS silently never runs.”

The palette's “Run JavaScript” step needs exactly that. So the manifest declares one sandboxed page, which gets its own CSP with `'unsafe-eval'` and the `allow-modals` sandbox flag — the second flag mattering because `window.alert()` is what a one-liner usually wants. The palettes embed the page as a hidden iframe and `postMessage` the code in; the handler validates `d.zjs === 1` and `typeof d.code === 'string'`, runs `(new Function('q', d.code))(d.arg || '')`, and posts back `{zjs:1, id, ok}` with `err` set to the message on a throw.

The header is equally clear about the price: “no `chrome.*` and no access to the host page's DOM — only this frame's own window/eval. That is inherent to how MV3 permits eval.” A user script that runs here can compute, and can talk to whoever posted to it, and can do nothing else. Both extensions declare the identical sandbox block, so both palettes get the same capability with the same limit.

# Chapter 66 — Two Bases, One Launcher

zwire runs on a browser it did not build, or on one it did, and the same launcher starts either. The two paths produce artifacts that differ in size by two orders of magnitude and in provenance completely, and the entire seam between them is two text files. This chapter is that seam and the packaging that sits on top of it.

## base.path and base.version

Everything downstream of “which browser” reads `$ZWIRE_STATE/base.path`: one line, an absolute path to an executable. `$ZWIRE_STATE/base.version` sits beside it and is never read by the launcher — it exists so a human can tell which path produced the binary. The two writers use distinguishable formats. `fetch-base.sh:58-59` writes `r<REVISION>`; `fork/package.sh:73-74` writes `fork-<TAG>`. A state directory whose `base.version` reads `r1547892` came from a Google-hosted snapshot; one that reads `fork-150.0.7871.46` came from a local compile.

## The snapshot path

`scripts/fetch-base.sh` is 60 lines and downloads a prebuilt Chromium continuous build. Its header is specific about why *this* artifact rather than the more obvious Chrome for Testing: a plain Chromium snapshot is “a real Chromium/Blink build with NO ‘Google Chrome’ product branding and NO ‘for automated testing’ banner (that stripe is exclusive to Chrome for Testing).” And a second reason that is a hard dependency rather than a cosmetic one: “Unbranded Chromium also retains the `--load-extension` switch that preloads `zpwrcrome` (removed only from *branded* Chrome 137+).” The entire extension-loading architecture of this project depends on that switch, so the base cannot be a branded build at all.

The platform table (`fetch-base.sh:25-28`) maps four host tuples to a snapshot platform id and a zip layout:

|                            |           |                  |              |
|----------------------------|-----------|------------------|--------------|
| Darwin/arm64               | Mac_Arm   | chrome-mac.zip   | chrome-mac   |
| Darwin/x86_64              | Mac       | chrome-mac.zip   | chrome-mac   |
| Linux/x86_64   Linux/amd64 | Linux_x64 | chrome-linux.zip | chrome-linux |
| MINGW*   MSYS*   CYGWIN*   | Win_x64   | chrome-win.zip   | chrome-win   |

With no revision argument it resolves `LAST_CHANGE` from the bucket (`fetch-base.sh:35`), so the default is “whatever Chromium shipped most recently.” Passing a revision pins it. After unzip it resolves the executable per platform — on macOS by finding a file at `*Chromium.app/Contents/MacOS/*` named `Chromium` (`fetch-base.sh:51`) rather than assuming the path — and fails loudly if the result is not executable.

## The fork path, and the bundle-picking problem

`fork/package.sh` installs a locally built app as the base, and its macOS branch spends most of its length on one question: which `.app` in the output directory is the one that was just built.

The naive answer fails, and the comment says how (`package.sh:24-28`). Patch 0006 renames the bundle to `zwire.app`; an unbranded build without it yields `Chromium.app`; and “A stale app from an earlier build/rename

can linger in the out dir alongside the fresh one (e.g. `zbrowser.app` next to `zwire.app`), and a plain `head -1` would grab the alphabetically-first STALE bundle.” So the selection is by modification time:

```
APP=$(find "$OUTDIR" -maxdepth 1 -name '*.app' -type d ! -name '*Helper*' \
    -exec stat -f '%m %N' {} \; | sort -rn | head -1 | cut -d' ' -f2-)
```

`-maxdepth 1` so nested framework bundles are invisible, `! -name '*Helper*'` so the renderer/GPU helper bundles are excluded, `stat -f '%m %N'` for an mtime-prefixed line, numeric reverse sort, most recent wins. The executable name is then derived rather than assumed — `PlistBuddy -c 'Print :CFBundleExecutable'` on the chosen bundle (`package.sh:34`) — because the branding patch renames the binary too, and the install goes in under the bundle’s own name with the comment noting the point: “no ‘fork-’ prefix, so Finder / Dock / ⌘-Tab all read ‘zwire’.” The Linux branch is four lines by comparison: copy `$OUTDIR/*` into `base/fork-linux/`, point at `chrome`.

Both branches converge on the same two `printf`s, and then the macOS branch calls `scripts/rebrand-macos.sh` — the same script the runtime path uses, invoked as `[[ $os == Darwin ]] && "$ROOT/scripts/rebrand-macos.sh" || true` (`package.sh:57`). One rebrand implementation, two callers.

## The refusal that is the most important line in the installer

`scripts/localinstall.sh` needs a base and will build one if `base.path` is missing (`localinstall.sh:60-63`). But if `base.path` exists and points somewhere that no longer does, it does two different things depending on what it finds, and the distinction is the interesting part.

First it tries to recover. The comment at `localinstall.sh:65-69` describes the common cause — a state-directory migration “moves `$STATE/base` but leaves that recorded path pointing at the old, now-deleted location” — and since “The bundle always lives under `$STATE/base/`,” the script globs for a `.app` there, verifies the executable of the same basename is present, rewrites `base.path`, and continues. No refetch.

If that fails, it stops. It does not fall back to `scripts/build.sh`, and the comment at `localinstall.sh:76-80` explains why in terms of an incident: “NOT a silent `scripts/build.sh` here: that fetches a STOCK Chromium snapshot, and swapping the engine under the same product name is invisible in the output — a fork base that had gone missing was replaced with stock Chromium, packaged, and shipped as a `zwire` release. Say what is missing and stop; the fork is rebuilt deliberately (`fork/build.sh` + `fork/package.sh`).”

Two bases that are interchangeable at the `base.path` interface are precisely what makes that substitution silent. Everything downstream — the launcher, the extension staging, the plist rebrand, the `.pkg` — behaves identically whether the binary was compiled from twenty-seven patches or downloaded from a bucket. The only observable difference is that none of the native chrome is themed and the H.264 streams do not play, which is not something a build script notices. The fix is not a check that the base is a fork; it is a refusal to guess.

## What is staged, and the one assertion after the copy

The three extensions are staged with `rsync -a` and five excludes (`localinstall.sh:231`): `node_modules`, `.git`, `tests`, `target`, `_metadata`. The first four are size; the fifth is correctness, and the comment gives it — `_metadata` is “a dev-profile-specific compiled index Chromium regenerates per-user at launch, so bundling it is dead weight that never gets loaded.”

Immediately after the loop comes a hard failure that is not about any of those (`localinstall.sh:237-238`): if `$RES/ext/hud-internal/lib/hooks-editor/hooks-editor.bundle.js` is not present and non-empty, the install aborts with “staged hud-internal is missing lib/hooks-editor/ — Hooks/Commands/Triggers would ship without Monaco.” Chapter 23 covers why that bundle is a gitignored build artifact and how its absence renders as an empty pane rather than an error. What this line adds is that the check is repeated *after* the

copy, not only after the build, because “an `--exclude` that starts matching it, or a `lib/` symlink, drops it” — a bundle that was built correctly can still fail to arrive.

## The observable end state

The staged tree under `dist/stage/` is what the packaging produces, and reading it back is the only way to confirm the assembly rather than infer it. The nested browser bundle’s `CFBundleShortVersionString` is `152.0.7946.0`, and its framework directory is `Chromium Framework.framework/Versions/152.0.7946.0/`. That is a snapshot-path base: the fork’s pin is `150.0.7871.46`, two milestones behind. The staged tree also contains `Resources/ext/` with `newtab`, `hud-internal`, and `zpwchrome` — three directories, each with a `manifest.json`, matching the three-extension load list the launcher passes to `--load-extension`.

Which is the honest summary of the packaging path: it is agnostic. It will build a correct, self-contained, branded, TCC-safe, signed application around whichever browser `base.path` names, and the one place it declines to be agnostic is the one place being agnostic already shipped the wrong thing.

## The two scripts named for compiling that do not compile

Two entry points in `scripts/` carry names that imply a build, and neither one produces an executable. Their headers say so up front, and the distinction they draw is the cleanest statement of what the snapshot path actually is.

`scripts/build.sh` (51 lines) opens: “Assemble the zwire browser bundle: fetch the base Chromium snapshot if it is missing, then rebrand it in place (Dock name + cyberpunk icon). There is no compile step — zwire runs a prebuilt Blink base against a fixed extension set — so ‘build’ == ‘make sure a rebranded base bundle exists and is current’.” It is a three-stage idempotent check: fetch if `base.path` is absent, re-fetch if `base.path` points at a directory that is gone (`build.sh:26-30`), then run `rebrand-macos.sh` on Darwin and warn-and-skip elsewhere. It ends by `du -sh`-ing the bundle and printing the size — a derived figure, not a constant.

Note the difference between that re-fetch and the refusal in `localinstall.sh` described above. `build.sh` re-fetches a missing base without asking, because `build.sh` is the snapshot path and a stock snapshot is exactly what it is supposed to produce. `localinstall.sh` refuses, because it might be packaging a fork base and cannot tell the difference after the fact. Same missing directory, opposite correct responses, and the deciding factor is which script the user invoked.

`scripts/install.sh` (47 lines) is the fresh-clone path and is three steps plus one that was added after a shipped failure. Step 1 fetches the base if `base.path` is absent. Step 2 symlinks `bin/zwire` into `$BINDIR` (default `~/local/bin`, overridable with `--bindir`) and, rather than editing a shell profile, checks whether that directory is on `PATH` and prints an instruction if it is not (`install.sh:37-40`). Step 3 runs `rebrand-macos.sh` on Darwin, tolerating failure with `|| true`.

Step 1b is the interesting one, and it is placed between the base fetch and the symlink for a specific reason. It runs `scripts/build-hooks-editor.sh` unconditionally, and its comment states what happens without it: `lib/hooks-editor/` “is a gitignored esbuild artifact and `bin/zwire` loads the extensions straight out of the repo, so on a fresh clone the Hooks / Commands / Triggers pages come up with NO editor at all: each only mounts one if (`window.HooksEditor`) and the `<script src>` 404s — silently, nothing on screen says why.”

The developer path and the packaged path fail differently here, which is why the same build is invoked from both. `localinstall.sh` stages a copy and can assert afterwards that the bundle arrived (`localinstall.sh:237-238`); `bin/zwire` loads the working tree directly, so there is no staged copy to assert against and the only defence is to have built it before the first launch. Step 1b is that defence, and it costs a fresh clone a dependency on node ≥ 20 and pnpm — stated in the comment rather than discovered at runtime.

# Chapter 67 — What a Version Bump Breaks

Chapter 21 gives the rebase economics: three risk tiers, the anchors that shift every milestone, the recurring tax. That chapter is about the twenty-seven patches. This one is about everything *else* that is pinned to a Chromium version — the parts that are not in `fork/patches/` at all, do not produce a `.rej`, and mostly fail by working differently rather than by failing.

## Two version numbers moving at different speeds

The project tracks Chromium twice, and the two clocks are independent by design. `fork/CHROMIUM_VERSION` reads `150.0.7871.46` and is the tag every patch was authored against. `fetch-base.sh` defaults to `LAST_CHANGE` and therefore floats. The consequence is directly observable in the current staged tree: the nested browser in `dist/stage/zwire.app/Contents/Resources/browser/Chromium.app` reports `CFBundleShortVersionString 152.0.7946.0`, two milestones ahead of the pin.

That gap is not a bug in either path; it is the shape of having both. But it means that a claim measured against one base is not automatically true of the other, and the book contains at least one measurement that is explicitly scoped: the MV3 cached-worker experiment recorded in `localinstall.sh:306-313` is stated as “Measured against this Chromium (150.0.7871.46)” — 197,490 bytes served over `chrome-extension://` while the worker actually parsed was the old 196,642-byte script, surviving a full restart. Whether Chromium 152 still does that is not something the repository knows. The mitigation (`stamp_service_worker`’s content-keyed shim) is version-independent, so nothing breaks either way; but the *justification* for the mitigation has a version attached to it.

## The extension-side surface, which has no `.rej`

Six things outside `fork/patches/` are bound to Chromium’s behaviour, and none of them fail the way a patch fails.

**The launch switches.** `bin/zwire:91-101` passes `--extensions-on-chrome-urls`, `--test-type`, `--disable-features=NtpFooter`, and `--enable-features=SplitViewHorizontal,SplitViewTabRestore`. Feature names are internal identifiers with no compatibility guarantee: when a feature graduates and its flag is removed, the switch is silently ignored, and `NtpFooter` reappearing at the bottom of the new tab is the only symptom. `--extensions-on-chrome-urls` is the switch that lets the HUD’s content scripts touch `chrome://` pages at all (Chapter 14’s debug adapter depends on it entirely), and it has no replacement if it goes.

**--load-extension itself.** `fetch-base.sh:7` records that it was “removed only from *branded* Chrome 137+.” The snapshot path is safe because the snapshot is unbranded, and the fork path is safe because `args.gn` sets `is_chrome_branded = false`. Both bases are unbranded for reasons that predate this concern, which is luck rather than design, and it is the single upstream decision that could end the project’s loading model outright.

**minimum\_chrome\_version.** `zpwchrome`’s manifest declares `127`; neither `hud-internal` nor `newtab` declares one. That floor moves only upward and only deliberately, so it is not a bump hazard — but it is the one

place the project states a compatibility range at all, and it means `zpwchrome` will refuse to load on a base older than 127 while the other two will attempt to load anywhere and fail at runtime instead.

**The two allowlists keyed by extension identity.** Patch `0016` widens `_permission_features.json` so `hud-internal` — identified by its id hash, `DCE846DF...` — gets `developerPrivate` and `settingsPrivate`. Patch `0023` adds a second allowlisted feature variant in `_api_features.json` for `tabCapture` without the manifest permission, and defaults `GetAllowlistedExtensionID()` to `omcggnjf...` when `--allowlisted-extension-id` is unset. These are JSON and C++ respectively, and both encode an assumption about *how Chromium spells allowlists* that is more fragile than the code around them. The failure mode is specific: if the allowlist schema changes shape, the patch may still apply and the privileged API is simply absent at runtime, so the HUD's Extensions and Settings pages come up empty with no build-time signal.

**The WebUI token map.** Appendix F lists the `--color-sys-*`, `--cr-fallback-color-*` and `--cr-*` custom properties the theme injector overrides on native `chrome://` pages. Every one of those is an internal design-system token name. Chromium renames them on its own schedule, and a renamed token does not error — it simply is not overridden, so one native page comes back in stock colours while the rest stay themed. There is no gate on this in the repository and no way to write one without rendering the pages.

**The DevTools patch.** `0007` edits three files in the DevTools front end, two of them TypeScript: `front_end/design_system_tokens.css` (an append), `front_end/entrypoints/main/main-meta.ts` (the `ui-theme` setting), and `front_end/ui/legacy/theme_support/ThemeSupport.ts` (`applyTheme`). Being TypeScript in a separately-built front end means this patch has a compile gate the CSS-only patches lack, but also that the front end can be restructured wholesale between milestones in ways the browser's own C++ never is.

## Inside the series, the parts that behave differently

Three properties of the patch set determine how a bump actually goes, and they are visible from the earlier chapter's inventory.

The first is concentration. Five patches — `0002`, `0020`, `0021`, `0025`, `0027` — edit `chrome/browser/ui/color/chrome_color_mixer.cc`, and four of them extend a single function, `AddZwireHudFinalMixer`, that `0002` introduces. A milestone that reworks the colour-mixer pipeline does not produce five independent conflicts; it produces one problem with five entangled instances, and they must be re-anchored in series order because each later patch's context includes the earlier patches' additions. `toolbar_view.cc` has the same structure at a smaller scale with `0004` and `0011`. The other thirty target files are single-owner and can be re-anchored in any order.

The second is that `git apply --3way (apply-patches.sh:33)` is doing real work here. Plain `git apply` fails on a context shift; `--3way` falls back to a merge using the blob ids in the diff and can succeed where the context has moved, leaving conflict markers rather than a `.rej` when it cannot. That is more forgiving and correspondingly quieter — a `--3way` apply that “succeeds” still warrants reading the result.

The third is the one the series file already tells you: twenty-two of the twenty-seven entries carry a `VERIFIED` note, six of those record `git-apply+reverse`, and only the three audio patches carry `compiled + runtime`. The reverse-verified ones are the patches the documented rebase procedure can safely back out of an old tree; for the rest, `fork/patches/README.md` step 2 offers “or start clean” as the alternative, which is the honest fallback.

## The one thing that survives a bump for free

`fork/tests/run_dsp_tests.sh` does not read a Chromium checkout. It extracts the DSP engine out of `0022`'s diff text, compiles it against a 73-line shim with `clang++ -std=c++20`, and runs nineteen invariant checks. Its subject is the patch, not the tree, so a Chromium bump cannot break it and cannot make it pass falsely either. If the audio patch has to be re-anchored, the test tells you whether the re-anchored engine still

behaves — which for the highest-risk patch in the series is the one question a `git apply --check` cannot answer.

That asymmetry is the practical lesson of the whole rebase. Chapter 21 notes that the worst outcome is a patch that “applies and is *wrong*.” The only defence built into this repository against that outcome is a test whose input is a diff.

## The order of operations

Assembling the above with the recipe in `fork/patches/README.md`, a bump is:

```
1 bump fork/CHROMIUM_VERSION
2 fork/apply-patches.sh SRC --reverse (reverse order; or start from a clean tree)
3 fork/fetch.sh (checkout tags/$VERSION -B zwire-$VERSION)
4 fork/apply-patches.sh SRC (--3way; read the successes, not only the
  ~ rejects)
5 scripts/check-patch-series.sh (series and disk still 1:1)
6 fork/tests/run_dsp_tests.sh (the audio engine still satisfies 19 invariants)
7 fork/build.sh SRC (the compile gate – the only proof the C++ is
  ~ valid)
8 fork/package.sh OUTDIR (writes base.path + a fork-<tag> base.version)
```

Steps 5 and 6 are the two that need no checkout and cost seconds. Step 7 is the one that costs hours and is the only one that proves the patches compile. Nothing in the list checks the six extension-side bindings of the first section, because nothing can, short of launching the browser and looking — which is what Chapter 20’s CDP verification layer exists to do, and the reason it exists at all.

## The only check that can see the extension-side bindings

Steps 5 through 7 of that list verify the patches. Nothing in it verifies the six bindings of this chapter’s first section, and nothing can, because every one of them fails at *runtime* in a browser that started successfully. A missing feature flag, a renamed design-system token, a reshaped allowlist, an API removed from a manifest key — none of these produce a compile error, a `.rej`, or a nonzero exit anywhere in the build.

The only instrument in the repository that operates at that layer is the CDP verification set: eight `.mjs` probes at the repo root — `vkeys`, `vovl`, `vpl`, `vpop`, `vpos`, `vsys`, `vterm`, `vwin` — totalling 176 lines, 15 to 28 lines each. Appendix L covers what each one asserts. What matters after a version bump is their shape, which is identical across all eight and visible in the first six lines of any of them: fetch `/json/version` from `127.0.0.1:9222`, open the returned `WebSocketDebuggerUrl`, and drive the session with a six-line request/reply multiplexer:

```
const cmd=(m,p,s)=>new Promise(r=>{const k=++i;w[k]=r;
  ws.send(JSON.stringify({id:k,method:m,params:p||{},sessionId:s}));});
```

No client library, no test framework, one `Host: localhost` header because the DevTools endpoint rejects other host headers. A probe is small enough to read in full and to write a new one in minutes, which is the property that matters here: after a bump, the questions that need answering are not the ones the existing probes already answer. They are new questions — is `--extensions-on-chrome-urls` still honoured, did `--disable-features=NtpFooter` still take, is `chrome.developerPrivate` still defined on the HUD’s Extensions page, did `--color-sys-surface` survive under that name — and each is a two-line `Runtime.evaluate` against a running browser.

That is the honest close to the rebase story. `check-patch-series.sh` proves the series is complete, `run_dsp_tests.sh` proves the largest patch still satisfies its invariants, `fork/build.sh` proves the C++ compiles, and none of the three has any visibility into whether the browser that results still behaves like the one the extensions were written against. Between the build gate and the user there is one tool, and it is eight small files that talk to a debugger port.

# Chapter 68 — The Stamp, Audited

Chapter 23 records a version stamp and the portability trap underneath it: an in-place `sed` in the installer that silently no-ops when the GNU build shadows the system one, which “has shipped a release whose System page read one version while its manifest read another.” Chapter 48 builds a whole classification on top of that incident. Neither chapter answers the question a reader of either would ask next, which is whether the stamp is right *now*.

This chapter answers it by reading every place a version appears at the commit this edition is set against — `439ecb6052`, `zwire/package.json` at `0.6.59` — and by re-deriving each value with a command rather than repeating one. It is a narrow chapter on purpose. The value of an audit is entirely in whether it was actually run.

## Every location, and what it holds

```
$ git -C zwire log -1 --format='%H %ci'
439ecb60528d474770a6dacf076a36cfe551bca1 2026-08-09 20:26:37 -0400
```

Fifteen files in the tree carry a version. Grouped by what consumes them:

| consumer                    | file                                                     | value                      |
|-----------------------------|----------------------------------------------------------|----------------------------|
| the release + the cover     | <code>package.json</code>                                | <code>0.6.59</code>        |
| the System page (dev path)  | <code>extensions/hud-internal/pages/version.js:10</code> | <code>0.6.59</code>        |
| Chromium's extension reload | <code>extensions/hud-internal/manifest.json</code>       | <code>0.6.29</code>        |
|                             | <code>newtab/manifest.json</code>                        | <code>1.0.13</code>        |
|                             | <code>extensions/zpwrchrome/manifest.json</code>         | <code>0.22.40</code>       |
|                             | <code>extensions/zpwrchrome/theme/manifest.json</code>   | <code>0.22.40</code>       |
|                             | <code>theme/manifest.json (never loaded)</code>          | <code>1.0.0</code>         |
| cargo                       | <code>native/zwire-host/Cargo.toml</code>                | <code>0.3.15</code>        |
|                             | <code>zpwrchrome/zpwrchrome-host/Cargo.toml</code>       | <code>0.10.3</code>        |
|                             | <code>lib/zgui-core/crate/Cargo.toml</code>              | <code>0.1.0</code>         |
| npm/pnpm only               | <code>extensions/hud-internal/package.json</code>        | <code>0.4.0</code>         |
|                             | <code>extensions/zpwrchrome/package.json</code>          | <code>0.22.40</code>       |
|                             | <code>lib/zgui-core/package.json</code>                  | <code>1.0.61</code>        |
| the fork's patch anchors    | <code>fork/CHROMIUM_VERSION</code>                       | <code>150.0.7871.46</code> |
| the host's own replies      | <code>src/lib.rs:89 env!("CARGO_PKG_VERSION")</code>     | <code>0.3.15</code>        |

The one pair that is *supposed* to agree is the first two, because `version.js` hardcodes the number the System page prints and an extension cannot read its containing bundle's `CFBundleVersion` at runtime. At this commit they agree: `package.json` line 3 reads `0.6.59` and `pages/version.js` line 10 reads `var ZWIRE_VERSION = '0.6.59';`. The stamp is consistent right now.

Everything else on that list is independently versioned by design and its disagreement with `package.json` carries no information. `hud-internal`'s manifest sitting at `0.6.29` against a release of `0.6.59` looks like drift and is not: Chapter 23 establishes that the installer overwrites the *staged* manifest with a content-derived version, so the tracked value is a floor nobody bumps. `zwire-host` at `0.3.15` is a submodule with its own release cadence. `theme/manifest.json` at `1.0.0` has never moved because the launcher has never loaded it.

## The drift that did happen, two releases ago

The interesting result is not the current state; it is what the same comparison shows across history. Walking every commit that touched `package.json` and reading both files out of each tree:

```
commit      package.json version.js hud-manifest
af7d557aab  0.6.59      0.6.59      0.6.29
4d58d6067a  0.6.58      0.6.58      0.6.29
af1b549041  0.6.57      0.6.56      0.6.29      <-- drift
ca30291b03  0.6.56      0.6.56      0.6.29
96e9950d9f  0.6.55      0.6.55      0.6.29
...
cb6a50a0b6  0.6.40      0.6.40      0.6.29
```

Twenty release commits, nineteen of them in step, one not. At `af1b549041` — the commit whose own subject line reads `chore(release): 0.6.57 - content-keyed shim worker + ...` — `package.json` moved to `0.6.57` and `pages/version.js` stayed at `0.6.56`. It was re-synced at the next release and has been correct since.

That is the same failure the earlier incident produced, arriving through a different door. The old one was a stamp that ran and silently did nothing. This one is a stamp that was never asked to run, because the developer path does not stamp at all: `bin/zwire` loads `extensions/hud-internal` straight out of the working tree, so the System page on a developer's machine prints whatever is committed in `version.js`. For the length of the 0.6.57 cycle, a `zwire` launched from the checkout reported `0.6.56`.

The size of that consequence is small and the shape of it is the point. Chapter 48's rule is to record, for each number, where a mismatch appears. For this one the answer is: on a HUD page, in a field labelled `Version`, on the developer's machine only, with no other symptom. There is nothing in the tree that would have caught it, and nothing that would catch the next one — the two files are joined by a comment on `version.js:9` that reads `// Keep in sync with package.json "version"`. and by nothing else.

## The sed census

The earlier incident's mechanism was `sed -i ''`. The correct statement about the tree today is a negative one, and negatives have to be probed rather than assumed:

```
$ git -C zwire grep -n -- 'sed -i'
NONE
```

There are no in-place `sed` invocations anywhere in the repository. Every in-place edit is `perl -i`, and there are seven of them:

```
scripts/localinstall.sh:245      var ZWIRE_VERSION = '...' -> $VERSION
scripts/localinstall.sh:255      manifest "version"        -> $VERSION    (if $. < 12)
scripts/localinstall.sh:299      manifest "version"        -> $base.$rev (if $. < 12)
scripts/localinstall.sh:344      manifest "service_worker" -> the content-keyed shim
scripts/localinstall-linux.sh:178 __HUD_ID__ / __ZPWR_ID__ / __NEWTAB_ID__ in the
    ~ launcher
.github/workflows/homebrew-cask.yml:67 the tap cask's version line
.github/workflows/homebrew-cask.yml:68 the tap cask's sha256 line
```

The `sed` that remains is stream-only, and it is worth listing rather than waving at, because “we removed the dangerous ones” is a claim a reader should be able to check:

```

bin/zwire:84          sed -n 's/.*"path".../1/p' read the host manifest's
  ~ path
fork/apply-patches.sh:23 sed -E strip comments from the series file
fork/tests/run_dsp_tests.sh:38,47 sed 's/^+//' strip diff markers off the extracted
  ~ engine
scripts/release-macos.sh:47 sed -n '2,26p' "$0" print the script's own header as
  ~ --help
scripts/test.sh:24,42 sed 's/^/ /' indent captured output
extensions/hud-internal/scripts/test.sh (17 more of the same indent form)

```

None of those substitutes into a file, so none of them can silently no-op the way the in-place form did — a stream `sed` that matches nothing produces its input unchanged and the caller sees an empty capture rather than an unmodified file. `bin/zwire:84` is the one in the launch path, and its failure mode is benign in a way worth stating: if the pattern misses, `NHPATH` is empty, the `-x` test fails, and the launcher re-runs the host setup — the same branch a missing manifest takes.

## The guard that cannot see what it is named for

One more artifact belongs in this audit because it looks like the answer to the whole problem and is not. `scripts/release-macos.sh` reads the built bundle's version back and compares it:

```

BUILT_VER="$(defaults read "$STAGE_APP/Contents/Info.plist" CFBundleShortVersionString ...)"
[[ "$BUILT_VER" == "$VERSION" ]] \
  || cyber_warn "built .app is $BUILT_VER but releasing $VERSION - version.js/package.json
  ~ out of sync?"

```

Two things are wrong with it as a stamp check, and the second is structural.

The first is that a mismatch is a `cyber_warn`, not a `cyber_fail` — the script continues into `pkgbuild`, `ditto`, and the release upload. A version disagreement it does detect still ships, with one coloured line in the middle of a long banner-heavy log.

The second is that it cannot detect the disagreement its own message names. `VERSION` is read from `package.json` at `release-macos.sh:65`; the plist it compares against was written by `localinstall.sh:550-551` from `VERSION`, read from `package.json` at `localinstall.sh:32`. Both sides of the comparison come from the same field of the same file, so `BUILT_VER != VERSION` only when the plist write itself failed. `version.js` is not an input to either side. The check is a plist-write assertion wearing a version-drift label, and the label is the reason nobody has noticed it is not one.

The honest form of the check the message promises is a one-line comparison of `package.json`'s version against the `ZWIRE_VERSION` literal in the *tracked* `pages/version.js` — the two values this chapter's history table showed diverging — run before the build rather than after it. Nothing in the tree does that today, on any path.

# Chapter 69 — One Installer Stamps; Two Do Not

Chapter 23 describes the install-time stamping as the answer to a class of silent failures: a System page that disagrees with its release, an extension Chromium declines to reload, and — the one that cost the most to find — a service worker served from cache while the pages around it were re-read from disk. It describes four rewrites and a cache eviction, and it presents them as what “the installer” does.

There are three installers and a launcher. This chapter counts which of them carry that machinery, because the answer is one, and the consequence is that the entire cached-worker defence is a property of macOS packaged builds rather than a property of zwire.

## The count

Grepping each installer for the four mechanisms Chapter 23 names:

| script                           | lines | stamp_<br>content_ver | stamp_<br>service_wkr | ZWIRE_<br>VERSION | SW-cache<br>eviction |
|----------------------------------|-------|-----------------------|-----------------------|-------------------|----------------------|
| scripts/localinstall.sh          | 599   | 4                     | 5                     | 2                 | 4                    |
| scripts/localinstall-linux.sh    | 216   | 0                     | 0                     | 0                 | 0                    |
| scripts/localinstall-windows.ps1 | 251   | 0                     | 0                     | 0                 | 0                    |
| bin/zwire                        | 101   | 0                     | 0                     | 0                 | 0                    |

`scripts/localinstall.sh` — the macOS self-contained installer — is the only file in the repository that contains any of it. The Linux installer assembles the same three extensions into `~/.local/opt/zwire`, and the Windows installer assembles them into `%LOCALAPPDATA%\zwire`, and neither one touches a manifest version, a worker filename, the System page’s literal, or the profile’s worker cache. Its own `perl -i` (line 178) rewrites the three extension ids into the generated launcher and nothing else.

The launcher is the case with the widest reach. `bin/zwire` is what the dev loop runs, what `install.sh` symlinks onto `PATH`, and what every developer uses every day. It contains no stamping and no cache eviction; the eviction lives at `scripts/localinstall.sh:494-507`, in the section that *generates* the bundle’s launcher, so it ships inside `/Applications/zwire.app/Contents/MacOS/zwire` and nowhere else.

## What that means on each path

Read against Chapter 23’s own account of the failure, the three paths sort cleanly.

**The packaged macOS path** has the full defence. The staged manifest gets a content-derived fourth component so identical code produces an identical version and changed code does not; the worker is pointed at a content-keyed shim so its registration URL moves whenever the code does; and the generated launcher deletes `$PROFILE/Default/Service Worker` and `$PROFILE/Service Worker` when the marker changes, so a registration left pointing at a shim `rsync --delete` has removed cannot survive.

**The Linux and Windows packaged paths** have none of it. Both stage the extensions with the tracked manifest versions — `0.6.29`, `1.0.13`, `0.22.40` — which are exactly the values that do not move between releases. Chapter 23’s measured symptom is therefore live on those platforms by construction: install a new build over an old one, and the extension version Chromium sees is unchanged, so its reload heuristic has

nothing to react to. Whether the browser then serves a cached worker is a question about that Chromium build; what the tree guarantees is only that nothing on those two paths is trying to prevent it.

The **developer path** is the interesting one, because it is the path with the highest change rate. `bin/zwire` loads the extension directories in place, so `background.js` is whatever is on disk this second, and the manifest version is whatever is committed. A worker edit therefore has no version signal at all behind it — which is precisely why `extensions/hud-internal/scripts/dev.sh` exists as a wrapper around `localinstall.sh`, and why Chapter 22’s dev-loop note that “a `background.js` change needs the reload glyph on `chrome://extensions`” is a manual step rather than an automated one. The glyph is the whole mechanism.

## Why the asymmetry is not obviously wrong

It would be easy to file this as three-quarters of a feature and move on, and that would misread it. Each of the four mechanisms exists to defeat a cache that a *reinstall over an existing install* leaves behind, and the three paths have genuinely different exposure to that.

The macOS path installs into `/Applications` over a previous copy, with a per-user staged tree the launcher `rsyncs` with `--delete` on every launch. That is the exact arrangement that produces a dangling worker registration, and Chapter 23 records that it did.

The developer path never has a stale staged copy, because there is no staged copy — the browser reads the working tree. Its cache problem is real but it is one file deep and one click wide, and a developer already has the extensions page open.

The Linux and Windows paths sit in between and are the ones where the gap is a gap rather than a difference: they do stage, they do overwrite, and they carry unchanged manifest versions while doing it.

What the tree does not have is any statement of that reasoning, which is the reportable part. The macOS installer carries thirty lines of comment explaining, with measured byte counts, why every one of these mechanisms is necessary. The Linux installer carries no comment saying it is unnecessary there, no comment saying it is missing, and no comment saying the question was considered. A reader comparing the two files cannot tell a decision from an omission, and that is the condition under which an omission becomes permanent.

## The smallest thing that would fix the reading

The mechanisms are four shell functions and a marker file. Two of them — `stamp_content_version` and `stamp_service_worker` — are self-contained enough to be sourced rather than re-implemented, which is the same reuse discipline Chapter 33 applies to JavaScript and Chapter 23 applies to the Monaco build (a nine-line forwarder at the repo root so “the four packaging paths ... run one build with one set of checks”). Nothing about the version stamps resists the same treatment. The Windows path would need a PowerShell port, which is the honest cost, and is the same cost the Monaco build already paid there.

The alternative that would also be defensible, and would be better than the current ambiguity, is a comment in each of the two other installers stating that the stamps are deliberately macOS-only and why. Chapter 48’s rule was to write down where a mismatch appears. This is the same rule applied one level out: write down where a mechanism *does not run*, because a mechanism whose absence is undocumented reads, to the next maintainer, as a mechanism that was forgotten.

# Chapter 10 – What the Frame Reader Does With a Lie

Appendix G gives the wire format in one line: a four-byte little-endian length followed by that many bytes of UTF-8 JSON. Chapter 15 gives the implementation in one sentence — `src/proto.rs` is “the whole framing.” Neither says what happens when the four bytes are a lie, and that is the only interesting question a framing has, because a length prefix is a promise the sender makes and the reader has no way to check before acting on it.

This chapter is that behaviour, measured. Every row below is a byte stream fed to the host’s native-messaging mode on stdin, with `HOME`, `ZWIRE_STATE`, `TMPDIR`, and `ZWIRE_HOST_SOCKET` redirected into a scratch tree and `ZWIRE_BUS_NO_DAEMON=1` set, so nothing here touched a live configuration.

## The reader, in eleven lines

```
pub fn read_native<R: Read>(r: &mut R) -> Option<Value> {
    let mut len = [0u8; 4];
    r.read_exact(&mut len).ok()?;
    let n = u32::from_le_bytes(len) as usize;
    if n == 0 || n > MAX_MSG {
        return None;
    }
    let mut buf = vec![0u8; n];
    r.read_exact(&mut buf).ok()?;
    serde_json::from_slice(&buf).ok()
}
```

That is `proto.rs:98-108`, with `MAX_MSG` defined at `proto.rs:19` as 128 MiB. Read it as a specification and three properties fall out before any byte is sent.

There is **no magic**. Nothing identifies the stream as this protocol; the first four bytes on the wire are consumed as a length whatever they are. A sender speaking a different protocol into this pipe is not rejected, it is misparsed.

There is **one failure value for six distinct failures**. EOF, a short prefix, a zero-length frame, an out-of-range length, a truncated body, and invalid JSON all return `None`, and `transport.rs:37` treats `None` as the loop’s terminating condition:

```
while let Some(msg) = read_native(&mut stdin) {
    sess.handle(&out, &msg);
}
```

So every malformed input ends the session. There is no branch that answers a bad frame with an error and continues.

And the **allocation precedes the validation of the body**. `vec![0u8; n]` is sized from the prefix alone, before a single byte of the body has been seen.

## The matrix

Seventeen streams, each piped into the host, each with the process's exit code, the byte count it wrote to stdout, and whatever it wrote to stderr:

| case            | rc | stdout | stderr |                                     |
|-----------------|----|--------|--------|-------------------------------------|
| valid           | 0  | 252    | (none) | one framed reply                    |
| two             | 0  | 504    | (none) | two framed replies                  |
| zerolen         | 0  | 0      | (none) | 0-length prefix, then a VALID frame |
| truncbody       | 0  | 0      | (none) | len=100, 10 bytes, EOF              |
| oversize_max    | 0  | 0      | (none) | len=0xFFFFFFFF                      |
| over_128mib     | 0  | 0      | (none) | len=MAX_MSG+1                       |
| shortprefix     | 0  | 0      | (none) | 2 bytes then EOF                    |
| badutf8         | 0  | 0      | (none) | 4 bytes of 0xff..0xfc               |
| notjson         | 0  | 0      | (none) | "not json at all"                   |
| jsonscalar      | 0  | 30     | (none) | 12345                               |
| jsonarray       | 0  | 30     | (none) | [1,2,3]                             |
| nocmd           | 0  | 37     | (none) | {"id":9}                            |
| unknowncmd      | 0  | 67     | (none) | {"cmd":"no_such_command_at_all"}    |
| good_then_bad   | 0  | 252    | (none) | valid, garbage, valid               |
| good_then_trunc | 0  | 252    | (none) | valid, then len=4096 with 5 bytes   |
| bigendian       | 0  | 0      | (none) | length written big-endian           |
| magicbytes      | 0  | 0      | (none) | "ZWIRE\0\0\0"                       |

Every row exits zero. No row writes anything to stderr. Nothing in this table distinguishes “the peer closed the port normally” from “the peer sent a length that could not possibly be real,” which is the single most consequential fact in the chapter: to the process that spawned this host, a clean shutdown and a protocol violation are the same event.

## The four results worth walking

**A zero-length frame is a kill switch.** The `zerolen` stream is a four-byte prefix of zero followed by a complete, valid `ping` frame. The reply count is zero: `n == 0` returns `None` at `proto.rs:102`, the loop ends, and the valid frame behind it is never read. Four zero bytes from any source in the stream silently discard everything after them.

**The connection dies mid-conversation, having already answered.** `good_then_bad` sends a valid frame, then eight bytes of garbage, then a second valid frame. The host writes 252 bytes — the first reply, in full — and then stops. A caller that got its first answer and is waiting on the second waits forever, or until it notices the pipe closed, and there is nothing in the protocol that tells it which frame was the last one honoured. `good_then_trunc` is the same shape with a truncated body instead of garbage and produces the identical 252 bytes.

**A wrong byte order is indistinguishable from a hang-up.** `bigendian` writes the correct length for a correct JSON body, big-endian. Read little-endian, `0x000000015` becomes `0x15000000` — 352,321,536, over the cap — so the host returns `None` and exits zero having said nothing. A client on a big-endian platform, or one that used `htonl` by habit, gets a host that appears to start cleanly and immediately close. `magicbytes` is the same outcome from a different cause: “ZWIR” read as a little-endian `u32` is 1,380,142,930, also over the cap.

**Well-formed JSON of the wrong shape is handled properly.** This is the one part of the surface that behaves the way a protocol should. A JSON scalar and a JSON array both get a real reply and the connection survives:

```
jsonscalar    -> {"err":"empty","ok":false}
jsonarray     -> {"err":"empty","ok":false}
nocmd         -> {"err":"empty","id":9,"ok":false}
unknowncmd    -> {"cmd":"no_such_command_at_all","err":"unknown_cmd","ok":false}
```

The `id` is echoed when present, per `proto.rs:83-88`, and `unknown_cmd` names the command back. Once a frame parses, the dispatcher is careful. The gap is entirely in the layer below it.

## The asymmetry with NDJSON, which is the design's own answer

The same file implements the socket framing, and it makes the opposite choice, with the reason in the doc comment (`proto.rs:110-125`):

```
/// Read one NDJSON message: the next non-blank line parsed as JSON. `None` on
/// EOF. Blank lines are skipped; malformed lines yield `Some(Value::Null)` so
/// the caller can reply with an error rather than dropping the connection.
```

A malformed line on the socket becomes `Value::Null`, reaches the dispatcher, and comes back as an error reply on a connection that is still open. A malformed frame on stdio becomes silence and a closed pipe. The two paths feed one dispatcher, as Chapter 15 says, but they disagree about what a client is owed when it is wrong, and the NDJSON side is the side that got it right.

The native side could adopt the same shape without changing the wire format, because the distinction is available at the point of failure: a body that was read completely and failed to parse is a *recoverable* error, and only a length that cannot be trusted forces the stream to end. `read_native` currently collapses both into `None` by applying `.ok()?`  to the parse at `proto.rs:107`. Separating them would cost one enum and would turn the most common malformation — a producer that emitted bad JSON — from a silent hang-up into an error frame the caller can log.

## What the 128 MiB cap actually costs

The cap is real and the guard's comment is accurate about its purpose: it stops a bogus prefix from asking the process to buffer the address space. It is worth stating what it does *not* cost, because “a four-byte input triggers a 128 MiB allocation” invites a conclusion the measurement does not support.

Feeding a prefix declaring exactly `MAX_MSG` and then holding the pipe open, the host's resident set three seconds later:

```
PID    RSS    VSZ  COMM
34911  7120 435439952  zwire-host
```

7.1 MB resident. `vec![0u8; n]` on this platform is a zeroed mapping the kernel backs lazily, so the pages become resident only as `read_exact` fills them — which means the memory cost tracks the bytes actually delivered, not the bytes declared. The virtual size is large and meaningless. The honest statement is that the cap bounds an allocation the allocator does not charge for until it is used, and the amplification a reader might expect from the code is not present in the behaviour.

What the in-range case *does* cost is the same silence as everything else: a prefix declaring 128 MiB followed by eight bytes and EOF exits zero with no output, because `read_exact` fails and `.ok()?`  swallows it. A sender that got its length arithmetic wrong by a factor of a million and a sender that finished talking produce identical observable behaviour.

# Chapter 71 – Six Ways In, and What Each One Returns

Chapter 15 describes the host's transports and Chapter 54 corrects the count from three to four. Those are the ways a *message* reaches the dispatcher. This chapter is one layer out: the ways a *process* is started, what each mode does, and what it hands back to whoever started it — because the launcher, the installers, and any wrapper script a user writes all depend on that exit code, and one of the six modes is the mode every mistake lands in.

The list is derived from the argument handling in `src/lib.rs`, not from the help screen, because the two do not agree.

## The six arms

| arm                  | lib.rs  | behaviour                                                                                                 |
|----------------------|---------|-----------------------------------------------------------------------------------------------------------|
| bus-daemon           | 209-211 | the detached singleton that owns the zgui bus; runs before <code>ensure_stryke_app</code> , never returns |
| serve                | 222-234 | seed <code>global.toml</code> , ensure the bus daemon, run the NDJSON socket/pipe daemon forever          |
| call                 | 235-245 | connect, send one request, print reply frames                                                             |
| version --version -V | 246-248 | <code>println!("zwire-host {VERSION}")</code>                                                             |
| help --help -h       | 249-251 | print the styled usage screen                                                                             |
| _ (everything else)  | 252-262 | seed <code>global.toml</code> , ensure the bus daemon, run native messaging on stdio                      |

Five named arms and a catch-all. The catch-all is the important one and the comment above `run` at `lib.rs:172-175` states why it exists: “Any unrecognised first token falls through to native-messaging mode, because Chrome launches the host with extension-origin arguments we must ignore.”

That is a correct requirement. Chromium invokes a native-messaging host with the host manifest path and the calling extension's origin as `argv`, and a host that rejected unknown arguments would refuse to start under the browser. The catch-all is the standard way to satisfy it.

The consequence is that the host has no invalid invocation.

## Measured exit codes

Each mode run with stdin closed, in the same redirected scratch environment as the previous chapter:

```

invocation          rc  stdout / stderr
zwire-host version    0  zwire-host 0.3.15
zwire-host -V         0  zwire-host 0.3.15
zwire-host --help     0  the usage screen
zwire-host call '{"cmd":...}' 1  zwire-host: connect ...: No such file or
    (no daemon running)      directory (os error 2) (is the daemon
                               running? `zwire-host serve`)

zwire-host frobnicate </dev/null 0  (nothing)
zwire-host --not-a-flag </dev/null 0  (nothing)
zwire-host serv      </dev/null 0  (nothing)

```

The last three rows are the finding. `frobnicate` is a nonsense subcommand, `--not-a-flag` is a nonsense flag, and `serv` is a one-character typo of the mode a person most often wants. All three exit zero, print nothing on either stream, and do nothing — because all three fell through to native messaging, read an immediate EOF from `/dev/null`, and returned from the loop.

Set beside the failure the same tree already documents, the shape is familiar. Chapter 23 records a Monaco bundle whose absence rendered as an empty pane because every mount site was written `if (window.HooksEditor)`. This is the process-level version: a mistyped mode renders as a command that ran and succeeded.

Concretely, a supervisor unit or a shell wrapper written as

```
zwire-host serv --socket "$SOCK" || { echo "host failed to start" >&2; exit 1; }
```

prints nothing, exits zero, and leaves no daemon. The error handling is correct and the diagnosis it produces is that the daemon started fine.

## The flag parser eats forward

Flags are pulled out of the argument list before the mode is chosen ([lib.rs:186-204](#)), and four of them consume the next token unconditionally:

```

"--tcp"    => tcp    = it.next(),
"--token"  => token  = it.next(),
"--name"   => name   = it.next(),
"--peer"   => { if let Some(p) = it.next() { peers.push(p); } }

```

`it.next()` takes whatever is there. It is not checked for a leading dash and it is not checked against the set of known flags, so a flag typed in the wrong place swallows the argument after it, whatever that argument was meant to be. The visible case is `call`:

```

$ zwire-host call --tcp '{"cmd":"hostinfo"}' </dev/null
zwire-host: connect .../h.sock: No such file or directory (os error 2)
rc=1

$ zwire-host call --tcp '{"cmd":"hostinfo"}'          # stdin is a pipe that stays open
(no output; still running after 6s)
rc=124 (killed by the harness)

```

`--tcp` consumed the JSON, so `positional[1..]` is empty, so [lib.rs:237-243](#) takes the read-from-stdin branch and calls `read_to_string`. With stdin closed that returns immediately and the empty request produces a connect error. With stdin open — a terminal, a pipe, a supervisor's null-but-not-closed handle — the process blocks until EOF, forever, with no prompt and no timeout.

The same `read_to_string` is why `zwire-host call` cannot be driven interactively at all: the documented form `echo '...' | zwire-host call` works because the pipe closes, and a session that wanted to send a request and then keep the connection open for streamed frames cannot use the stdin form, because the request is not sent until the caller's stdin reaches EOF.

There is a second, quieter effect of parsing flags before modes: the flags are global, so `zwire-host serve --stream` is accepted and silently ignored (`serve` never reads `follow`), and `zwire-host version --socket /nonexistent` is accepted and prints the version. Nothing warns that a flag was irrelevant to the mode it was given to.

## The undocumented mode

`bus-daemon` is dispatched at `lib.rs:209` and appears nowhere in the help screen's MODES section, which lists five entries: the bare invocation, `serve`, `call`, `call --stream`, and `version | help`. Its absence is defensible — it is an internal subcommand that `zbus::ensure_daemon` spawns, and Chapter 54 explains at length why a user should never run it by hand. It is worth recording here only because the help screen is the surface a reader would use to enumerate the modes, and enumerating them from the help screen gives five of six.

The reverse discrepancy is smaller and points the other way: the help screen advertises `-s/--socket`, `-f/--stream`, `--tcp`, `--token`, `--name`, `--peer`, `-h`, and `-V`, which is exactly what the parser accepts, plus the aliases `--follow` (accepted at `lib.rs:193`, undocumented) and `help/version` as bare words (accepted, and documented).

## What a correct wrapper has to do

The practical consequence for anyone scripting the host is that the exit code is not a mode check, so a wrapper has to assert the *effect* instead. For the daemon that is a liveness probe rather than a return value:

```
zwire-host serve --socket "$SOCK" &
for _ in $(seq 40); do
    zwire-host call --socket "$SOCK" '{"cmd":"ping"}' >/dev/null 2>&1 && break
    sleep 0.1
done
```

which is, not coincidentally, the shape `zbus::ensure_daemon` already uses for the other socket — Chapter 54 records it spinning “up to 200 iterations of a 10 ms sleep, a bounded two seconds — until a connect succeeds,” precisely because starting a process is not evidence that the process is serving. The host applies that discipline to the daemon it spawns and does not offer it to anyone spawning the host.

# Chapter 12 — Two Daemons, One Path

Chapter 54 is one of the best-argued chapters in this book. It walks an intermittent `Connection refused` (`os error 61`) bug down to its cause — ownership of a socket handed, at random, to the shortest-lived process in the system — and it walks the fix up from there: a dedicated `bus-daemon` singleton, an advisory `lock`, a private temp bind, and an atomic `rename`, with the reasoning stated in the source:

The lock serializes check-and-bind “so two starters can’t both bind (the loser’s `remove_file` would otherwise unlink the winner’s live socket — last writer wins the path).”

That is `zbus.rs:645-651`, and it is exactly right. This chapter is about the other socket — the one the daemon transport actually serves on, the one `zwire-host` `serve` binds, the one Chapter 15 calls “what lets tmux, emacs, shell scripts, and any language talk to the host without a client library.” It never received that fix, and the sentence above describes its behaviour precisely.

## Nine lines

```
pub fn serve(sock: &Path) -> ! {
    if let Some(parent) = sock.parent() {
        let _ = std::fs::create_dir_all(parent);
        let _ = std::fs::set_permissions(parent, std::fs::Permissions::from_mode(0o700));
    }
    // A stale socket file from a previous run blocks bind(); clear it.
    let _ = std::fs::remove_file(sock);
    let listener = match UnixListener::bind(sock) {
        Ok(l) => l,
        Err(e) => die(&format!("bind {}: {e}", sock.display())),
    };
};
```

`transport.rs:99-109`. The comment is true of a *stale* file and says nothing about a live one, and `remove_file` cannot tell the difference — a Unix domain socket is a filesystem entry that outlives its listener, which is the whole reason the stale case exists and the whole reason the live case is indistinguishable from it. There is no liveness probe before the unlink, no lock around the check-and-bind, and no temp-bind-and-rename. The second starter wins the path.

## The transcript

Two daemons, one socket, one PTY session and one connected client on the first:

```
A=72784 A's pty child=73642
who answers (pid): 72784
--- start daemon B on the SAME socket path ---
B=74592 B stderr: zwire-host: listening on /private/tmp/.../h.sock
A alive: YES B alive: YES
who answers now (pid): 74592
A's pty child still alive: YES
A's streaming client still connected: YES
```

Read it in order. A is serving and a `hello` sent to the path is answered by pid 72784. B is started on the same path and prints the same success line A printed — the identical `zwire-host: listening on ... at transport.rs:113`, with nothing to distinguish a first bind from a hijack. Both processes are alive. The path now resolves to B. A still holds its PTY child and its connected client, and no new client can ever reach it again.

Neither process has any way to learn this. A does not know its path was unlinked; its listener is still valid and still accepting on an inode with no name. B does not know it displaced anything, because from B's side there was no socket at the path by the time it bound — B removed it.

## Two corrections this makes to earlier chapters

Chapter 36's field guide says, of a daemon that will not start: "the cause there is almost always permissions: the socket is `0600` inside a `0700` dir, so a socket owned by another user, or a stale socket file from a crashed daemon, blocks a new one until it is removed." Chapter 40 repeats it: "a stale socket file from a crashed daemon blocks a new one until it is removed, which Chapter 36 lists as a real failure mode."

A stale socket file does not block a new daemon. `transport.rs:105` removes it unconditionally before every bind, so the stale case — the one both chapters describe as the failure — is the one case the code handles automatically. The genuine failure is the opposite one, and neither chapter names it: a *live* daemon's socket is removed just as readily, and the symptom is not a daemon that will not start but two daemons that both did.

The other-user case in Chapter 36 survives the correction and is worth restating precisely, because the mechanism is different from what the sentence implies: a socket owned by another user is not removed (the `remove_file` fails, discarded by `let _ =`), so the subsequent `bind` fails with `EACCES`, and *that* is what produces the `die("bind ...")` at `transport.rs:108` — the only path in the daemon that exits non-zero.

## Why the fix that exists next door was not applied here

The two sockets are in one binary, and one of them is hardened. The most likely reading is chronological rather than deliberate: `zbus.rs` was written to solve a bug that had already been observed in production, with a 30-line comment recording it, while the daemon socket has never produced a report — because starting two `serve` daemons by hand is not something that happens in the browser's own use of the host, which is native messaging on stdio.

That reading also says where it *would* happen. Chapter 40 walks the multi-instance case and concludes that "the daemon path is the one where two instances genuinely share a process rather than each having their own — and where a stale socket file from a crashed daemon blocks a new one." Replace the last clause with what the code does and the paragraph gets sharper: the daemon path is where two instances share a *path*, and the second one takes it. Two `ZWIRE_STATE` trees do not help, because the socket location is deliberately not under the state dir — `lib.rs:91-99` puts it in `$XDG_RUNTIME_DIR` or `$TMPDIR`, per-user rather than per-install, with the doc comment explaining that "a socket is ephemeral runtime state, so it deliberately does not live under the app-data/state dir." Two independent `zwire` installs on one machine therefore share one daemon socket path by design, and the second `serve` silently evicts the first.

The remedy is already written, forty lines away, in a function that takes a directory and a socket path and returns an `Option<UnixListener>`. `bind_bus` at `zbus.rs:653-677` probes liveness under the lock, binds a private `<name>.<pid>.tmp`, `chmods` it `0600`, and renames it over the target — and its comment notes that "the listener survives the rename on macOS/Linux (verified)." Every one of those steps applies unchanged to `transport.rs`'s socket. What would have to be decided is the policy the bus already chose: `bind_bus` returns `None` when a live owner exists, and the caller exits zero. For `serve` the correct policy is arguably the opposite — a person who typed `zwire-host serve` and finds a daemon already running should be told, on `stderr`, with a non-zero exit — but either policy is a policy, and the current behaviour is not one.

## The TCP listener fails in the other direction

The same daemon can take `--tcp <addr>`, and a bind failure there is handled, but handled quietly. `peer.rs:152-165`:

```
pub fn listen_tcp(addr: String) {
    std::thread::spawn(move || {
        let listener = match TcpListener::bind(&addr) {
            Ok(l) => l,
            Err(e) => {
                eprintln!("zwire-host: tcp bind {addr}: {e}");
                return;
            }
        };
    });
};
```

The error is printed and the *thread* returns. The daemon carries on. Started against a loopback port already held by another process:

```
--- stderr ---
zwire-host: listening on /private/tmp/.../h.sock
zwire-host: tcp bind 127.0.0.1:65033: Address already in use (os error 48)
--- daemon alive? YES ---
--- local socket still works? ---
{"id":3,"ok":true,"peers":[],"self":"codelabs-arm"}
```

Two properties are visible there. The daemon is fully functional locally with its peering listener silently absent, and the state that reports on peering cannot express the difference: `{"peers":[],"ok":true}` is what a host with no peers yet returns and what a host that failed to open its peering port returns. Chapter 61 establishes that a peer link carries bus federation and remote requests; a fleet configured for federation whose TCP bind lost a race comes up looking healthy, answers `peers` truthfully-but-uselessly, and simply never federates.

The other property is the ordering. The bind happens on a spawned thread while the main thread proceeds to the local accept loop, so the two `stderr` lines race — here the success line for the local socket printed first and the TCP failure second, and the reverse ordering is equally available. A wrapper that reads the last line of `stderr` to decide whether the daemon came up correctly is reading a coin flip.

# Chapter 13 – SIGINT, and What Dies With the Host

Chapter 15 lists what a session holds: PTY sessions keyed by id, file watchers keyed by id, pub/sub subscriptions, background jobs, peer links. Chapter 45 adds that the daemon is “the longest-lived thing in the system” and holds all of it “until something cancels them.” The teardown story the book tells is the drop story — [transport.rs:54](#) notes that when a connection closes, “[sess](#) drops here, tearing down its PTYs/streams/subs.”

That is true of a connection ending. It is not true of the process ending, and the difference is a signal.

## There are no signal handlers

```
$ grep -rn 'signal\\|SIGINT\\|SIGTERM\\|ctrlc\\|sigaction' src/
src/procs.rs:53 the `kill` command's own signal parameter
src/procs.rs:66 ...
src/exec.rs:16 a doc comment about a child killed by a signal
src/api.rs:27 the same
src/zbus.rs:709 a comment about the bus daemon's process group
```

Every hit is either the host signalling something else or prose. Nothing installs a handler, and nothing needs to for correctness in the ordinary case — but it means the five [Drop](#) implementations the crate does carry never run when a signal arrives:

```
src/session.rs:66 impl Drop for Session
src/sysmon.rs:39 impl Drop for Monitor
src/stryke_lsp.rs:111 impl Drop for StrykeLsp
src/watch.rs:32 impl Drop for Watcher
src/pty.rs:151 impl Drop for PtySession
```

A default-disposition SIGINT terminates the process without unwinding. Every one of those destructors is skipped.

## What actually happens

The daemon, started under job control so it keeps default signal dispositions, holding one PTY session running a marker process, with a streaming client attached:

```

[before] daemon=61235  pty child: 61964
[before] reply: {"id":1,"ok":true,"spawned":true}
[after INT] daemon alive: NO
[after INT] pty child: GONE
[after INT] streaming client exit code: 0
[after INT] socket file: srw----- /private/tmp/.../h.sock
[after INT] call against the orphaned socket file:
zwire-host: connect .../h.sock: Connection refused (os error 61) (is the daemon running?
↳ `zwire-host serve`)
rc=1

```

Four results, and they are not all the same kind of thing.

**The PTY child dies, and not because the code killed it.** `PtySession`'s destructor did not run. The child went away because the process exiting closed the pseudo-terminal master, and the kernel delivered `SIGHUP` to the session leader on the slave side. The outcome is correct and the mechanism is the operating system's, which matters for one reason: it is the outcome on Unix, and the Windows path has no equivalent — there is no controlling-terminal hangup to inherit. The book cannot claim from this transcript that a signalled host reaps its terminals; it can claim that on macOS and Linux the OS reaps them for it.

**The socket file survives.** No handler, no destructor, no cleanup — the `srw-----` entry outlives the process that bound it. This is the stale-socket case Chapter 36 describes, and Chapter 72 establishes it is harmless for the next `serve`, which removes it. It is not harmless for a client: `call` against it gets `Connection refused (os error 61)`, which is the exact error string `zbus.rs`'s crate documentation cites as the symptom of the bug it was written to fix. The same string, from the same cause — a socket path outliving its owner — is still produced by the other socket, and the message the client prints, “is the daemon running? `zwire-host serve`”, is the right advice for it.

**The streaming client exits zero.** This is the one that will mislead someone. The client had an open stream, the server was killed under it mid-session, and the process reports success. `transport.rs:70-88` is why:

```

loop {
    let mut line = String::new();
    match reader.read_line(&mut line) {
        Ok(0) | Err(_) => break,
        Ok(_) => { ... if !follow { break; } }
    }
}
std::process::exit(0);

```

`Ok(0)` is a clean EOF and `Err(_)` is a broken connection, and both `break` into the same unconditional `exit(0)`. There is no path by which `zwire-host call --stream` reports a severed stream. A monitoring script that streams `sysinfo_start` and restarts on failure will never restart, because there is never a failure — the process simply returns, having streamed for an hour or for one frame, with the same status.

That is the sharpest instance of a pattern this part of the book keeps finding, and it has a one-line fix with a real design question behind it: `exit(0)` on `Ok(0)` and a non-zero exit on `Err(_)` would distinguish them, but only for the client, because the server closing cleanly and the connection breaking both surface as `Ok(0)` to a reader on a socket whose peer went away. Distinguishing them properly requires the protocol to say goodbye — a terminating frame the client can look for — which the NDJSON path has no notion of.

## The daemon does not need a handler; the report needs one

It is worth resisting the conclusion that a `SIGINT` handler should be added. On Unix the outcome is already correct: the terminals die, the connections close, the process exits. A handler that ran the destructors would

remove the socket file and would make the shutdown marginally tidier, and it would introduce the class of bug where a handler runs while a background thread is mid-write.

What is missing is not a handler but an *exit status that carries information*, and that is a property of the client, not the daemon. The daemon dying is observable to anyone watching the process. A stream ending is observable only to the client, and the client currently declines to say.

# Chapter 74 – The Suite, Counted and Run

Chapter 22 describes zwire’s testing posture as a syntax gate plus each extension’s own suite, and defends the modesty of that at length. Chapter 20 describes the CDP probes. Chapter 59 describes the wire-conformance corpus, Chapter 63 the DSP extraction test, and Chapter 64 the toolkit’s own tests. Each of those is accurate about the thing it describes. What no chapter does is put a number on the whole, and a testing posture is not really stated until someone has run it and reported what came back.

Everything below was run at the commit named in the front matter, on this machine, with the wall-clock times as measured. Nothing is quoted from a CI badge.

## The count

| suite                                  | files | tests | passed | failed | wall   |
|----------------------------------------|-------|-------|--------|--------|--------|
| zwire-host cargo test                  | 6     | 60    | 60     | 0      | 4.76s  |
| src/*.rs unit tests                    | 4     | 26    | 26     | 0      |        |
| tests/protocol.rs                      | 1     | 25    | 25     | 0      |        |
| tests/wire_conformance.rs              | 1     | 8     | 8      | 0      |        |
| doc-tests (src/api.rs)                 | –     | 1     | 1      | 0      |        |
| hud-internal scripts/test.sh           | 22    | 22    | 22     | 0      | 6.32s  |
| + node --check gate                    | 68    | –     | –      | 0      |        |
| zwire root scripts/test.sh             | –     | –     | –      | 0      | 39.80s |
| + node --check gate                    | 13    | –     | –      | 0      |        |
| fork/tests/run_dsp_tests.sh            | 1     | 19    | 19     | 0      | 3.05s  |
| scripts/check-patch-series.sh          | –     | 27    | 27     | 0      | 0.23s  |
| zgui-core node --test webui/*.test.cjs | 25    | 203   | 203    | 0      | 0.59s  |

Everything passes. There is no red anywhere in the tree at this commit, and the total wall time to run every one of the rows above is under a minute on an M-series laptop.

Two rows need reading carefully, and they are the two rows that are not what they look like.

## The 39.8 seconds that contain the 6.3

`scripts/test.sh` at the repository root is a wrapper. Its own JS gate checks 13 files; its EXTENSION SUITES section then shells into each bundled extension and runs that extension’s `pnpm test`. So the root suite’s 39.8 seconds is its own gate plus `hud-internal`’s 6.3 seconds plus `zpwirchrome`’s suite, which runs its own JavaScript tests and then `cargo test --manifest-path zpwirchrome-host/Cargo.toml`. The three numbers are nested, not additive, and the top-level figure is dominated by a submodule’s Rust build.

`cargo test` for `zwire-host` is *not* in that chain. `extensions/hud-internal/scripts/test.sh` builds the host — its final step reports `[DONE] // zwire-host builds` — and does not test it. The host’s 60 tests run only when someone runs `cargo test` in the host directory by hand. That is a defensible split, since the host is a submodule with its own CI, and it is worth stating because “run the tests” at the repository root does not run them.

## The 13 files, and what the gate says about itself

The root gate's header (`scripts/test.sh:2-4`) says it is “a JS syntax gate over the repo's own front-end (new-tab + theme + root scripts).” Chapter 22 repeats it: “It runs `node --check` over the repo's own front-end (the new-tab page, the theme, the root scripts).”

The `find` that produces the file list is `scripts/test.sh:26-27`:

```
done < <(command find newtab theme -name '*.js' \
  -not -path '*/node_modules/*' -not -path '*/lib/zgui-core/*' 2>/dev/null)
```

Two search roots, one glob, and the result is thirteen files — all of them under `newtab/`:

```
newtab/background.js      newtab/palette-cmds.js
newtab/cmd-defaults.js    newtab/palette.js
newtab/lib/zgui/command-palette.js  newtab/sandbox/js-run.js
newtab/lib/zgui/fzf.js     newtab/scheme-sync.js
newtab/lib/zgui/util.js    newtab/schemes.js
newtab/newtab.js           newtab/tmux-host.js
                           newtab/tmux-pane.js
```

`theme/` contributes zero, because a Chrome theme extension is a manifest and images and contains no JavaScript at all — the search root has never matched anything and never will. And “root scripts” are not in the search path: `find` is given `newtab` and `theme` and not `.`, so the repository root is never walked, and even if it were, `-name '*.js'` does not match `.mjs`.

The eight CDP probes are therefore never syntax-checked:

```
vkeys.mjs 22  vovl.mjs 21  vpl.mjs 17  vpop.mjs 25
vpos.mjs 15  vsys.mjs 25  vterm.mjs 23  vwin.mjs 28      176 lines total
```

Running `node --check` on them by hand at this commit, all eight parse cleanly. The gap is latent, not live — which is the right thing to report, and also the reason it has stayed open. Chapter 22's argument for why a parse check is disproportionately valuable in this project is that “a content script with a syntax error does not throw into anything the user sees; Chromium declines to inject it and the feature is simply absent.” That argument is about content scripts and does not transfer to the probes, which are Node scripts that fail loudly. The honest correction is not that the probes urgently need gating; it is that the script and the chapter both describe a scope two-thirds larger than the one the code searches.

The extension's own gate is the one doing the work, and it is scoped correctly: `extensions/hud-internal/scripts/test.sh:26-28` walks `.` excluding `node_modules`, `lib/zgui-core`, `dist`, and `.git`, which is 68 files.

## The 22 that run are the 22 that exist

The one place the count is unambiguous and clean. Comparing the files on disk with the files the runner invokes:

```
on disk: 22   invoked: 22   on disk but never run: (none)   run but missing: (none)
```

`bracenav`, `browser-tools`, `cleardata`, `compute`, `exec-chain`, `expose`, `feeds`, `history-delete`, `history-render`, `history`, `hud-accel`, `notes`, `pageactions`, `pipes`, `settings-render`, `settings-sections`, `spec-roundtrip`, `tabquery`, `translate`, `trigger-revert`, `undo`, `urlsurgery`. No orphans in either direction. For a suite invoked by a hand-written shell script rather than a glob, that is worth recording, because the failure mode of hand-written invocation lists is a test file that stops being run and keeps being maintained.

## The 203 tests nothing here runs

Chapter 64 describes the shared toolkit's test suite and gives its size as "22 `.test.cjs` files totalling 2,691 lines." At the pointer this superproject records, it is 25 files and 3,162 lines, and running them:

```
i tests 203    i pass 203    i fail 0    i skipped 0    i duration_ms 590.263167
```

Two hundred and three tests, all passing, in six-tenths of a second. Nothing in `zwire` runs them. `lib/zgui-core/package.json` has no `scripts` block at all, so there is no `pnpm test` to delegate to; both syntax gates explicitly exclude the directory (`scripts/test.sh:27` and `extensions/hud-internal/scripts/test.sh:27`); and CI does not enter it.

That is the correct division of labour under Chapter 33's reuse rules — a submodule's tests are the submodule's CI's problem, and re-running them here would produce failures nobody in this repository can fix. It is worth writing down anyway, for two reasons. The first is that Chapter 64 reads, as written, as though the suite is part of `zwire`'s verification story, and it is not part of anything that runs in this tree. The second is the number itself: 203 tests at 590 milliseconds is the cheapest suite in the entire project by two orders of magnitude, and the argument for excluding it — cost, ownership, noise — is weakest exactly where the suite is cheapest. A single `node --test` line would turn "the toolkit was fine when its own CI last ran" into "the toolkit is fine at the pointer this superproject records," which is a different and more useful claim, and the one Chapter 48 spends a section arguing nothing currently checks.

## What none of it tests

The row that is absent from the table at the top is the one Chapter 20 already names. None of the sixty-plus Rust tests, twenty-two JavaScript suites, nineteen DSP assertions, or two hundred and three toolkit tests drives a browser. They test the host's dispatcher against an in-memory connection, the extension's pure functions against stubs, the DSP engine against a shim, and the patch series against the filesystem. The eight probes are the only artifacts in the repository that assert on a running `zwire`, they require a browser launched by hand with a debugging port, they are excluded from every gate, and — per Chapter 20's own accounting — two of their assertions are dead.

The suite is green and it is worth what it costs. What it does not do is what Chapter 67 already concluded: "between the build gate and the user there is one tool, and it is eight small files that talk to a debugger port."

# Chapter 75 — What CI Does Not Look At

Chapter 22 gives CI as three checks and defends their modesty: shellcheck over the shell scripts, JSON validation over the tracked manifests, and the patch-series consistency gate. It is right that the third is the valuable one and right that the first two are hygiene that matters unusually much in a project made mostly of shell and JSON.

This chapter reads the workflow rather than the description, runs each gate locally, and reports what falls outside them. Two of the three are narrower than they read; one is exactly what it claims, and the reason to say so is that it is the one a reader would guess is broken.

## The whole workflow

```
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: false

      - name: shellcheck (all repo shell scripts)
        run: |
          shellcheck --version
          shellcheck -S warning \
            bin/zwire \
            scripts/*.sh \
            fork/*.sh

      - name: validate JSON
        run: |
          git ls-files '*.json' '!:extensions/zpwrchrome/**' | while read -r f; do
            python3 -c "import json,sys; json.load(open(sys.argv[1]))" "$f"
            echo "ok: $f"
          done

      - name: patch series ↔ disk consistency
        run: scripts/check-patch-series.sh
```

That is [.github/workflows/ci.yml](#) in full — forty lines, one job, three steps, no matrix, no cache, no Rust, no Node. `submodules: false` is deliberate and Chapter 22 explains it well: nothing CI checks needs the power-tool, and running without it incidentally exercises the graceful-skip path a non-recursive clone produces.

## The JSON gate does fail, which is worth verifying rather than assuming

A `while` loop whose last statement is `echo "ok: $f"`, at the tail of a pipeline, is the classic shape that swallows an error — the loop's exit status is the status of the last command it ran, and that command always succeeds. It looked like a finding. Replicating the step verbatim under `bash -e`, which is the shell GitHub Actions uses for a `run` block on Linux, with a deliberately malformed file in the middle of the list:

```
json.decoder.JSONDecodeError: Expecting property name enclosed in double quotes: line 1
  ~ column 2 (char 1)
STEP EXIT (clean)=1
```

Exit 1. The `-e` propagates into the pipeline's subshell, the failing `python3` aborts it before the `echo`, and the pipeline's status is the subshell's. The gate works. It is recorded here because the alternative — leaving a plausible-looking hazard unexamined in a chapter about what CI misses — would have been exactly the kind of unverified claim this book keeps flagging elsewhere.

The step's real limit is its scope, and the scope is stated honestly in its own comment: every tracked `*.json` outside the power-tool submodule, discovered with `git ls-files` and never hardcoded. Since submodules are not checked out, `newtab/`, `extensions/hud-internal/`, `theme/`, and `fork/` manifests are covered and the toolkit's are not.

## shellcheck's glob misses eleven tracked scripts

The step lints three globs: `bin/zwire`, `scripts/*.sh`, and `fork/*.sh`. Tracked shell scripts outside those globs:

```
extensions/hud-internal/scripts/build-hooks-editor.sh
extensions/hud-internal/scripts/build.sh
extensions/hud-internal/scripts/bust.sh
extensions/hud-internal/scripts/clean.sh
extensions/hud-internal/scripts/cyberpunk.sh
extensions/hud-internal/scripts/dev.sh
extensions/hud-internal/scripts/localinstall.sh
extensions/hud-internal/scripts/nuke.sh
extensions/hud-internal/scripts/rebuild.sh
extensions/hud-internal/scripts/test.sh
fork/tests/run_dsp_tests.sh
```

Eleven files, ten of them a parallel dev-loop family inside the extension and one of them the DSP test runner, which sits in `fork/tests/` rather than `fork/` and so falls outside `fork/*.sh` by one directory level.

Running the CI gate's own command at the CI gate's own severity over the covered set passes clean. Running it over the uncovered set does not:

```
extensions/hud-internal/scripts/bust.sh:6:1: warning: Use 'cd ... || exit' ... [SC2164]
extensions/hud-internal/scripts/clean.sh:4:1: warning: Use 'cd ... || exit' ... [SC2164]
extensions/hud-internal/scripts/nuke.sh:6:1: warning: Use 'cd ... || exit' ... [SC2164]
extensions/hud-internal/scripts/test.sh:6:1: warning: Use 'cd ... || exit' ... [SC2164]
```

Four findings, all the same one: an unguarded `cd`. In three of the four it is cosmetic. In `nuke.sh` it is not, and it is worth spelling out because it is the case SC2164 exists for. The script opens with

```
cd "${dirname "$0"}/.."
```

and later runs

```
command rm -rf dist
```

with a relative path. If the `cd` fails — a deleted working directory, a permissions change, an invocation through a broken symlink — the shell continues in whatever directory it was already in and the `rm -rf` names a `dist` there instead. The repository's own top-level scripts get this right: `scripts/test.sh:6` is `cd "$(dirname "$0")/.." || exit 1`. The extension's copy of the same family does not, and CI has never looked at it because of a glob.

The fix is a one-word change to the workflow — `git ls-files '*.sh'` in place of three hardcoded globs, which is the same discovery discipline the JSON step already uses two lines below it and the same discipline `check-patch-series.sh` uses to derive its own count. That the two steps in one file disagree about whether to enumerate or to discover is the most concrete thing in this chapter.

## What CI runs of the previous chapter's table: none of it

Set the workflow against the suites that exist:

| suite                                                | tests | run in CI                 |
|------------------------------------------------------|-------|---------------------------|
| <code>scripts/check-patch-series.sh</code>           | 27    | yes                       |
| shellcheck over three globs                          | —     | yes (partial scope)       |
| JSON validation over tracked manifests               | —     | yes                       |
| zwire-host cargo test                                | 60    | no                        |
| hud-internal 22 .mjs suites                          | 22    | no                        |
| zgui-core 203 toolkit tests                          | 203   | no                        |
| <code>fork/tests/run_dsp_tests.sh</code>             | 19    | no                        |
| <code>scripts/test.sh</code> (the repo's own runner) | —     | no                        |
| the 8 CDP probes                                     | —     | no (needs a live browser) |

Three hundred and four assertions exist, all pass, all run in under a minute, and CI runs none of them. Two of the omissions are structural and defensible: the probes need a browser someone launched, and the toolkit is a submodule CI does not check out. The other three are not structural. `run_dsp_tests.sh` needs `clang++` and no Chromium checkout and takes three seconds — Chapter 63 calls it “the only part of the fork that can be verified without a 100 GB checkout” and “the only patch in the series whose *behaviour*, not merely its context, is under test,” and it runs nowhere automatically. `cargo test` for the host needs a Rust toolchain the runner image already has. `scripts/test.sh` needs Node and npm.

Chapter 22 argues that CI “is intentionally lightweight: lint, validate, check consistency. It does not build the browser (there is nothing to build) or compile the fork (that needs the 100 GB checkout).” Both of those parentheticals are true and neither is a reason not to run tests that require neither. The gap between the stated rationale and the actual scope is one workflow step wide, and the three-second DSP test is the clearest instance: it is cited in two chapters as the fork's only behavioural proof, and nothing ever runs it but a person who remembers to.

# Chapter 76 – Exit Zero, and the Things It Does Not Mean

The previous eight chapters found the same thing from eight directions, and it is worth collecting rather than leaving distributed, because the pattern is more useful than any instance of it. In each case a component finished, reported success, and had not done the thing its caller believed it had done.

None of these is a crash, and that is the point. A crash is self-reporting. Everything below is a process that returned zero.

## The inventory

|                                          |                                     |
|------------------------------------------|-------------------------------------|
| what returned 0                          | what it did not do                  |
| zwire-host serv                          | start a daemon                      |
| zwire-host <any typo>                    | anything at all                     |
| zwire-host (malformed frame on stdin)    | report that the frame was malformed |
| zwire-host call --stream (server killed) | report that the stream was severed  |
| zwire-host serve (socket already served) | refuse, or warn, or notice          |
| zwire-host serve --tcp <bound port>      | open the peering listener           |
| setup-native-host.sh                     | verify the binary it wired          |
| release-macos.sh (version mismatch)      | stop the release                    |
| scripts/test.sh                          | check the eight probes, or theme/   |

Four of those are recorded above in their own chapters. The remaining two are worth their own transcripts.

## The native-host setup wires whatever it is told

`scripts/setup-native-host.sh` builds the host and writes the profile-relative manifest that Chromium reads. It is the step `bin/zwire:86` invokes when the manifest is missing or its target is gone, and Chapter 19 describes it as what “makes a bare `zwire` self-sufficient on Linux and in dev.”

It has no idea what it is wiring. `BIN` is `${ZWIRE_HOST_BIN:-$HOST_DIR/target/release/zwire-host}` at `setup-native-host.sh:21`, the build at line 33 is skipped when that path is already executable, and the manifest at lines 46-61 is written from `$BIN` with nothing in between:

```
$ ZWIRE_STATE=../nhstate ZWIRE_HOST_BIN=/bin/echo bash scripts/setup-native-host.sh
setup-native-host: manifest -> /bin/echo
EXIT=0

$ cat ../nhstate/profile/NativeMessagingHosts/com.zwire.hud.json
{
  "name": "com.zwire.hud",
  "description": "zwire HUD native host",
  "path": "/bin/echo",
  "type": "stdio",
  "allowed_origins": [
    "chrome-extension://omcgnnjfbmpdlofkfbpddkhnfibfhgg/",
    ...
  ]
}
```

Exit zero, a valid manifest, and three extension origins granted a native-messaging channel to `/bin/echo`. The launcher then reads that manifest back with the `sed` at `bin/zwire:84`, finds the path executable, and skips the rebuild — permanently, because the only condition that triggers a re-run is the recorded path not being executable.

The reachable version of this is not an operator setting `ZWIRE_HOST_BIN` to a joke. It is the gap between lines 39 and 48. The build's success is checked; the *product* of the build is not:

```
39: ( cd "$HOST_DIR" && cargo build --release ) >/dev/null 2>&1 || {
40:   echo "setup-native-host: cargo build --release failed" >&2
48:   cat > "$d/com.zwire.hud.json" <<JSON
```

If `cargo` succeeds and the binary lands somewhere other than the path `BIN` names — a workspace `target` inherited from a parent directory, a `CARGO_TARGET_DIR` in the environment, a cross-compilation triple — the manifest is written pointing at a file that does not exist, and the script exits zero having printed `manifest -> <that path>`. The next launch finds the path non-executable, re-runs the setup, rebuilds (a no-op), rewrites the same dangling manifest, and exits zero again, indefinitely. A single `[[ -x $BIN ]] || die` between those two lines would convert it into a one-line error.

The redirection on line 39 deserves its own sentence. `>/dev/null 2>&1` discards the compiler's entire output, so the diagnostic for any Rust compile error in the host is exactly one line — `setup-native-host: cargo build --release failed` — followed, from `bin/zwire:87`, by `zwire: native host unavailable (terminal/sysinfo disabled)`. Two lines of prose and zero lines of compiler output for a failure whose cause is a compiler error. Chapter 36's field guide sends a reader with a silent host to check the manifest and `allowed_origins`; it cannot send them to the error message, because there is not one.

## The release guard that cannot see the drift it names

Covered in Chapter 68 and repeated here because it belongs in this list: `release-macos.sh` compares the built bundle's `CFBundleShortVersionString` against `$VERSION` and, on a mismatch, emits a warning whose text blames `version.js/package.json` being out of sync. Both sides of that comparison are read from `package.json`, so `version.js` cannot affect it; and the mismatch it *could* detect is a warning rather than a failure, so the release proceeds to `pkgbuild` and upload either way. It is a check that runs, prints, and cannot fail the thing it exists to protect.

## Why this class and not another

The through-line is not carelessness. Every instance above is a deliberate tolerance that outlived the case it was written for.

The catch-all subcommand exists because Chromium passes arguments a native-messaging host must ignore, which is true and necessary — and it makes typos silent. The `remove_file-before-bind` exists because a stale socket really would block a bind, which is true — and it makes a live daemon evictable. `let _ =` on the permission calls in the same function exists because a failed `chmod` should not stop a daemon from serving, which is defensible — and it means the `0700` and `0600` modes Chapter 25 relies on as “the load-bearing security in the whole system” are best-effort. The non-fatal native-host step exists because Chapter 24’s degradation ladder is real and a browser without a terminal is still a browser — and it means a mis-wired host is indistinguishable from an absent one.

Each tolerance is correct in its own frame. What they share is that the tolerance was written where the failure occurs and the consumer of the exit code is somewhere else, so the decision to keep going was made by the component least able to know whether going on was acceptable.

The rule that falls out is Chapter 48’s, one level down. That chapter says: for every version number, record what enforces it and where a mismatch appears. The same discipline applied to a status code reads: **for every path that returns success, record what a caller is entitled to conclude from it.** Applied to the list at the top of this chapter, the answers are short and none of them is what a caller would assume. `zwire-host serve` returning means the process ran, not that it owns the socket. `call --stream` returning zero means the loop ended, not that the stream completed. `setup-native-host.sh` returning zero means a file was written, not that anything works. Written down, each of those is a one-line docstring. Left undocumented, each is a wrapper script that reports a healthy system.

# Chapter 11 – The New Tab Becomes a Layout

Chapter 6 described the new tab as a page: a clock, an omnibox, a row of tiles, and a `localStorage` key you could edit if you wanted different tiles. That is what it was. It is now a **layout** — one of several named designs you keep and switch between, each owning a navigation rail of Speed Dial groups, a background, the Speed Dial appearance block, and its own grid of widgets per group. The tiles chapter is not wrong about the page it described; it describes a page that no longer exists, and this chapter replaces it.

The model is Vivaldi's Start Page, ported setting for setting rather than reinterpreted, which is worth saying plainly because the temptation in a port is to improve the parts you do not understand yet. Where zwire deviates — and it deviates twice, both in wrapping geometry — the deviation is named below with the reason it was necessary.

## What a layout is

A layout is a plain object, and the whole schema fits in a paragraph. It carries a `nav` (which edge the group rail sits on, and whether it shows at all), a `bg` (none, a colour, a CSS gradient, or an uploaded image), a `dial` block (maximum columns, thumbnail size, when titles show, whether the add button shows, whether drag-reorder is on) and a list of `pages` — Vivaldi calls them Speed Dial groups. Each group holds its own dials *and* its own widget grid, which is the one structural liberty this port takes with the original: in Vivaldi the dashboard widgets are a property of the start page, here they are a property of the group, so “work” and “home” can be different surfaces inside one layout rather than one surface with different bookmarks.

The library is a list of those objects plus an `activeId`. Switching layouts is switching that one field.

## One pure module, and the reason it is duplicated

Everything a layout *means* lives in `zntp-core.js`, and it is pure: no DOM, no `chrome.*`, no timers. That is not a stylistic preference. The file is loaded by four different things — the new-tab page that renders it, the inline editor that mutates it, the HUD page in the *other* extension that edits the same data, and the new-tab service worker that re-validates every write — and any behaviour that lived in one of them rather than in the model would be behaviour the other three disagreed about.

It exists as byte-identical copies in `newtab/` and `extensions/hud-internal/`, for the reason Chapter 17 gives about `newtab/lib/`: Chromium loads each unpacked extension from its own directory and an extension cannot read a sibling's files. What is new is that the duplication is now *enforced*. `scripts/test.sh` grew a shared-file parity section that diffs all four duplicated sources — `schemes.js`, `palette-cmds.js`, `cmd-defaults.js`, `zntp-core.js` — and fails the run when the copies drift. Before that gate, editing one copy and forgetting the other silently forked the two surfaces, and nothing said so.

## `normalize()` is the only gate

`normalize()` (`zntp-core.js:204`) is the single path between stored or imported JSON and the renderer, and it is written as a gate rather than a formatter. It clamps every number, enum-checks every string, re-issues

duplicate ids, drops widget types it does not recognise, and guarantees the invariants the UI relies on — a library is never empty, a layout always has at least one group, an add-once widget appears at most once per group.

Two of its checks are security boundaries rather than tidiness, and they run in opposite directions:

```
safeUrl    (zntp-core.js:85)    scheme allow-list: https, http, ftp, chrome,
                                chrome-extension, file. A javascript: dial would run
                                in the new tab's own extension origin; data: and blob:
                                would smuggle markup past the same check. Scheme-less
                                input gets https://, matching the omnibox heuristic.

safeImage  (zntp-core.js:96)    the exact inverse: base64 image data URLs only. A
                                background is an UPLOAD, never a fetch — the page has to
                                paint offline and under the extension CSP — so a remote
                                URL is rejected here and a data:text/html payload with it.
```

That is why import is safe to expose. `importJSON` runs arbitrary pasted JSON through the same `normalize()` the renderer trusts and throws rather than half-applying; a layout file carrying a `javascript:` dial imports as a layout with one fewer dial.

## The widget catalog, and three kinds of data

Eleven widget types, each declaring whether it can be added more than once per group and whether it offers Vivaldi's `Widget Size → Regular | Tall` (`zntp-core.js:40`). Date, Speed Dial and Search are add-once and unsized; Top Sites, Bookmarks, History, Feeds, Notes, Reading List, Sessions and Webpage are the list widgets, and the last four of those are zwire's own — Vivaldi has no widget for a tmux session.

What separates them in implementation is not what they look like but where their rows come from, and `widgets.js` keeps the three classes apart deliberately:

- **Local.** Clock, search, speed dial, webpage. No API, no async, no failure mode.
- **Own permission.** Top sites, bookmarks, history — this extension's own `topSites`, `bookmarks` and `history` permissions, which is why the new-tab manifest grew four entries this edition (Appendix O).
- **The other extension's.** Feeds, notes, reading list, sessions. That data lives in `hud-internal` — its storage, its `readingList` permission, its `<all_urls>` fetch — and an extension cannot read a sibling's storage. Those widgets ask over the same `externally_connectable` channel `palette.js` already used, with one verb: `zbNtpData` (`widgets.js:83`), answered by `ntpData` in the HUD worker (`background.js:125`).

The last one has a detail worth pulling out, because it is the reuse rule of Chapter 33 applied under pressure. The worker needs to parse a feed and title a note to answer those requests, and both parsers already exist — in the HUD *pages*. Rather than grow a second copy in the worker, `background.js:113` `importScripts` the pages themselves: `pages/feeds.js` and `pages/notes.js` now hang their pure halves on `self` when a worker loads them and bail before touching the DOM, exactly as they already did for the headless tests. A feed item is titled identically whether you read it on the Feeds page or in a new-tab widget, because it is the same function.

## Where the layouts live, and who may write them

`chrome.storage.local` under `zb_ntp`, in the **new-tab** extension, is the truth. `newtab.js:21`. That placement is forced: the page that renders layouts needs them on first paint, and it cannot read another extension's storage.

Two consequences follow. First, a `localStorage` mirror (`zb.ntp.cache`, `newtab.js:22`) exists purely so the first paint is synchronous — `chrome.storage` is async, and a new tab that waits for it flashes an

empty page — with the real read landing a moment later and re-rendering. Second, the HUD's layout manager, being in the other extension, cannot write `zb_ntp` directly; it goes through `zbNtpGet / zbNtpSet` (`newtab/background.js:35` and `:46`), and the worker re-runs `normalize()` on every write it accepts. The manager therefore cannot store a layout the page would refuse to draw, even if it were buggy — which is the same argument the fork makes about never trusting a value merely because it came from inside the fleet.

Open new tabs repaint through `chrome.storage.onChanged` (`newtab.js:241`), so a switch made in one tab lands in every other one without a message bus.

Pre-layout installs migrate once, by copy: `migrate()` (`zntp-core.js:253`) reads the old `zb.tiles` array into the default layout's first group and never writes or clears the legacy key, so rolling the feature back leaves the old page exactly as it was.

## The two geometry deviations

Vivaldi's Speed Dial appearance settings port directly: `Maximum Columns` (0 meaning "No limit"), five thumbnail sizes (`THUMB_PX`, `zntp-core.js:501` — 96 through 248 px), titles always / when-needed / never, the add button, drag-to-reorder, and per-dial custom thumbnails. Two things about *wrapping* are not ports, and both were written after looking at the result.

**Rows are balanced, not packed** (`dialColumns`, `zntp-core.js:479`). Six dials plus the add button under a six-column cap packs 6+1, and the stranded tile sits alone under a full row. Balancing lays the same seven out 4+3. The cap is still a maximum — balancing only ever returns fewer columns, and never adds a row.

**The cap is only half of what decides a row** (`dialFit`, `zntp-core.js:492`). Under "No limit", or any cap wider than the window, it is the viewport that wraps, and the model cannot see the viewport. `dialFit` takes measured space and balances against whatever actually fits; the page measures, the model decides. It is re-run on a debounced resize rather than re-rendering, because a full repaint would re-query top sites and history on every pixel of a window drag.

The honest residual: a uniform column count cannot always balance. Seven dials in a window with room for three lay out 3+3+1. The ideal 3+2+2 needs per-row counts, which a CSS wrap does not have, and inventing them would mean laying out rows by hand in JavaScript to fix a case that appears at 620 px.

## Two editors, and why the preview cannot lie

Editing is inline, the way Vivaldi does it: a Widgets picker, a Customize dialog that applies live, and an edit mode with drag-to-reorder for widgets and dials (`layout-edit.js`). Nothing there decides what a valid layout is; it decides what the user is asked, and every mutation goes through a `zntp-core` op.

The HUD gains `pages/newtab.html`, which is the part Vivaldi has no equivalent for: a library. Every layout as a card with an SVG preview, plus duplicate, per-layout export and JSON import. The preview is drawn from `gridPlace` (`zntp-core.js:519`), the same routine that reproduces CSS grid's sparse auto-placement for the page — row-major, the cursor never moves backwards, a widget wider than the row's remainder wraps, a Tall widget occupies two rows that later widgets flow around. The alternative was for the preview to guess, and a preview that guesses is worse than no preview: it is a picture that disagrees with the page and gives no sign that it is disagreeing.

## What is tested, and what is not

`tests/ntp-layout.mjs` carries 141 assertions and runs as its own section of the HUD extension's suite. It loads `zntp-core.js` headlessly through `new Function`, so what is tested is what the shipped file computes, not a mirror of it. The assertions weigh toward the gate — a `javascript:` dial, a `data:text/html` background and an unknown widget type each have to not survive `normalize()`; a stale drag report must not delete a dial;

the balancing must never add a row or end on a row of one. The last two assertions in the file are a contract *across* the two trees: every widget in the catalog has a renderer, and no renderer outlives its catalog entry.

What that suite does not cover is everything with a pixel in it, and the layout bugs this feature shipped with were all pixels: an add button stranded on its own row, a clock left-aligned against a centred page, an omnibox whose GO button escaped its own border because a flex item defaults to `min-width: auto` and would not shrink. Each was found by driving the real browser over CDP and measuring, in the manner of Chapters 68 through 76, and none of them was reachable from the model tests. A pure core is worth having; it is not a substitute for looking at the page.

# Appendix A – File and Directory Map

A single reference for where each thing this book discussed actually lives in the [zwire](#) repository.

```

bin/zwire                                the launcher - resolve base, build ext list, ensure
├─ host, exec                             edition source (version), npm-script aliases
package.json                             zwire.icns, zwire.ico, icon.svg, icon-1024.png
branding/

scripts/
  fetch-base.sh                          download + pin the Chromium snapshot base
  install.sh                             PATH symlink + fetch + Monaco bundle + (macOS) rebrand
  rebrand-macos.sh                       in-place Dock name + icon + re-sign
  setup-native-host.sh                   build zwire-host + write profile-relative manifest
  localinstall.sh                        self-contained /Applications/zwire.app
  localinstall-linux.sh                  self-contained ~/.local/opt/zwire + .desktop
  localinstall-windows.ps1              self-contained %LOCALAPPDATA%\zwire + registry host
  dev / build / rebuild / clean         dev-loop helpers (no compile - assemble base)
  bust / nuke / test / cyberpunk        cache-bust · full reset · syntax gate · banner lib
  build-hooks-editor.sh                  forwarder -> hud-internal/scripts/ (Monaco esbuild)
  check-patch-series.sh                  CI: series file <-> patch files consistency

newtab/
  newtab.{html,css,js}                  the HUD new-tab extension (chrome_url_overrides)
  palette.js  schemes.js                 page, style, logic (clock/tiles/omnibox/focus)
  scheme-sync.js  tmux-pane.js           new-tab command palette + the 8-scheme table
  fonts/                                       scheme follow via native file + tiling surface
                                           locally-vendored display faces (offline-correct)

extensions/
  hud-internal/
    manifest.json                        the internal-HUD content-script suite
    background.js                       MV3: content-script order, commands, permissions
    theme.js  schemes.js                 scheme relay + tab bus + term-open state + cmd seed
    redirect.js  rules/frame_bust.json   chrome:// WebUI skin + the 8-scheme table
    zvim.js  ztmux-config.js  zpowerline.js  vim keys · tmux overlay (ZGui.tmux) ·
├─ statusbar (ZGui.powerline)
  zpalette.js  zfind.js                  global ⌘K palette · fuzzy find (Custom Highlight API)
  zkeys.js  cmd-defaults.js              keybinding registry · seeded ⌘K keyword commands
  pages/
    extensions.js                       extension manager (management + developerPrivate)
    settings.js                         all Chrome prefs (settingsPrivate) + scheme/effects
    history/bookmarks/downloads         the three list pages (shared zg-list.js)
    sessions.js                        durable tmux-session CRUD (zb_tmux_sessions)
    version.js                         HUD chrome://version + live native-host probe
    keys.js  commands.js                keybinding editor · ⌘K keyword-command editor
    extshortcuts.js  ci.js               extension shortcuts · GitHub Actions dashboard
    zg-boot.js  zg-list.js  fzf.js       shared shell · list factory · fuzzy matcher
  lib/term/
  lib/hooks-editor/                     xterm.js terminal (native-PTY bridged)
                                           Monaco bundle for hooks/commands/triggers
                                           (GITIGNORED build artifact -
├─ scripts/build-hooks-editor.sh)
  vendor/zpwr-hooks-editor/
  native/zwire-host                     the esbuild source that bundle is built from
├─ (Rust)                               scheme + ui + sysinfo + PTY + fs/exec native host
  native/zwire-host/src/
                                           the Rust host: transport/proto/session/store/
                                           sysmon/pty/exec/fsops/jobs/watch/procs/osops/
                                           bus/peer/api (submodule, packaged path)
                                           shared fuzzy/palette/fx/fonts toolkit (submodule)
                                           the MV3 power-tool (submodule)
  lib/zgui-core/
  zpwrchrome/

theme/
  a colors-only Chrome theme - present, NOT loaded

fork/
  CHROMIUM_VERSION                      the source-build path (optional alternate base)
  args.gn                               pinned tag: 150.0.7871.46
  fetch.sh  apply-patches.sh            unbranded release GN config
  build.sh  package.sh                  depot_tools + checkout · apply/reverse series
  patches/0001..0027 + series           gn gen + 256 oninja · install as base + rebrand
                                           the twenty-seven HUD patches, apply-clean at the tag

v{keys,ovl,pl,pop,pos,sys,term,win}.mjs  CDP verification probes (port 9222)
github/workflows/ci.yml                   shellcheck + JSON validate + patch-series check

```

# Appendix B — The State Directory (\$ZWIRE\_STATE)

Everything zwire writes at runtime, in one place. The default is the OS application- data location — `~/Library/Application Support/com.menketotechnologies.zwire` on macOS (named by the reverse-DNS bundle id), `${XDG_CONFIG_HOME:-- ~/.config}/zwire` on Linux — resolved by `scripts/state-dir.sh`; the old `~/.zwire` dotdir and the brief bare-zwire macOS folder are legacy locations, auto-migrated once. `$ZWIRE_STATE` overrides it on every platform.

```
$ZWIRE_STATE/                                (default: OS app-data dir; ~/.zwire + bare zwire are
├── legacy, migrated)                         the extracted Chromium snapshot, or the packaged fork
├── base/                                     the extracted Chromium snapshot, or the packaged fork
├── (~325MB)
├── base.path                               absolute path to the resolved binary (launcher reads
├── this)
├── base.version                           "r<REV>" for a snapshot, "fork-<TAG>" for a compiled
├── fork
├── profile/                               the dedicated --user-data-dir
├── (bookmarks/history/sessions)
├── NativeMessagingHosts/
│   ├── com.zwire.hud.json                 the HUD native host manifest (path + allowed_origins)
│   └── Default/NativeMessagingHosts/     second location Chromium also checks
├── ext/                                   per-user writable copy of the loaded extensions (staged
├── at launch)
```

The extension-owned keys inside `profile/` that this book referenced:

```
zb_scheme      the mirrored active scheme (worker <-> content scripts)
zb_keys        user keybinding remaps { action: key }
zb_custom_cmds the %K keyword-command rules
zb_sys         the streamed system-stats sample (statusbar reads this)
zb_term_open   whether the terminal overlay should re-open on the next page
zb_cmd         the storage command bus (wakes the worker for tab ops)
zb.tiles/zb.engine new-tab tiles + search engine (page localStorage, not chrome.storage)
```

The two files at the top level — `base.path` and `base.version` — are the entire contract between the base layer and the launcher. The theme itself lives *outside* this per-install tree, in the fleet-wide `~/.zwire/global.toml` (with the host's `hud-scheme/hud-light` text projections beside it for the fork's native color mixer) — the one channel through which the new-tab extension, the internal HUD, `zpwichrome`, the wider app suite, and (on the fork) the native chrome all agree on one look. `profile/` and `ext/` are the isolation boundary that keeps zwire a guest on the machine. Delete the whole tree and the next `zwire` launch fails with “no base browser installed” until you re-fetch — there is no hidden state anywhere else.

# Appendix C – Keybinding Reference

The full default keymap from [zkeys.js](#), the single source of truth both [zvim.js](#) and the Keyboard settings page read. Every key here is remappable in the settings page (which writes [zb\\_keys](#)), except the fixed native prefix.

```
— Scroll —
j scroll down      k scroll up
h scroll left      l scroll right
d half-page down  u half-page up
G bottom of page  H top of document
M middle of document L bottom of document

— Tabs & history —
J previous tab    K next tab
x close tab      t new tab      r reload
[ history back   ] history forward

— Jump & hints —
f link hints (click)    F link hints (new tab)
g g-prefix: gg top · gt/gT tab next/prev · gi focus input
z z-prefix: zt/zz/zb position ref element top/center/bottom
y y-prefix: yy yank URL
m set mark m<x>        ` jump to mark `<x> (also ')

— Palette & find —
o command palette      : command palette
/ find in page         \ toggle vim off (Ctrl/⌘+\\ re-enables)
⌘K / Ctrl+K open palette    ⌘F / Ctrl+F open find

— Tmux overlay (prefix Ctrl-b, or ⌘b) —
% split right    " split down    o next pane      ; last (prev) pane
h select pane ← j select pane ↓ k select pane ↑ l select pane →
H resize pane ← J resize pane ↓ K resize pane ↑ L resize pane →
z zoom          x close pane      { swap ←        } swap →
O rotate        ! break to window q pane numbers  space cycle layout
F grid 4        G grid 8          I grid 16
c new window    n next window    p prev window   Tab last window
, rename window . move window    ' go to window  w window list
& kill window  f find window    i window info   Enter open a document
m mark / swap  e sync panes    E toggle pane in sync
[ copy mode    ] paste buffer  = choose buffer
s sessions    S save session  M manage saved layouts (editor)
r reload pane  t clock          : palette      d detach (commits the layout) ? help

0-9 select window      ↔↑↓ focus by direction
Ctrl/Alt + ↔↑↓ resize the active pane

— Fixed (native fork) —
Ctrl-b the tmux split prefix (rebindable in the Keyboard page)
```

# Appendix D — The Eight-Scheme Palette

The full dark palettes from `schemes.js`. Every HUD surface styles itself in terms of these twenty CSS variables; each scheme also ships a matched `lightVars` set (omitted here for space) for light mode. Colors are the `vars` (dark) values.

```
cyberpunk  - Hot pink + cyan (default)
  accent #ff2a6d  cyan #05d9e8  magenta #d300c5  green #39ff14  red #ff073a
  text #e0f0ff / dim #7a8ba8 / muted #3d4f6a
  bg #05050a / #0a0a14 / card #0d0d1a / hover #12122a  border #1a1a3e

midnight  - Deep blue + electric purple
  accent #7c3aed  cyan #38bdf8  magenta #6366f1  green #34d399  red #f87171
  text #e2e8f0 / dim #94a3b8 / muted #475569
  bg #020617 / #0f172a / card #1e293b / hover #334155  border #334155

matrix    - Green phosphor terminal
  accent #00ff41  cyan #00ffcc  magenta #008f39  green #00ff41  red #ff3333
  text #d4ffd4 / dim #7ccc7c / muted #406640
  bg #000500 / #001a00 / card #002a00 / hover #003a00  border #004d00

ember     - Burnt orange + amber
  accent #f97316  cyan #fbbf24  magenta #dc2626  green #84cc16  red #ef4444
  text #fed7aa / dim #d97706 / muted #78350f
  bg #0a0503 / #1a0f05 / card #27150a / hover #3a1f0f  border #4a2c17

arctic    - Pale ice + steel blue
  accent #60a5fa  cyan #22d3ee  magenta #818cf8  green #2dd4bf  red #f87171
  text #e0f2fe / dim #7dd3fc / muted #0369a1
  bg #020617 / #0c1424 / card #0f1e33 / hover #1e2f4d  border #1e3a5f

crimson   - Rose-red accent + teal highlight
  accent #e11d48  cyan #2dd4bf  magenta #f43f5e  green #22c55e  red #ff073a
  text #ffe4e6 / dim #b08a92 / muted #6b4a52
  bg #0a0506 / #140a0c / card #1a0d10 / hover #2a1318  border #3e1a22

toxic     - Acid-lime accent + magenta
  accent #c6ff00  cyan #00e5ff  magenta #ff00aa  green #39ff14  red #ff073a
  text #e8ffd0 / dim #8a9a6a / muted #4a5a32
  bg #07090a / #0c0f0a / card #0f130c / hover #161b10  border #2a3a1a

vapor     - Vaporwave pastel pink + cyan
  accent #ff6ec7  cyan #72f1ff  magenta #c792ea  green #5af2b0  red #ff6b8b
  text #f0e6ff / dim #a99cc4 / muted #6a5f86
  bg #0d0814 / #140d1f / card #1a1228 / hover #241836  border #2e2142
```

The twenty keys in full order (`SCHEME_VAR_KEYS`): `--accent`, `--accent-light`, `--accent-glow`, `--cyan`, `--cyan-glow`, `--cyan-dim`, `--magenta`, `--magenta-glow`, `--green`, `--green-bg`, `--red`, `--text`, `--text-dim`, `--text-muted`, `--bg-primary`, `--bg-secondary`, `--bg-card`, `--bg-hover`, `--border`, `--border-glow`.

## The light palettes (lightVars)

Each scheme ships a matched light variant under `lightVars`, and light mode swaps these in for the dark `vars` above. The light sets invert the depth relationship — `--bg-primary` becomes a near-white, `--bg-card` becomes `#ffffff`, the text darkens — while keeping each scheme’s hue identity, with accents deepened for contrast on a light ground. The full light key colors, in the same layout as the dark table:

```
cyberpunk  accent #d6196e  cyan #0891b2  magenta #a300a3  green #15803d  red #dc2626
text #1e293b / dim #475569 / muted #94a3b8
bg #f0f2f5 / #e4e7ec / card #ffffff / hover #f7f8fa  border #cbd5e1

midnight   accent #6d28d9  cyan #0284c7  magenta #4f46e5  green #047857  red #dc2626
text #0f172a / dim #475569 / muted #94a3b8
bg #f1f5f9 / #e2e8f0 / card #ffffff / hover #f8fafc  border #cbd5e1

matrix     accent #16a34a  cyan #059669  magenta #15803d  green #16a34a  red #dc2626
text #14532d / dim #4d7c0f / muted #86efac
bg #f0fdf4 / #dcfce7 / card #ffffff / hover #f7fee7  border #bbf7d0

ember      accent #c2410c  cyan #ca8a04  magenta #b91c1c  green #65a30d  red #dc2626
text #431407 / dim #9a3412 / muted #fdb74
bg #fff7ed / #ffedd5 / card #ffffff / hover #fef3c7  border #fed7aa

arctic     accent #2563eb  cyan #0891b2  magenta #4f46e5  green #0f766e  red #dc2626
text #0c4a6e / dim #0369a1 / muted #7dd3fc
bg #f0f9ff / #e0f2fe / card #ffffff / hover #f0f9ff  border #bae6fd

crimson    accent #be123c  cyan #0d9488  magenta #be185d  green #15803d  red #dc2626
text #1e293b / dim #475569 / muted #94a3b8
bg #faf0f1 / #f2e4e6 / card #ffffff / hover #fdf6f7  border #e0c5cb

toxic      accent #5c8a00  cyan #0891b2  magenta #a3006e  green #15803d  red #dc2626
text #1e293b / dim #475569 / muted #94a3b8
bg #f3f7ec / #e7eedb / card #ffffff / hover #f8fbf2  border #d0dcc0

vapor      accent #d6469e  cyan #0e9bb0  magenta #8b5cf6  green #10a37f  red #e11d48
text #2a1e3a / dim #5a4a72 / muted #9a8ab0
bg #f7f0fb / #ede2f5 / card #ffffff / hover #faf4fd  border #ddc9ec
```

Two families collapse toward a shared light red: every scheme except vapor uses `#dc2626` for `--red` in light mode (vapor keeps a rose `#e11d48`), where the dark palettes each carried their own. That is the one place the light variants converge — a bright neutral red reads as “error” on any light background, so the schemes stop differentiating it.

# Appendix E – Launch Flag Reference

Every flag `bin/zwire` passes to the base, and why. User arguments ("`$@"`") are appended after these, so any Chromium flag you pass overrides or adds to them.

```
--user-data-dir="$PROFILE"      isolate to ~/.zwire/profile (never system Chrome)
--load-extension="$LOAD"        preload newtab + zpwrchrome + hud-internal (the
                                switch unbranded Chromium keeps, branded Chrome
                                dropped in 137) – only dirs with a manifest.json
                                let hud-internal content scripts run on chrome://
--extensions-on-chrome-urls     suppress the automation-restriction warnings the
--test-type                     above flags would otherwise raise
--no-first-run                  skip Chromium's onboarding
--no-default-browser-check      skip the default-browser nag
--homepage="chrome://newtab"    the home button opens the HUD new tab
--disable-features=NtpFooter    drop the new-tab footer under the HUD page
--enable-features=SplitViewHorizontal,SplitViewTabRestore
                                side-by-side split view + restore-on-reopen
--autoplay-policy=no-user-gesture-required
                                let page audio start without a click, so the
                                Audio dashboard's engine (Ch 52) has a signal
"$@"                            everything you typed, passed straight through
                                (e.g. a URL, --incognito,
❧ --remote-debugging-port=9222)
```

That is the whole launcher contract: **ten** HUD-defining flags, then your arguments. (Earlier printings said nine and omitted `--autoplay-policy`; it is `bin/zwire:100`, and Chapter 60 already counted ten. The three generated launchers each pass a different set — `localinstall.sh` substitutes `--restore-last-session` for `--autoplay-policy`, the Linux one adds `--no-sandbox` and carries neither extra, and the Windows `.lnk` carries nine. Chapter 62 tabulates the divergence.) The launcher owns what makes `zwire` `zwire` and gets out of the way of every other Chromium flag — which is why `zwire --remote-debugging-port=9222` (Chapter 20), `zwire --incognito`, and `zwire https://github.com` all just work.

# Appendix F – The Chrome WebUI Token Map

`theme.js` skins the `chrome://` pages that have no bespoke HUD replacement by overriding Chrome's own CSS custom properties, each with `!important`, so the WebUI components recolor without any markup change. It deliberately maps *only* palette and the mono font — dropping the grid lattice, scanlines, glow, and floating picker — so these pages inherit the theme with no HUD chrome on top. The full override map, token to HUD variable:

```

— Chrome design-system tokens (--color-sys-*) —————
--color-sys-surface          -> --bg-primary
--color-sys-surface-variant  -> --bg-card
--color-sys-surface-subtle   -> --bg-secondary
--color-sys-base             -> --bg-primary
--color-sys-base-container   -> --bg-card
--color-sys-base-container-elevated -> --bg-hover
--color-sys-header           -> --bg-secondary
--color-sys-header-container -> --bg-secondary
--color-sys-on-surface        -> --text
--color-sys-on-surface-subtle -> --text-dim
--color-sys-on-surface-variant -> --text-dim
--color-sys-on-surface-secondary -> --text-dim
--color-sys-primary          -> --cyan
--color-sys-on-primary        -> --bg-primary
--color-sys-primary-container -> --bg-hover
--color-sys-on-primary-container -> --accent
--color-sys-secondary         -> --accent
--color-sys-outline           -> --border
--color-sys-tonal-container   -> --bg-hover
--color-sys-tonal-outline     -> --border
--color-sys-neutral-container -> --bg-card
--color-sys-inverse-surface    -> --text
--color-sys-state-hover-on-subtle -> --bg-hover

— Fallback tokens (--cr-fallback-color-*) —————
--cr-fallback-color-base-container -> --bg-card
--cr-fallback-color-surface         -> --bg-primary
--cr-fallback-color-primary         -> --cyan
--cr-fallback-color-on-surface      -> --text
--cr-fallback-color-on-surface-subtle -> --text-dim
--cr-fallback-color-outline         -> --border
--cr-fallback-color-neutral-container -> --bg-card
--cr-fallback-color-tonal-container -> --bg-hover
--cr-fallback-color-divider         -> --border

— Component tokens (--cr-*) —————
--cr-card-background-color -> --bg-card
--cr-primary-background-color -> --bg-primary
--cr-secondary-background-color -> --bg-secondary
--cr-primary-text-color -> --text
--cr-secondary-text-color -> --text-dim
--cr-toolbar-background-color -> --bg-secondary
--cr-hover-background-color -> --bg-hover
--cr-link-color -> --cyan
--cr-focus-outline-color -> --accent
--cr-toggle-color -> --cyan
--cr-separator-line -> 1px solid var(--border)

— Legacy Google tokens (--google-*) —————
--google-grey-900 -> --bg-primary
--google-grey-700 -> --bg-hover
--google-blue-300 -> --cyan
--google-blue-500 -> --accent
--google-grey-800 -> --bg-card
--google-grey-500 -> --text-dim
--google-blue-400 -> --cyan
--google-blue-600 -> --accent

— Also set —————
color-scheme: dark
font-family: "Share Tech Mono","Monaco",monospace (on the full element list)

```

The stylesheet is injected into `document.head` and recursively into every open shadow root, because Chrome's

WebUI is a tree of custom elements and a `:root`, `:host` rule must reach inside each shadow boundary to recolor the components within it. The scheme it reads is `zb_scheme` from `chrome.storage.local` (a content script cannot call the native host itself), falling back to a `zbhud-get` worker message, and it applies cyberpunk first to avoid a flash of the stock theme before the real scheme resolves.

# Appendix G – The Native-Messaging Protocol

Every message the browser exchanges with the native host, framed as a 4-byte little-endian length followed by that many UTF-8 bytes of JSON. The **legacy path** is what zwire's browser side actually invokes — the scheme bridge, the sysinfo stream, and the PTY relay of Chapter 15. The **extended** commands are the larger local-machine agent **zwire-host** also exposes; zwire's front end does not call them, but they ship in the same binary and are reachable over its socket daemon.

```
— Legacy zwire path (what the browser uses) —————
{cmd:"get"}           -> {scheme, ui}   read the scheme + UI-pref files
{scheme:"<name>"}     validate vs 8 names, atomically write hud-scheme
{ui:{...}}            merge visual-effect prefs into hud-ui.json (atomic)
{cmd:"sysinfo_start"} stream {sys:{...}} per interval until the port closes
{cmd:"pty_spawn"}     fork a login shell in a PTY (becomes a terminal port)
  {cmd:"pty_write", ...} keystrokes to the shell
  {cmd:"pty_resize", ...} TIOCSWINSZ the window size
  {cmd:"pty_kill"}     end the shell
  <- {ev:"output", b64:...} shell output, base64, 64KB chunks
  <- {ev:"exit"}       the shell died

— Extended agent (same binary, not called by zwire UI) —
hello / ping          {ok,host,version,os,arch,pid,caps:[...]}
kv_get/set/merge/del/keys namespaced JSON key-value store (~/.<app>/kv/)
fs_read/write/append  file contents (64 MiB read cap)
fs_list/walk/stat     directory listing + bounded recursive crawl
fs_mkdir/rm           make (-p) / remove (recursive for dirs)
fs_watch / fs_tail    stream change events / follow a file (rotation-aware)
watch_stop / watch_list drop / list active observers
exec                 run a program to completion -> {ok,code,stdout,stderr}
job_start/poll/list/result background jobs (+ desktop notify on completion)
ps / kill / which     process list / signal / PATH resolve
sub / unsub / pub     process-global publish-subscribe bus
open / clipboard_get/set OS open + clipboard
notify / hostinfo     desktop notification / host facts
sysinfo_once / sysinfo_stop one-shot sample / stop a running stream
auth / peer_hello     token auth / peer handshake (TCP mesh)
peer_connect/peers/remote dial a peer / list peers / run a request remotely
```

The **sys** sample carries: **cpu**, **mem{u,t,p}**, **swap{u,t,p}**, **load[3]**, **uptime**, **disk{u,t,p}**, **net{up,down}**, **io{r,w}** (disk read/write rate), **temp**, **host**, **lip** (LAN IP), **pip** (public IP, refreshed sparingly), and **batt{p,c}** — the battery read natively per platform (**pmset** on macOS, **/sys/class/power\_supply** on Linux, **GetSystemPowerStatus** on Windows), the disk-I/O and network figures as per-second rates over the elapsed interval. The stream cadence is a configurable **interval\_ms**, with one-shot (**sysinfo\_once**) and **sysinfo\_stop** variants.

# Appendix H – The Seeded Keyword Commands

The `⌘K` palette's keyword commands are seeded from `cmd-defaults.js` (`ZWIRE_CMD_DEFAULTS`) on first use and thereafter live in the user-editable `zb_custom_cmds` array, edited on the Commands page. New defaults added to the file appear automatically; a default you delete is remembered in `zb_cmds_removed` and never re-added. Each `url` entry substitutes your typed argument for `{q}`; the three `action` entries take no argument. The full seed set:

```
— AI & search —
cg ChatGPT          cl Claude          pp Perplexity
gm Gemini           wa WolframAlpha    kagi Kagi
bing Bing          def Define word    dd DevDocs

— Google —
gm Gmail            gd Google Drive    cal Google Calendar
tr Google Translate img Google Images  sch Google Scholar
fl Google Flights  wx Weather

— Code & dev —
gh GitHub           gist GitHub Gist    pr GitHub PRs
iss GitHub Issues  cop GitHub Copilot  gl GitLab
bb Bitbucket       grep grep.app       rex regex101
ciu Can I Use      bp Bundlephobia    npt npm trends
mvn Maven Central  hf Hugging Face    cp CodePen
csb CodeSandbox    repl Replit        aw Arch Wiki
uni Emoji / Unicode

— Cloud & tools —
aws AWS Console    gcp GCP Console    vc Vercel
cf Cloudflare      no Notion           fig Figma
lin Linear         trl Trello

— Comms —
li LinkedIn        sl Slack            dc Discord
tg Telegram        wsp WhatsApp

— Media & shopping —
imdb IMDb          nf Netflix          sp Spotify
ytm YouTube Music  tw Twitch           sc SoundCloud
eb eBay            etsy Etsy           abnb Airbnb
yelp Yelp

— Reading —
hn Hacker News     nyt New York Times  arx arXiv

— Actions (no argument) —
rl Reload page     cu Copy page URL    cs Cycle color scheme
```

The list is intentionally a starting kit, not a fixed set: it is seeded once, yours to prune and extend, and the seeder's job is only to make sure a fresh profile has a sensible default without ever overwriting what you have

changed. (The worker carries a shorter first-run seed of the same shape; the `cmd-defaults.js` list is the fuller one that backfills the rest by id on the next load.)

# Appendix I – The Storage Keys

The HUD's cross-script state lives in `chrome.storage.local` under a family of `zb_*` keys, plus a couple of page-local `localStorage` keys and the two native files. This is the whole set the book referenced, in one place — the shared memory the content scripts, the worker, and the HUD pages reconcile through:

```
— chrome.storage.local (shared across the extension) —
zb_scheme           the mirrored active scheme (worker <-> content scripts)
zb_sys              the streamed system-stats sample (statusbar reads this)
zb_tabs             live list of open tabs (palette reads it, worker writes it)
zb_exts             enabled extensions (palette Tweak/Manage entries)
zb_frecent          frecenty-scored history, top 30 (palette + new tab)
zb_shortcuts        every extension's keyboard shortcuts (via developerPrivate)
zb_cmd              the command bus – content scripts write, worker executes
zb_keys             user keybinding remaps { action: key }
zb_custom_cmds      the %K keyword-command rules
zb_cmds_seeded      one-shot guard: the worker's first-run seed ran
zb_cmds_removed     tombstones for deleted default commands (never re-seed)
zb_status           whether the powerline statusbar is shown
zb_term_open        whether the terminal overlay should re-open on the next page
zb_tmux             statusbar readout of the in-page overlay (counts, prefix armed)
zb_tmux_state       the worker window-tiling engine's state (eviction recovery)
zb_tmux_sessions    durable named tmux layouts (the Sessions page); each window is
                    { name, panes[], tree } – content plus tiling shape. One reserved
                    entry, id __auto_last__ / name "last", is overwritten by every
                    detach of an unattached layout

zb_tmux_load        trigger the live overlay watches to attach a saved session
zb_tmux_prefix      the tmux prefix chord (Keyboard page)
zb_tmux_opts        tmux options, e.g. { timeout } (Keyboard page)
zb_ci               CI-monitor config { user, token, repolimit }
zb_ci_status        the CI tally { fail, running, ok, total, at }
zb_ci_cache         cached CI run data (the CI page)

— chrome.storage.local, NEW-TAB extension (not hud-internal's) –
zb_ntp              the new-tab layout library: { v, activeId, layouts[] }, each
                    layout { nav, bg, dial, pages[] } and each page its own dials
                    plus widget grid (Chapter 77). Owned here because the page
                    needs it on first paint; the HUD manager reaches it over
                    zbNtpGet / zbNtpSet and every write is re-normalized

— page localStorage (per-origin, not shared) —
zb.ntp.cache        first-paint mirror of zb_ntp – chrome.storage is async and a new
                    tab that waits for it flashes empty. Never the truth
zb.tiles            new-tab quick-launch tiles [{ label, url }] – LEGACY. Read once
                    by migrate() into the default layout, then never written or
                    cleared, so a rollback finds it intact
zb.engine           new-tab search engine template with %s

— native files (~/.zwire, the cross-extension bus) —
hud_scheme          the active scheme (fork mixer + new-tab both read it)
hud-ui.json         shared visual-effect + light-mode prefs
```

The distinction that matters: `chrome.storage.local` is shared among the internal HUD's own scripts but *not*

across extensions, and `localStorage` is not shared even across origins — so anything the new-tab extension or the power-tool must also see travels through the two native files, which are the only channel two extensions share. That is the whole reason `hud-scheme` and `hud-ui.json` exist as files rather than storage keys.

# Appendix J – The Rust Host Modules

`zwire-host` is organized into twenty-five files under `src/` — 6,518 lines — and the split is the map of the superset described in Chapter 23. For the reader who opens the crate, each module and its one job:

```
main.rs      entry point: pick the transport, run the session loop
lib.rs       the crate root; re-exports the api facade for embedders
transport.rs three entry points – Chrome stdio, the NDJSON socket daemon,
              and a one-shot client call
proto.rs     framing (4-byte LE length + JSON; NDJSON for the socket),
              the 128-MiB message guard, id-tagged replies
session.rs   the per-connection router: dispatch, held PTYs/watchers/subs,
              teardown on drop
store.rs     file-backed state under ~/.<app>/ – the kv store and the
              legacy scheme/ui files, all atomic tmp-and-rename (no SQLite)
sysmon.rs    the system sample via the `sysinfo` crate (not psutil),
              streamed every interval
pty.rs       PTY terminals via `portable-pty`, multiplexed by id, base64 I/O
exec.rs      run a program to completion (base64 stdout/stderr + exit code)
fsops.rs     read/write/append/list/walk/stat/mkdir/rm, with size + count caps
jobs.rs      background jobs + completion notifications
watch.rs     poll-based fs_watch and rotation-aware fs_tail observers
procs.rs     ps / kill / which (reusing the sysinfo crate)
osops.rs     open / clipboard / notify / hostinfo (shelling to platform tools)
bus.rs       the process-global publish-subscribe broker
peer.rs      the TCP host-to-host mesh: auth, dial, bus federation, remote
api.rs       an in-process library facade (exec/walk/read/write) for embedders
zbus.rs      the GUI Automation Bus endpoint: `App::open("zwire")` from a
              stryke script (871 lines – the largest module; Chapters 54-58)
txn.rs       transactional compensation for that bus – the journal, the
              inverses, the abort path (Chapters 56-57)
hooks.rs     lifecycle scripting hooks: bind an event to a stryke script
hostlog.rs   the shared tx/rx log, because Chrome spawns a separate host
              process per sendNativeMessage and no one process sees it all
stryke_runner.rs subprocess bridge to the `stryke` interpreter (not embedded:
              `VMHelper` is !Send/!Sync and pulls a second bundled rusqlite)
stryke_lsp.rs a long-lived `stryke --lsp` child, LSP JSON-RPC over stdio with
              Content-Length framing, for the Hooks editor
theme_watch.rs cross-process live theme sync off ~/.zwire/global.toml
tauri_theme.rs a ready-made Tauri v2 plugin for that same fleet-wide sync
```

The release profile is tuned for a small artifact — `opt-level="z"`, LTO, symbols stripped, `panic="abort"` — which is how a host that carries a kv store, a filesystem API, a PTY multiplexer, a job runner, and a TCP mesh still fits in roughly half a megabyte with no external runtime. `zwire`'s browser uses only `store.rs`, `sysmon.rs`, and `pty.rs`; the rest is the larger local agent the same binary offers any app that speaks its socket.

# Appendix K – The Launcher and Build Scripts

The runtime path is a handful of shell scripts. Their contracts, in one place:

```
— bin/zwire – the launcher, in sequence —————
1 source state-dir.sh; STATE = $ZWIRE_STATE or the OS app-data dir
2 one-time migrate a leftover ~/.zwire into STATE (if no $ZWIRE_STATE)
3 base = executable $ZWIRE_BASE, else $STATE/base.path, else FAIL
  "zwire: no base browser installed – run: ../scripts/fetch-base.sh"
4 ext list = { newtab, extensions/zpwrchrome, extensions/hud-internal }
  filtered to dirs with a manifest.json (theme/ deliberately excluded)
5 read "path" from com.zwire.hud.json; if missing/dangling run
  setup-native-host.sh (never fatal – browser runs without the host, and
  never verifies the binary it wires: Chapter 76)
6 exec base with the ten flags + "$@"

— the ten launch flags (Appendix E has the why) —————
--user-data-dir --load-extension --extensions-on-chrome-urls --test-type
--no-first-run --no-default-browser-check --homepage=chrome://newtab
--disable-features=NtpFooter
--enable-features=SplitViewHorizontal,SplitViewTabRestore
--autoplay-policy=no-user-gesture-required
  (the packaged macOS launcher swaps the last one for --restore-last-session;
  the Linux one drops both and adds --no-sandbox – Chapter 62)

— fetch-base.sh – host -> snapshot —————
Darwin/arm64 -> Mac_Arm    chrome-mac.zip    chrome-mac
Darwin/x86_64 -> Mac       chrome-mac.zip    chrome-mac
Linux/x86_64  -> Linux_x64 chrome-linux.zip chrome-linux
MINGW/MSYS   -> Win_x64   chrome-win.zip   chrome-win
revision = arg, else the platform's LAST_CHANGE
writes base.path (abs binary) + base.version ("r<REV>")

— the self-contained install layouts —————
macOS    /Applications/zwire.app/Contents/{Resources/{browser,ext,native},MacOS/zwire}
Linux    ~/.local/opt/zwire/{browser,ext,native} + PATH link + .desktop
Windows  %LOCALAPPDATA%\zwire\{browser,ext,native} + zwire.cmd + registry host
  each stages extensions per-user (rsync --delete, excluding _metadata) at launch

— install-time stamps, macOS localinstall.sh ONLY (Ch 23, Ch 69) —
version.js ZWIRE_VERSION  <- package.json "version"          (perl -i, never sed -i)
manifest "version"        <- <base>.<sha256 of the ext tree % 65536>
manifest "service_worker" <- background.sw-<hash>.js (one-line import of background.js)
launcher SW-cache marker  <- every staged manifest's version AND worker filename;
  on change, rm -rf $PROFILE/{Default/,}Service Worker
  none of these four run in localinstall-linux.sh, localinstall-windows.ps1,
  or bin/zwire – the whole cached-worker defence is macOS-packaged-only
```

The three pinned extension ids appear identically in four scripts and each extension's own **key**: HUD `omcggnjfbmpdlofkbpddkhnfibfhgg`, zpwrchrome `hpppdchpnphmijdeanibpcadgknmaja`, new-tab `gpoepnekoiplhkegjpcocnpeijiefgieb`. The native host name is `com.zwire.hud`, its manifest lists all

three as `allowed_origins`, and the pinned fork tag is `150.0.7871.46`.

# Appendix L – The CDP Verification Probes

The eight `v*.mjs` probes attach to a running `zwire` over the Chrome DevTools Protocol on `127.0.0.1:9222` (opened only when you pass `--remote-debugging-port=9222`). Each shares one bootstrap — `fetch /json/version`, open the `websocketDebuggerUrl`, `Target.attachToTarget` with `flatten:true`, `Runtime.enable`, then evaluate expressions with `returnByValue:true` — and prints booleans and counts that a human or a CI grep reads as pass or fail. What each verifies:

| probe              | target                                  | asserts                                                                                                                                       |
|--------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>vkeys</code> | <code>createTarget keys.html</code>     | <code>ZWIRE_KEYMAP</code> present · chip/category counts · writes <code>zb_keys.scrollDown='s'</code> + reads it back                         |
| <code>vterm</code> | <code>createTarget terminal.html</code> | <code>Terminal</code> global · <code>#terminalPane</code> · <code>.xterm</code> · waits 5s for <code>zsh</code> · rows of shell text appeared |
| <code>vovl</code>  | <code>attach github/amazon/wiki</code>  | <code>synth Ctrl+`</code> · <code>#terminalPane</code> becomes <code>.active</code> · <code>.xterm</code> rendered                            |
| <code>vpop</code>  | <code>attach a stable page</code>       | <code>Ctrl+`</code> · <code>.active</code> · waits 7s for the <code>SW-&gt;host PTY</code> shell · reads back visible shell text              |
| <code>vwin</code>  | <code>attach a stable page</code>       | <code>Ctrl+`</code> opens <code>terminal.html</code> popup window · attach it · <code>.xterm</code> · <code>connectNative</code> reachable    |
| <code>vsys</code>  | <code>attach a stable page</code>       | waits 5s for <code>zb_sys</code> · reads <code>CPU/MEM/NET/HOST/LAN/UP/BATT</code> segments · <code>.plr</code> chevron count · clock         |
| <code>vpos</code>  | <code>attach a stable page</code>       | left-to-right segment order · <code>C-b</code> prefix sits left of the scheme segment                                                         |
| <code>vpl</code>   | <code>attach a stable page</code>       | the powerline <code>::</code> after chevron borders + the prefix block background render as intended                                          |

`vkeys` and `vterm` open an extension page directly; the other six attach to an existing `github/amazon/wikipedia` tab (and exit cleanly if none is open, since that is a setup gap, not a fault). Two probe lines are dead — `vterm` reaches for a non-existent `window.__zbTermInstance`, and `vovl` prints a placeholder instead of reading `zb_term_open` — noted here so the suite is not oversold.

# Appendix M – The zpwrcrome Manifest

The power-tool's manifest, as the concrete proof of Chapter 2 (this book does not otherwise document `zpwrcrome`, which has its own):

```
manifest_version 3 · minimum_chrome_version 127 · module service worker
version 0.22.39 · <all_urls> host access · 6 content scripts · 55 commands

permissions (19):
  userScripts · declarativeNetRequestWithHostAccess · nativeMessaging ·
  webRequest · webRequestAuthProvider · tabs · tabGroups · sessions ·
  bookmarks · history · scripting · webNavigation · cookies · downloads ·
  contextMenus · notifications · clipboardWrite · alarms · storage

the four that require a genuine Chromium engine (Chapter 2):
  userScripts · declarativeNetRequestWithHostAccess · nativeMessaging · webRequest

feature surface (one toolbar icon): browserpass-compatible `pass` (fill/copy/
OTP/autofill via a Rust host over ~/.password-store) · segmented download
manager · MRU tab switcher (scenes/opener-tree/fzf history) · userscript
engine · full-page screenshot · tech detector · theme injector · lights-off
dimmer · JSON/XML viewers · UA switcher · find-in-all-tabs
```

# Appendix N – The Fork Patch Targets

The twenty-seven HUD patches, each apply-clean against [150.0.7871.46](#), and the native surface each reaches that a theme extension cannot:

|                             |                |                                            |                                                                                                  |
|-----------------------------|----------------|--------------------------------------------|--------------------------------------------------------------------------------------------------|
| 0001                        | tab shape      | tab_style_views.cc (GetPath)               | trapezoid tabs, hud_slope 0.28                                                                   |
| 0002                        | UI colors      | chrome_color_mixer.{cc,h}+mixers.cc        | HUD palette + 8 schemes on frame/<br>toolbar/tabs/omnibox + separator                            |
| 0003                        | UI font        | resource_bundle.cc (InitDefault...)        | Share Tech Mono, Monaco fallback                                                                 |
| 0004                        | toolbar line   | toolbar_view.cc (Init, kNormal)            | 2px cyan under-toolbar border                                                                    |
| 0005                        | omnibox        | location_bar_view.cc (ComputeBorderRadius) | clamp field radius to 2px                                                                        |
| 0006                        | product str.   | chrome/app/theme/chromium/BRANDING         | Chromium -> zwire, bundle id                                                                     |
| 0007                        | DevTools       | 3 front_end files                          | 8 HUD schemes in Theme dropdown                                                                  |
| 0008                        | allow framing  | ancestor_throttle.cc                       | bypass X-Frame-Options / CSP                                                                     |
| 0009                        | ext focus      | extension_keybinding_registry_views.cc     | hand focus to page for %K                                                                        |
| 0010                        | force newtab   | search.cc (HandleNewTabURLRewrite)         | chrome://newtab -> zwire NTP                                                                     |
| 0011                        | pinned exts    | toolbar_view.cc                            | pinned actions never overflow                                                                    |
| 0012                        | frame-src      | navigation_request.cc                      | host CSP never blocks subframe                                                                   |
| 0013                        | restore sess.  | session_startup_pref.cc                    | default startup = last session                                                                   |
| 0014                        | no crash bubl  | session_crashed_bubble_view.cc             | suppress "Restore pages?" bubble                                                                 |
| 0015                        | no nav block   | extension_navigation_throttle.cc           | allow NTP nav pre-registration                                                                   |
| 0016                        | ext priv API   | _permission_features.json                  | hud-internal:                                                                                    |
| ~ developer/settingsPrivate |                |                                            |                                                                                                  |
| 0017                        | no hold-quit   | app_controller_mac.mm                      | immediate %Q (no hold-to-quit)                                                                   |
| 0018                        | null-pol fix   | extension_function_dispatcher.cc           | gate is_sandboxed on IsActive                                                                    |
| ~ (crash fix)               |                |                                            |                                                                                                  |
| 0019                        | store script   | chrome_extensions_client.cc                | content scripts run on Web Store                                                                 |
| 0020                        | menu colors    | chrome_color_mixer.cc (final mixer)        | native menus -> HUD Sys tokens                                                                   |
| 0021                        | primitive col  | chrome_color_mixer.cc (final mixer)        | native primitives -> HUD Sys                                                                     |
| ~ tokens                    |                |                                            |                                                                                                  |
| 0022                        | audio EQ       | output_controller.{cc,h}+client.cc         | browser-wide output EQ (every<br>~ stream)                                                       |
| 0023                        | audio capture  | tab_capture_api.cc + _api_features         | Audio page taps a tab, no prompt                                                                 |
| 0024                        | audio live     | audio_service.mojom + service.{cc,h}       | live EQ config + output meters                                                                   |
| 0025                        | custom palette | chrome_color_mixer.cc (final mixer)        | paint native chrome from the<br>hud-palette base colours, not<br>the baked scheme-name table     |
| 0026                        | reading list   | toast_service.cc                           | "added to reading list" toast<br>Open -> the HUD page, not the<br>native side panel              |
| 0027                        | strip buttons  | chrome_color_mixer.cc (final mixer)        | new-tab +, tab search, combo<br>button -> HUD scheme (ink drop,<br>bg/fg, focus ring, separator) |

The series file records each patch's apply-clean verification and keeps a commented-out [0008-tmux-split-prefix](#) tombstone (the native two-pane split that ate the [C-b](#) prefix, superseded by the JS overlay); [scripts/check-patch-series.sh](#) gates the file against the patches on disk. [fork/args.gn](#) builds it unbranded ([is\\_chrome\\_branded=false](#)), official ([is\\_official\\_build=true](#)), with symbols, PGO, NaCl, and ThinLTO all off for a workstation-tolerable build time.

# Appendix O — The Three Extension Manifests

zwire loads three unpacked extensions, each pinned to a stable id by an embedded [key](#). Their manifests side by side:

```
— newtab — "zwire New Tab" —————
manifest_version 3 · version 1.0.17 · id gpoepnekoiplhkegjpcnpeiijefgieb
chrome_url_overrides.newtab -> newtab.html
permissions (7): nativeMessaging · tabs · history · storage · topSites ·
  bookmarks · favicon
  (the last four arrived with layouts: storage holds zb_ntp, the middle two feed the
  Top Sites and Bookmarks widgets, and favicon serves dial icons out of Chrome's own
  store so a dial never fetches its icon from the site it points at)
background: background.js — importScripts zntp-core.js and answers zbNtpGet/zbNtpSet
externally_connectable.ids: [ hud-internal ] (so the worker can open its palette)
web_accessible_resources: newtab.html/tmux.html · newtab.css/.js · zntp-core.js ·
  widgets.js · layout-edit.js · palette.js ·
  tmux-pane.js · tmux-host.js · scheme-sync.js · schemes.js · fonts/* · lib/*, <all_urls>

— hud-internal — "zwire HUD Internal" —————
manifest_version 3 · version 0.6.29 · id omcggnjfbmpdlofklbpddkhnfibfhgg
permissions (20): storage · nativeMessaging · management · tabs · scripting ·
  webNavigation · history · downloads · bookmarks · sessions · alarms ·
  notifications · developerPrivate · settingsPrivate · system.display ·
  declarativeNetRequest · browsingData · tabGroups · readingList · power
host_permissions: <all_urls>
declarative_net_request: rules/frame_bust.json
background: background.js (classic service worker, not a module)
commands: open-palette = ⌘K / Ctrl+K
content_scripts: 11 blocks (Appendix P / Chapter 7)
externally_connectable.ids: [ zpwrchrome, newtab ]
web_accessible_resources: pages/* · schemes.js · sandbox/js-run.html · lib/term/* ·
  lib/zgui-core/webui/*.css
  (no minimum_chrome_version, no chrome_url_overrides)

— zpwrchrome — the power-tool (submodule) —————
manifest_version 3 · version 0.22.40 · minimum_chrome_version 127
id hpppdchpnphmiijdeanibpcadgknmaja
permissions (19, incl. userScripts · declarativeNetRequestWithHostAccess ·
  nativeMessaging · webRequest) · host_permissions <all_urls>
background: module service worker · 6 content scripts · 55 commands (Alt+T)
options_ui: scripts-manager/dashboard.html (open_in_tab)
externally_connectable.ids: [ hud-internal, newtab ]
```

The `externally_connectable` fields are the cross-extension messaging graph, and they are not symmetric. The new-tab extension names one id — the HUD worker, which is what opens its palette on ⌘K. The HUD and the power-tool each name two: each other, for the scheme bridge, and the new-tab extension. So of the six ordered pairs among three extensions, five are allowed and one is not: nothing accepts a message *from* the power-tool except the HUD, and the new-tab extension is the only one of the three that grants no inbound edge to it. Read the other way, the new-tab page can originate a message to either of the other two, but only

the HUD can originate one back to it. These are the only three extensions on the machine that speak to one another, wired by id, with the native host's `allowed_origins` completing the figure so any of the three can reach `com.zwire.hud` directly.

# Appendix P – The Content-Script Injection Order

The `hud-internal` manifest registers eleven content-script blocks; the order within a block is load-bearing because later scripts read globals earlier ones set. Each script also guards against double injection with a `window.__zb...Loaded` flag.

| block | run_at         | frames | scripts (in order)                                                                                                                                                                                                                                                                          | defines                                                                     |
|-------|----------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1     | document_start | top    | redirect.js                                                                                                                                                                                                                                                                                 | (redirect)                                                                  |
| 2     | document_idle  | top    | blank-newtab.js                                                                                                                                                                                                                                                                             | (about:blank<br>-> new tab)                                                 |
| 3     | document_start | top    | schemes.js, theme.js                                                                                                                                                                                                                                                                        | window.ZWIRE_HUD                                                            |
| 4     | document_start | top    | debug-cloak.js                                                                                                                                                                                                                                                                              | (hide native<br>debug markup)                                               |
| 5     | document_idle  | top    | zgui data-table/card/json-view,<br>debug-adapter.js                                                                                                                                                                                                                                         | zgui debug pages                                                            |
| 6     | document_idle  | top    | schemes.js, pages/fzf.js, zfind.js                                                                                                                                                                                                                                                          | ZBFzf, find bar                                                             |
| 7     | document_start | top    | schemes.js, zgui util/fzf/command-palette,<br>zkeys.js, cmd-defaults.js, palette-cmds.js,<br>zpalette.js, zgui expose + zexpose.js,<br>zpageactions, zreader, zreload, zbreak,<br>zgestures, zpanels, zextras, ztrash,<br>zcookies, zspatial, zspeak, zpeek, zzap,<br>zvim.js, ztriggers.js | window.ZGui,<br>ZWIRE_KEYMAP,<br>palette, page<br>actions, vim,<br>triggers |
| 8     | document_idle  | top    | schemes.js, powerline.js, zpowerline.js                                                                                                                                                                                                                                                     | statusbar                                                                   |
| 9     | document_idle  | ALL    | zpipes-core.js, ztmux-pane.js                                                                                                                                                                                                                                                               | pane forwarder,<br>pipe runtime                                             |
| 10    | document_idle  | top    | zgui util/split-pane/modal/toast/<br>button-bar/tmux, zpipes-core.js,<br>ztmux-config.js                                                                                                                                                                                                    | tmux overlay                                                                |
| 11    | document_idle  | top    | modal-drag.js, xterm.js, terminal.js,<br>zterm-boot.js                                                                                                                                                                                                                                      | terminal                                                                    |

Block 1 matches only the five redirected `chrome://` pages; block 3 matches every other `chrome://` page (excluding the redirected set and the new-tab URLs); blocks 4 and 5 match one fixed list of twenty-eight *static* `chrome://` debug pages (`local-state`, `gpu`, `policy`, `histograms`, the `*-internals` family) and are the cloak-then-rebuild pair that turns those pages into `ZGui` widgets; blocks 6, 7, 8, and 11 match `http/https/file/chrome://` (excluding the new-tab URLs); blocks 2, 9, and 10 match only `http/https/file`, and block 9 is the sole `all_frames` block, so the `tmux` forwarder and the pipe runtime run inside framed panes while the shared window manager stays in the top frame in block 10. `schemes.js` is re-listed in every block that needs it because each block is an isolated injection that needs the palette table present.

# Appendix Q – The Fuzzy Matcher

`ZGui.fzf` (`fzf.js`) is the fzf-style matcher behind the palette, the find-bar fallback, and every fuzzy-filtered HUD page — extracted verbatim from Audio-Haxor so the whole suite ranks identically. Its scoring:

```
— weights (defaults, all tunable + localStorage-persisted) —
SCORE_MATCH          16    per matched character
SCORE_GAP_START      -3    first char of a gap between matches
SCORE_GAP_EXTENSION  -1    each further gap character
BONUS_BOUNDARY        9    match right after a non-word char (start of word)
BONUS_NON_WORD        8    match across a word/non-word class change
BONUS_CAMEL           7    lowercase -> UPPERCASE transition (camelCase)
BONUS_CONSECUTIVE     4    match adjacent to the previous match
BONUS_FIRST_CHAR      x2    the first matched char's bonus is doubled

— extended query sigils (space = AND, | = OR within a group) —
!term    negate (exclude matches)
'term    or term' exact substring
^term    prefix
term$    suffix
term     fuzzy (subsequence)
```

The matcher tries every start position of the needle's first character, greedily takes the earliest subsequent matches, scores each run with match + position bonus + consecutive/gap adjustment, and keeps the best. Matched characters are wrapped in `<mark class="fzf-hl">` for the cyan emphasis. The name field of a row gets a `+500` field bonus so a hit in the label outranks a hit in the detail.

# Appendix R – Visual Effects and Readline

The cyberpunk look and the terminal-style input editing both come from `zgui-core`, on two clean conventions. Effects (`ZGui.fx`): an effect is *on* when its body class is *absent*.

```
— effects: pref -> body class -> CSS technique —————
scanlines  no-scanlines    repeating-linear-gradient on body::after (+ animated beams)
vignette    no-vignette    radial-gradient ellipse on body::before (transparent->dark)
glow        no-neon-glow   pervasive text-shadow / box-shadow (off = none !important)
anim        no-anim        all animation/transition (off = none !important)
light       [data-theme=light] flips the palette (a root attribute, not a body class)
grid        two linear-gradient hairlines, 40px fixed cell (always on)
           prefs persist to zg-fx-* localStorage and to native hud-ui.json (cross-extension)

— readline (installReadline, on every text input) —————
^A start-of-line  ^E end-of-line    ^B back char    ^F forward char
^H backspace      ^D delete char    ^K kill to EOL  ^U kill to BOL
^W kill word back ^Y yank           ^T transpose    M-b/M-f word back/fwd
M-d kill word fwd (^K/^U/^W/M-d feed a shared kill-ring that ^Y pastes)
  also: autocorrect/autocapitalize/spellcheck forced off; ⌘F/Ctrl+F focuses the filter
```

# Appendix S – The System-Stats Sample

Every tick of the sysinfo stream writes one `zb_sys` object; the statusbar renders one field per segment. The sample, and which host produces each field:

| field  | meaning                       | shape     | both / note                |
|--------|-------------------------------|-----------|----------------------------|
| cpu    | CPU percent                   | number    | both                       |
| mem    | memory used/total/percent     | {u,t,p}   | both                       |
| swap   | swap used/total/percent       | {u,t,p}   | both                       |
| load   | load averages (3)             | [a,b,c]   | both                       |
| uptime | seconds since boot            | number    | both                       |
| disk   | root-fs used/total/percent    | {u,t,p}   | both                       |
| net    | throughput (bytes/s)          | {up,down} | both (delta / interval)    |
| io     | disk I/O (bytes/s)            | {r,w}     | both (delta / interval)    |
| temp   | max sensor temperature (°C)   | number    | both (hidden if absent)    |
| batt   | battery percent + charging    | {p,c}     | both (macOS/Linux/Windows) |
| host   | short hostname                | string    | both                       |
| lip    | LAN IP (UDP-connect trick)    | string    | both                       |
| pip    | public IP (ipify, every 120s) | string    | both                       |

The stream cadence is a configurable `interval_ms` (floored at 250 ms), with one-shot (`sysinfo_once`) and `sysinfo_stop` variants and a background-thread pump that keeps the connection free for other RPCs. The one-shot `sysinfo_once` double-refreshes so its `net/io` rates report a near-zero delta rather than a lifetime total mislabelled as a per-second rate.

# Appendix T – The Palette Destinations

Beyond your tabs, history, schemes, and keyword commands, the `⌘K` palette carries a static catalog of destinations. The HUD pages and the `chrome://` diagnostics tail:

```
— HUD replacement pages —
Extensions · Settings · History · Bookmarks · CI runs · Shortcuts (keys) ·
Extension shortcuts · Custom commands · System info (version)

— chrome:// destinations (≈44) —
newtab · downloads · history · bookmarks · extensions · settings · version ·
system · flags · discards · net-internals/#dns · gpu · net-internals ·
password-manager · inspect · net-export · policy · components · management ·
accessibility · omnibox · media-internals · webrtc-internals ·
serviceworker-internals · indexeddb-internals · quota-internals ·
blob-internals · histograms · tracing · crashes · device-log · gcm-internals ·
sync-internals · process-internals · autofill-internals · download-internals ·
signin-internals · translate-internals · user-actions · ukm · predictors ·
memory-internals · webui-gallery · chrome://about (all pages)

— chrome://settings sub-pages (19) —
You & Google · Appearance · Autofill · Payments · Addresses · Privacy ·
Security · Cookies · Site settings · Clear browsing data · Performance ·
Search engine · Default browser · On startup · Languages · Downloads ·
Accessibility · System · Reset

— external —
Chrome Web Store · zwire app store (menketechnologies.github.io/app-store/)
```

The new-tab palette carries the same static catalog (it is a separate extension and must re-carry it), which is why the two palettes list identical destinations even though they reach `chrome.tabs` by different paths (Chapter 12).

## Appendix U – The Command Bus

Content scripts cannot call the privileged tab, window, and session APIs, so they write a `zb_cmd` object to `chrome.storage.local`; the `storage.onChanged` event reliably wakes the MV3 worker, which performs the action. Every `zb_cmd` carries an action field `a`, a monotonic tag `n` (so a repeated identical command still registers as a change), and any parameters. The worker's dispatch:

| <code>a</code>            | <code>params</code>   | <code>effect</code>                                                 |
|---------------------------|-----------------------|---------------------------------------------------------------------|
| <code>ping</code>         | <code>-</code>        | wake + refresh <code>zb_tabs/zb_exts/zb_frecent/zb_shortcuts</code> |
| <code>open</code>         | <code>url</code>      | navigate the active tab, else create a tab                          |
| <code>openTab</code>      | <code>url</code>      | create a new tab                                                    |
| <code>activate</code>     | <code>tabId</code>    | switch to a tab + focus its window                                  |
| <code>newTab</code>       | <code>-</code>        | create an empty tab                                                 |
| <code>newWindow</code>    | <code>-</code>        | create a new window                                                 |
| <code>duplicateTab</code> | <code>-</code>        | duplicate the active tab                                            |
| <code>reopenTab</code>    | <code>-</code>        | <code>sessions.restore</code> (reopen last closed)                  |
| <code>pinTab</code>       | <code>-</code>        | toggle the active tab's pinned state                                |
| <code>muteTab</code>      | <code>-</code>        | toggle the active tab's muted state                                 |
| <code>closeTab</code>     | <code>-</code>        | close the active tab                                                |
| <code>closeOthers</code>  | <code>-</code>        | close every non-active, non-pinned tab in the window                |
| <code>nextTab</code>      | <code>-</code>        | cycle to the next tab (last-focused window)                         |
| <code>prevTab</code>      | <code>-</code>        | cycle to the previous tab                                           |
| <code>tmux</code>         | <code>sub, ...</code> | drive the worker's OS-window tiling engine (Chapter 8)              |

The vim script issues `prevTab/nextTab/closeTab/newTab/openTab` this way; the palette adds `duplicateTab`, `reopenTab`, `pinTab`, `muteTab`, `closeOthers`, `newWindow`, and `activate`. A parallel path — `runtime.onMessage` — carries the scheme set (`zbhud-scheme`), the scheme get (`zbhud-get`), a content-script open (`zbopen`), and a wake ping (`zbping`); these are request/response where the bus is fire-and-forget, and the worker uses whichever suits the reliability the caller needs.

## Appendix V — The Frecency Model

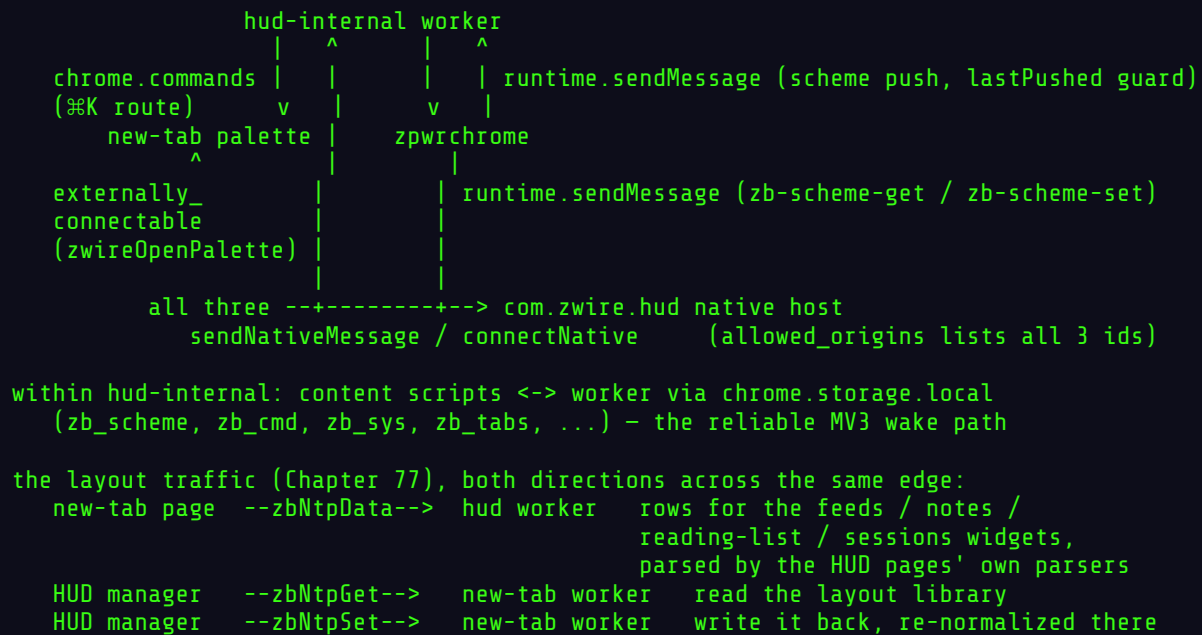
Three surfaces — the worker's `zb_frecent`, the new-tab palette, and the HUD pages' `zg-boot.js` — surface your “frequent + recent” history, and all three use the same formula, so a site ranks identically wherever it appears:

```
window  chrome.history.search over the last 90 days (startTime = now - 90d)
score    (visitCount + 2 * typedCount) / (1 + ageDays * 0.3)
        ageDays = (now - lastVisitTime) / 86_400_000
filter   drop URLs beginning with "chrome"
keep     the top 30 by score, descending
```

A typed visit counts double a clicked one (you meant to go there), and the divisor decays a site's score by roughly a third per day of age, so recency and frequency trade against each other rather than one dominating. The worker recomputes the list on every `history.onVisited`, so the palette's history rows are never stale.

# Appendix W – The Extension Messaging Graph

Three extensions and one native host coordinate across boundaries that neither `chrome.storage` nor `localStorage` can cross. Who reaches whom:



The rule underneath the graph: storage is shared only *within* an extension, so every cross-extension value (the scheme, the UI prefs) travels through the native files, and every cross-extension *event* (open the palette, push a scheme) travels through `externally_connectable` or `runtime.sendMessage` between the specific ids each manifest allows.

# Appendix X – The zgui-core Toolkit Surface

zwire loads a slice of the shared [zgui-core](#) toolkit; the modules and their public surface:

```
util.js    -> ZGui.util    escapeHtml · throttle · debounce · clamp · formatBytes ·
                                formatDuration · slugify · copyToClipboard · lineEdit ·
                                installReadline · focusFilterBar · dampenInput ·
                                installInputDefaults (auto-installs readline + input
                                defaults on load; binds ⌘F/Ctrl+F to the filter bar)
fzf.js     -> ZGui.fzf     fzfMatch · getMatchIndices · highlightWithIndices ·
                                searchScore · parseFzfQuery · setParam/defaults/bounds
command-palette.js -> ZGui.palette register · registerProvider · clear · open ·
                                close · isOpen · bindHotkey · create (three-tier ranking:
                                static+strong / secondary / fallback-last)
fx.js      -> ZGui.fx      set · toggle · get · all · load · buildToggles (the four
                                body.no-* effect classes)
tmux.js    -> ZGui.tmux    init · open · toggle · isOpen · isInitd · status ·
                                setStatus · suppressKeys ·
                                prefix / key / relaySync / syncOf / yank (the external
                                feed a cross-origin-iframe host drives it with) ·
                                editSessions · editKeymap · loadSession(id) ·
                                layout.{ rects(panes,tree,w,h) · dirs(win) ·
                                    split(win,i,dir) · close(win,i) }
                                (the saved-window geometry + the two structural edits,
                                shared so a host's own session manager draws and edits
                                exactly what Load tiles)
powerline.js -> ZGui.powerline the shared statusbar (segments, chevrons, feeds)
crt.js     -> ZGui.crt / ZGui.neonGlow the legacy scanline + glow layers
data-table.js / card.js / json-view.js the widgets the chrome:// debug adapter
                                rebuilds native tables and JSON blobs into
split-pane.js / modal.js / toast.js / button-bar.js / modal-drag.js / expose.js
                                the dialog, tiling, and overview primitives the overlay,
                                the terminal, and the HUD pages share
fonts.css + webui/fonts/ Orbitron (display) + Share Tech Mono (body), SIL OFL
```

The guidance from [zgui-core](#)'s own docs is to consume it as a submodule and load the modules directly — never copy [webui/\\*](#) — which [zwire](#) follows for the content-script extension; the new-tab extension, a distinct origin, bundles its own copy of the [util/fzf/command-palette](#) trio under [lib/zgui/](#).

# Appendix Y – The HUD Internal Pages

Each replacement page, the privileged API it stands on, and the state it reads or writes:

| page         | API                           | reads / writes                                                                                    |
|--------------|-------------------------------|---------------------------------------------------------------------------------------------------|
| extensions   | developerPrivate + management | enable/disable/reload/pack/site-access                                                            |
| settings     | settingsPrivate               | every Chrome pref + scheme/effects -> hud-ui                                                      |
| history      | history.search                | recency-sorted list (zg-list)                                                                     |
| bookmarks    | bookmarks.getTree             | flattened folder-path table                                                                       |
| downloads    | downloads.search              | open-on-click (native page owns the manager)                                                      |
| version      | runtime + navigator + host    | live com.zwire.hud probe                                                                          |
| keys         | ZWIRE_KEYMAP                  | writes zb_keys remaps + zb_tmux_prefix/opts                                                       |
| commands     | -                             | writes zb_custom_cmds (+ zb_cmds_removed)                                                         |
| sessions     | -                             | zb_tmux_sessions CRUD -> zb_tmux_load trigger                                                     |
| extshortcuts | developerPrivate              | updateExtensionCommandKeybinding                                                                  |
| ci           | GitHub REST API               | zb_ci config -> per-repo run accordions                                                           |
| readinglist  | readinglist                   | add/read/remove, unread-first ordering                                                            |
| terminal     | native host (pty_*)           | a PTY per tab, keyed by tab id                                                                    |
| host         | native host (every cmd)       | the console over the one JSON protocol                                                            |
| dashboard    | -                             | tile order per category (ZGui.drag)                                                               |
| hooks        | native host (hooks_*)         | hooks.json + <id>.st, hook-result bus topic                                                       |
| triggers     | -                             | zb_triggers CRUD (ztriggers.js compiles it)                                                       |
| pipes        | -                             | zb_pipes CRUD (zpipes-core.js runs it)                                                            |
| audio        | native host (fs_read/write)   | \$STATE/audio-eq spec + \$STATE/meters frames                                                     |
| notes        | -                             | zb_notes folder tree, autosaved                                                                   |
| feeds        | fetch (<all_urls>)            | zb_feeds URLs; RSS/Atom parsed in-page                                                            |
| translate    | fetch (<all_urls>)            | keyless endpoint, no stored state                                                                 |
| store        | tabs.create                   | static catalog (store-catalog.js) + welcome                                                       |
| newtab       | runtime.sendMessage (ext)     | the new-tab LAYOUT library, over zbNtpGet/zbNtpSet into the other extension's zb_ntp (Chapter 77) |

All twenty-four mount through `zg-boot.js` (`ZBHUD.mount`), share the `zb-*` shell chrome, carry the same `⌘K` palette, and filter through the same `fzf.js` — one surface, not twenty-four. The newest is the odd one: every other page reads state its own extension owns, and that one edits a sibling extension's storage through a message pair, because the layouts have to live where the page that paints them can read them synchronously.

# Appendix Z – The Dev-Loop Scripts

The `scripts/` family assembles and resets the base; none of them compile the browser (the base is a prebuilt snapshot). Each sources `cyberpunk.sh` for its banner:

```
dev.sh      launch from the checkout, foreground (content scripts hot-reload)
build.sh    ensure a current rebranded base exists (fetch if missing, then rebrand)
rebuild.sh  clean + build
clean.sh    remove the /Applications wrapper + staged dist/ (keeps base + profile)
bust.sh     re-seal + re-register the bundle with LaunchServices (macOS only)
nuke.sh     wipe base + wrapper, re-fetch, rebrand, reinstall (keeps the profile)
test.sh     node --check the front-end (excl. node_modules + lib/zgui-core),
            then each extension's own pnpm test

install.sh  PATH symlink + fetch + hooks-editor bundle + (macOS) rebrand
localinstall*.sh  self-contained app: browser + ext + Rust host in one location
setup-native-host.sh  cargo build --release zwire-host + write the host manifest
fetch-base.sh  download + pin the Chromium snapshot base
build-hooks-editor.sh  esbuild the Monaco bundle into hud-internal/lib/hooks-editor
                    (forwards to extensions/hud-internal/scripts/ - one impl);
                    every packaging path calls it and hard-fails on a bad build
check-patch-series.sh  CI gate: fork/patches/series <-> the *.patch files on disk
```

The destructive ones are careful about what they keep: `clean` and `nuke` never touch the profile, and `clean` does not even touch the 325 MB base — that is `nuke`'s job. The npm scripts in `package.json` alias the common ones (`pnpm dev`, `pnpm build`, `pnpm nuke`) for anyone who reaches for the package manager instead.

# Appendix AA – Three Paths, End to End

The HUD's design is a set of multi-hop paths across isolation boundaries. Three of them, traced in full, tie the book together.

Switching the color scheme:

```
pick a scheme (palette / picker / Settings page)
-> runtime.sendMessage {zbhud-scheme} to the worker + storage.local zb_scheme
-> worker: sendNativeMessage {scheme} to com.zwire.hud
    -> host writes ~/.zwire/hud-scheme (atomic tmp + rename)
-> worker: pushToZpwr (runtime.sendMessage, lastPushed guard)
renderers reconcile to the file / the mirror:
content scripts + chrome:// skin    <- storage.local zb_scheme (observed)
new-tab extension                  <- polls hud-scheme every 1.5s
zpwchrome                          <- the runtime.sendMessage push
fork native color mixer            <- reads hud-scheme natively
```

Opening the command palette with ⌘K:

```
ordinary page    zpalette.js catches ⌘K in a capture listener -> opens
new-tab page     Chrome reserves ⌘K -> worker owns it as a chrome.commands
                  shortcut -> sendMessage(newtab, zwireOpenPalette) -> palette opens
inside a pane    the forwarder postMessages {palette:1} up -> top frame opens,
                  navigates the active pane instead of a stray tab
```

A keystroke in the terminal overlay:

```
xterm onData -> content-script port {pty_write} -> worker (zwire-pty, per-tab)
-> connectNative com.zwire.hud -> host os.write(master, bytes)
shell output -> host reads 64KB -> base64 {ev:output,b64} -> worker
-> (buffer if detached, else) content-script port -> atob -> xterm.write
navigation: content script disconnects, worker keeps sess.nat alive,
next page re-injects, re-attaches to the same tab's shell – session survives
```

Each path crosses at least one boundary a single page cannot cross on its own — a suspended worker, a second extension, a native process — and each is routed around its guard by the mechanism the earlier chapters named: the storage bus for the worker, [externally\\_connectable](#) for the other extension, the per-tab port for the PTY. That routing, repeated for every feature, is the whole of what makes a stack of extensions behave like one browser.

# Appendix AB – Glossary

The recurring names, in one place.

```
— native files + host —————
hud-scheme      the active scheme; the cross-extension source of truth (a file)
hud-ui.json     shared visual-effect + light-mode prefs (a file)
com.zwire.hud   the native-messaging host manifest -> the Rust zwire-host binary
zwire-pty       the runtime.connect port name for the terminal's PTY relay

— storage keys (chrome.storage.local, Appendix I) —————
zb_scheme      mirror of the active scheme (worker <-> content scripts)
zb_cmd         the command bus (content script -> worker, Appendix U)
zb_sys        the streamed system-stats sample (statusbar, Appendix S)
zb_tabs/_exts/_frecency/_shortcuts  worker-maintained mirrors for the palette
zb_keys       keybinding remaps { action: key }
zb_custom_cmds the %K keyword commands (Appendix H)
zb_term_open   whether the terminal re-opens on the next page
zb_tmux*      overlay readout, window-engine state, sessions, prefix, opts
zb_status     whether the powerline statusbar is shown
zb_ci*       CI-monitor config, status, cache

— globals (in the page / content scripts) —————
window.ZWIRE_HUD  the 8-scheme palette table (VAR_KEYS, ORDER, SCHEMES)
window.ZWIRE_KEYMAP the keybinding registry (Appendix C)
window.ZGui      the shared toolkit (fzf, palette, fx, util – Appendix X)
window.ZBHUD     the HUD-page shell (mount, palette, publishUi)

— the three extension ids —————
hud-internal    omcgnnjfbmpdlofklbpddkhnfibfhgg
zpwichrome     hpppdchpnphmiijdeanibpcadgknmaja
new-tab        gpoepnekoiplhkegjpcocnpeijiefgieb

— other terms —————
frecency       frequency+recency history score (Appendix V)
the fork       fork/, a compiled patched Chromium as an alternate base (Ch 21)
the rebrand    the runtime, no-compile skinning path (the default, Ch 5)
```

# Appendix AC – The Statusbar Segments

The powerline statusbar (shared `ZGui.powerline` fed by `zpowerline.js`, top frame only, `position:fixed` bottom, 22px) is a left group of HUD state and a right group of live machine stats, split by a flex spacer, with true CSS-border powerline chevrons between blocks. Reading left to right:

```
— left: HUD state —————
ZW      the badge (cyan block, chevron)
C-b     the tmux prefix indicator (dim; flips to accent for 1.6s when armed)
<scheme> the active scheme's label
i·panes per-window tmux readout (active emphasized, ± when syncing)
VIM     static mode marker
%K      clickable; opens the palette

— right: live machine (from zb_sys, Appendix 5) —————
seg      data-*      format
CPU      data-cpu     <n>%
MEM      data-mem     used/total   (humanized B/K/M/G/T, 1 decimal < 100)
SWP      data-swap    used/total
DSK      data-disk    <n>%
IO       data-io      R<rate> W<rate>          (bytes/s)
NET      data-net     ↑<rate> ↓<rate>        (bytes/s)
LD       data-load    a b c              (load averages)
UP       data-up      Xd Yh Zm           (humanized uptime)
°        data-temp    <n>°C              (segment hidden if no sensor)
BATT     data-batt    ⚡/battery + <n>%
LAN      data-lip     private IP
WAN      data-pip     public IP
@        data-host    short hostname
clock    data-clock   MM-DD HH:MM:SS      (own 1-second interval)
```

The chevrons are real triangles: an 11-pixel transparent top/bottom border plus a solid side border on a pseudo-element, left-of-spacer blocks pointing right and right-of-spacer blocks pointing left, consecutive blocks alternating two background shades. A stats refresh rewrites the `data-*` spans in place; a scheme change re-themes the bar from the same twenty variables as everything else.

# Appendix AD — Grounding and Edition

Every claim in this book traces to a file in the `zwire` repository. Where the text says “the code comments say,” it is quoting the source verbatim; where it gives a number — 65 seeded commands, 44 `chrome://` destinations, a 0.28 tab slope, a 90-day frequency window, nine launch flags, twenty-seven patches — that number was read out of the code, not recalled. Where the runtime path and the source fork differ, the book marks the boundary rather than rounding up; the honest gaps (the two dead CDP-probe lines) are named where they occur because a companion grounded in the code owes the reader its seams as much as its surfaces.

Two corrections this edition makes to earlier printings are worth flagging: the state directory default is the OS application-data location (with `~/.zwire` a legacy path that is migrated once, not the live one), and the Python reference host has been removed — the native host is now the Rust `zwire-host` alone, and where earlier printings speak of a “Python reference,” read that as the historical first implementation of the same scheme/stats/PTY path `zwire-host` now owns outright.

This edition also grows with the code. The patch series is twenty-seven rather than twenty-five (0026 retargets the reading-list toast at the HUD page; 0027 binds the tab-strip control buttons to the scheme), against 67 hunks across 32 Chromium files; `hud-internal` registers eleven content-script blocks rather than seven, the four new ones being the `about:blank` redirect and the cloak-then-rebuild pair over the static `chrome://` debug pages; the HUD is twenty-three pages rather than eleven; the automation bus is 109 `browser.*` verbs; and five chapters — 49 through 53 — document the layers that did not exist when the earlier printings were set: lifecycle hooks, output triggers, pane pipelines, the audio dashboard, and the later HUD pages. Where an earlier printing gave a smaller number for any of these, the smaller number was true when it was printed.

Three corrections in this edition are retractions rather than growth, and they are recorded here because a companion that only ever revises upward is not being read against the code. The superseded `ztmux.js` and `zstatus.js` are not “left in the tree for reference” — they were deleted, and the shared `ZGui.tmux` and `powerline.js` in `zgui-core` are the only implementations. The extension manifest versions were classified in Chapter 48 as *labels*; they are resolution inputs, consumed by Chromium’s extension reload and, through the `service_worker` filename, by its service-worker registration — which is why the install-time stamping of Chapter 23 exists at all. And Chapter 14’s Sessions page was described as rendering its preview “using the same even-split geometry the overlay uses”; it was in fact a hand-written copy of that geometry, it disagreed with the overlay for any window that was not split horizontally, and it now calls `ZGui.tmux.layout` rather than mirroring it.

## The measured pass (Chapters 68–76)

Nine chapters were added by running commands against the tree named in the Provenance section rather than by reading it, and they retract seven earlier statements. Each is recorded with the file:line that settles it, because a correction without a citation is just a second opinion.

**The launcher passes ten flags, not nine.** `bin/zwire:100` adds `--autoplay-policy=no-user-gesture-required`, which Appendix E omitted entirely and which Appendix K, Chapter 19, and Chapter 46 all counted out. Chapter 60 had it right at ten. Appendix E and Appendix K are corrected; the prose chapters are left as printed and the count is stated here.

**A stale socket file does not block a new daemon.** Chapter 36 and Chapter 40 both say a stale socket “blocks a new one until it is removed.” `transport.rs:105` removes it unconditionally before every bind, so the stale case is the case that works. The real failure is the inverse and neither chapter names it: a *live*

daemon's socket is removed just as readily, so a second `zwire-host serve` on the same path silently takes the endpoint while the first keeps running unreachable. Chapter 72 has the transcript and the contrast with `zbus.rs:653-677`, which solved exactly this for the other socket.

**The stamping is macOS-only.** Chapter 23 describes four install-time rewrites and a worker-cache eviction as what “the installer” does. `scripts/localinstall.sh` is the only file that contains any of them; `localinstall-linux.sh`, `localinstall-windows.ps1`, and `bin/zwire` contain none. The eviction in particular lives at `localinstall.sh:494-507`, inside the generated *bundle* launcher, not in `bin/zwire`. Chapter 69 counts it; Appendix K is corrected.

**The syntax gate's scope is a third of what it claims.** `scripts/test.sh:2-4` and Chapter 22 both describe it as covering “the new-tab page, the theme, the root scripts.” The `find` at `scripts/test.sh:26-27` searches `newtab` and `theme` for `*.js`, which yields thirteen files, all under `newtab/`; `theme/` contains no JavaScript, and the repository root is not a search path, so the eight `v*.mjs` probes are never checked. They all parse at this commit. Chapter 74 has the enumeration.

**The toolkit suite is 25 files, not 22.** Chapter 64 gave “22 `.test.cjs` files totalling 2,691 lines.” At the recorded submodule pointer it is 25 files, 3,162 lines, 203 tests, all passing in 590 ms — and nothing in `zwire` runs them: `lib/zgui-core/package.json` has no `scripts` block, both syntax gates exclude the directory, and CI does not check it out. Chapter 64 is corrected in place.

**CI's shellcheck misses eleven tracked scripts.** Chapter 22 describes the step as covering “every shell script the project ships.” Its globs are `bin/zwire`, `scripts/*.sh`, and `fork/*.sh`; eleven tracked `*.sh` files sit outside them, four of which have SC2164 findings at CI's own severity, including an unguarded `cd` above an `rm -rf` with a relative path. Chapter 75 lists them.

**The version guard in the release script cannot see version drift.** Chapter 23 records the stamp and the incident behind it. `release-macos.sh:99-101` reads the built plist's version back and warns on a mismatch, naming `version.js/package.json` — but both sides of that comparison originate in `package.json`, so `version.js` is not an input, and a mismatch is a warning rather than a failure either way. Chapter 68 has the derivation, and records the one real drift the history contains: at `af1b549041`, released as 0.6.57, `pages/version.js` still read `0.6.56`.

One earlier statement was *checked and upheld*, and it is recorded because leaving a plausible hazard unexamined is its own kind of error. The JSON validation step in `ci.yml:33-37` has the shape that normally swallows failures — a `while` loop ending in `echo`, at the tail of a pipeline — and it does not: replicated verbatim under `bash -e` with a malformed file, the step exits 1. Chapter 75 shows the run.

## The 0.6.60 edition (Chapter 77)

One chapter is added, and unlike the chapters before it, it *replaces* rather than extends. The new tab described in Chapter 6 — a clock, an omnibox, six tiles and a `zb.tiles` key — is gone. It is a layout library now: named designs you switch between, each with Speed Dial groups, a widget grid per group, a navigation rail on any edge and a background, ported from Vivaldi's Start Page. Chapter 6 keeps the parts that remain true (the extension's shape, the vendored fonts, the palette module, the focus trick) and carries a note at its head pointing forward; Chapter 77 is the current account.

Four appendices move with it. Appendix I gains `zb_ntp`, which is the first key in the book that lives in the *new-tab* extension's storage rather than `hud-internal`'s, and demotes `zb.tiles` to a legacy key read once by migration. Appendix O's new-tab manifest goes from three permissions to seven and from version 1.0.13 to 1.0.17. Appendix W gains the three verbs that cross the extension boundary in both directions — `zbNtpData` one way, `zbNtpGet/zbNtpSet` the other. Appendix Y is twenty-four pages rather than twenty-three, and the new one is noted as the only HUD page that edits a *sibling* extension's storage.

Two figures elsewhere in the book are now low by one and are left as printed, in the convention this appendix has followed since the first edition: where an earlier printing gives a smaller number, the smaller number was

true when it was printed. The HUD page count in the Chapter 14 prose and the “twenty-three pages rather than eleven” line above both predate the layouts page.

The edition string on the cover tracks [zwire/package.json](#), which this edition reads at **0.6.60**. `zwire` is fast-moving software, a MenkeTechnologies project built on the Chromium project’s engine; this book will move with the code, and where the two ever disagree, the code is right and the book is the bug.

# Colophon

This book was written against the browser it describes, and it is generated from one source. `docs/book.md` is the manuscript; `scripts/book_pdf.sh` renders it to PDF through pandoc and LuaLaTeX with the project's own theme, and `scripts/book_epub.sh` renders the same manuscript — plus this apparatus — to a reflowable EPUB 3. The cover is not hand-placed either: `scripts/book_cover.sh` draws it at 1600×2560 from the same palette the PDF theme uses, so the cover on a storefront and the chapter headings inside the book cannot drift apart.

The display faces are Orbitron and Share Tech Mono; body text is set in Arimo. All three are licensed under the SIL Open Font License 1.1.

The architecture diagrams in `docs/diagrams/` are authored as SVG and converted to vector PDF at build time, so they scale rather than pixelate and they carry the same palette as the rest of the book.

Every figure in these pages — patch counts, verb counts, flag counts, test counts, version numbers — was read out of the tree named in the front matter by running the tool that produces it, and is cited to the file and line that settles it. The later chapters were produced by running commands against that tree rather than by reading it, and where they contradict an earlier chapter the earlier one is corrected in place with the citation that overturned it. Where a capability needs the compiled fork rather than the runtime rebrand, the text marks the boundary rather than rounding up; the two dead probe lines are named where they occur for the same reason.

# Reproducing the Book's Claims

The book is written to be checkable. With the repository cloned at the commit named in the front matter, these are the commands behind its central claims.

Standing the browser up at all, which is the two commands the first chapter opens with:

```
scripts/install.sh      # fetch base + link `zwire` on PATH + rebrand (macOS)
zwire                   # launch with the HUD new tab
```

The front-end gate — `node --check` across the shipped JavaScript, then each extension's own test suite. Chapter 74 measures exactly which files this reaches and which tracked scripts sit outside its globs:

```
bash scripts/test.sh
```

The patch series against the files on disk, which is the CI gate that keeps the fork's twenty-seven patches and their manifest from drifting apart:

```
bash scripts/check-patch-series.sh
```

The native host, built and registered — the other end of every native-messaging claim in the book:

```
bash scripts/setup-native-host.sh
```

And the live verification layer: the CDP probes attach to a running `zwire` over the Chrome DevTools Protocol and assert on what the page actually rendered rather than on what the source says it should. They need the debugging port opened explicitly:

```
zwire --remote-debugging-port=9222 &
node vkeys.mjs
```

Appendix L lists what each of the eight probes asserts, including the two lines that are dead, and Appendix Z lists the rest of the dev-loop scripts.

## Also from MenkeTechnologies

zwire is one program in a stack that is documented the same way — from source, with the numbers cited to the tools that produce them. Companion volumes cover the shell, the terminal emulator, the multiplexer, the shared audio stack whose DSP chain zwire's audio service carries, and the bytecode VM the language frontends share.

Titles and availability change; the current catalog is published at [github.com/MenkeTechnologies](https://github.com/MenkeTechnologies).